UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

SISTEMA WEB PARA LA GESTIÓN DE CITAS MÉDICAS DE LA CLÍNICA "VIDA SALUD"

Grupo #13

Docente: MSc. Ing. Angélica Garzón Cuéllar

Materia: Sistemas de Información I — SA

N°	Apellidos y nombres	Registro
1	Ayala Ayala Edison	222153512
2	Saldaña Zurita Pablo Andres	223044253

Repositorio: github.com/edisonayala000-netizen/clinica-vida-salud

Santa Cruz – Bolivia

2026

ÍNDICE

1. PERFIL.....7
1.1 Introducción.....7
1.2 Objetivos.....7
Objetivo General.....7
Objetivos Específicos.....7
1.3 Descripción del problema.....8
1.3.1 Antecedentes.....8
1.3.2 Situación actual y problemas identificados.....8
1.3.3 Formulación del problema.....8
1.4 Alcance.....8
2. MARCO TEÓRICO.....10
2.1 Sistemas de información.....10
2.2 Proceso Unificado de Desarrollo de Software (PUDS).....10
2.3 Lenguaje Unificado de Modelado (UML).....10
2.4 Arquitectura en tres capas y patrón MVC.....10
2.5 PHP y PDO.....10
2.6 PostgreSQL.....11
2.7 HTML5, CSS3 y JavaScript.....11
2.8 Servidor web Apache y XAMPP.....11
2.9 Seguridad en aplicaciones web.....11
2.10 Git y GitHub.....11
3. MODELO DE NEGOCIO.....12
3.1 Diagrama de casos de uso del negocio.....12
3.2 Actores y trabajadores del negocio.....12
3.3 Diagrama de actividades.....13
3.4 Reglas del negocio.....14
4. FLUJO DE TRABAJO: CAPTURA DE REQUISITOS.....16
4.1 Requisitos funcionales.....16
4.2 Requisitos no funcionales.....18
4.3 Encontrar actores y casos de uso.....19
4.3.1 Actores.....19
4.3.2 Casos de uso.....20
4.4 Priorizar casos de uso.....20

4.5 Detallar casos de uso....21
CU01: Iniciar sesión....21
CU02: Gestionar usuarios y permisos....23
CU03: Gestionar pacientes....24
CU04: Gestionar perfil del paciente....26
CU05: Consultar historial de citas....27
CU06: Gestionar especialidades....28
CU07: Gestionar médicos....29
CU08: Gestionar horarios de atención....30
CU09: Programar cita....32
CU10: Reprogramar cita....33
CU11: Cancelar cita....34
CU12: Registrar asistencia....35
CU13: Consultar agenda médica....36
CU14: Generar reportes....37
4.6 Prototipar la interfaz de usuario....39
Prototipo CU01: Iniciar sesión....39
Prototipo CU02: Gestionar usuarios y permisos....41
Prototipo CU03: Gestionar pacientes....43
Prototipo CU06: Gestionar especialidades....45
Prototipo CU07: Gestionar médicos....46
Prototipo CU08: Gestionar horarios de atención....48
Prototipo CU09: Programar cita....49
Prototipo CU04: Gestionar perfil del paciente....53
Prototipo CU05: Consultar historial de citas....55
Prototipo CU10: Reprogramar cita....56
Prototipo CU11: Cancelar cita....57
Prototipo CU12: Registrar asistencia....58
Prototipo CU13: Consultar agenda médica....59
Prototipo CU14: Generar reportes....61
4.7 Estructurar el modelo de casos de uso....64
Ciclo 1 — Seguridad, registros base y programación de citas....64
Ciclo 2 — Seguimiento de citas, agenda, portal del paciente y reportes....65
Escenario completo....66
5. FLUJO DE TRABAJO: ANÁLISIS....68
5.1 Análisis de arquitectura....68

5.2 Análisis de casos de uso....68
CU01: Iniciar sesión....69
CU02: Gestionar usuarios y permisos....70
CU03: Gestionar pacientes....71
CU06: Gestionar especialidades....71
CU07: Gestionar médicos....72
CU08: Gestionar horarios de atención....72
CU09: Programar cita....73
CU04: Gestionar perfil del paciente....74
CU05: Consultar historial de citas....74
CU10: Reprogramar cita....75
CU11: Cancelar cita....75
CU12: Registrar asistencia....76
CU13: Consultar agenda médica....76
CU14: Generar reportes....77
5.3 Análisis de clases....77
Clases de análisis CU01: Iniciar sesión....79
Clases de análisis CU02: Gestionar usuarios y permisos....80
Clases de análisis CU03: Gestionar pacientes....81
Clases de análisis CU06: Gestionar especialidades....81
Clases de análisis CU07: Gestionar médicos....82
Clases de análisis CU08: Gestionar horarios de atención....82
Clases de análisis CU09: Programar cita....83
Clases de análisis CU04: Gestionar perfil del paciente....84
Clases de análisis CU05: Consultar historial de citas....84
Clases de análisis CU10: Reprogramar cita....85
Clases de análisis CU11: Cancelar cita....85
Clases de análisis CU12: Registrar asistencia....86
Clases de análisis CU13: Consultar agenda médica....86
Clases de análisis CU14: Generar reportes....87
5.4 Análisis de paquetes....87
5.4.1 Relación entre paquetes: acoplamiento y cohesión....88
5.5 Reglas de negocio por paquete....88
6. FLUJO DE TRABAJO: DISEÑO....90
6.1 Diseño de arquitectura (física y lógica)....90
6.1.1 Arquitectura física....90

6.1.2 Arquitectura lógica....91
6.2 Diseñar casos de uso (diagramas de clases de diseño y de secuencia)....93
CU01: Iniciar sesión....93
CU02: Gestionar usuarios y permisos....95
CU03: Gestionar pacientes....97
CU06: Gestionar especialidades....99
CU07: Gestionar médicos....101
CU08: Gestionar horarios de atención....103
CU09: Programar cita....105
CU04: Gestionar perfil del paciente....107
CU05: Consultar historial de citas....109
CU10: Reprogramar cita....111
CU11: Cancelar cita....113
CU12: Registrar asistencia....115
CU13: Consultar agenda médica....117
CU14: Generar reportes....119
6.3 Diseño de datos (lógico y físico)....122
6.3.1 Diseño de datos lógico — diagrama de clases....122
Mapeo de clases a tablas....126
Normalización....127
6.3.2 Diseño de datos físico....127
7. FLUJO DE TRABAJO: IMPLEMENTACIÓN....137
7.1 Herramientas de desarrollo de la aplicación web....137
7.2 Implementación de la arquitectura del sistema....138
7.3 Implementación de la arquitectura de los subsistemas....143
7.3.1 Subsistema seguridad....143
7.3.2 Subsistema pacientes....145
7.3.3 Subsistema medicos....146
7.3.4 Subsistema citas....149
7.3.5 Subsistema reportes....152
7.4 Trazabilidad del análisis al código....153
7.5 Instalación y puesta en marcha....157
8. FLUJO DE TRABAJO: PRUEBAS....158
8.1 Pruebas mínimas del examen....158
Prueba 1: Registrar un paciente nuevo....159
Prueba 2: Registrar un paciente con CI duplicado....160

Prueba 3: Programar una cita en horario disponible....161
Prueba 4: Programar otra cita superpuesta para el mismo médico....161
Prueba 5: Intentar reservar fuera del horario médico....162
Prueba 6: Cancelar una cita....163
Prueba 7: Consultar el historial de un paciente....164
Prueba 8: Buscar las citas de un médico por fecha....164
Prueba 9: Ingresar a una página interna sin iniciar sesión....165
Prueba 10: Ejecutar una operación con datos inválidos....166
8.2 Pruebas de la base de datos....167
8.3 Pruebas de validación y permisos por caso de uso....168
8.4 Resultado de las pruebas....169
CONCLUSIÓN....170
RECOMENDACIÓN....170
BIBLIOGRAFÍA....170
CÓDIGO FUENTE....171
ANEXOS....172
Anexo A. Usuarios de demostración....172
Anexo B. Reportes adicionales....172
Anexo C. Paneles por rol....174
Anexo D. Glosario....176

1. PERFIL

1.1 Introducción

La Clínica Vida Salud es un establecimiento médico privado que brinda atención en medicina general, pediatría, ginecología, odontología y cardiología. Su recepción registra a los pacientes, consulta la disponibilidad de los médicos, programa las citas y comunica a cada paciente la fecha y hora de su atención. Hasta ahora este trabajo se realiza con cuadernos, llamadas telefónicas y hojas de cálculo, lo que provoca citas duplicadas, datos de pacientes perdidos o mal escritos y demoras para responder preguntas tan simples como «¿qué horarios tiene libre el doctor el martes?».

El presente documento describe el análisis, diseño, implementación y pruebas del **Sistema Web de Gestión de Citas Médicas**, desarrollado con **PHP** como lenguaje del servidor, **PostgreSQL** como gestor de base de datos (conexión **PDO** con el controlador **pdo_pgsql**), **HTML5**, **CSS3** y **JavaScript**, y ejecutado sobre el servidor **Apache de XAMPP**. El desarrollo sigue el **Proceso Unificado de Desarrollo de Software (PUDS)** y se modela con **UML**, recorriendo los flujos de trabajo de captura de requisitos, análisis, diseño, implementación y pruebas en dos ciclos (iteraciones).

El sistema centraliza la información de pacientes, médicos, especialidades y horarios de atención; calcula automáticamente los horarios disponibles; impide en el servidor y en la base de datos que un médico tenga dos citas en el mismo horario; controla los estados de cada cita (programada, confirmada, atendida, cancelada e inasistencia) sin borrar su historial, y genera los reportes administrativos que la dirección necesita. Además ofrece un portal para que el paciente cree su cuenta, mantenga su perfil médico, reserve sus citas en línea y reciba recordatorios.

1.2 Objetivos

Objetivo General

Desarrollar un sistema web de gestión de citas médicas para la Clínica Vida Salud, utilizando PHP como lenguaje de programación del lado del servidor y PostgreSQL como sistema gestor de base de datos, con el propósito de mejorar el registro, la programación, el seguimiento y el control de las citas médicas.

Objetivos Específicos

- Analizar los procesos de registro y programación de citas de la clínica mediante el modelo de negocio y la captura de requisitos.
- Diseñar una base de datos relacional normalizada en PostgreSQL, con tablas relacionadas mediante claves primarias y foráneas.
- Implementar formularios para el registro, consulta, modificación y desactivación de pacientes y médicos.
- Gestionar las especialidades médicas y los horarios de atención de cada médico.
- Programar citas verificando la disponibilidad del médico en el servidor y en la base de datos.
- Controlar los estados de las citas médicas con transiciones válidas y trazabilidad de cada cambio.
- Generar reportes de citas por fecha, médico, especialidad y estado, con totales y exportación.
- Aplicar validaciones de datos (JavaScript y PHP) y mecanismos básicos de seguridad (contraseñas cifradas, sesiones, permisos por rol, consultas parametrizadas).
- Realizar pruebas para verificar el correcto funcionamiento del sistema.

1.3 Descripción del problema

1.3.1 Antecedentes

La recepción de la Clínica Vida Salud atiende diariamente llamadas y visitas de pacientes que solicitan una cita. Para programarla, la recepcionista revisa un cuaderno por médico, anota a mano los datos del paciente y, cuando necesita información de un paciente antiguo, la busca en hojas de cálculo distintas para cada especialidad. Los horarios de los médicos se comunican de palabra o en papeles pegados en la recepción, y los cambios no siempre llegan a todos.

1.3.2 Situación actual y problemas identificados

Tabla 1. *Problemas del proceso actual de gestión de citas*

N°	Problema	Consecuencia para la clínica
P1	Duplicación de citas para un mismo médico y horario.	Dos pacientes llegan a la misma hora; uno espera o se reprograma con molestia.
P2	Pérdida o registro incorrecto de los datos de los pacientes.	CI repetidos o mal escritos, teléfonos inválidos y pacientes que no se pueden contactar.
P3	Dificultad para conocer los horarios disponibles de los médicos.	La recepcionista tarda en responder y ofrece horarios que el médico no atiende.
P4	Falta de control sobre las citas canceladas y las inasistencias.	Horarios bloqueados por citas canceladas y sin registro de quién faltó.
P5	Demoras en la búsqueda del historial de citas de un paciente.	Hay que revisar varios cuadernos para saber cuándo y con quién se atendió.
P6	Dificultad para obtener reportes de las atenciones programadas.	Los reportes se arman a mano al final del mes y con errores.
P7	Falta de información actualizada para la administración.	Las decisiones (contratar médicos, ampliar horarios) se toman sin datos.

1.3.3 Formulación del problema

¿Cómo desarrollar un sistema web utilizando PHP y PostgreSQL que permita administrar pacientes, médicos, especialidades y citas médicas, evitando conflictos de horarios y facilitando el control de las atenciones de la Clínica Vida Salud?

1.4 Alcance

El sistema es una aplicación web accesible desde un navegador (computadora o celular), destinada principalmente al personal administrativo de la clínica, y con un portal de autoservicio para los pacientes. Se organiza en cinco paquetes (subsistemas) que agrupan 14 casos de uso:

Tabla 2. *Paquetes y casos de uso del sistema*

Paquete	Responsabilidad	Casos de uso
Seguridad	Inicio de sesión seguro, usuarios, roles y permisos básicos.	CU01 Iniciar sesión CU02 Gestionar usuarios y permisos
Pacientes	Registro y consulta de pacientes, perfil	CU03 Gestionar pacientes

Paquete	Responsabilidad	Casos de uso
	del paciente e historial de citas.	CU04 Gestionar perfil del paciente CU05 Consultar historial de citas
Médicos	Médicos, especialidades y horarios de atención (incluye feriados).	CU06 Gestionar especialidades CU07 Gestionar médicos CU08 Gestionar horarios de atención
Citas	Programación, reprogramación, cancelación, asistencia y agenda.	CU09 Programar cita CU10 Reprogramar cita CU11 Cancelar cita CU12 Registrar asistencia CU13 Consultar agenda médica
Reportes	Reportes administrativos con filtros, totales y exportación.	CU14 Generar reportes

El sistema SÍ permite: registrar y consultar pacientes; registrar médicos y sus especialidades; definir horarios de atención; consultar la disponibilidad de citas; registrar, reprogramar y cancelar citas; registrar la asistencia o inasistencia del paciente; consultar el historial de citas; generar reportes administrativos; administrar usuarios y permisos básicos; y, para el paciente, registrarse en línea, gestionar su perfil, reservar citas y recibir recordatorios en su panel.

Fuera del alcance: historia clínica completa (diagnósticos, recetas, resultados de laboratorio), facturación y cobros, pagos en línea, envío de SMS o correos reales (los recordatorios se muestran dentro del sistema), videollamada integrada para la modalidad virtual y gestión de inventario de farmacia.

Usuarios del sistema: Administrador, Recepcionista, Médico y Paciente. Cada usuario inicia sesión con su nombre de usuario o correo y su contraseña; las opciones disponibles dependen de los permisos de su rol.

2. MARCO TEÓRICO

2.1 Sistemas de información

Un sistema de información es un conjunto de personas, datos, procedimientos y tecnología que recolecta, procesa, almacena y distribuye información para apoyar las operaciones y la toma de decisiones de una organización. En una clínica, el sistema de citas transforma datos sueltos (pacientes, horarios, solicitudes) en información útil: agenda del día, disponibilidad de cada médico, historial del paciente y reportes de atención.

2.2 Proceso Unificado de Desarrollo de Software (PUDS)

El PUDS es un proceso de desarrollo dirigido por casos de uso, centrado en la arquitectura, iterativo e incremental. Cada iteración (ciclo) recorre los flujos de trabajo de **captura de requisitos, análisis, diseño, implementación y pruebas**. En este proyecto se definieron dos ciclos: el Ciclo 1 construye la seguridad, los registros base y la programación de citas (módulo principal), y el Ciclo 2 completa el seguimiento de citas, la agenda, el portal del paciente y los reportes.

Artefactos principales: modelo de casos de uso (requisitos), modelo de análisis (clases de interfaz, control y entidad, diagramas de comunicación y paquetes), modelo de diseño (arquitectura, diagramas de secuencia, diseño de datos), modelo de implementación (componentes y código) y modelo de pruebas.

2.3 Lenguaje Unificado de Modelado (UML)

UML es el lenguaje gráfico estándar del Object Management Group para especificar, visualizar y documentar sistemas. En este documento se usan: diagramas de casos de uso y de actividades (requisitos y negocio), diagramas de comunicación y de clases de análisis con estereotipos de interfaz (IU), control (CTR) y entidad (CE), diagramas de paquetes, diagramas de secuencia (diseño de casos de uso), diagrama de clases (diseño lógico de datos), diagrama de despliegue (arquitectura física) y diagramas de componentes (implementación).

2.4 Arquitectura en tres capas y patrón MVC

La arquitectura en capas separa la **presentación** (lo que ve el usuario), la **lógica de negocio** (reglas y casos de uso) y el **acceso a datos** (consultas a la base). El patrón Modelo-Vista-Controlador aplica esa idea en la web: la **vista** genera el HTML, el **controlador** recibe la petición y decide qué hacer, y el **modelo** lee y escribe en la base de datos. En este sistema cada caso de uso es una clase controladora (por ejemplo `CU09ProgramarCita.php`), lo que permite ubicar el código de cada requisito con rapidez.

2.5 PHP y PDO

PHP es un lenguaje de programación del lado del servidor, ampliamente usado para aplicaciones web. Se utiliza PHP 8 sin frameworks, con programación orientada a objetos, espacios de nombres (namespaces) y un autocargador de clases.

PDO (PHP Data Objects) es la capa de acceso a bases de datos de PHP. Con el controlador **pdo_pgsql** se conecta a PostgreSQL. Sus **sentencias preparadas** (`prepare` + `execute` con parámetros) separan el código SQL de los datos que escribe el usuario y evitan la **inyección SQL**. PDO también ofrece

transacciones (`beginTransaction`, `commit`, `rollBack`) para que un conjunto de operaciones se guarde completo o no se guarde nada.

2.6 PostgreSQL

PostgreSQL es un sistema gestor de bases de datos relacional de código abierto, robusto y compatible con el estándar SQL. Además de llaves primarias y foráneas, permite restricciones `CHECK`, índices únicos sobre expresiones, restricciones de exclusión (`EXCLUDE`) y funciones en PL/pgSQL ejecutadas por **triggers** (disparadores).

La restricción `EXCLUDE USING gist` con la extensión **btree_gist** impide que dos filas tengan el mismo médico y rangos de tiempo que se cruzan (&&). Es la última barrera contra las citas superpuestas aunque dos usuarios guarden al mismo tiempo. La sentencia `SELECT ... FOR UPDATE` bloquea una fila durante una transacción para serializar operaciones concurrentes.

2.7 HTML5, CSS3 y JavaScript

HTML5 define la estructura semántica de las páginas y ofrece validaciones nativas de formularios (`required`, `pattern`, `type="email"`, `type="date"`). **CSS3** define la presentación: variables de color, cuadrículas (grid), diseño adaptable a celulares (media queries) y estilos de impresión. **JavaScript** mejora la interacción: valida los formularios antes de enviarlos y consulta al servidor con `fetch` los médicos y horarios disponibles sin recargar la página.

2.8 Servidor web Apache y XAMPP

XAMPP es un paquete que instala Apache, PHP y otras herramientas para desarrollo local. **Apache HTTP Server** recibe las peticiones del navegador y las entrega a PHP. El módulo `mod_rewrite` y los archivos `.htaccess` dirigen todas las URL al controlador frontal `backend/public/index.php` y dejan fuera de la web las carpetas internas del proyecto.

2.9 Seguridad en aplicaciones web

Contraseñas: se guardan con `password_hash()` (algoritmo bcrypt con sal aleatoria) y se comprueban con `password_verify()`; nunca se almacenan en texto plano. **Sesiones:** la cookie de sesión es HttpOnly y SameSite, el identificador se regenera al ingresar y la sesión expira por inactividad. **Control de acceso:** cada ruta exige una sesión válida y un permiso del rol. **Inyección SQL:** se evita con consultas parametrizadas. **XSS:** toda salida se escapa con `htmlspecialchars`. **CSRF:** cada formulario incluye un token secreto que el servidor compara. **Manejo de errores:** el usuario ve mensajes claros y los detalles técnicos se registran en un archivo, sin mostrar credenciales ni consultas internas.

2.10 Git y GitHub

Git es un sistema de control de versiones que registra los cambios del código; GitHub aloja el repositorio en la nube para compartirlo y presentarlo. El código fuente organizado y comentado se publica en github.com/edisonayala000-netizen/clinica-vida-salud.

3. MODELO DE NEGOCIO

El modelo de negocio describe cómo funciona hoy la atención de citas en la clínica, quiénes participan y qué reglas deben cumplirse, antes de pensar en el software. Sirve para entender el contexto y validar que el sistema resuelva el proceso real.

3.1 Diagrama de casos de uso del negocio

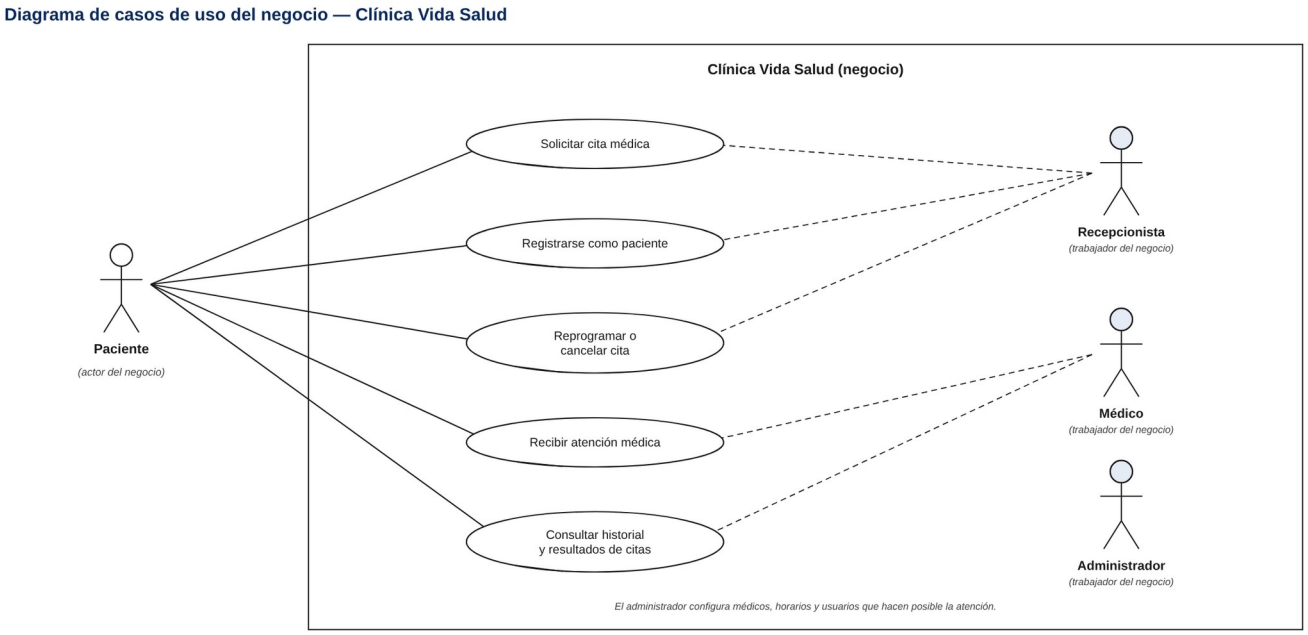

Figura 1. *Diagrama de casos de uso del negocio de la Clínica Vida Salud.*

El **paciente** es el actor del negocio: solicita su cita, se registra, la reprograma o cancela, recibe la atención y consulta sus citas. La **recepcionista**, el **médico** y el **administrador** son trabajadores del negocio que hacen posible esos servicios.

3.2 Actores y trabajadores del negocio

Tabla 3. *Actores y trabajadores del negocio*

Actor / trabajador	Tipo	Responsabilidades en el negocio
Paciente	Actor del negocio	Solicita citas, entrega sus datos, asiste a la consulta, avisa cambios o cancelaciones.
Recepcionista	Trabajador	Registra pacientes, consulta horarios, programa, reprograma y cancela citas, confirma asistencia y registra inasistencias.
Médico	Trabajador	Atiende a los pacientes según su agenda y registra que la consulta se realizó.
Administrador	Trabajador	Organiza médicos, especialidades y horarios, administra las cuentas y revisa los reportes.

3.3 Diagrama de actividades

El siguiente diagrama de actividades describe el flujo principal: desde que el paciente solicita una cita hasta que la consulta queda registrada como atendida, cancelada o con inasistencia. Los carriles muestran qué hace cada participante y qué operaciones realiza automáticamente el sistema sobre la base de datos.

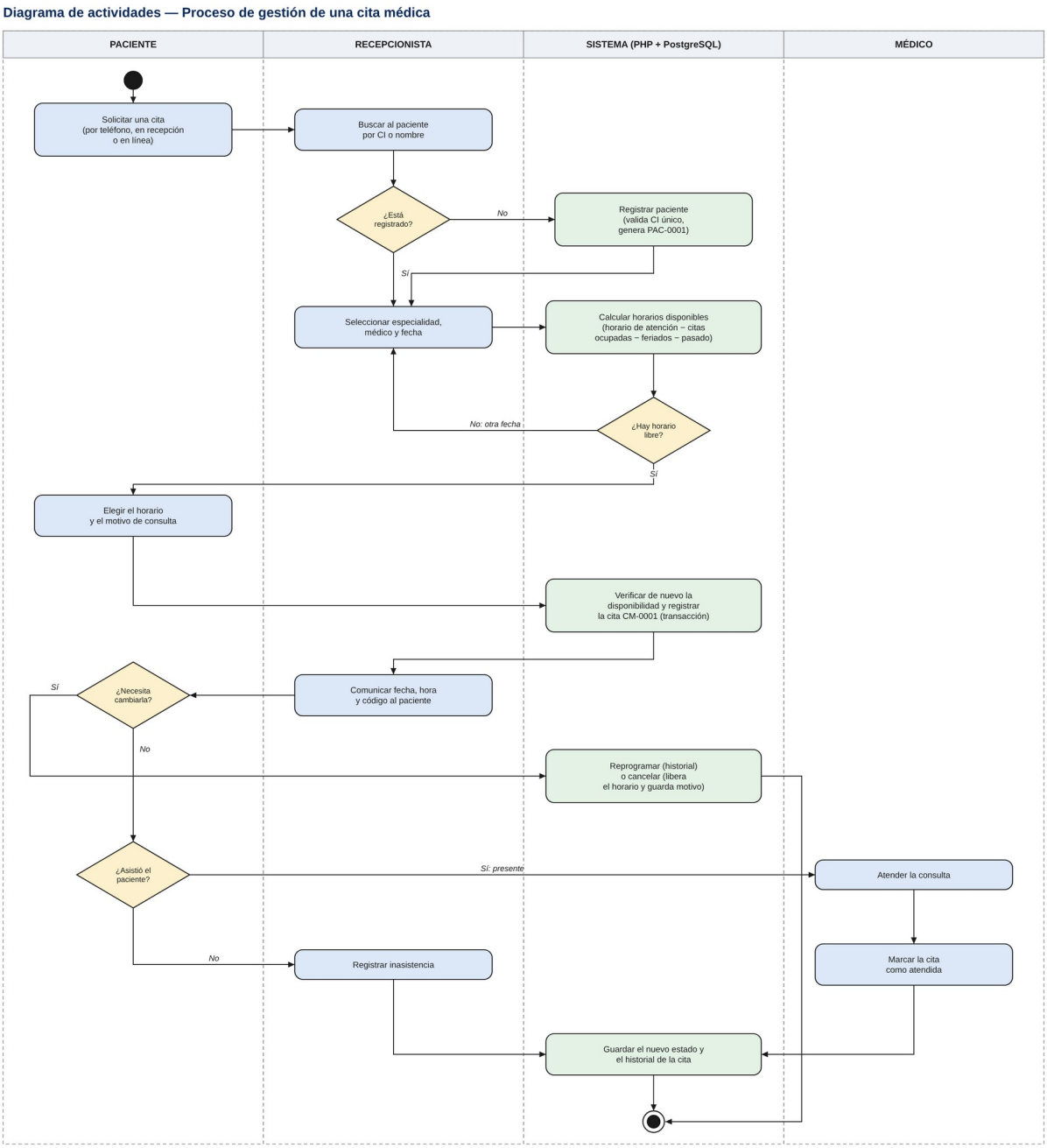

Figura 2. *Diagrama de actividades del proceso de gestión de una cita médica.*

El flujo inicia cuando el paciente solicita una cita. La recepcionista lo busca por CI o nombre; si no está registrado, el sistema lo registra validando que el CI sea único. Luego se elige especialidad, médico y fecha, y el sistema calcula los horarios disponibles restando del horario de atención las citas ocupadas, los feriados y las horas ya pasadas. Si no hay horario libre se propone otra fecha. Elegido el horario, el sistema vuelve a verificar la disponibilidad dentro de una transacción y registra la cita con su código único. La recepcionista comunica fecha, hora y código al paciente. Si el paciente necesita cambiarla, la cita se reprograma (queda en el historial) o se cancela (libera el horario). El día de la cita, si el paciente no se presenta se registra la inasistencia; si asiste, el médico lo atiende y marca la cita como atendida.

3.4 Reglas del negocio

Estas reglas se aplican en el servidor independientemente de lo que el usuario escriba en los formularios; varias de ellas están además garantizadas por la base de datos.

Tabla 4. *Reglas del negocio*

Código	Regla	Dónde se garantiza
RN01	Cada paciente tiene un identificador único (código PAC-0001) y un CI que no se repite.	CU03 (PHP) + UNIQUE uq_paciente_ci
RN02	Cada médico tiene un identificador único (código MED-001) y un registro profesional único.	CU07 + UNIQUE
RN03	Una cita está asociada a un paciente y a un médico válidos, y el médico atiende la especialidad.	CU09 + FK compuesta
RN04	El médico debe estar activo para recibir nuevas citas.	Disponibilidad + trigger fn_tg_validar_cita
RN05	La cita debe pertenecer al horario de atención del médico.	Disponibilidad::verificar + trigger
RN06	No se registran citas en fechas u horas pasadas.	Validador + Disponibilidad + trigger
RN07	Un médico no puede tener citas superpuestas.	Disponibilidad + FOR UPDATE + EXCLUDE
RN08	La duración de la cita respeta la configuración del horario (intervalos).	Disponibilidad (intervalos)
RN09	Las citas canceladas no ocupan disponibilidad.	EXCLUDE ... WHERE estado <> cancelada
RN10	Las citas atendidas conservan su registro histórico; ninguna cita se elimina.	EstadoCita + trigger que impide DELETE
RN11	No se eliminan pacientes ni médicos con citas; se desactivan.	CU03, CU07 + FK RESTRICT
RN12	Solo los usuarios autorizados realizan cada operación.	Enrutador + Autenticacion (permisos)
RN13	La programación y la reprogramación verifican la disponibilidad en el servidor.	CU09, CU10 (transacción)
RN14	Los datos se almacenan de forma	BaseDatos (PDO pdo_pgsql)

Código	Regla	Dónde se garantiza
	persistente en PostgreSQL.	

4. FLUJO DE TRABAJO: CAPTURA DE REQUISITOS

La captura de requisitos se realizó a partir del caso de estudio entregado por la docente, del análisis del proceso actual de la recepción (modelo de negocio) y de la revisión de los formatos que hoy usa la clínica (cuaderno de citas, hojas de cálculo de pacientes y horarios de los médicos). Con esa información se definieron los requisitos funcionales y no funcionales, los actores y los casos de uso.

4.1 Requisitos funcionales

Tabla 5. *Requisitos funcionales*

Código	Requisito	Descripción	Casos de uso
RF01	Inicio de sesión y control de acceso	Validar usuario o correo y contraseña (password_hash / password_verify), iniciar una sesión segura, identificar el rol, mostrar el menú según los permisos, cerrar la sesión e impedir el acceso a páginas internas sin sesión válida. Registrar en una bitácora los inicios y cierres de sesión, los intentos fallidos y cada operación, consultable por el administrador.	CU01, CU02
RF02	Gestión de pacientes	Registrar pacientes (código, CI, nombres, apellidos, fecha de nacimiento, sexo, teléfono, correo, dirección, estado), listarlos, buscarlos por CI, nombre o apellido, modificarlos, consultar sus citas anteriores y desactivarlos sin eliminar su historial. No se permiten dos pacientes con el mismo CI.	CU03, CU04
RF03	Gestión de médicos	Registrar médicos (código, registro profesional, nombres y apellidos, teléfono, correo, especialidades, estado), consultar el listado, buscar por nombre o especialidad, modificar, consultar su agenda y activar o desactivar. Un médico inactivo no recibe nuevas citas.	CU07
RF04	Gestión de especialidades	Administrar las especialidades (identificador, nombre, descripción opcional, estado) sin repetir nombres normalizados.	CU06
RF05	Gestión de horarios de atención	Registrar los horarios de cada médico por día, con la duración	CU08

Código	Requisito	Descripción	Casos de uso
		de cada cita, y calcular los horarios disponibles. Hora final posterior a la inicial, sin horarios inválidos, sin citas fuera del horario ni en fechas pasadas, respetando los intervalos; bloqueo de feriados.	
RF06	Registro de citas médicas	Seleccionar paciente, especialidad, médico, fecha, hora disponible y motivo opcional; generar un código único de cita y registrar la fecha de creación; mostrar solo horarios disponibles y volver a comprobar la disponibilidad al guardar.	CU09
RF07	Prevención de citas duplicadas	Impedir que un médico tenga dos citas incompatibles en el mismo horario o fuera de su horario, validando en el servidor y con transacciones y restricciones de base de datos.	CU09, CU10
RF08	Reprogramación de citas	Cambiar fecha y hora de una cita programada: recuperar sus datos, mostrar nuevas fechas y horas disponibles, verificar el nuevo horario, evitar conflictos y guardar manteniendo la integridad. Las citas atendidas o canceladas no se reprograman.	CU10
RF09	Cancelación de citas	Cancelar citas programadas registrando el estado y el motivo; no se eliminan físicamente, liberan el horario y quedan para consultas históricas. Una cita atendida no se cancela.	CU11
RF10	Control de asistencia	Manejar los estados programada, confirmada, atendida, cancelada e inasistencia, con transiciones válidas.	CU12
RF11	Consulta de agenda médica	Consultar la agenda de un médico para una fecha (hora, código, paciente, especialidad, estado y motivo si tiene permiso) o para toda la semana (vista semanal), y filtrar por médico, fecha y estado.	CU13
RF12	Historial de citas por paciente	Visualizar fecha y hora,	CU05

Código	Requisito	Descripción	Casos de uso
		médico, especialidad, estado y fecha de registro de cada cita, incluidas las pasadas y canceladas.	
RF13	Reportes	Citas programadas por fecha, por médico, por especialidad y por estado; pacientes registrados; atendidas, canceladas e inasistencias del período; agenda diaria de cada médico; filtros por fecha o rango con totales; exportación a CSV o PDF.	CU14
RF14	Perfil del paciente	Registro e inicio de sesión del paciente, edición de datos personales, de contacto e información médica, contacto de emergencia, reserva y seguimiento de citas, recordatorios y preferencias de privacidad.	CU04, CU05, CU09, CU11

4.2 Requisitos no funcionales

Tabla 6. *Requisitos no funcionales*

Código	Atributo	Descripción
RNF01	Seguridad — autenticación	Contraseñas con password_hash() (bcrypt); bloqueo de 15 minutos tras 5 intentos fallidos; sesión con cookie HttpOnly y SameSite, regenerada al ingresar y cerrada tras 30 minutos de inactividad.
RNF02	Seguridad — autorización	Cada ruta exige sesión y permiso; el menú se arma según los permisos del rol; un médico solo ve su agenda y un paciente solo sus datos.
RNF03	Seguridad — datos	Consultas SQL parametrizadas (PDO), escape de salida contra XSS, token CSRF en todos los formularios y errores sin detalles internos.
RNF11	Seguridad — auditoría	Bitácora de acciones con usuario, acción, módulo, detalle, IP, fecha y hora; nunca guarda contraseñas ni datos de formularios y la base de datos impide modificarla o borrarla (trigger fn_tg_bitacora_inalterable).
RNF04	Integridad	PK, FK, CHECK, UNIQUE y EXCLUDE en PostgreSQL; triggers que validan agenda, transiciones y que impiden borrar citas; transacciones en operaciones

Código	Atributo	Descripción
		compuestas.
RNF05	Usabilidad	Interfaz en español, clara para personal con conocimientos básicos de informática; mensajes que explican qué pasó y cómo corregirlo; adaptable a celulares.
RNF06	Rendimiento	Las consultas frecuentes responden en menos de 2 segundos con índices sobre llaves foráneas y fechas.
RNF07	Mantenibilidad	Código organizado en paquetes, un archivo por caso de uso, comentado; separación entre interfaz, lógica de negocio y acceso a datos.
RNF08	Portabilidad	Funciona con XAMPP (Apache + PHP 8.1 o superior) y PostgreSQL 14 o superior en Windows o Linux; también con el servidor integrado de PHP.
RNF09	Compatibilidad	Navegadores modernos: Google Chrome, Microsoft Edge, Mozilla Firefox.
RNF10	Fecha y hora	Todas las fechas se manejan en la zona horaria de Bolivia (America/La_Paz) tanto en PHP como en PostgreSQL.

4.3 Encontrar actores y casos de uso

4.3.1 Actores

Tabla 7. *Actores del sistema*

Actor	Descripción	Rol en el sistema
Administrador	Configura y supervisa el sistema: registra usuarios y asigna roles y permisos, registra médicos, especialidades y horarios de atención, activa o desactiva registros y consulta los reportes generales.	administrador
Recepcionista	Gestiona las citas de los pacientes: registra y busca pacientes, consulta médicos y horarios disponibles, programa, reprograma y cancela citas, registra la asistencia o inasistencia y consulta citas por fecha y estado.	recepcionista
Médico	Consulta su agenda de atención (citas asignadas, fecha y hora, datos básicos del paciente), identifica citas pendientes, atendidas y canceladas y marca una cita como atendida si tiene ese permiso.	medico
Paciente	Se registra en línea, mantiene su perfil (datos personales, contacto, información médica, contacto de emergencia y	paciente

Actor	Descripción	Rol en el sistema
	preferencias), reserva y cancela sus citas y sigue su historial con recordatorios.	

Los cuatro actores son especializaciones del actor **Usuario**, que es quien inicia sesión (CU01).

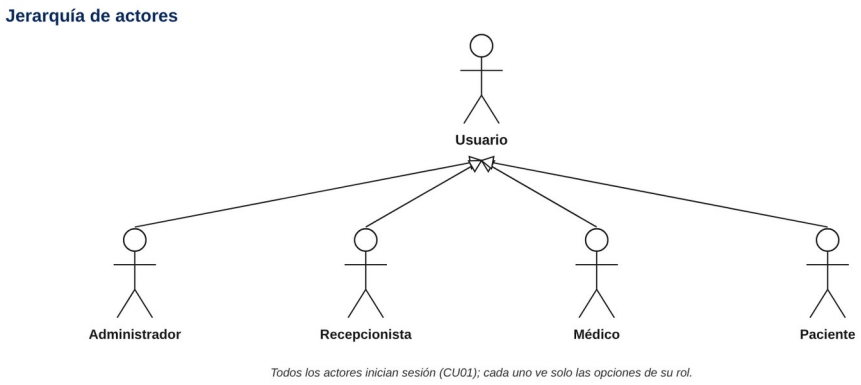

Figura 3. *Jerarquía de actores.*

4.3.2 Casos de uso

Paquete seguridad: Acceso, usuarios, roles, permisos y bitácora

- CU01. Iniciar sesión
- CU02. Gestionar usuarios y permisos

Paquete pacientes: Registro de pacientes, perfil e historial

- CU03. Gestionar pacientes
- CU04. Gestionar perfil del paciente
- CU05. Consultar historial de citas

Paquete medicos: Médicos, especialidades y horarios de atención

- CU06. Gestionar especialidades
- CU07. Gestionar médicos
- CU08. Gestionar horarios de atención

Paquete citas: Programación, cambios, estados y agenda de citas

- CU09. Programar cita
- CU10. Reprogramar cita
- CU11. Cancelar cita
- CU12. Registrar asistencia
- CU13. Consultar agenda médica

Paquete reportes: Reportes administrativos

- CU14. Generar reportes

4.4 Priorizar casos de uso

Los casos de uso se priorizaron según su importancia para el negocio y sus dependencias: primero lo necesario para entrar al sistema y registrar los datos base (pacientes, médicos, especialidades y horarios) y

el módulo principal (programar cita); después el seguimiento de la cita, la agenda, el portal del paciente y los reportes. Esto coincide con las dos presentaciones del examen: Presentación 1 (avance de al menos 50 %, Ciclo 1) y Presentación 2 (100 %, Ciclo 2).

Tabla 8. *Priorización de casos de uso por ciclo*

ID	Caso de uso	Paquete	Prioridad	Requisito
Ciclo 1 · Seguridad, registros base y programación de citas				
CU01	Iniciar sesión	Seguridad	Alta	RF01
CU02	Gestionar usuarios y permisos	Seguridad	Alta	RF01
CU03	Gestionar pacientes	Pacientes	Alta	RF02
CU06	Gestionar especialidades	Médicos	Alta	RF04
CU07	Gestionar médicos	Médicos	Alta	RF03
CU08	Gestionar horarios de atención	Médicos	Alta	RF05
CU09	Programar cita	Citas	Alta	RF06
Ciclo 2 · Seguimiento de citas, agenda, portal del paciente y reportes				
CU04	Gestionar perfil del paciente	Pacientes	Media	Punto 3.4 Perfil de pacientes
CU05	Consultar historial de citas	Pacientes	Media	RF12
CU10	Reprogramar cita	Citas	Alta	RF08
CU11	Cancelar cita	Citas	Alta	RF09
CU12	Registrar asistencia	Citas	Alta	RF10
CU13	Consultar agenda médica	Citas	Alta	RF11
CU14	Generar reportes	Reportes	Media	RF13

4.5 Detallar casos de uso

Para cada caso de uso se presenta su diagrama y su especificación: propósito, actores, actor iniciador, flujo de proceso, precondición, postcondición y excepciones.

CU01: Iniciar sesión

Paquete: Seguridad · Ciclo 1 · Prioridad Alta · Requisitos: RF01

Diagrama de caso de uso — CU01: Iniciar sesión

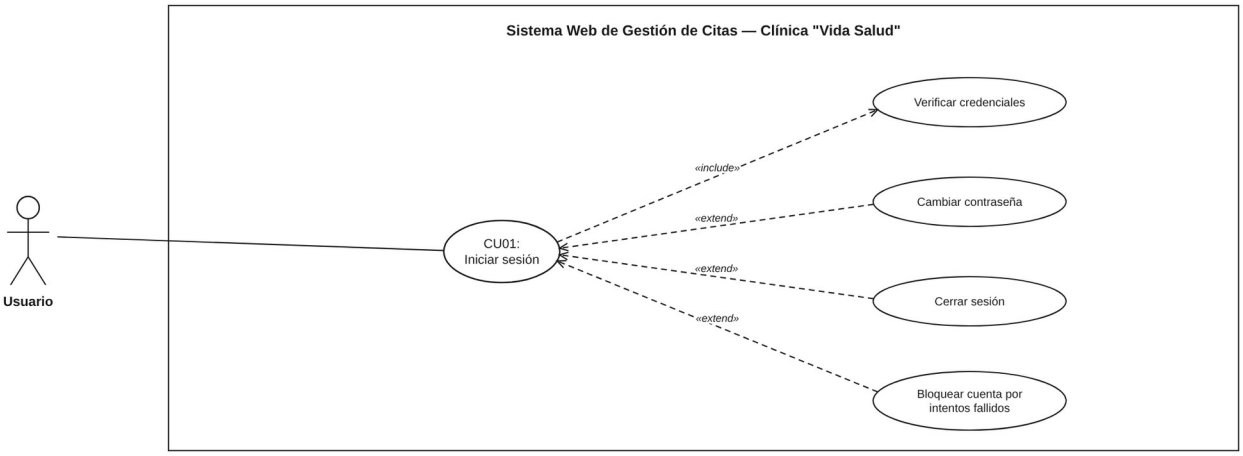

Figura 4. *Diagrama de caso de uso — CU01: Iniciar sesión.*

Caso de uso	CU01: Iniciar sesión
Propósito	Permitir que el usuario ingrese al sistema con su nombre de usuario o correo y su contraseña, identificar su rol, mostrar el menú según sus permisos y cerrar la sesión de forma segura.
Actores	Administrador, Recepcionista, Médico y Paciente (actor Usuario).
Actor iniciador	Usuario del sistema (Administrador, Recepcionista, Médico o Paciente)
Flujo de proceso	1. El usuario abre la pantalla de inicio de sesión. 2. Escribe su nombre de usuario o correo y su contraseña y presiona «Ingresar». 3. El sistema valida que los datos no estén vacíos (en el navegador y en el servidor). 4. Busca la cuenta y verifica la contraseña con password_verify() contra el hash guardado con password_hash(). 5. Verifica que la cuenta (y el médico o paciente vinculado) esté activa y no bloqueada. 6. Inicia una sesión segura (nuevo identificador de sesión), registra el último acceso e identifica el rol. 7. Registra en la bitácora el inicio de sesión (o el intento fallido o el bloqueo de la cuenta), con usuario, fecha, hora e IP. 8. Muestra el panel principal con el menú según los permisos del rol. 9. Flujo alternativo — cambiar contraseña: el usuario indica la contraseña actual y la nueva (8 a 64 caracteres con letras y números); el sistema la guarda cifrada.

	10. Flujo alternativo — cerrar sesión: el usuario elige «Cerrar sesión» y el sistema destruye la sesión.
Precondición	El usuario tiene una cuenta registrada en estado «activo».
Postcondición	El usuario accede solo a las opciones de su rol. Sin sesión válida no se puede entrar a ninguna página interna. Cada ingreso, intento fallido y cierre de sesión queda registrado en la bitácora.
Excepciones	Usuario o contraseña incorrectos: se muestra el mismo mensaje en ambos casos. Cinco intentos fallidos: la cuenta se bloquea 15 minutos. Cuenta inactiva o médico/paciente vinculado inactivo. Sesión vencida tras 30 minutos sin uso: se pide ingresar nuevamente.

CU02: Gestionar usuarios y permisos

Paquete: Seguridad · Ciclo 1 · Prioridad Alta · Requisitos: RF01 · RNF de seguridad (auditoría) · Alcance: administrar usuarios y permisos básicos

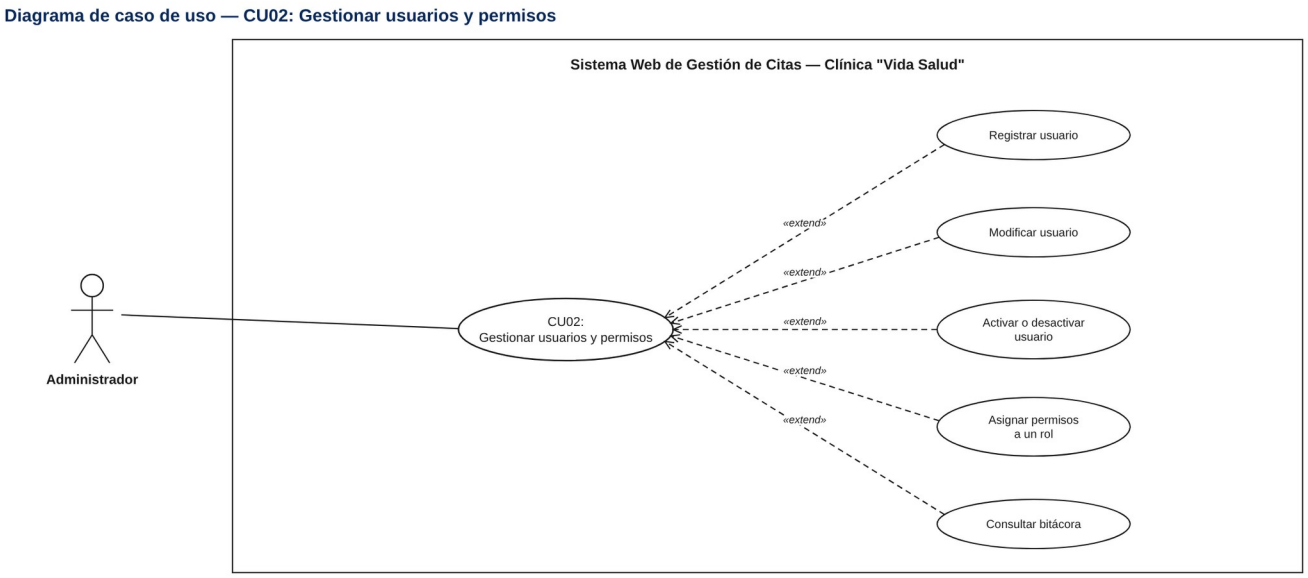

Figura 5. *Diagrama de caso de uso — CU02: Gestionar usuarios y permisos.*

Caso de uso	CU02: Gestionar usuarios y permisos
Propósito	Registrar y administrar las cuentas de usuario, asignarles un rol, vincularlas con el médico o paciente correspondiente, definir los permisos básicos de cada rol y consultar la bitácora de acciones de los usuarios.
Actores	Administrador
Actor iniciador	Administrador
Flujo de proceso	1. El administrador abre «Usuarios»; el sistema lista las

	cuentas con su rol, estado y último acceso. 2. Puede buscar por nombre, usuario o correo y filtrar por rol y estado. 3. Para registrar, elige «Nuevo usuario» e ingresa nombre completo, nombre de usuario, correo, rol y contraseña inicial. 4. Si el rol es Médico o Paciente, selecciona el médico o paciente que se vincula a la cuenta. 5. El sistema valida los datos en el servidor, cifra la contraseña con password_hash() y guarda la cuenta. 6. Para modificar, cambia los datos y, si lo desea, una nueva contraseña. 7. Puede activar o desactivar una cuenta (no la suya). 8. En «Permisos por rol» marca en una matriz qué puede hacer cada rol y guarda; el cambio se aplica de inmediato. 9. En «Bitácora» consulta quién hizo qué, cuándo y desde qué IP (inicios y cierres de sesión, intentos fallidos y cada operación enviada al sistema), filtrando por usuario, módulo, acción y rango de fechas; puede exportarla a CSV.
Precondición	El administrador inició sesión (CU01) y tiene el permiso «usuarios.gestionar» (y «bitacora.ver» para la bitácora).
Postcondición	Las cuentas y los permisos quedan registrados; el menú y el acceso de cada usuario dependen de su rol. La bitácora solo se consulta: nadie puede modificarla ni borrarla.
Excepciones	Nombre de usuario o correo ya registrados. Rol Médico/Paciente sin vincular a un médico/paciente. No se puede desactivar la propia cuenta ni dejar el sistema sin administradores activos. El administrador siempre conserva el permiso de gestionar usuarios. Intento de modificar o borrar la bitácora: la base de datos lo rechaza (trigger fn_tg_bitacora_inalterable).

CU03: Gestionar pacientes

Paquete: Pacientes · Ciclo 1 · Prioridad Alta · Requisitos: RF02 · RN01 · RN11

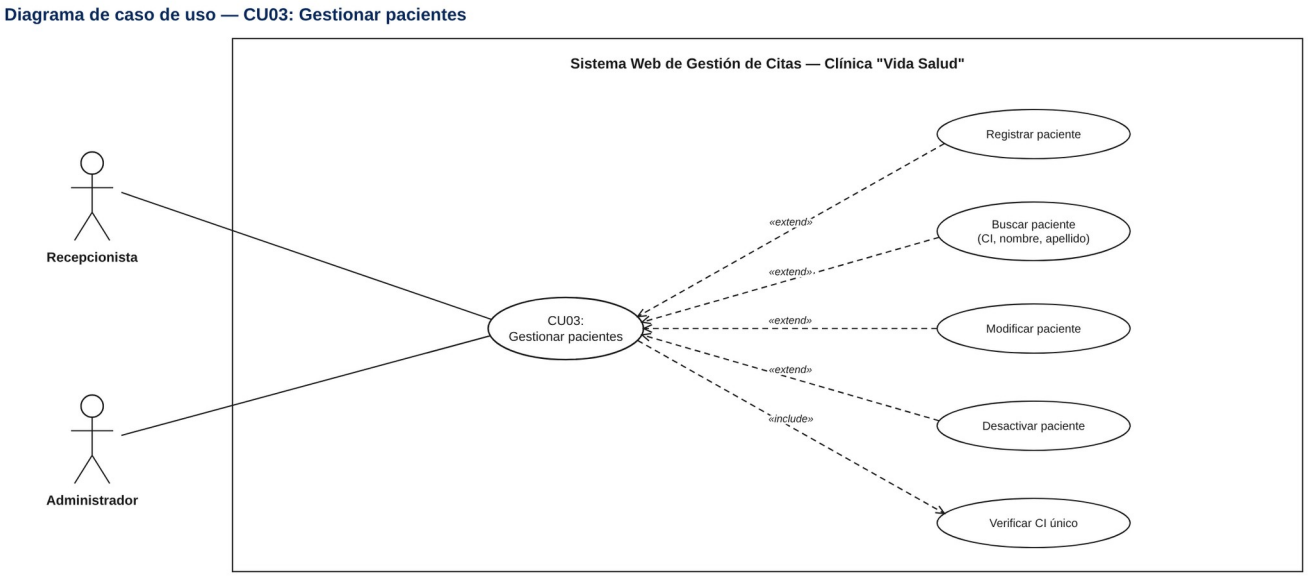

Figura 6. *Diagrama de caso de uso — CU03: Gestionar pacientes.*

Caso de uso	CU03: Gestionar pacientes
Propósito	Registrar, listar, buscar, modificar y desactivar pacientes sin perder su historial, garantizando que no existan dos pacientes con el mismo CI.
Actores	Recepcionista, Administrador
Actor iniciador	Recepcionista
Flujo de proceso	1. La recepcionista abre «Pacientes»; el sistema lista los pacientes con código, CI, edad, teléfono, próxima cita y estado. 2. Busca por CI, nombre o apellido (sin importar tildes ni mayúsculas). 3. Para registrar, elige «Registrar paciente» e ingresa CI, nombres, apellidos, fecha de nacimiento, sexo, teléfono, correo y dirección; opcionalmente la información médica y un contacto de emergencia. 4. El sistema valida los datos en el navegador (JavaScript) y en el servidor (PHP). 5. Verifica que el CI no pertenezca a otro paciente. 6. Guarda el paciente con un código único (PAC-0001) y muestra su ficha. 7. Desde la ficha puede modificar los datos, consultar sus citas anteriores (CU05) o programarle una cita (CU09). 8. Para desactivar, confirma la acción: el paciente queda inactivo, sus citas pendientes se cancelan y su historial se conserva.
Precondición	La recepcionista inició sesión y tiene los permisos «pacientes.ver» y «pacientes.gestionar».

Postcondición	El paciente queda registrado o actualizado. Un paciente inactivo no recibe nuevas citas y nunca se elimina físicamente.
Excepciones	CI ya registrado: «Ya existe un paciente registrado con ese CI». Datos obligatorios vacíos o con formato inválido (CI, teléfono, correo, fecha de nacimiento futura). Búsqueda sin resultados: se sugiere registrar al paciente.

CU04: Gestionar perfil del paciente

Paquete: Pacientes · Ciclo 2 · Prioridad Media · Requisitos: Punto 3.4 Perfil de pacientes

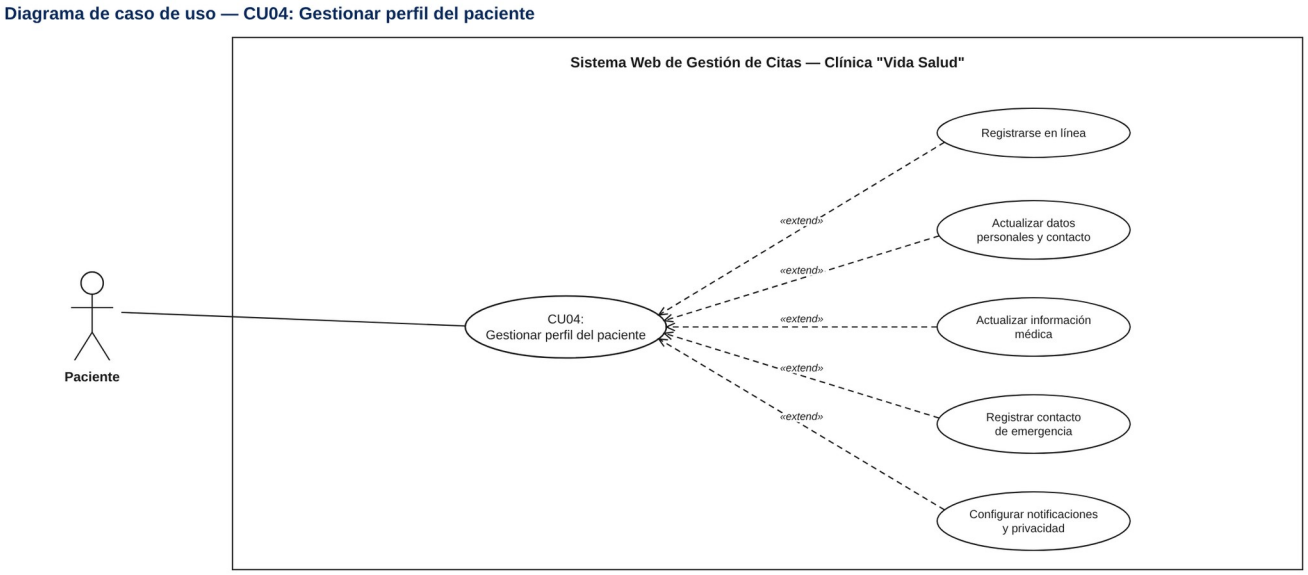

Figura 7. *Diagrama de caso de uso — CU04: Gestionar perfil del paciente.*

Caso de uso	CU04: Gestionar perfil del paciente
Propósito	Permitir que el paciente cree su cuenta en línea y mantenga actualizado su perfil: datos personales, contacto, información médica, contacto de emergencia y preferencias de notificaciones y privacidad.
Actores	Paciente
Actor iniciador	Paciente
Flujo de proceso	1. El paciente elige «Crear mi cuenta» en la pantalla de inicio de sesión. 2. Ingresa CI, nombres, apellidos, fecha de nacimiento, sexo, teléfono, correo, nombre de usuario y contraseña. 3. Si el CI es nuevo, el sistema crea el paciente y su cuenta con rol «paciente». 4. Si el CI ya fue registrado por recepción y no tiene cuenta,

	la vincula solo si la fecha de nacimiento coincide. 5. El sistema inicia la sesión y muestra el panel del paciente. 6. En «Mi perfil» el paciente actualiza por secciones: datos personales y de contacto, información médica (tipo de sangre, alergias, enfermedades crónicas, medicamentos, antecedentes) y contacto de emergencia. 7. En «Cuenta y privacidad» activa o desactiva los recordatorios y decide si comparte su información médica con sus médicos.
Precondición	Para el registro, el paciente no tiene cuenta. Para editar, inició sesión con rol Paciente.
Postcondición	El perfil queda actualizado; el CI y la fecha de nacimiento solo los corrige recepción.
Excepciones	El CI ya tiene una cuenta. El CI existe con otra fecha de nacimiento: se pide acudir a recepción. Correo o nombre de usuario ya usados. Contraseña débil o confirmación distinta.

CU05: Consultar historial de citas

Paquete: Pacientes · Ciclo 2 · Prioridad Media · Requisitos: RF12 · RN10

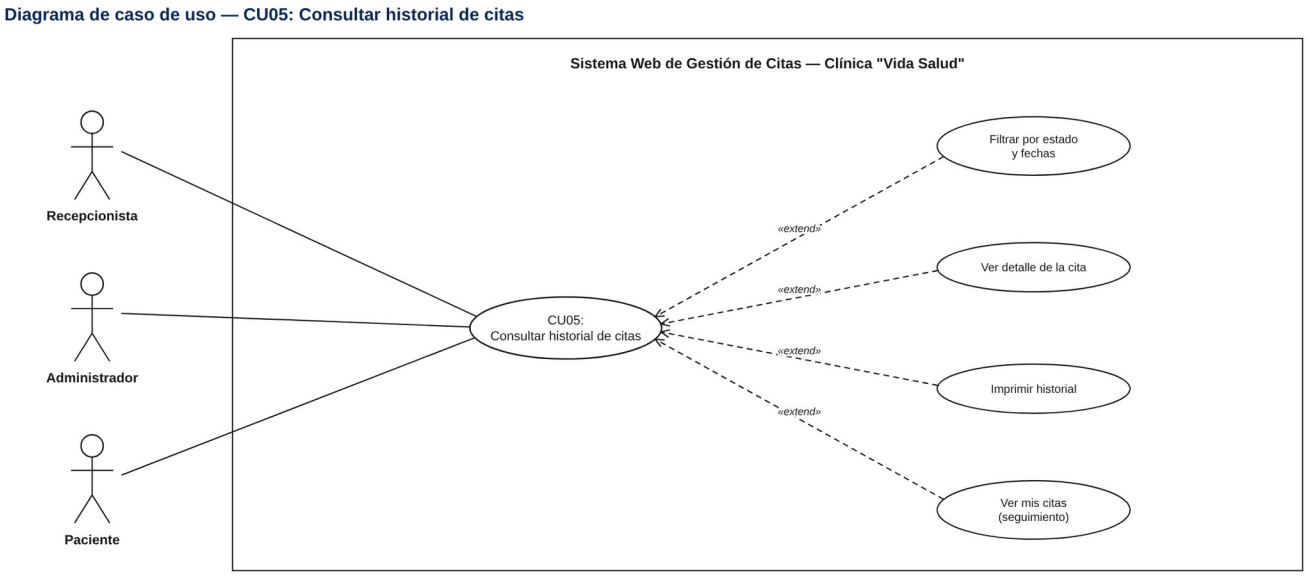

Figura 8. *Diagrama de caso de uso — CU05: Consultar historial de citas.*

Caso de uso	CU05: Consultar historial de citas
Propósito	Visualizar el historial de citas de un paciente (fecha y hora, médico, especialidad, estado y fecha de registro), incluidas las pasadas y canceladas, y el seguimiento de sus citas por parte del propio paciente.

Actores	Recepcionista, Administrador, Paciente
Actor iniciador	Recepcionista o Paciente
Flujo de proceso	1. La recepcionista abre la ficha del paciente y elige «Historial de citas». 2. El sistema muestra el resumen (total, atendidas, canceladas, inasistencias) y la tabla de citas ordenada de la más reciente a la más antigua. 3. Puede filtrar por estado y rango de fechas, abrir el detalle de una cita o imprimir el historial. 4. El paciente, desde «Mis citas», ve sus citas separadas en próximas, pasadas, canceladas e inasistencias. 5. Desde las próximas puede cancelar una cita (CU11).
Precondición	Usuario con sesión y permiso «pacientes.historial» (personal) o «portal.mis_citas» (paciente).
Postcondición	Se muestra la información sin modificarla; las citas canceladas siguen visibles (trazabilidad).
Excepciones	Paciente inexistente: «El paciente que busca no existe». Un paciente solo ve sus propias citas.

CU06: Gestionar especialidades

Paquete: Médicos · Ciclo 1 · Prioridad Alta · Requisitos: RF04

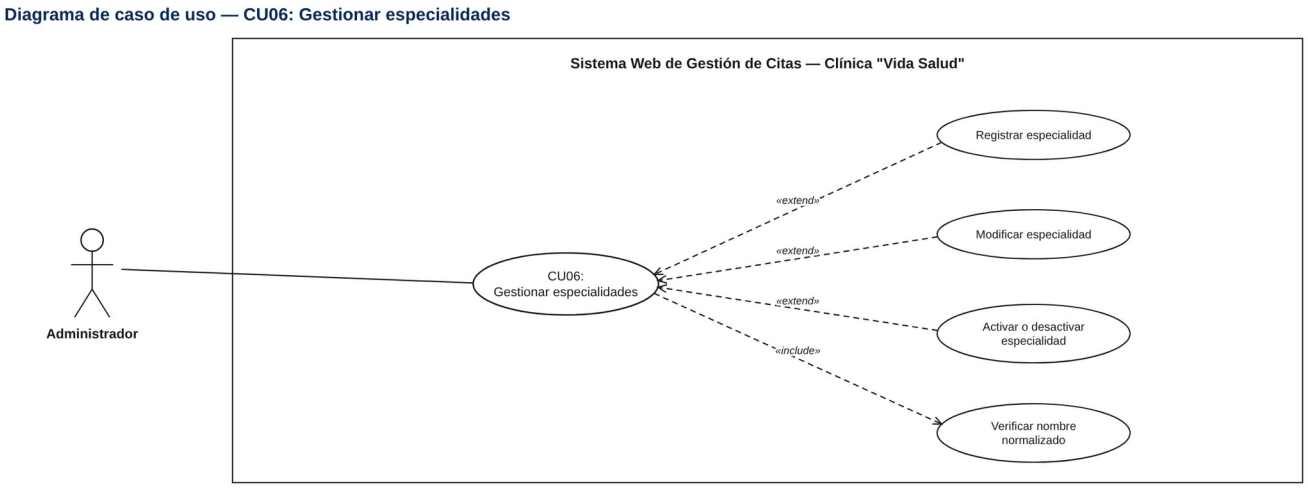

Figura 9. *Diagrama de caso de uso — CU06: Gestionar especialidades.*

Caso de uso	CU06: Gestionar especialidades
Propósito	Administrar las especialidades que ofrece la clínica (identificador, nombre, descripción opcional y estado) sin permitir nombres repetidos.
Actores	Administrador
Actor iniciador	Administrador

Flujo de proceso	1. El administrador abre «Especialidades»; el sistema lista cada especialidad con sus médicos activos, citas registradas y estado. 2. Para registrar, elige «Nueva especialidad» e ingresa nombre y descripción opcional. 3. El sistema normaliza el nombre (sin tildes, minúsculas y espacios simples) y verifica que no exista otro igual. 4. Guarda la especialidad en estado activo. 5. Para modificar, edita el nombre o la descripción en la misma ventana. 6. Para desactivar, confirma la acción; la especialidad deja de ofrecerse para nuevas citas.
Precondición	El administrador inició sesión y tiene el permiso «especialidades.gestionar».
Postcondición	La especialidad queda registrada o actualizada; las desactivadas conservan sus citas históricas.
Excepciones	Nombre repetido (p. ej. «PEDIATRIA» y «Pediatría»). Nombre vacío o con caracteres inválidos. No se desactiva una especialidad con citas pendientes.

CU07: Gestionar médicos

Paquete: Médicos · Ciclo 1 · Prioridad Alta · Requisitos: RF03 · RN02 · RN04 · RN11

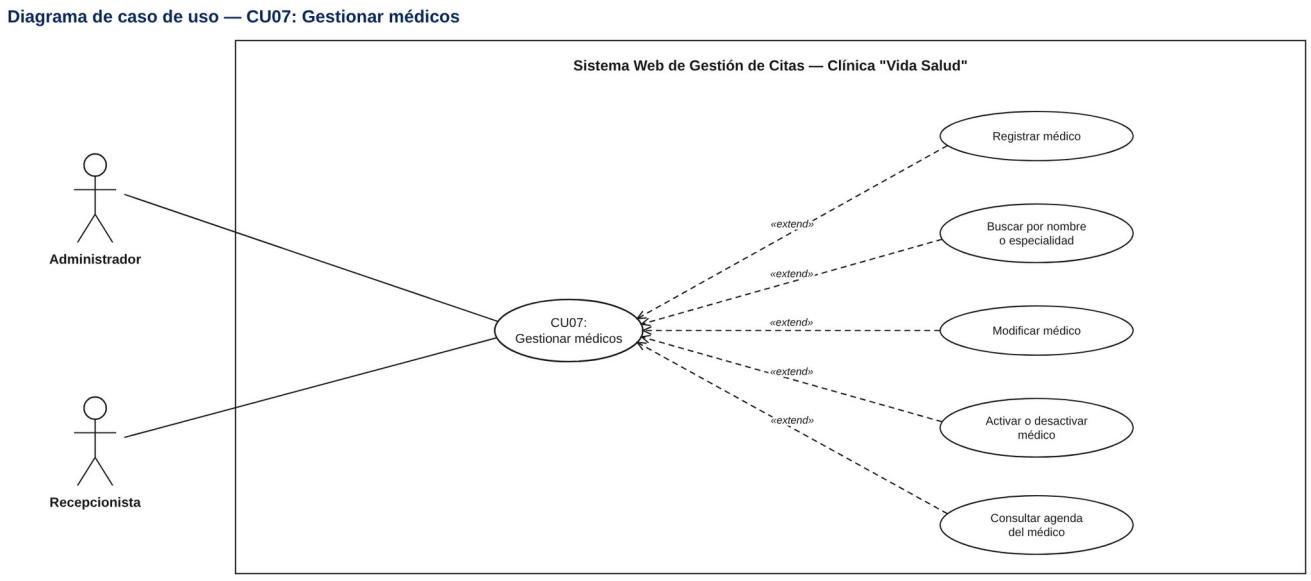

Figura 10. *Diagrama de caso de uso — CU07: Gestionar médicos.*

Caso de uso	CU07: Gestionar médicos
Propósito	Administrar la información de los médicos (código, registro profesional, nombres y apellidos, teléfono, correo,

	especialidades y estado) y consultar su agenda.
Actores	Administrador, Recepcionista
Actor iniciador	Administrador
Flujo de proceso	1. El administrador abre «Médicos»; el sistema lista los médicos con sus especialidades, horarios y citas pendientes. 2. Busca por nombre, código, registro profesional o especialidad. 3. Para registrar, ingresa registro profesional, nombres, apellidos, teléfono, correo y una o varias especialidades. 4. El sistema valida los datos, genera el código (MED-001) y guarda el médico y sus especialidades. 5. Luego lo lleva a definir sus horarios de atención (CU08). 6. Puede modificar los datos y especialidades o consultar la agenda del médico (CU13). 7. Para desactivar, confirma: el médico deja de recibir citas nuevas, sus citas pendientes se cancelan y su historial se conserva.
Precondición	El administrador inició sesión con «medicos.gestionar» (la recepcionista solo consulta con «medicos.ver»).
Postcondición	El médico queda registrado con sus especialidades; un médico inactivo no recibe nuevas citas.
Excepciones	Registro profesional o correo repetidos. Sin especialidad seleccionada. Quitar una especialidad con citas registradas.

CU08: Gestionar horarios de atención

Paquete: Médicos · Ciclo 1 · Prioridad Alta · Requisitos: RF05 · RN05 · RN08

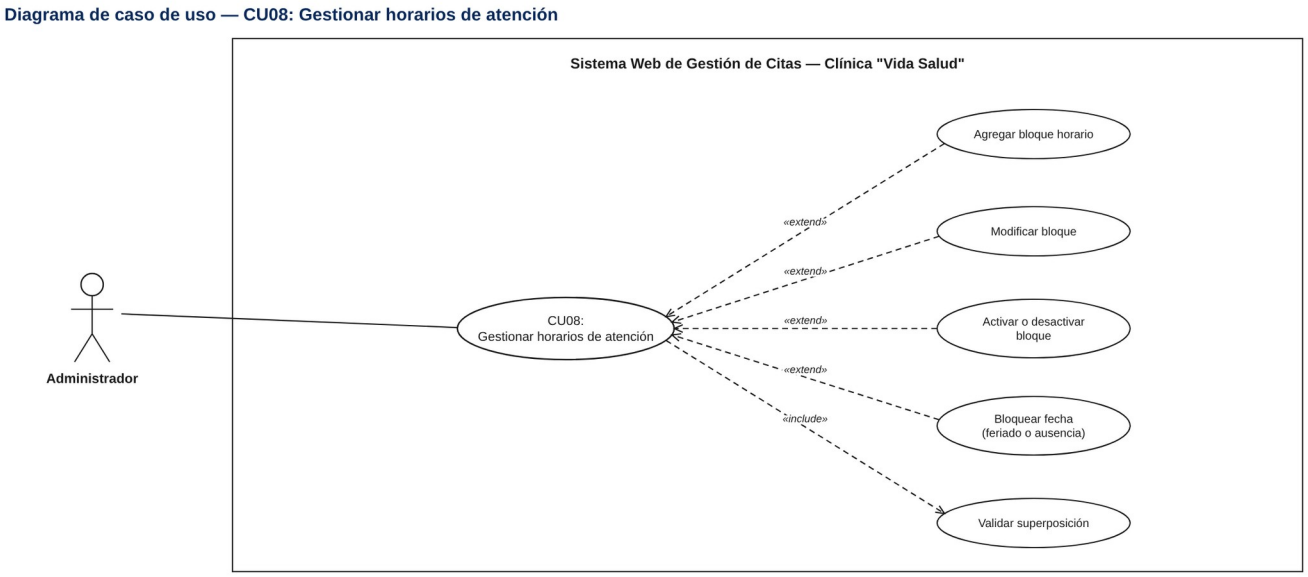

Figura 11. *Diagrama de caso de uso — CU08: Gestionar horarios de atención.*

Caso de uso	CU08: Gestionar horarios de atención
Propósito	Registrar los horarios semanales de atención de cada médico con la duración de cada cita, y bloquear fechas por feriados o ausencias, para que el sistema calcule los horarios disponibles.
Actores	Administrador
Actor iniciador	Administrador
Flujo de proceso	1. El administrador abre «Horarios» y elige un médico; el sistema muestra su semana con los bloques de atención. 2. Elige «Agregar horario» e indica día, hora inicial, hora final y duración de cada cita (15, 20, 30, 45 o 60 min). 3. El sistema valida que la hora final sea posterior a la inicial, que el bloque esté entre 06:00 y 22:00 y alcance para al menos una cita. 4. Verifica que el bloque no se superponga con otro del mismo médico ese día y lo guarda. 5. Puede modificar o desactivar un bloque si no deja fuera citas pendientes. 6. En «Feriados y ausencias» bloquea una fecha para toda la clínica o para un médico.
Precondición	El administrador inició sesión con «horarios.gestionar»; el médico está activo.
Postcondición	La agenda del médico queda definida; solo se ofrecerán citas dentro de sus horarios y fuera de las fechas bloqueadas.
Excepciones	Hora final anterior o igual a la inicial.

	Bloque superpuesto con otro del mismo día. El cambio deja fuera citas pendientes. Fecha a bloquear con citas pendientes.

CU09: Programar cita

Paquete: Citas · Ciclo 1 · Prioridad Alta · Requisitos: RF06 · RF07 · RN03–RN09 · RN13

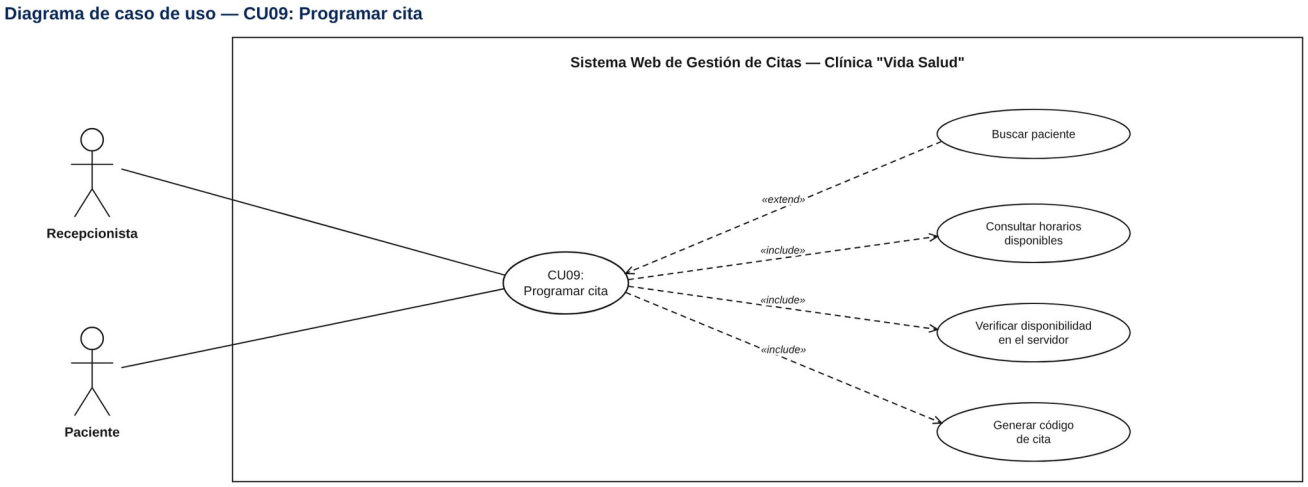

Figura 12. *Diagrama de caso de uso — CU09: Programar cita.*

Caso de uso	CU09: Programar cita
Propósito	Registrar una cita médica eligiendo paciente, especialidad, médico, fecha y una hora disponible, evitando citas duplicadas o superpuestas y generando un código único de cita.
Actores	Recepcionista, Paciente
Actor iniciador	Recepcionista (o el Paciente en línea)
Flujo de proceso	1. La recepcionista abre «Programar cita» y busca al paciente por CI, nombre o apellido (el paciente en línea ya está identificado). 2. Selecciona la especialidad; el sistema carga solo los médicos activos de esa especialidad con horario. 3. Selecciona el médico; el sistema muestra los próximos 14 días con la cantidad de horarios libres. 4. Selecciona la fecha; el sistema muestra ÚNICAMENTE los horarios disponibles según los intervalos configurados. 5. Elige la hora, la modalidad (presencial o virtual) y escribe un motivo breve opcional. 6. Confirma; el servidor bloquea al médico y vuelve a comprobar la disponibilidad dentro de una transacción. 7. El sistema registra la cita en estado «programada», genera el código único (CM-0001) y guarda la fecha de creación.

	8. Muestra el detalle de la cita para comunicar la fecha y hora al paciente.
Precondición	Usuario con permiso «citas.programar» (o «portal.reservar»); el paciente, el médico y la especialidad están activos y el médico tiene horarios.
Postcondición	La cita queda registrada sin conflictos de horario y aparece en la agenda del médico y en el historial del paciente.
Excepciones	RN07: el horario fue ocupado por otro usuario (el sistema rechaza y explica). RN05: hora fuera del horario de atención o que no respeta los intervalos. RN06: fecha u hora pasada. RN04: médico inactivo. Fecha bloqueada por feriado. El paciente ya tiene otra cita a esa hora.

CU10: Reprogramar cita

Paquete: Citas · Ciclo 2 · Prioridad Alta · Requisitos: RF08 · RN13

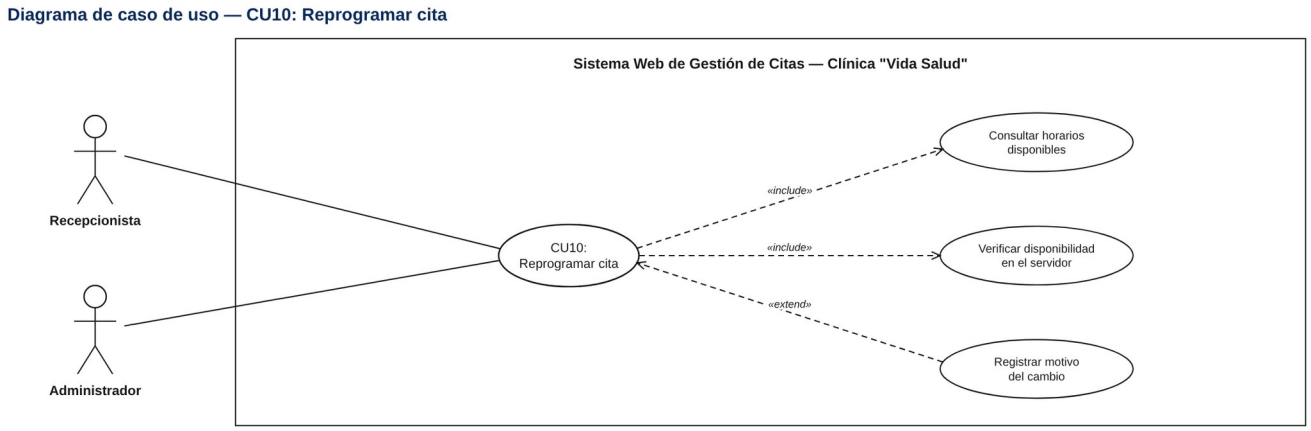

Figura 13. *Diagrama de caso de uso — CU10: Reprogramar cita.*

Caso de uso	CU10: Reprogramar cita
Propósito	Cambiar la fecha y la hora de una cita programada o confirmada, mostrando los nuevos horarios disponibles del mismo médico y manteniendo la integridad y la trazabilidad de la información.
Actores	Recepcionista, Administrador
Actor iniciador	Recepcionista
Flujo de proceso	1. La recepcionista abre el detalle de la cita y elige «Reprogramar». 2. El sistema recupera los datos de la cita y verifica que esté programada o confirmada y que su hora no haya pasado.

	3. Muestra los próximos días del mismo médico; el horario actual de la cita se considera libre. 4. La recepcionista elige la nueva fecha y hora, y opcionalmente el motivo del cambio. 5. El servidor bloquea al médico, verifica que pueda atender en el nuevo horario y que no haya conflictos. 6. Guarda la modificación; el trigger registra en el historial la fecha y hora anterior y la nueva.
Precondición	La cita existe en estado programada o confirmada y el usuario tiene «citas.reprogramar».
Postcondición	La cita queda con la nueva fecha y hora; el historial conserva el cambio.
Excepciones	Política: una cita atendida, cancelada o con inasistencia no se reprograma; se programa una nueva (CU09). Nuevo horario ocupado o fuera de horario. Misma fecha y hora que la actual.

CU11: Cancelar cita

Paquete: Citas · Ciclo 2 · Prioridad Alta · Requisitos: RF09 · RN09 · RN10

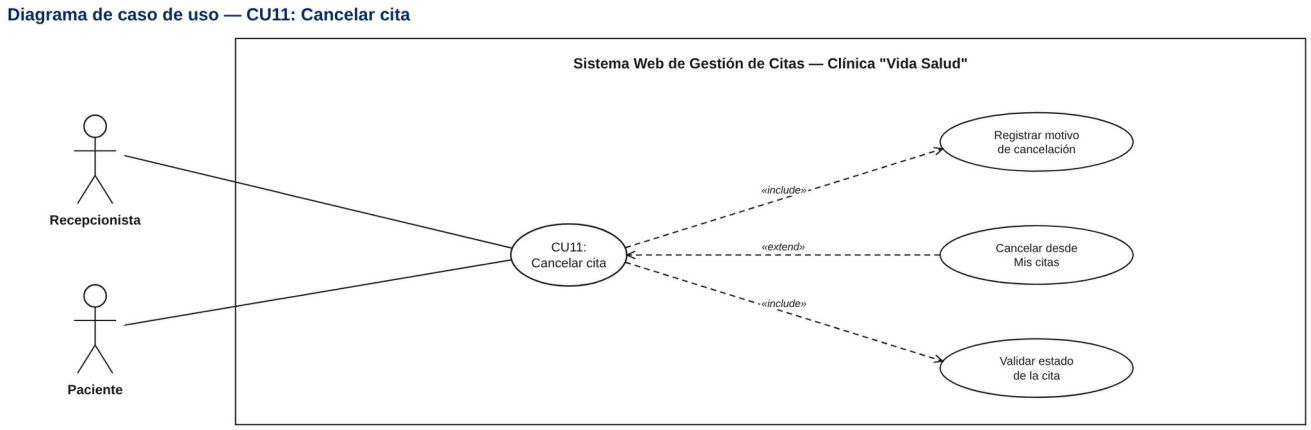

Figura 14. *Diagrama de caso de uso — CU11: Cancelar cita.*

Caso de uso	CU11: Cancelar cita
Propósito	Cancelar una cita programada o confirmada registrando el motivo, liberando el horario y conservando la cita para consultas históricas.
Actores	Recepcionista, Paciente
Actor iniciador	Recepcionista (o el Paciente en línea)
Flujo de proceso	1. La recepcionista abre el detalle de la cita y elige «Cancelar cita». 2. El sistema pide el motivo de la cancelación. 3. Verifica que la cita esté programada o confirmada y que

	aún no haya comenzado. 4. Cambia el estado a «cancelada» y guarda el motivo; NO elimina el registro. 5. El horario queda libre para otros pacientes y el historial registra el cambio de estado. 6. Flujo alternativo — paciente: desde «Mis citas» cancela con al menos 2 horas de anticipación.
Precondición	La cita existe y el usuario tiene «citas.cancelar» (o es el paciente dueño de la cita).
Postcondición	La cita queda «cancelada» con su motivo; deja de ocupar el horario y permanece en el historial.
Excepciones	Una cita atendida no puede cancelarse como si nunca hubiera ocurrido. La cita ya comenzó. Motivo vacío o muy corto. Paciente fuera del plazo de 2 horas o cita ajena.

CU12: Registrar asistencia

Paquete: Citas · Ciclo 2 · Prioridad Alta · Requisitos: RF10 · RN12

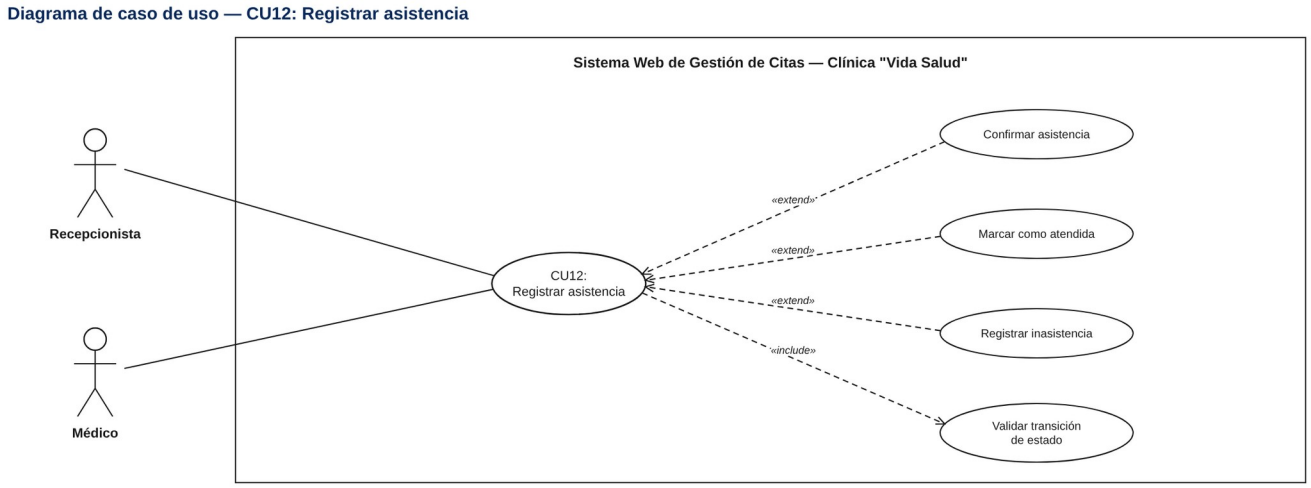

Figura 15. *Diagrama de caso de uso — CU12: Registrar asistencia.*

Caso de uso	CU12: Registrar asistencia
Propósito	Controlar los estados de la cita: confirmar la asistencia, marcarla como atendida o registrar la inasistencia, aplicando solo las transiciones válidas y los permisos de cada rol.
Actores	Recepcionista, Médico
Actor iniciador	Recepcionista o Médico
Flujo de proceso	1. La recepcionista abre la cita (o la agenda) y elige

	«Confirmar asistencia», «Marcar atendida» o «Inasistencia». 2. El sistema verifica que el usuario tenga el permiso de esa acción (el médico solo marca atendida si tiene «citas.atender» y solo sus citas). 3. Valida la transición según la máquina de estados: programada → confirmada / atendida / inasistencia / cancelada; confirmada → atendida / inasistencia / cancelada. 4. Valida la fecha: no se marca atendida o inasistencia una cita futura y no se confirma una cita que ya comenzó. 5. Cambia el estado; el trigger registra el cambio en el historial con el usuario y la hora.
Precondición	La cita existe y el usuario tiene el permiso correspondiente.
Postcondición	La cita queda en el nuevo estado; los estados atendida, cancelada e inasistencia son finales.
Excepciones	Transición no permitida (p. ej. una cita cancelada no puede marcarse como atendida). Sin permiso para la acción. Cita de otro médico. Cita futura marcada como atendida.

CU13: Consultar agenda médica

Paquete: Citas · Ciclo 2 · Prioridad Alta · Requisitos: RF11 · RF12

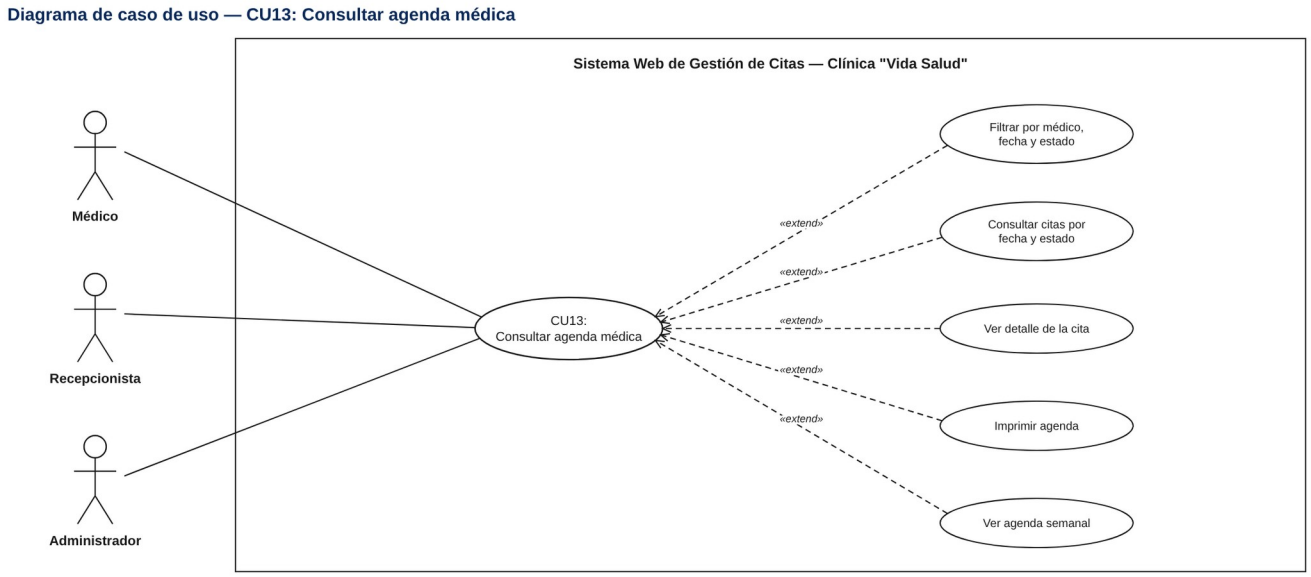

Figura 16. *Diagrama de caso de uso — CU13: Consultar agenda médica.*

Caso de uso	CU13: Consultar agenda médica
Propósito	Consultar la agenda de un médico para una fecha (hora, código de cita, paciente, especialidad, estado y motivo breve) o para toda la semana, y las citas por fecha, médico,

	especialidad y estado.
Actores	Médico, Recepcionista, Administrador
Actor iniciador	Médico o Recepcionista
Flujo de proceso	1. El usuario abre «Agenda médica»; el médico ve directamente su propia agenda. 2. Elige médico, fecha y estado (o navega al día anterior/siguiente). 3. El sistema muestra las citas del día en orden de hora con código, paciente, especialidad, estado y motivo (si tiene permiso), junto con los horarios libres. 4. La recepcionista también puede abrir «Citas» y filtrar por rango de fechas, médico, especialidad, estado o paciente, con totales por estado. 5. Desde cualquier cita abre el detalle con su historial de cambios. 6. Puede cambiar a la «Vista semanal»: el sistema muestra los siete días (lunes a domingo) de la semana elegida con las citas de cada día, marca los días que el médico no atiende o que están bloqueados y permite ir a la semana anterior o siguiente. 7. Puede imprimir la agenda del día.
Precondición	Usuario con «agenda.ver» (todos los médicos) o «agenda.propia» (médico).
Postcondición	Se muestra la información; no se modifican datos.
Excepciones	Sin citas para los filtros: se muestra un mensaje. Un médico no puede ver la agenda de otro médico.

CU14: Generar reportes

Paquete: Reportes · Ciclo 2 · Prioridad Media · Requisitos: RF13

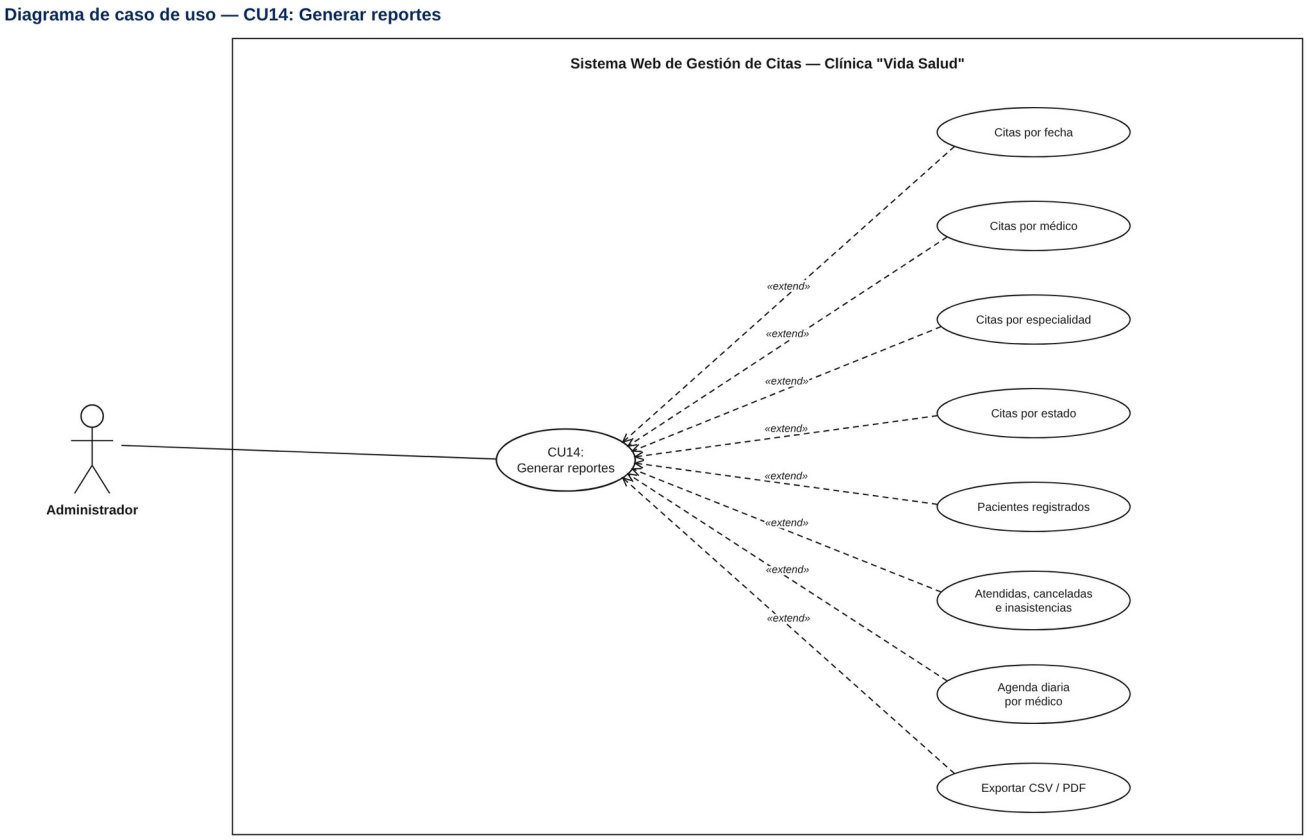

Figura 17. *Diagrama de caso de uso — CU14: Generar reportes.*

Caso de uso	CU14: Generar reportes
Propósito	Generar los reportes administrativos: citas programadas por fecha, por médico, por especialidad y por estado, pacientes registrados, cantidad de atendidas, canceladas e inasistencias del período y agenda diaria de cada médico, con filtros por fecha o rango y totales.
Actores	Administrador
Actor iniciador	Administrador
Flujo de proceso	1. El administrador abre «Reportes» y elige el tipo de reporte. 2. Indica la fecha o el rango de fechas (y el estado, si corresponde) y presiona «Generar». 3. El sistema consulta la base de datos y muestra la tabla con sus totales y, según el reporte, un gráfico de barras. 4. Puede descargar el reporte en CSV (se abre en Excel). 5. Puede imprimirlo o guardarlo como PDF; la versión impresa incluye el encabezado de la clínica y el período.
Precondición	El administrador inició sesión con «reportes.ver».
Postcondición	Se obtiene la información del período seleccionado sin modificar los datos.

Excepciones	Sin datos en el período: se muestra un mensaje. Fechas inválidas: se usa el mes en curso.

4.6 Prototipar la interfaz de usuario

Las siguientes pantallas corresponden al sistema implementado (HTML5, CSS3 y JavaScript). La interfaz usa la identidad visual de la clínica (verde pino y menta), un menú lateral que muestra solo las opciones permitidas para el rol y mensajes que explican qué ocurrió en cada operación. Todas las pantallas se adaptan al celular.

Ciclo 1 — Seguridad, registros base y programación de citas

Prototipo CU01: Iniciar sesión

Figura 18. *CU01 — Pantalla de inicio de sesión.*

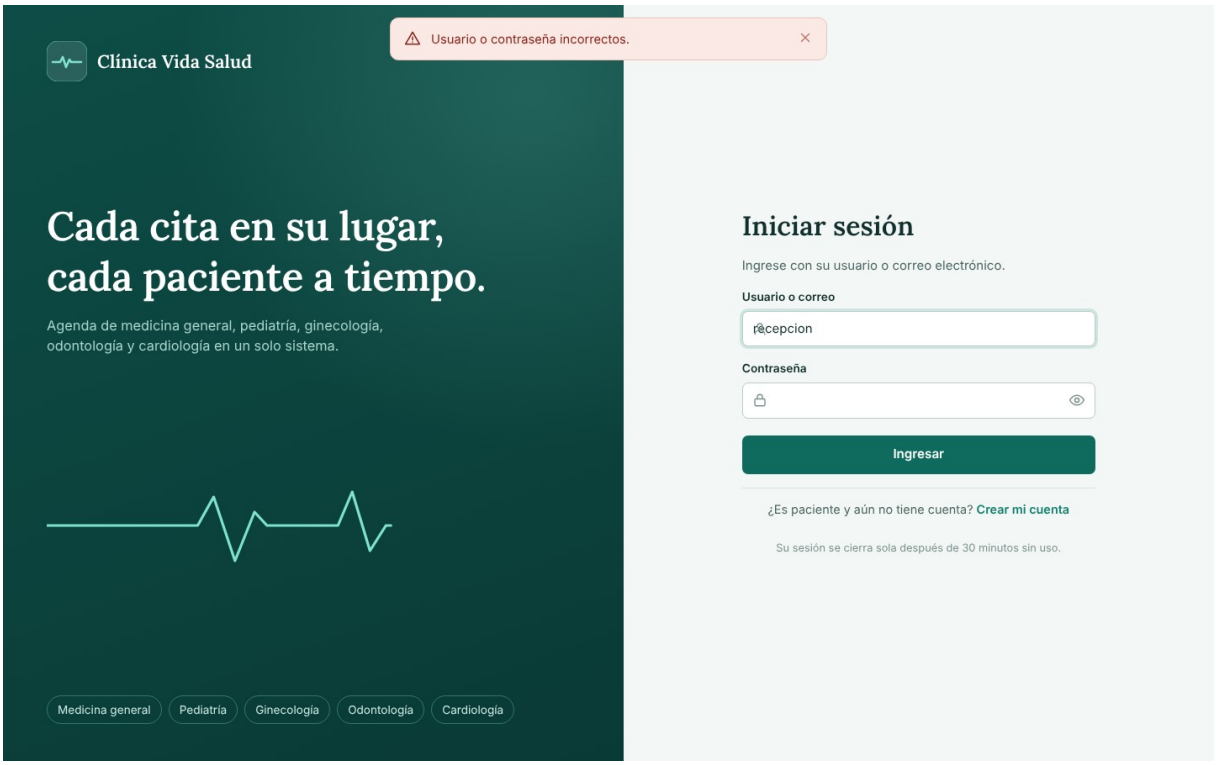

Figura 19. *CU01 — Mensaje ante credenciales incorrectas.*

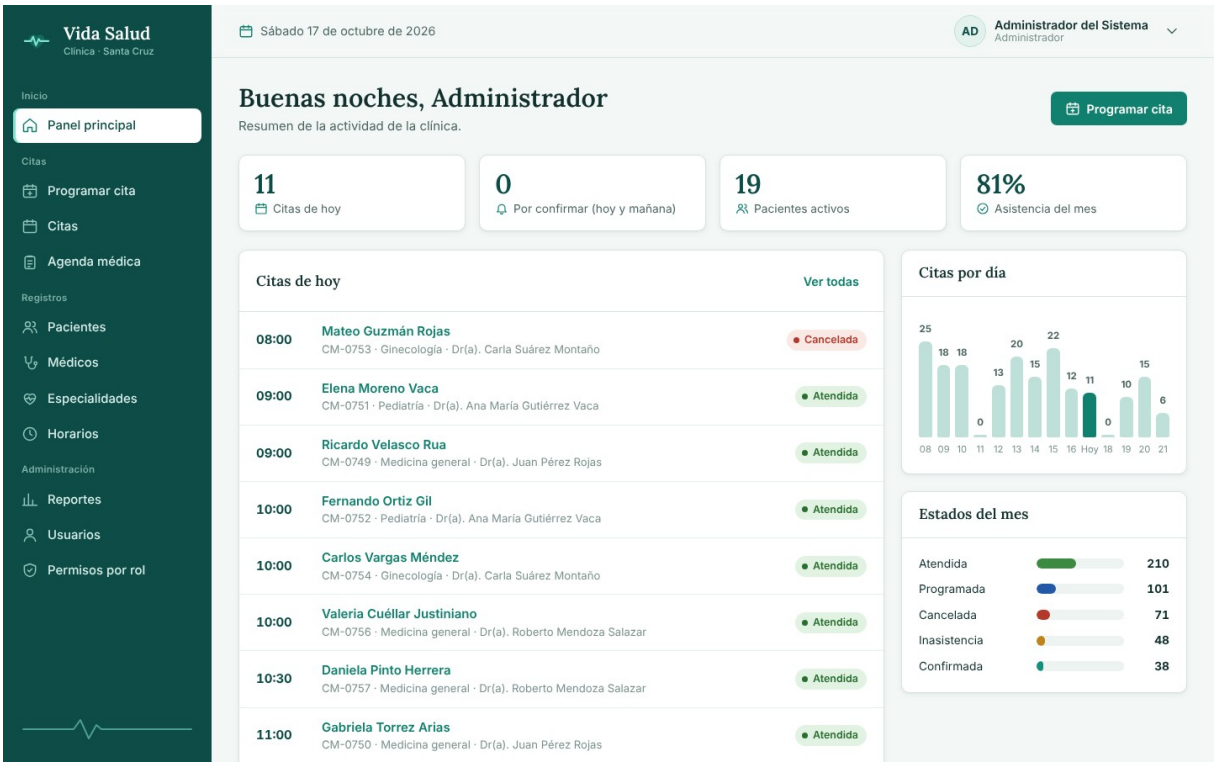

Figura 20. *CU01 — Panel principal del administrador con el menú según sus permisos.*

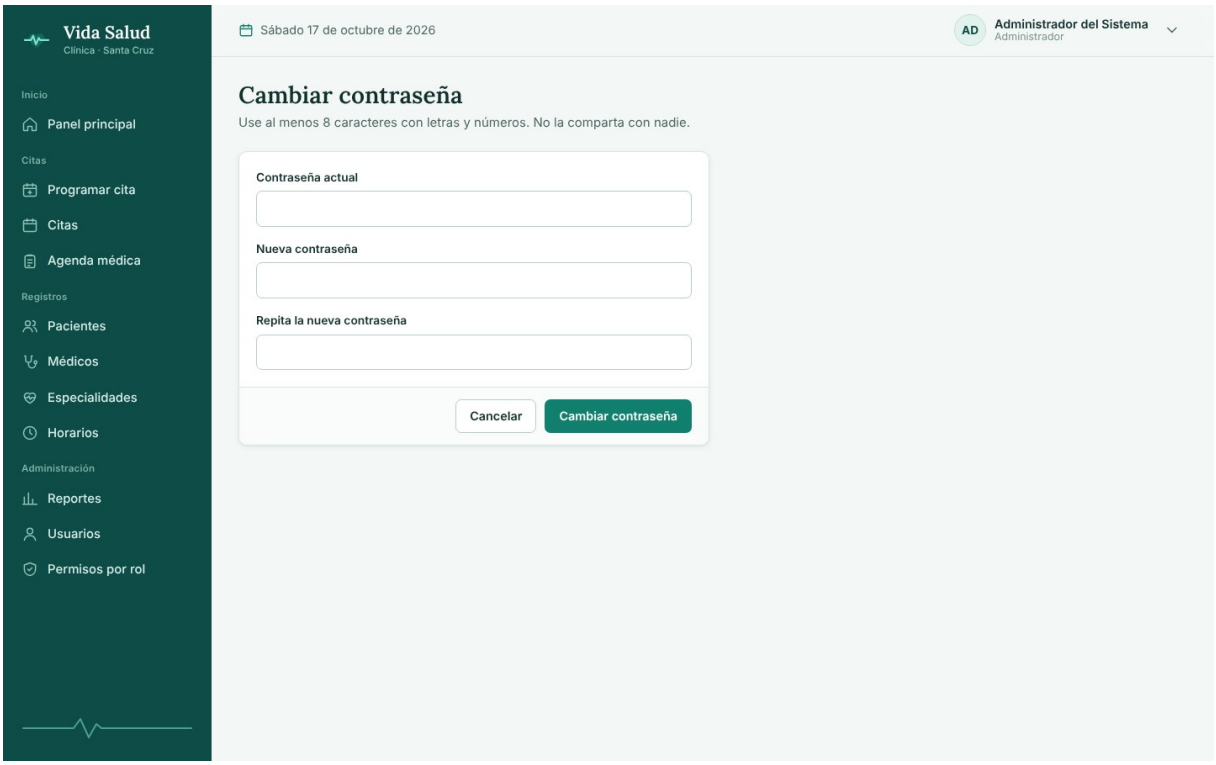

Figura 21. *CU01 — Cambiar contraseña (flujo alternativo).*

Prototipo CU02: Gestionar usuarios y permisos

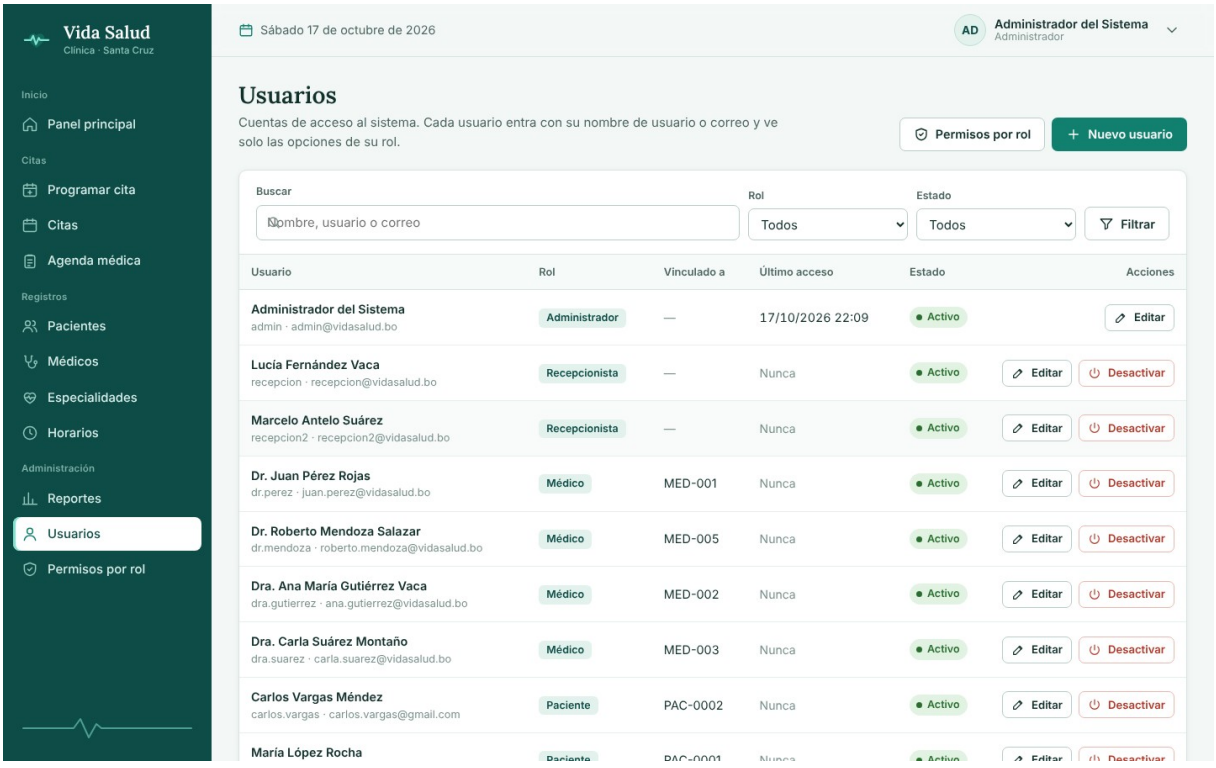

Figura 22. *CU02 — Lista de usuarios con rol, vínculo, último acceso y estado.*

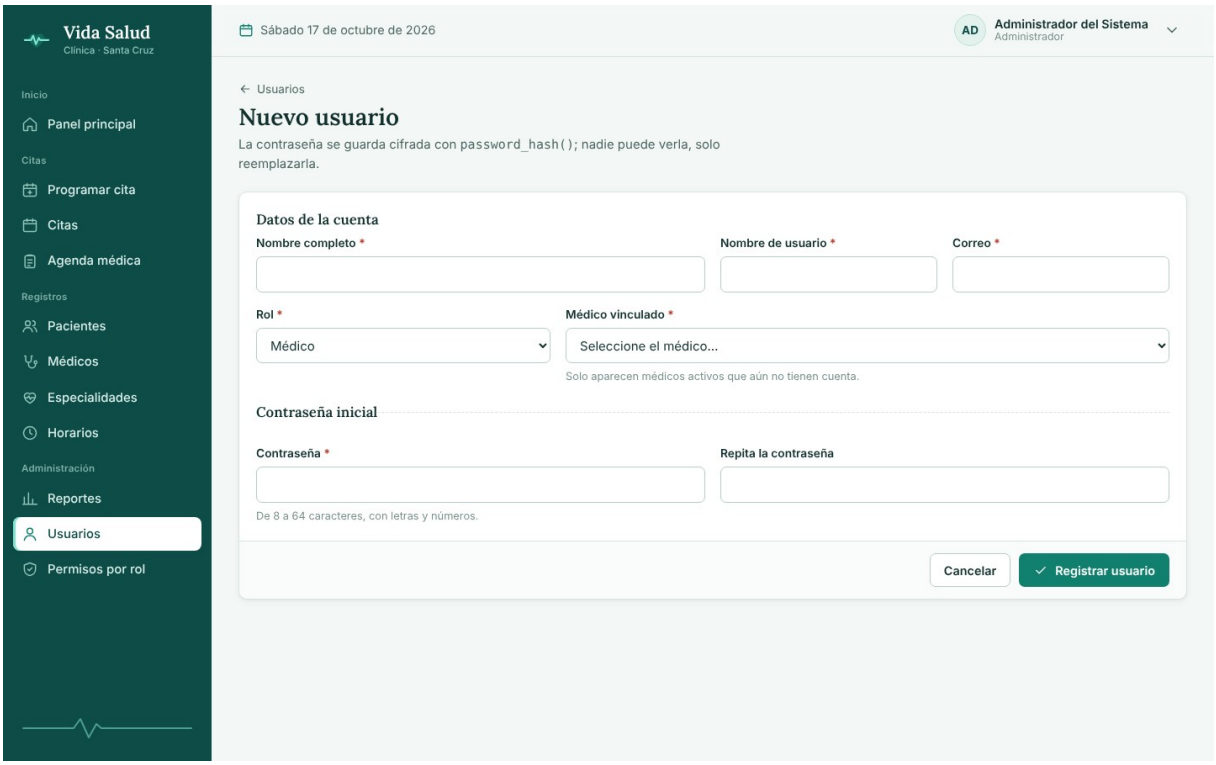

Figura 23. *CU02 — Registro de usuario con rol Médico vinculado a un médico.*

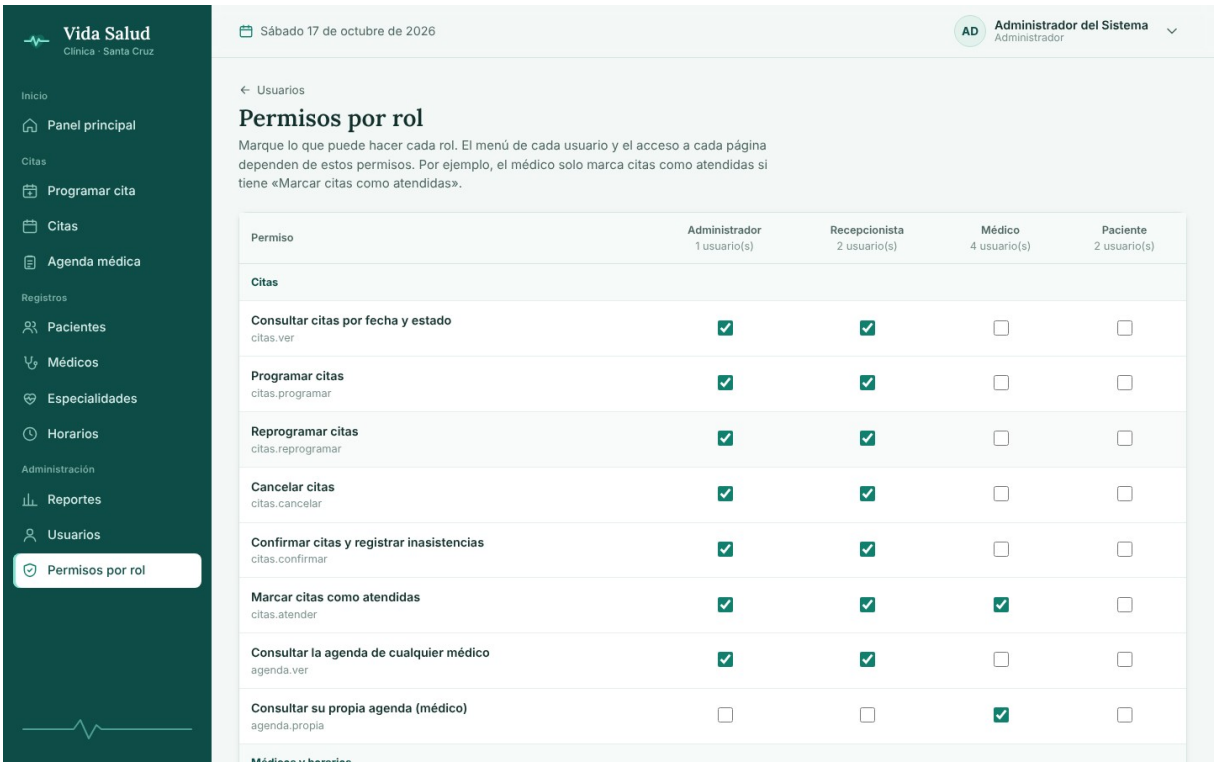

Figura 24. *CU02 — Matriz de permisos por rol.*

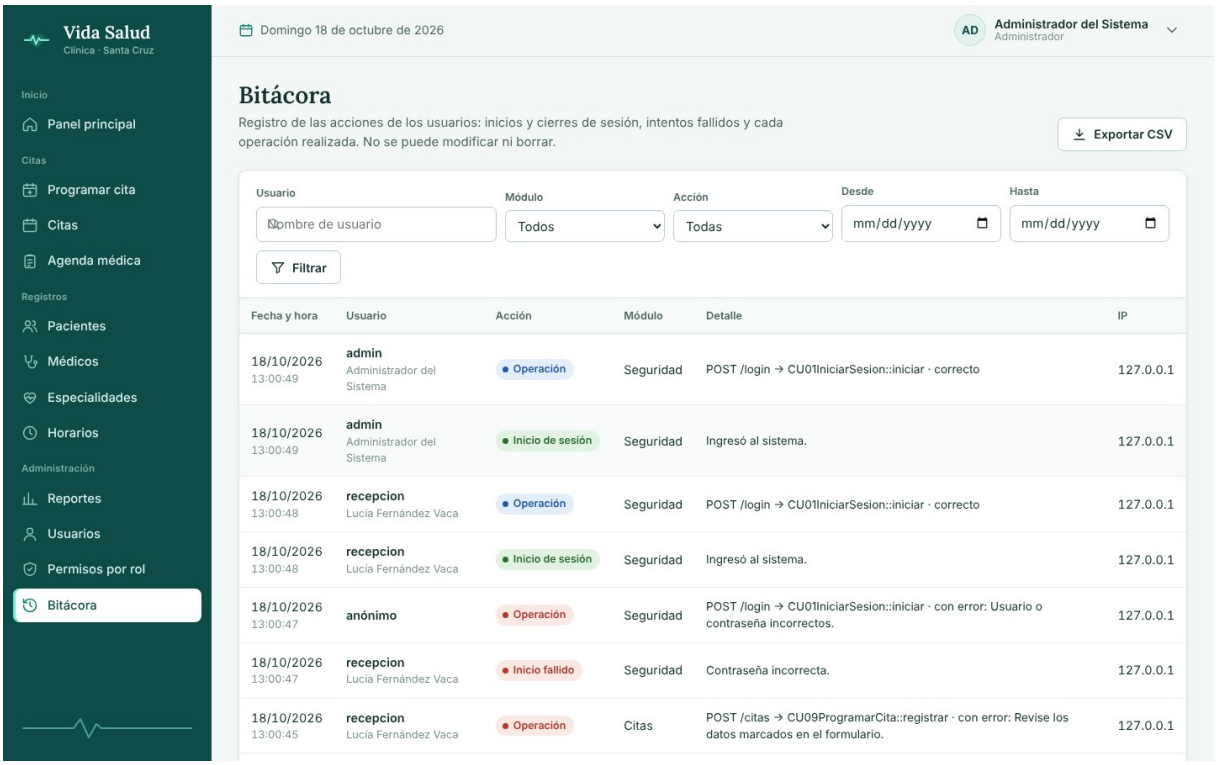

Figura 25. *CU02 — Bitácora de acciones: quién hizo qué, cuándo y desde qué IP, con filtros y exportación a CSV.*

Prototipo CU03: Gestionar pacientes

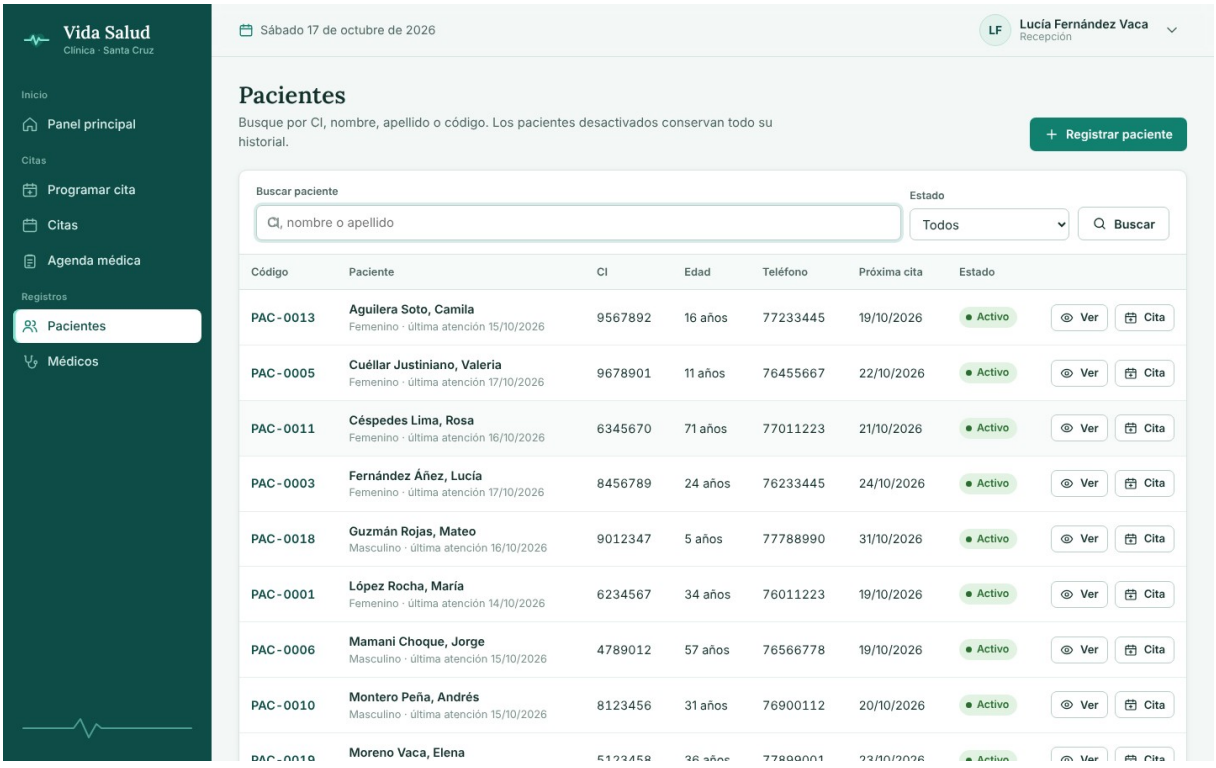

Figura 26. *CU03 — Listado y búsqueda de pacientes.*

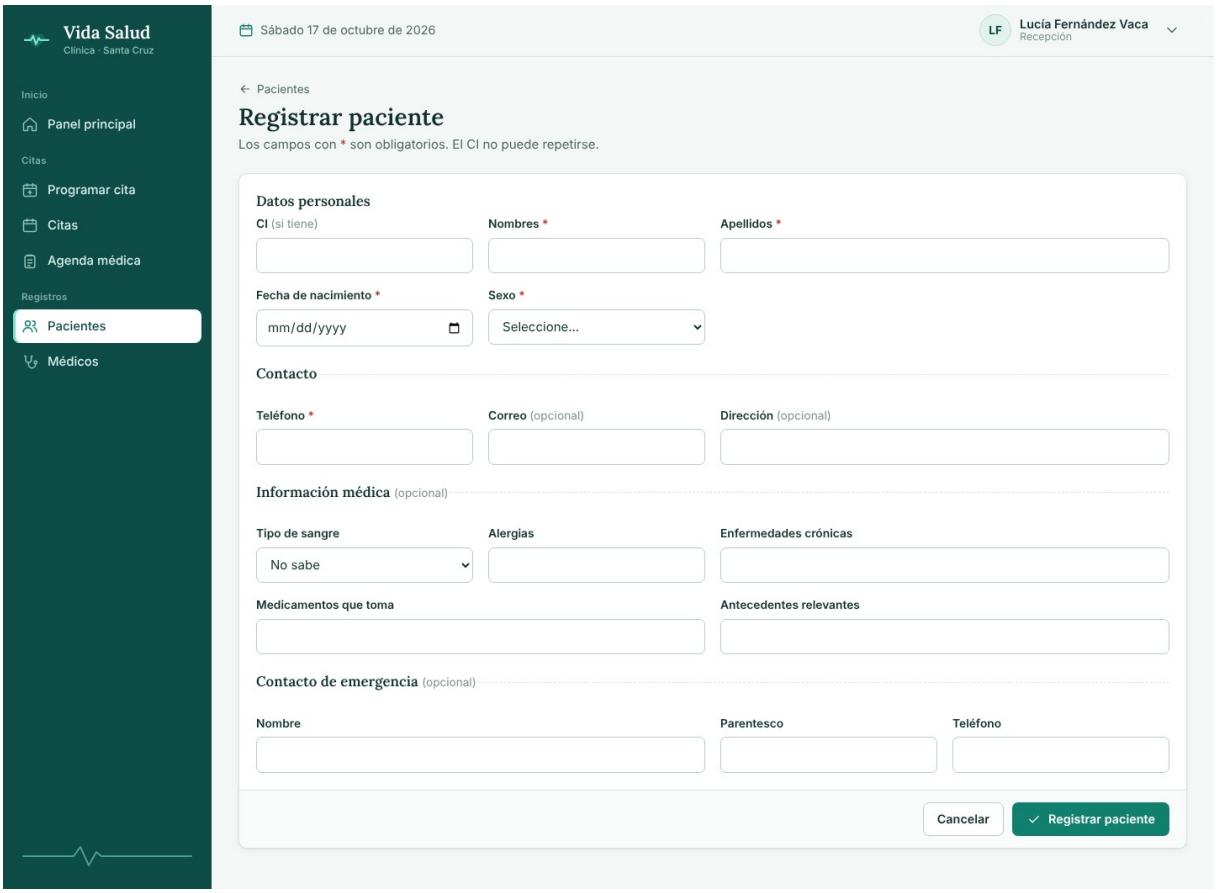

Figura 27. *CU03 — Formulario de registro de paciente.*

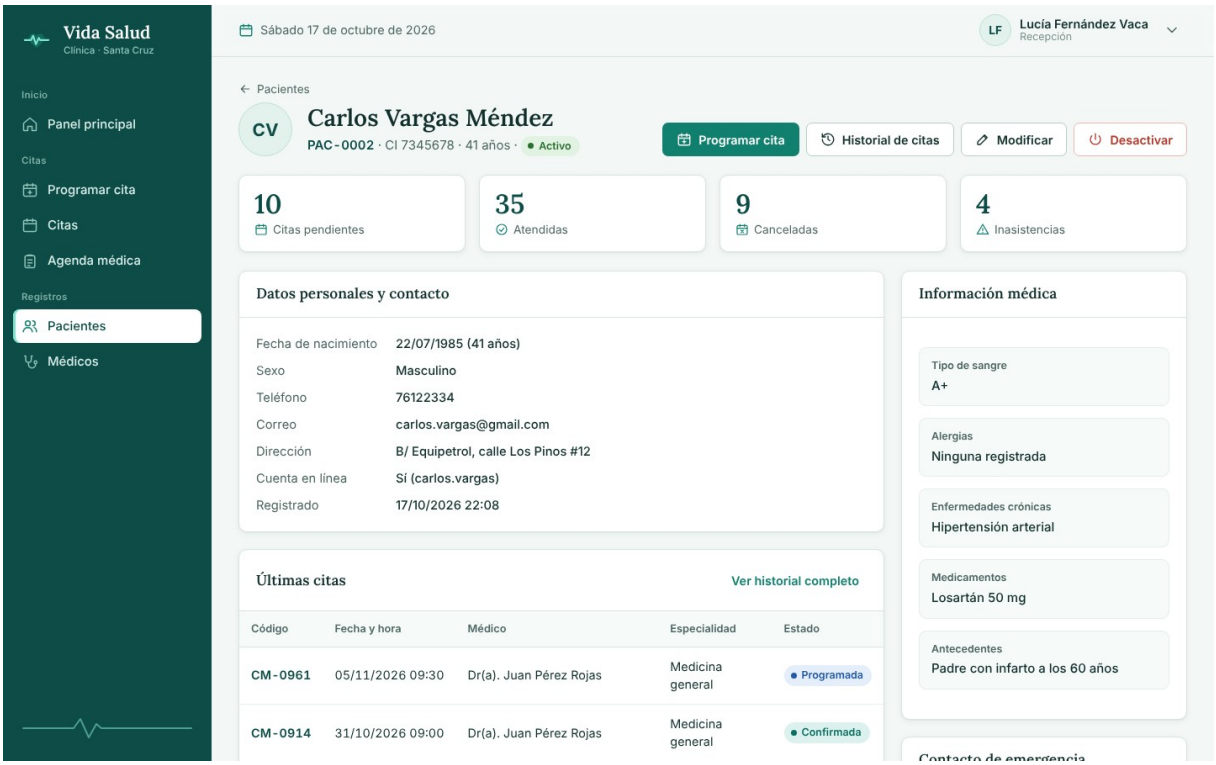

Figura 28. *CU03 — Ficha del paciente con resumen e información médica.*

Prototipo CU06: Gestionar especialidades

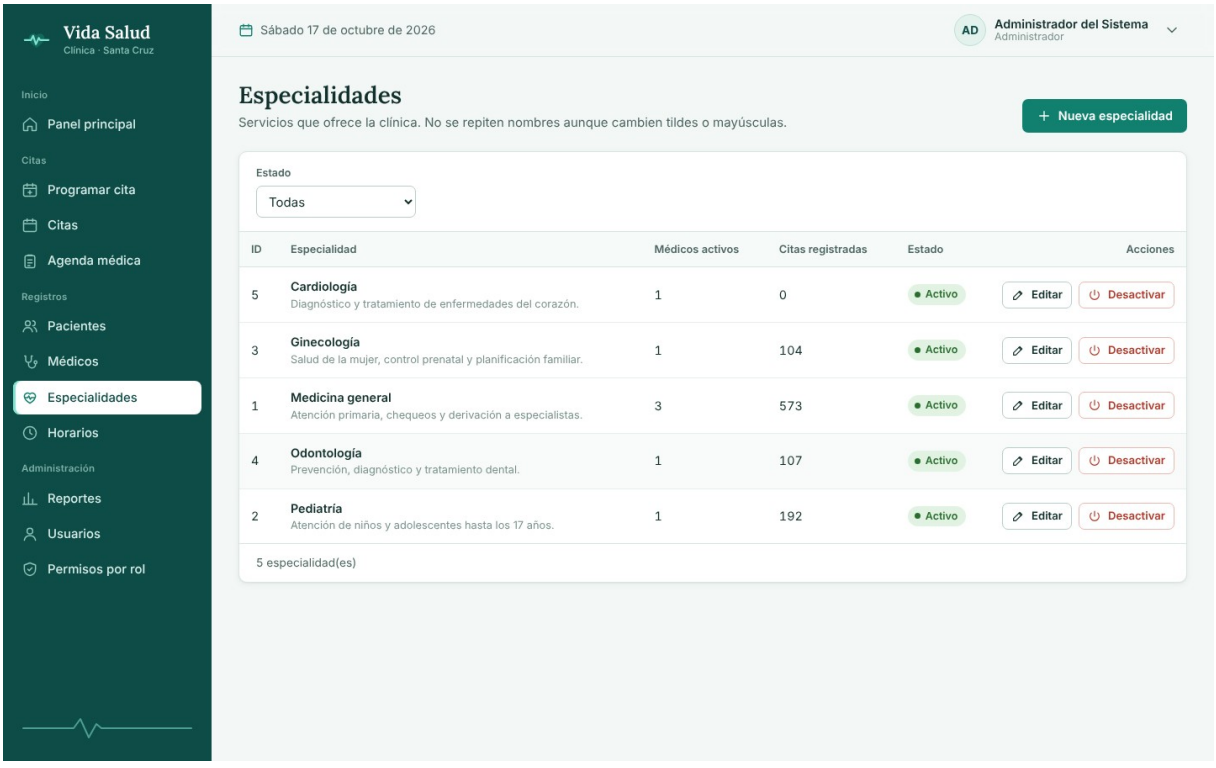

Figura 29. *CU06 — Gestión de especialidades.*

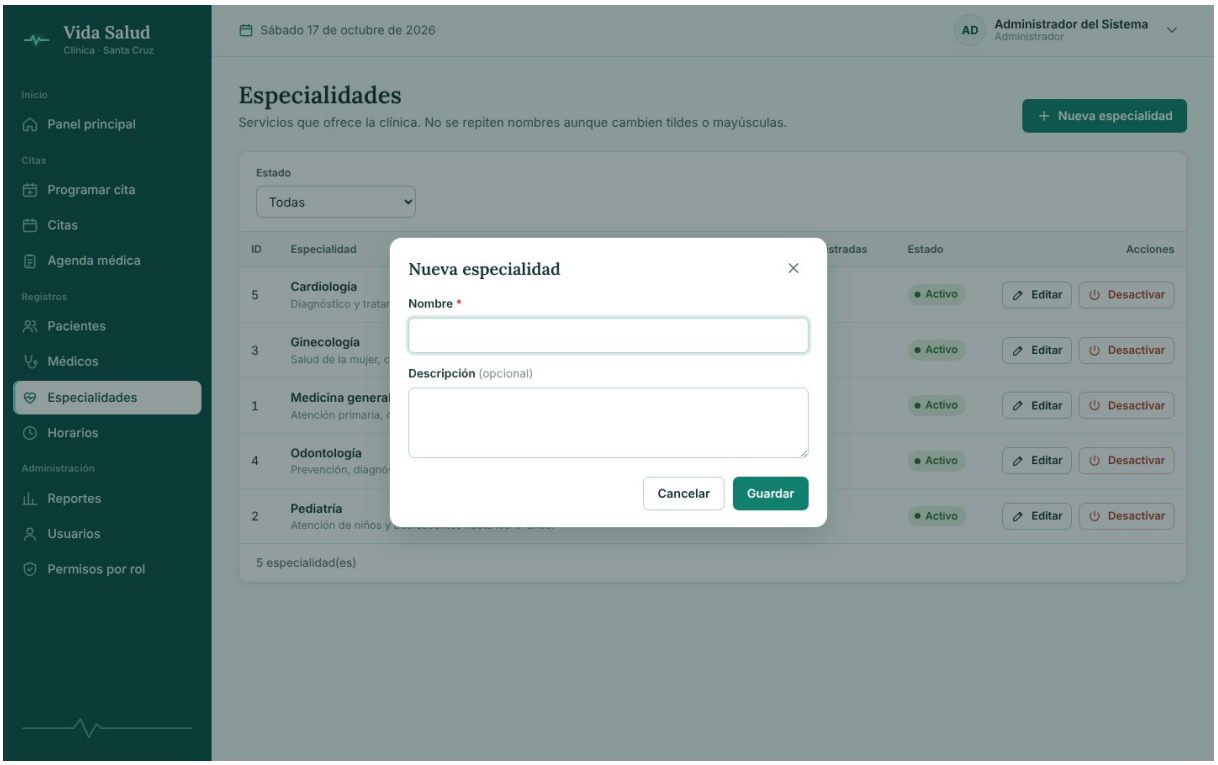

Figura 30. *CU06 — Ventana para registrar una especialidad.*

Prototipo CU07: Gestionar médicos

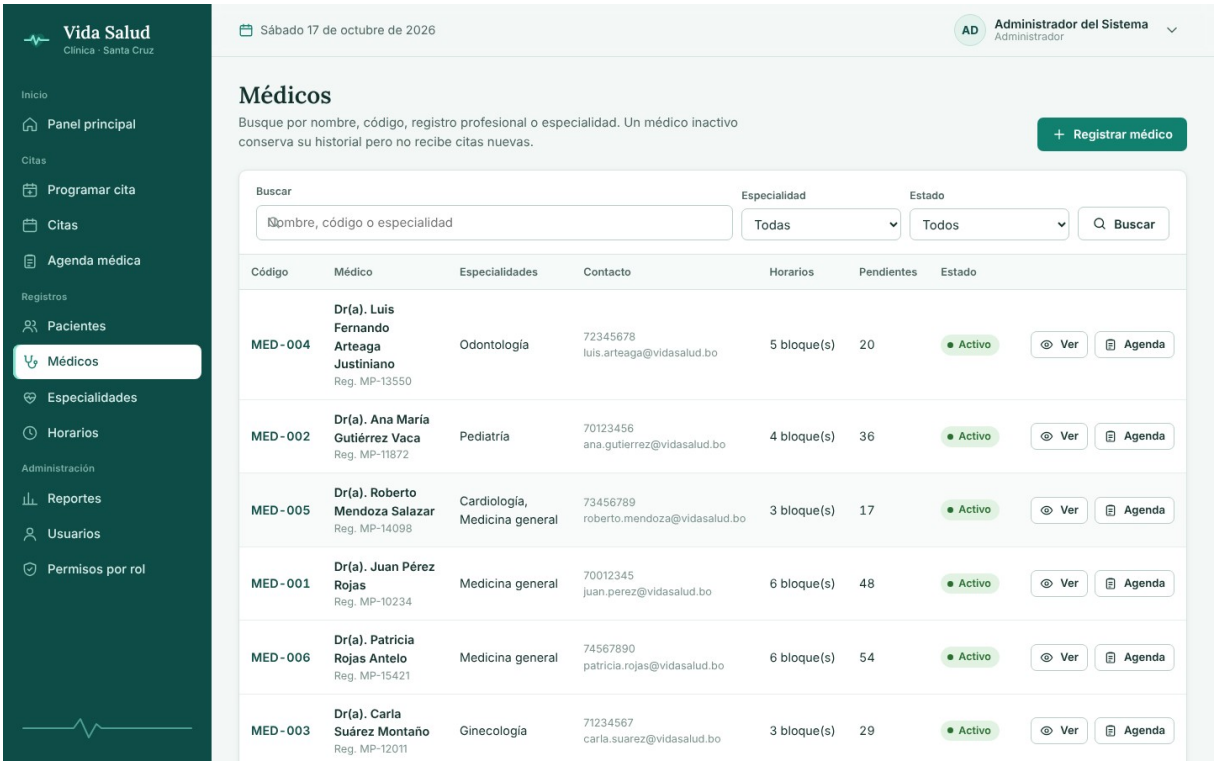

Figura 31. *CU07 — Listado de médicos con búsqueda por nombre o especialidad.*

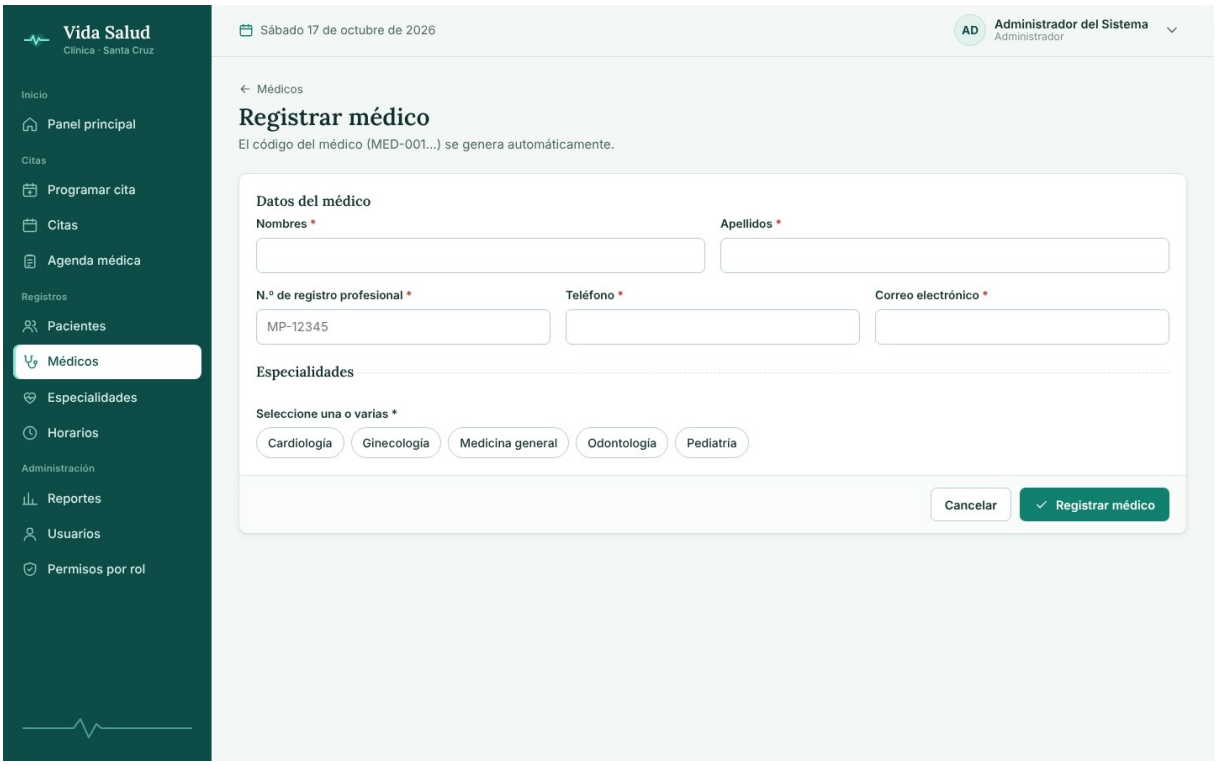

Figura 32. *CU07 — Registro de médico con sus especialidades.*

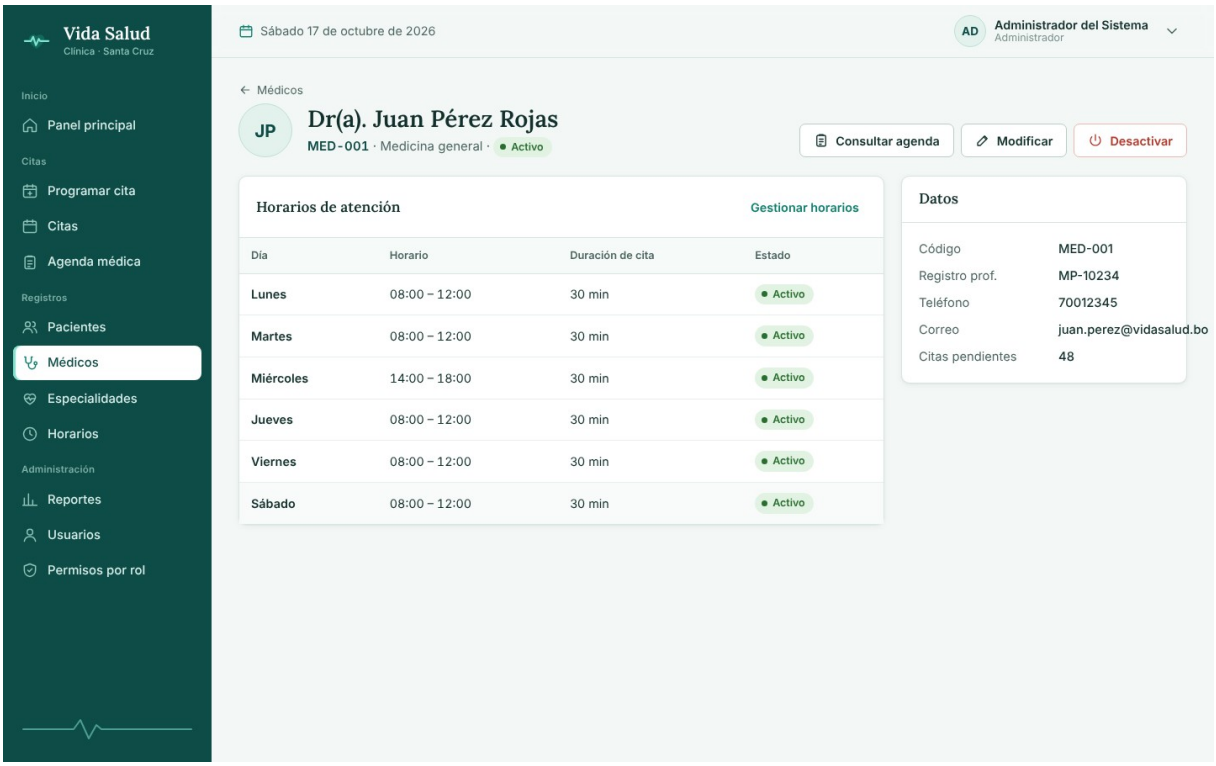

Figura 33. *CU07 — Ficha del médico con sus horarios.*

Prototipo CU08: Gestionar horarios de atención

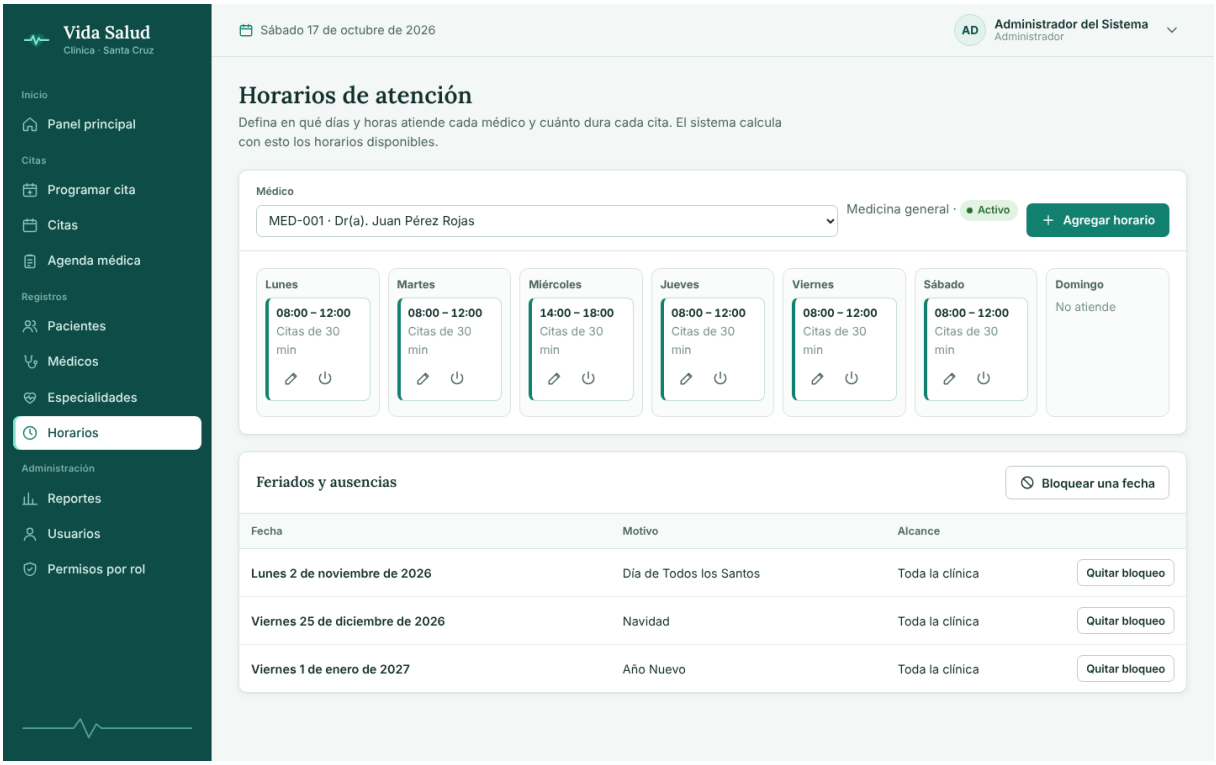

Figura 34. *CU08 — Semana de atención del médico y feriados.*

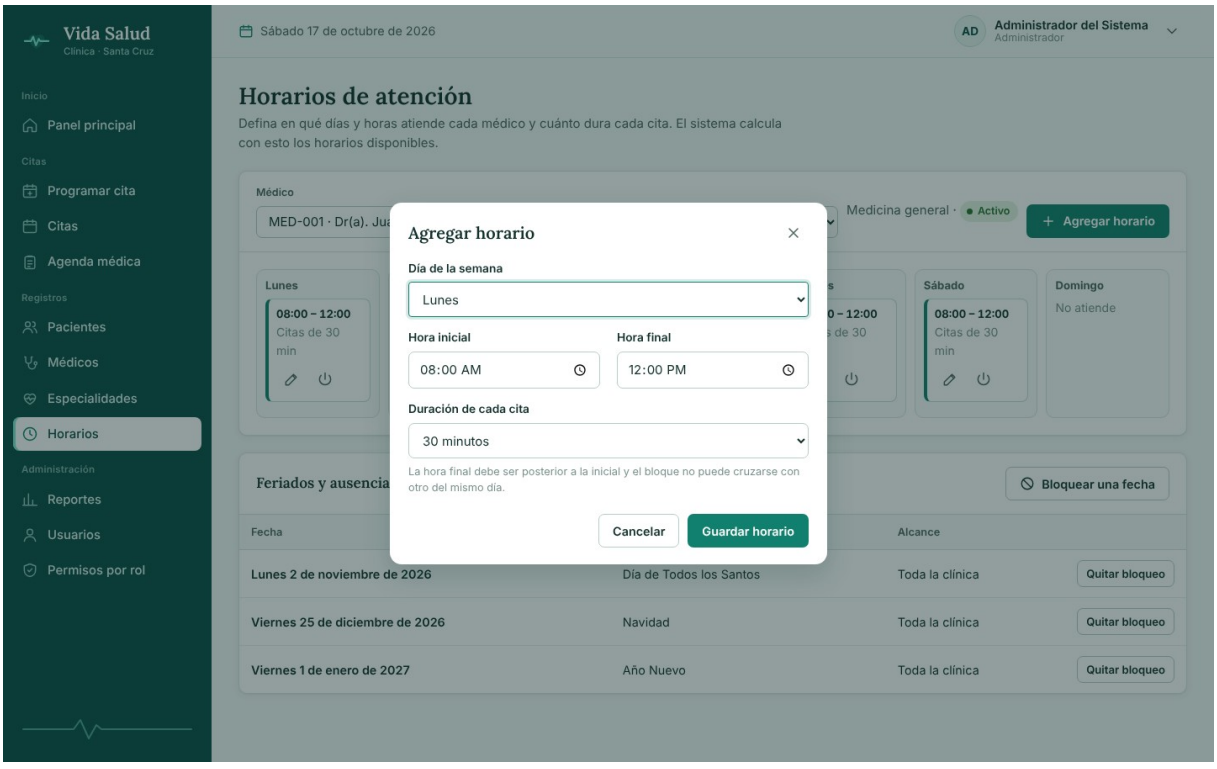

Figura 35. *CU08 — Agregar un bloque de horario con la duración de cita.*

Prototipo CU09: Programar cita

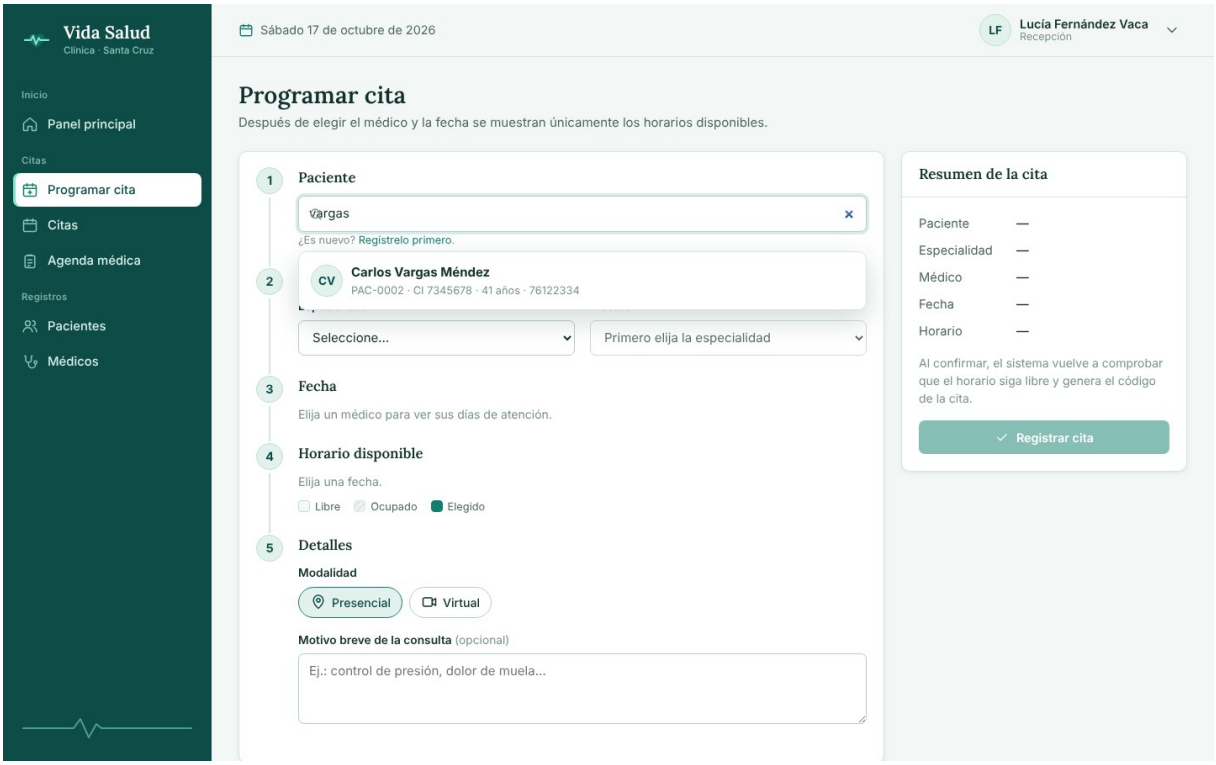

Figura 36. *CU09 — Búsqueda del paciente al programar una cita.*

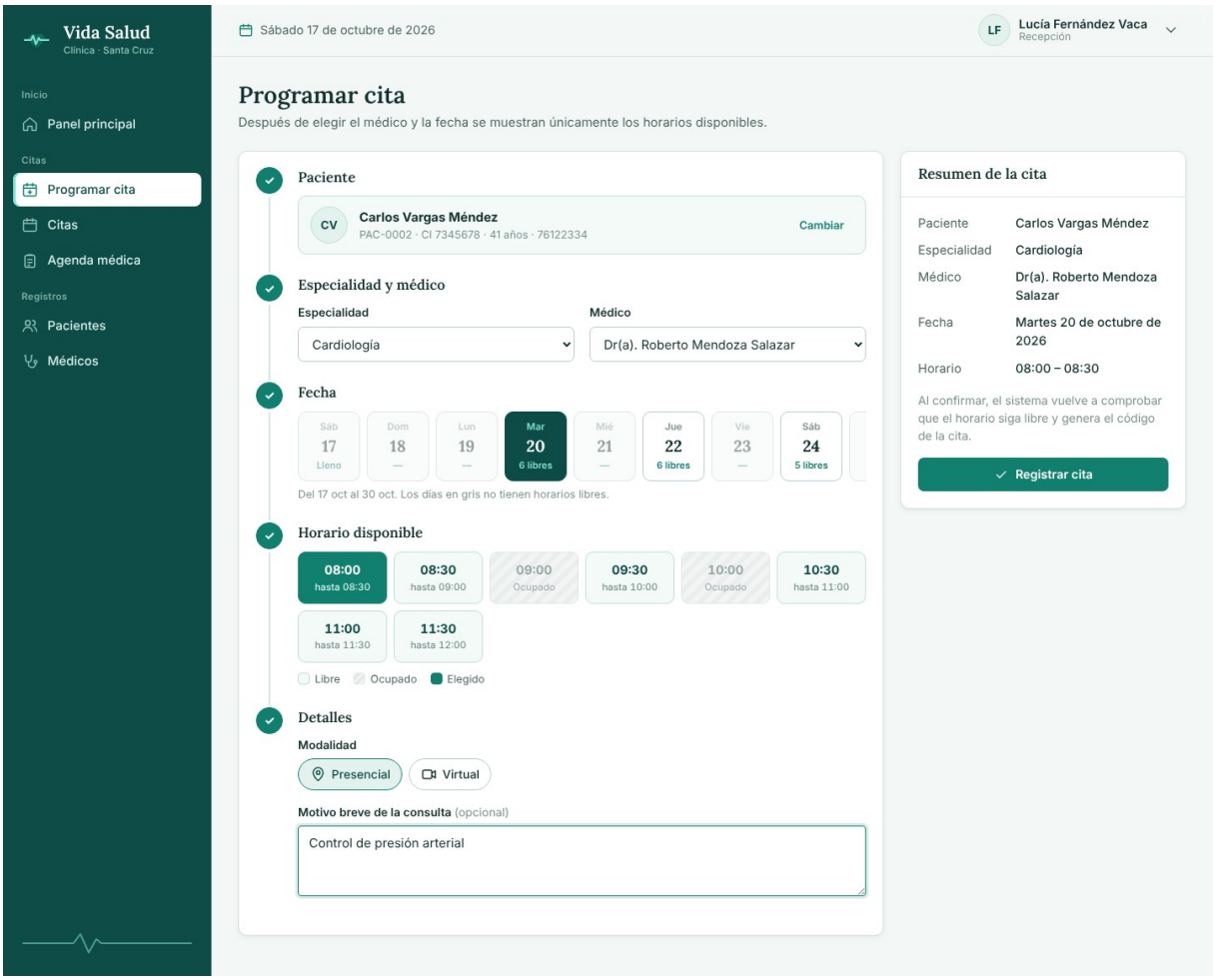

Figura 37. *CU09 — Selección de especialidad, médico, fecha y solo horarios disponibles.*

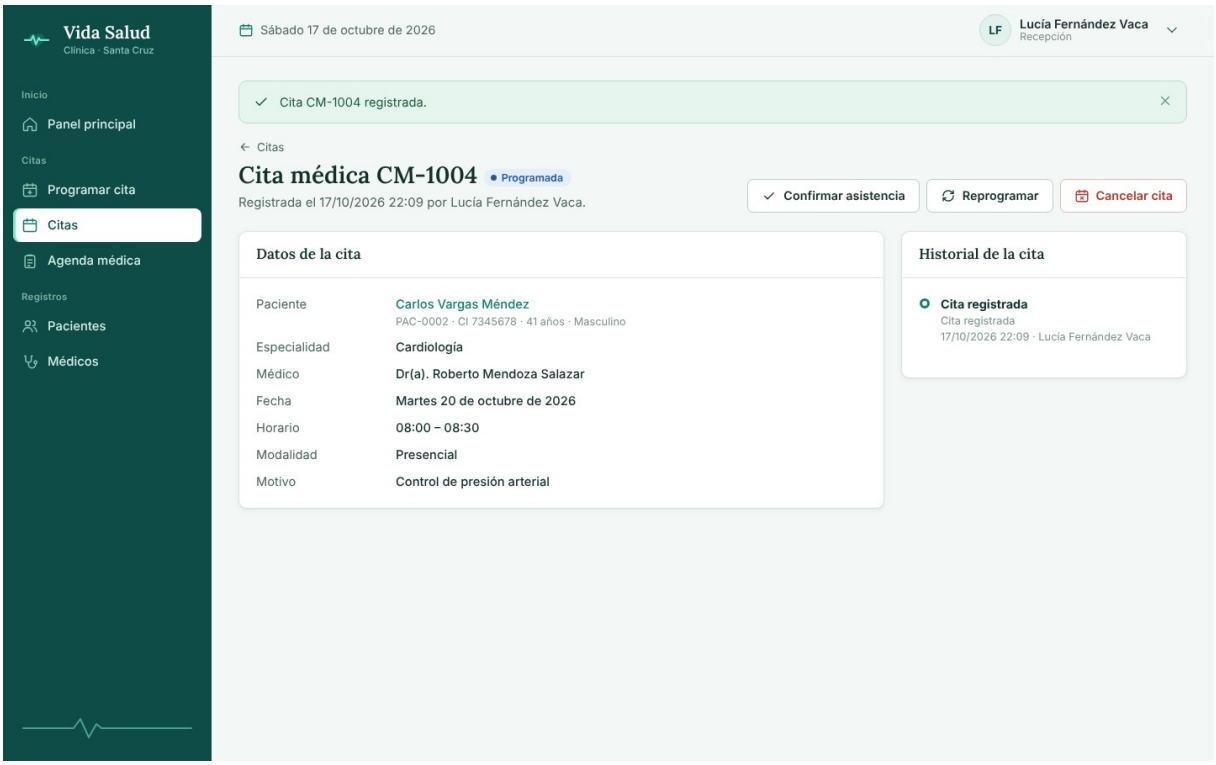

Figura 38. *CU09 — Cita registrada con su código único.*

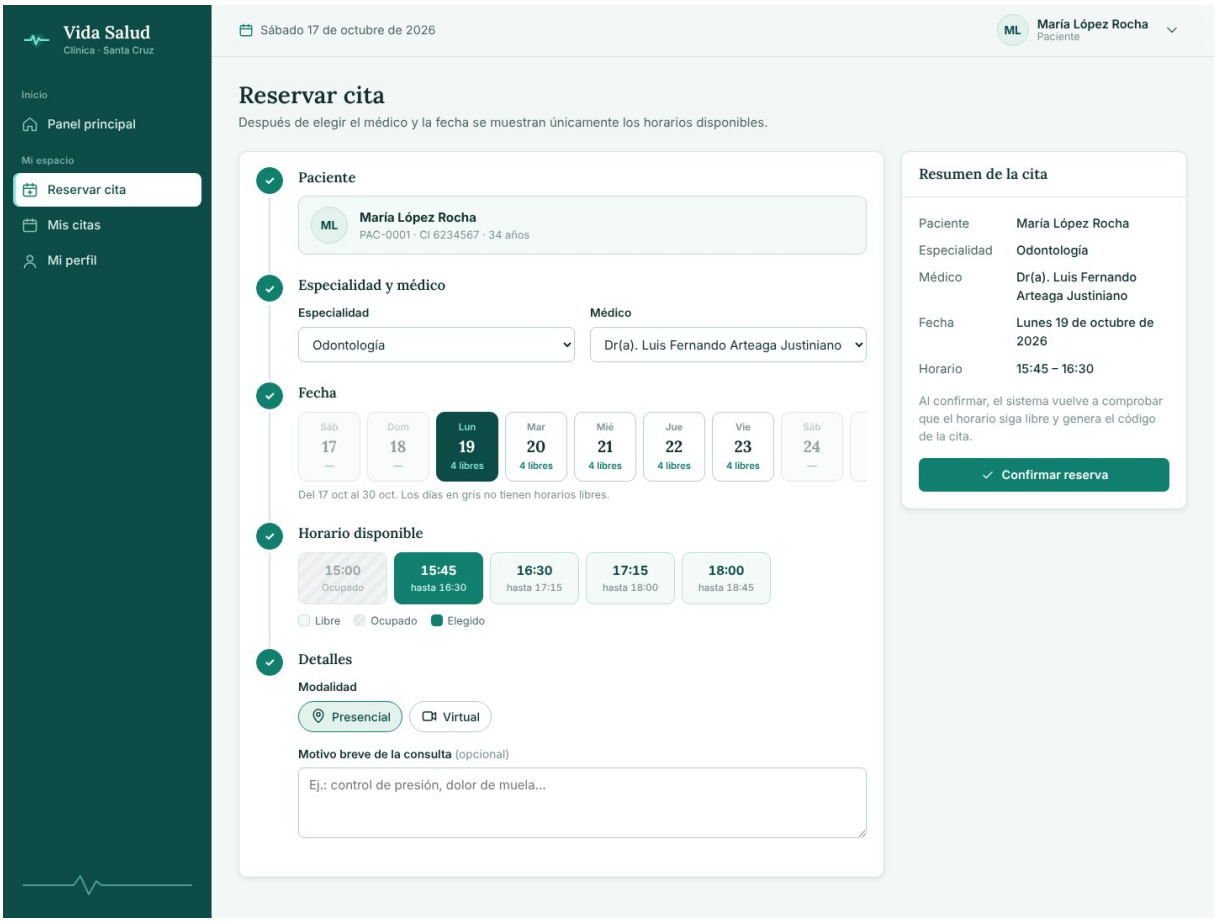

Figura 39. *CU09 — Reserva en línea desde el portal del paciente.*

Ciclo 2 — Seguimiento de citas, agenda, portal del paciente y reportes

Prototipo CU04: Gestionar perfil del paciente

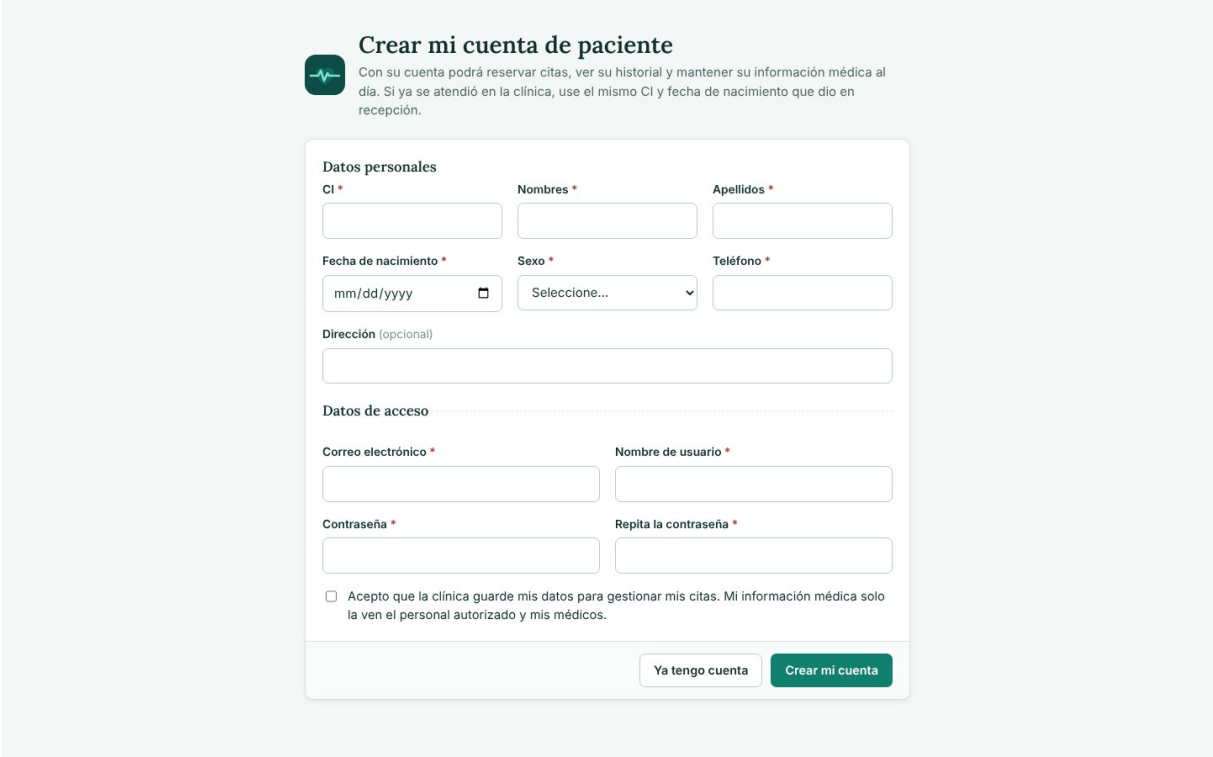

Figura 40. *CU04 — Registro en línea del paciente.*

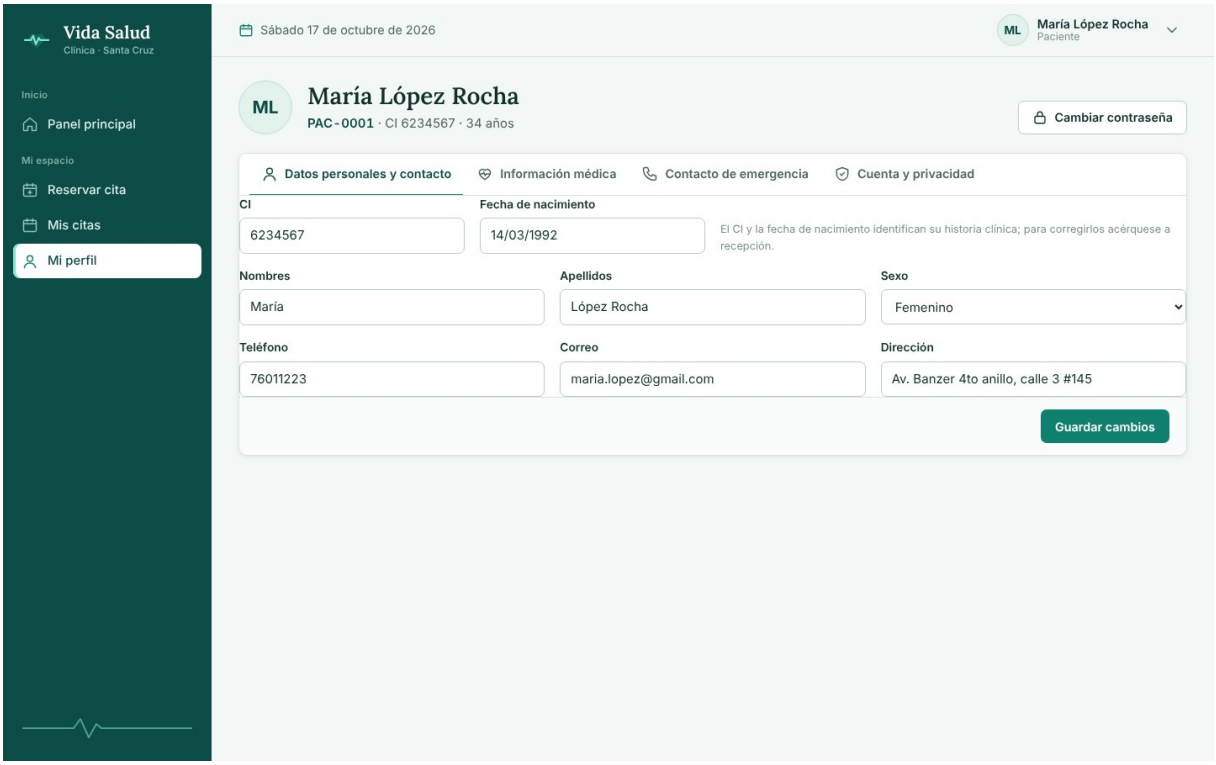

Figura 41. *CU04 — Mi perfil: datos personales y de contacto.*

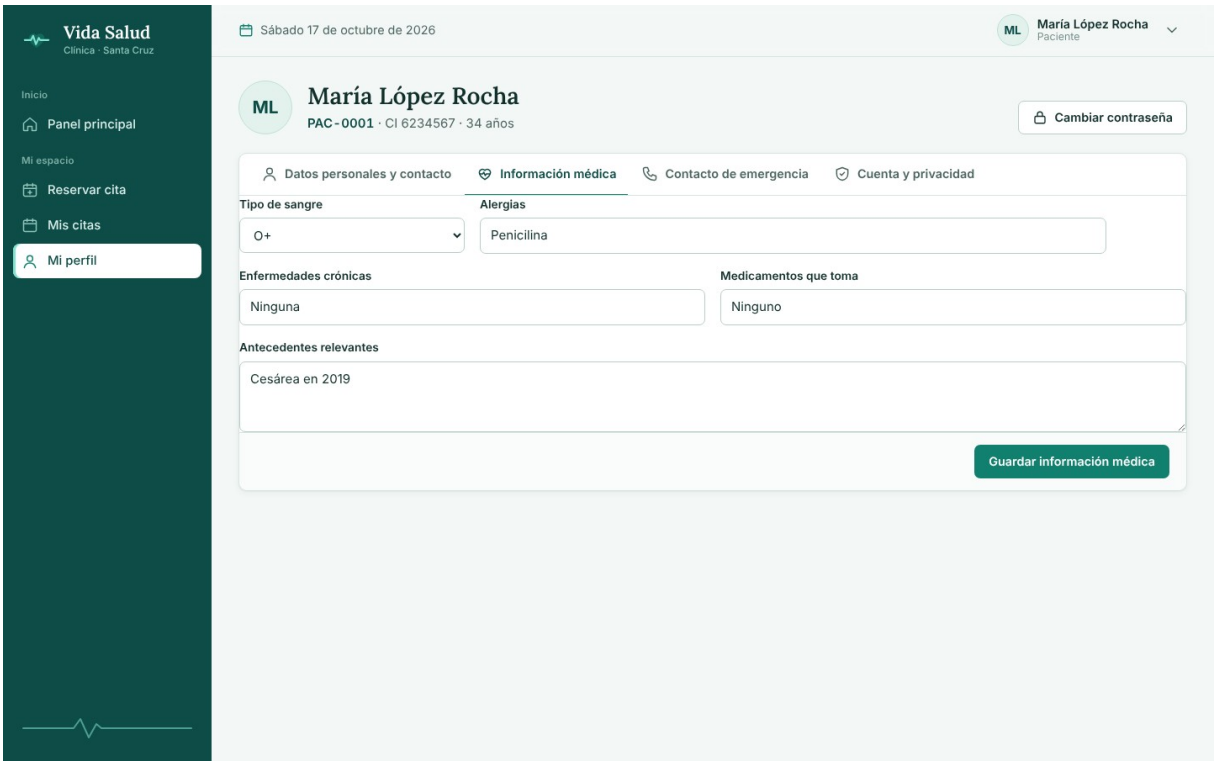

Figura 42. *CU04 — Mi perfil: información médica.*

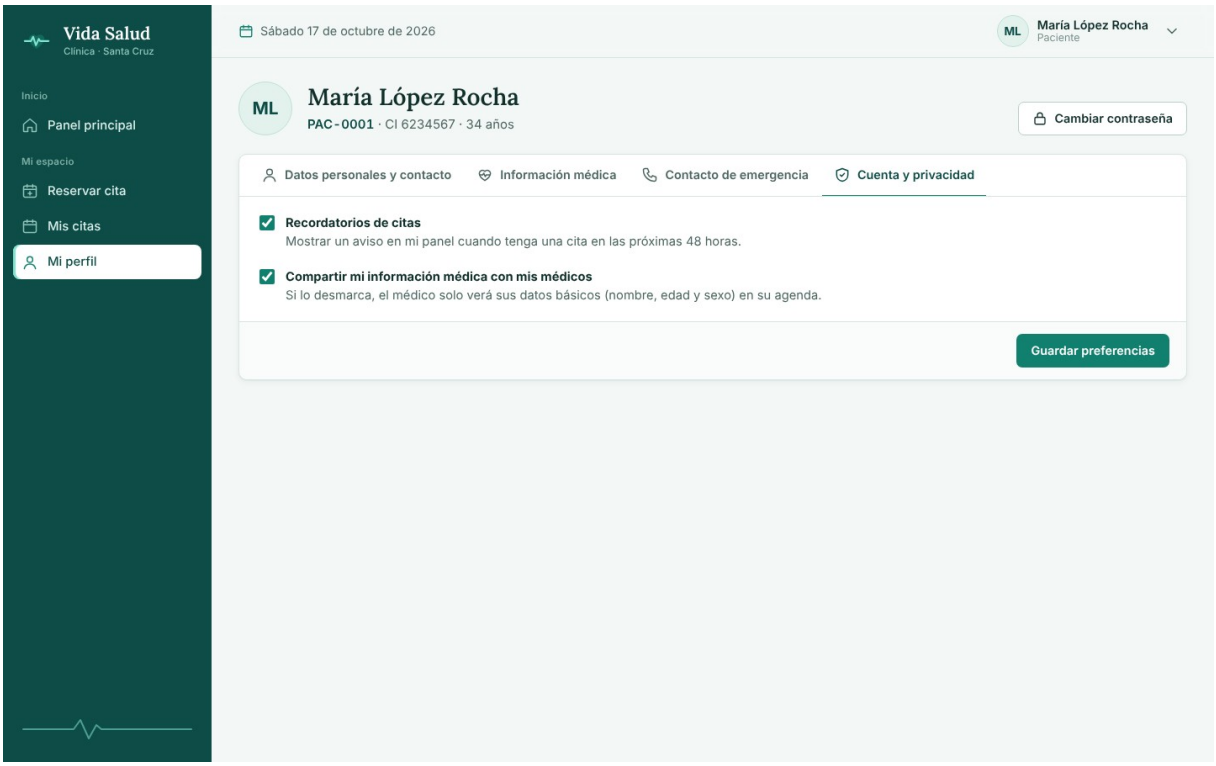

Figura 43. *CU04 — Mi perfil: notificaciones y privacidad.*

Prototipo CU05: Consultar historial de citas

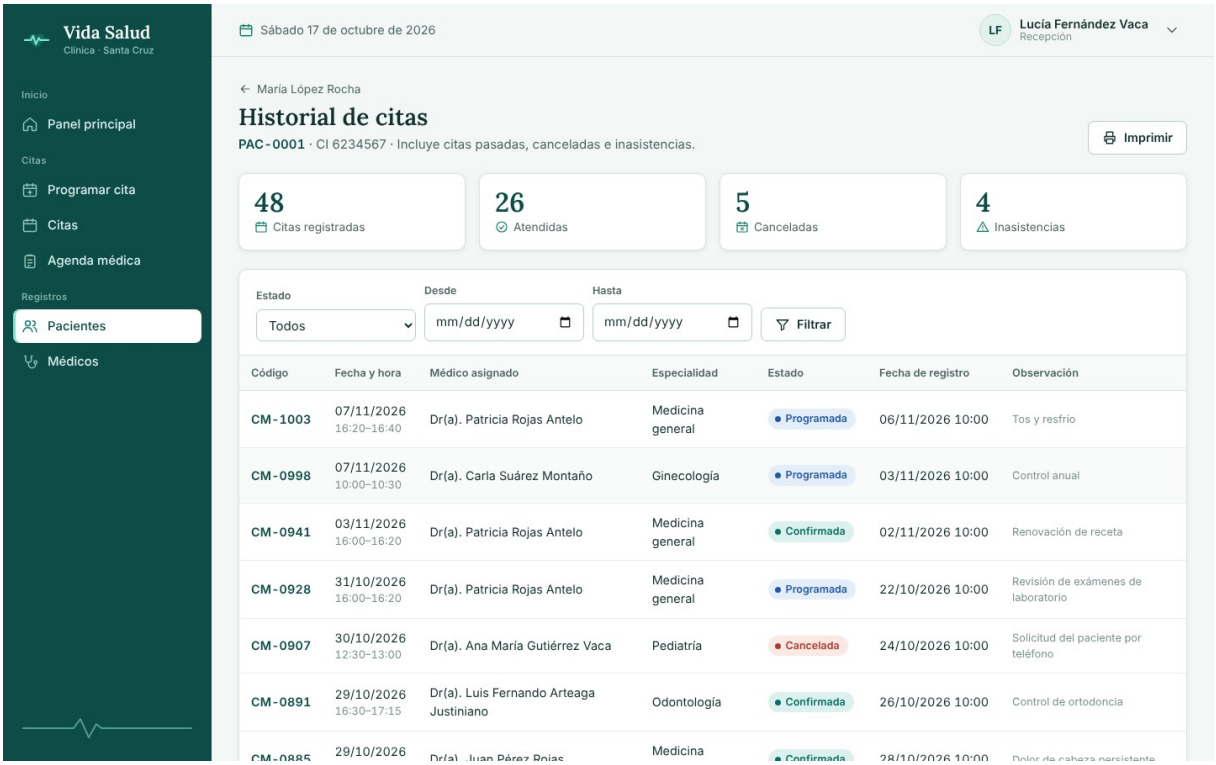

Figura 44. *CU05 — Historial de citas de un paciente con filtros.*

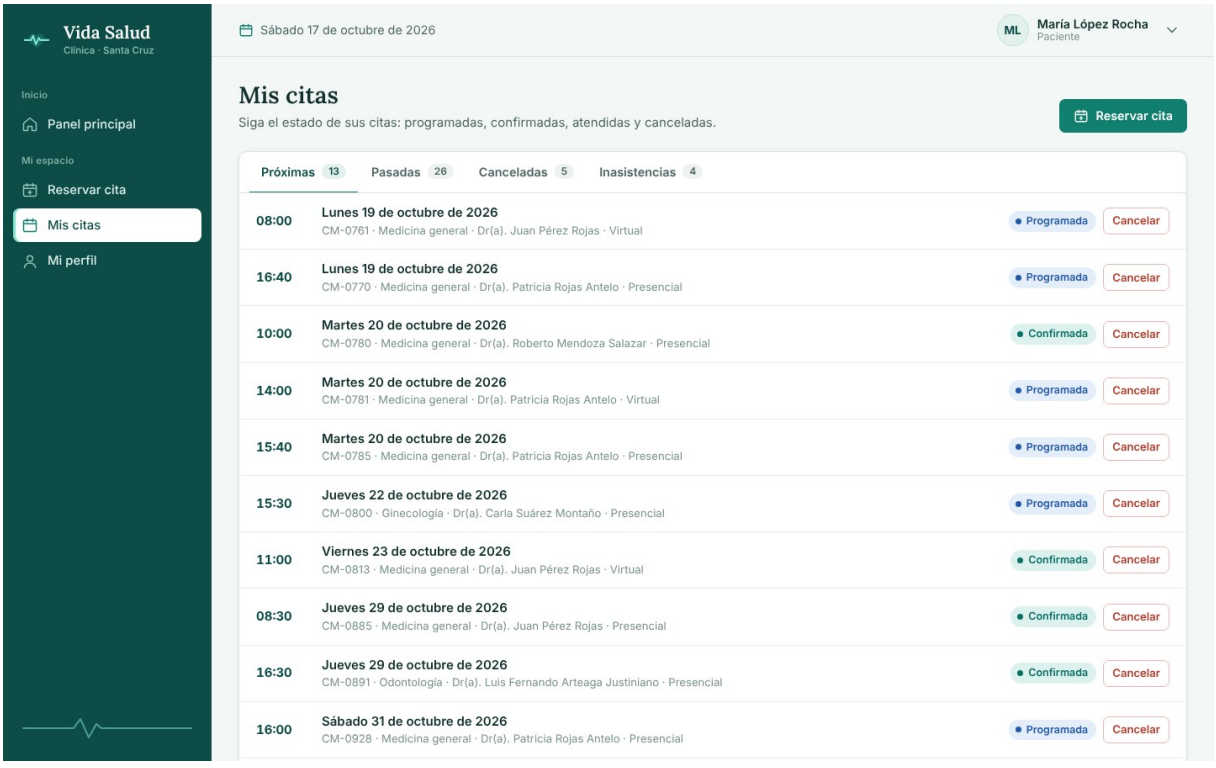

Figura 45. *CU05 — Mis citas: seguimiento del paciente.*

Prototipo CU10: Reprogramar cita

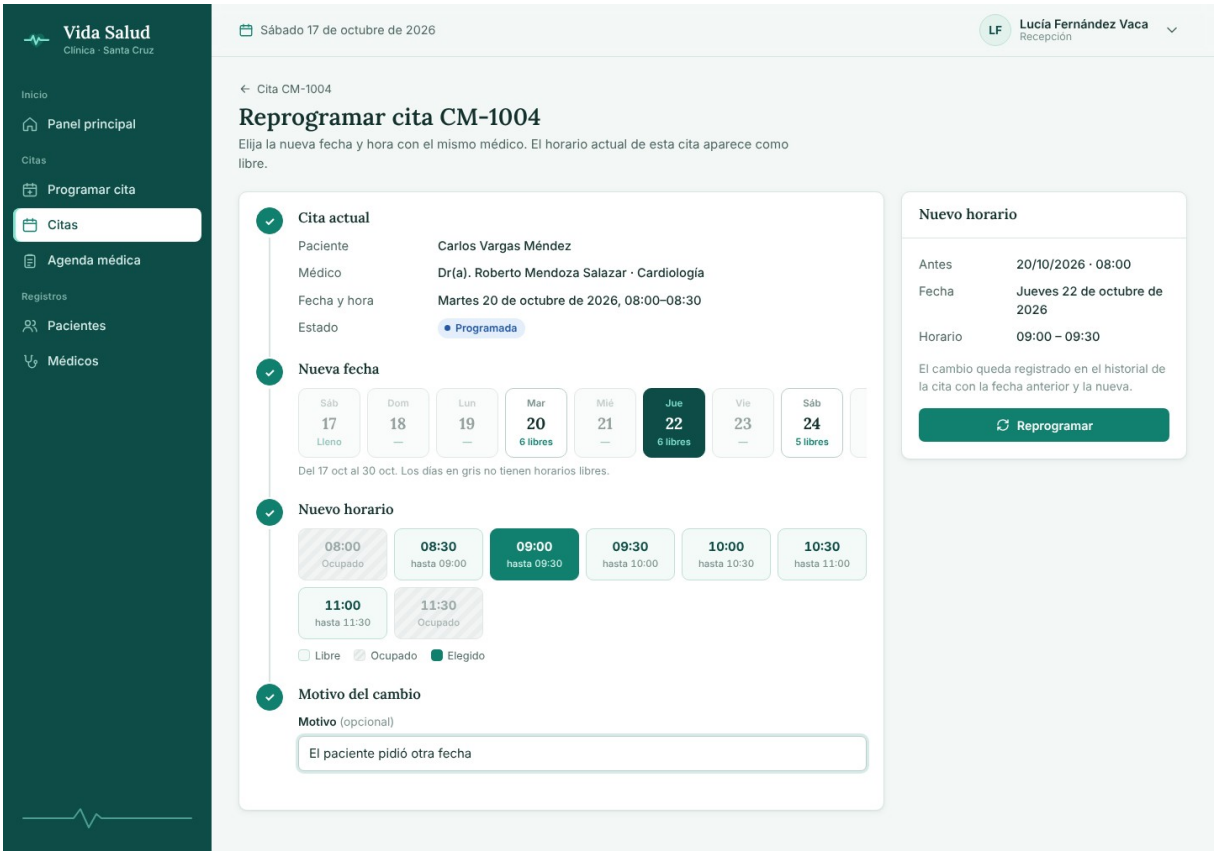

Figura 46. *CU10 — Reprogramar: nueva fecha y hora del mismo médico.*

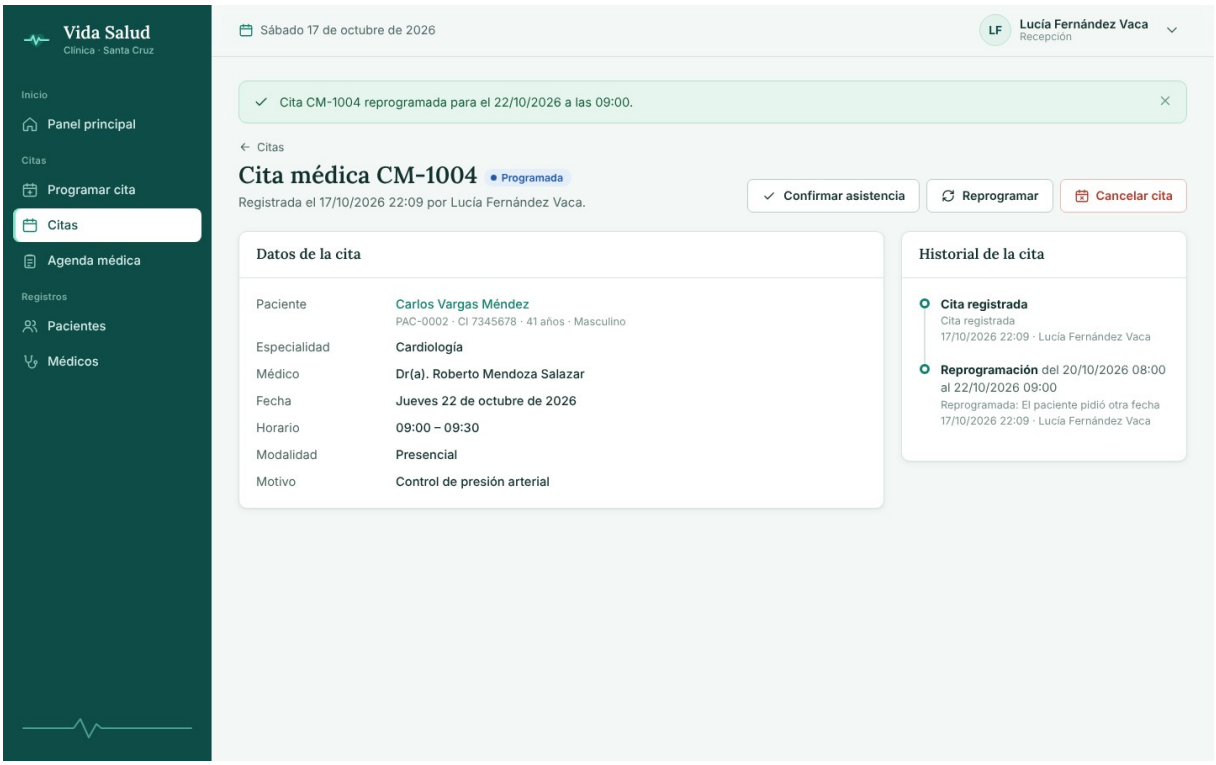

Figura 47. *CU10 — Cita reprogramada con el cambio en su historial.*

Prototipo CU11: Cancelar cita

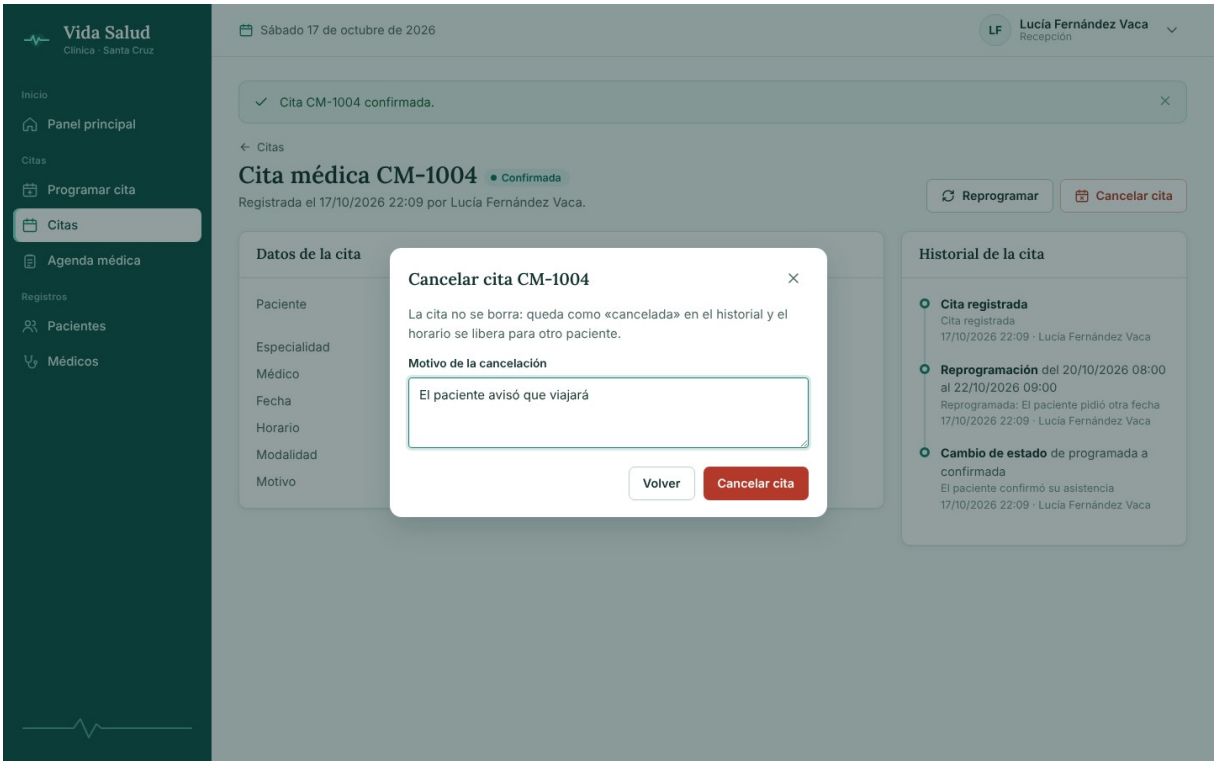

Figura 48. *CU11 — Cancelación con motivo obligatorio.*

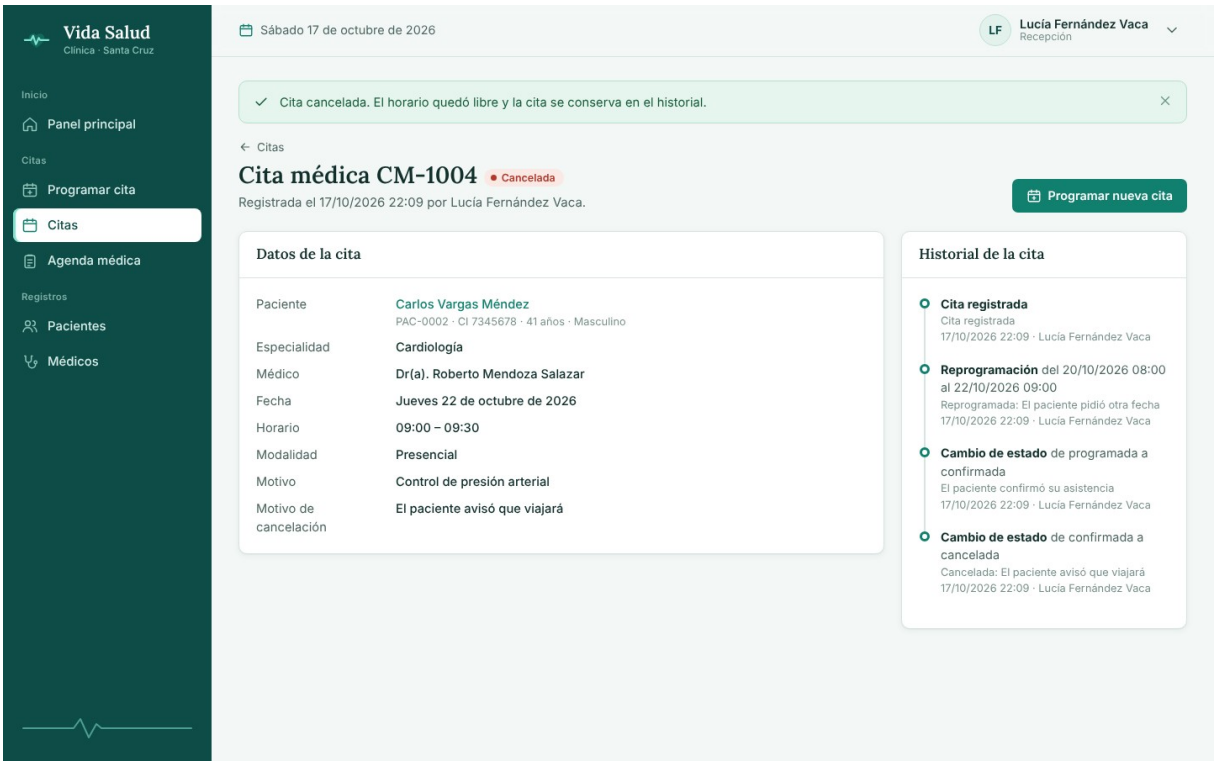

Figura 49. *CU11 — Cita cancelada: el registro se conserva y el horario se libera.*

Prototipo CU12: Registrar asistencia

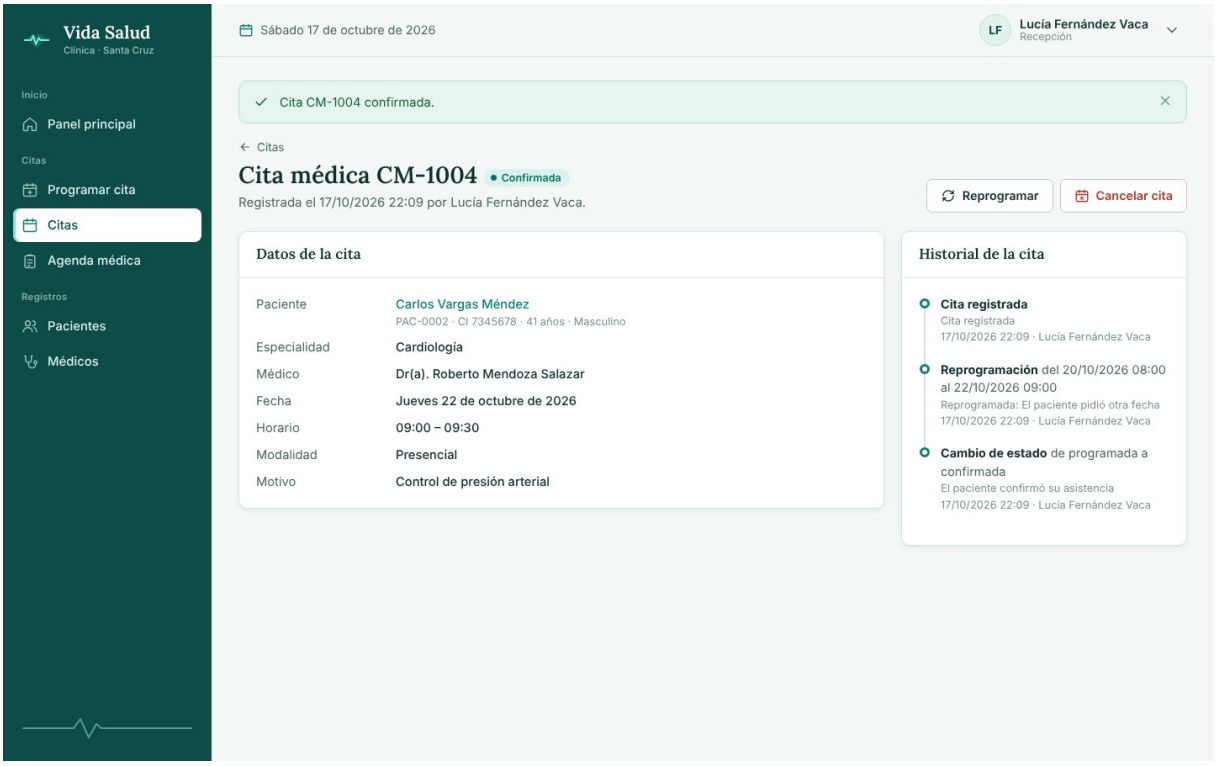

Figura 50. *CU12 — Cita confirmada desde su detalle.*

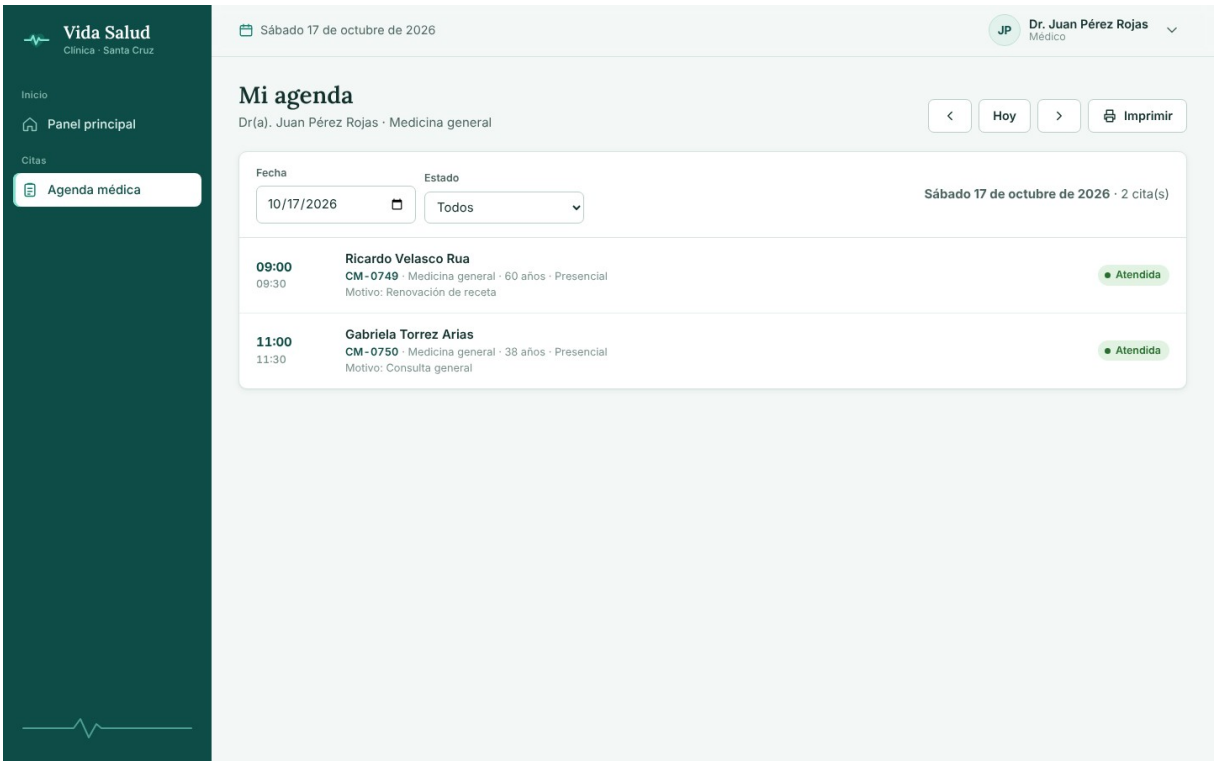

Figura 51. *CU12 — Agenda del médico con la acción «Atendida».*

Prototipo CU13: Consultar agenda médica

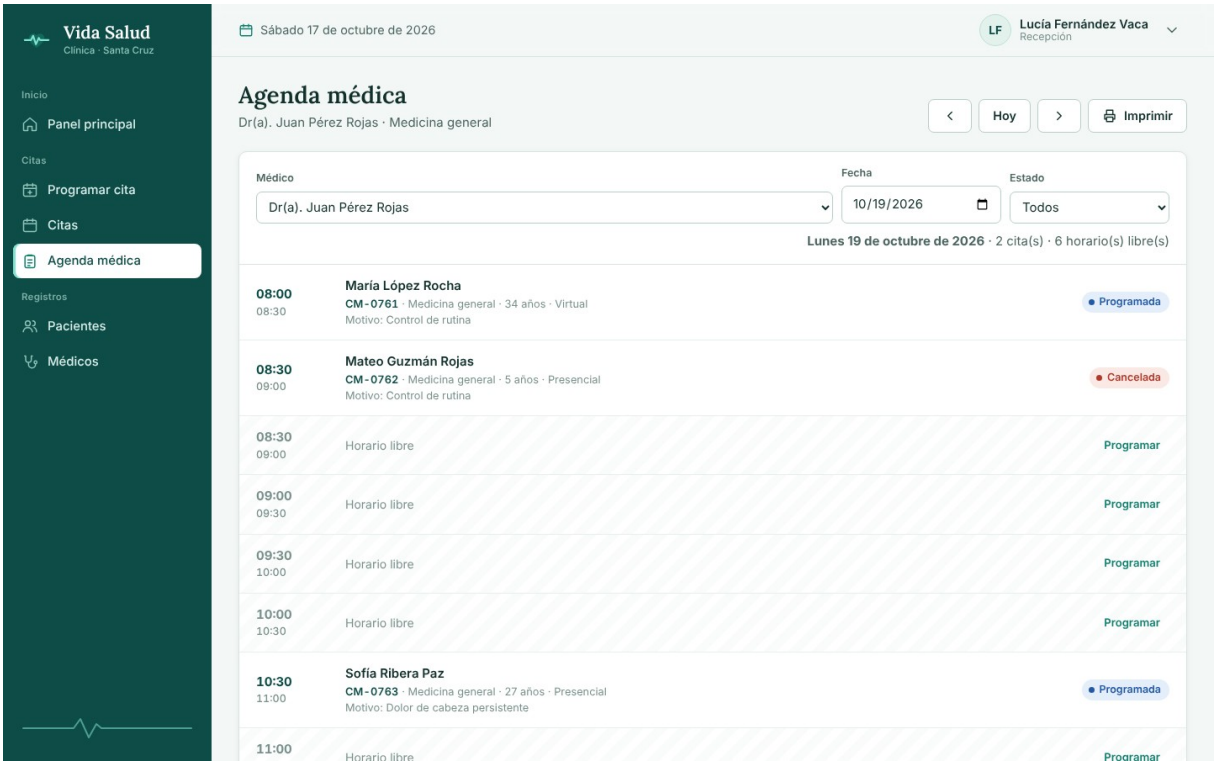

Figura 52. *CU13 — Agenda médica de un día con citas y horarios libres.*

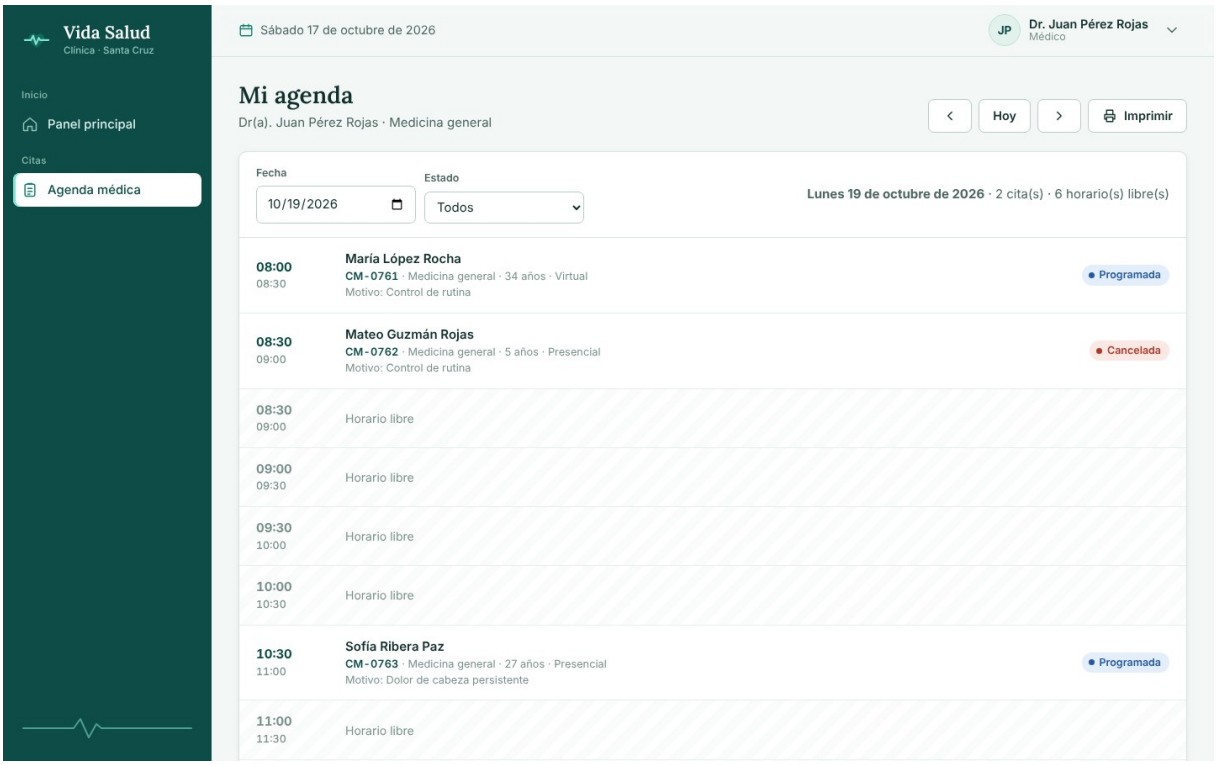

Figura 53. *CU13 — Mi agenda (vista del médico).*

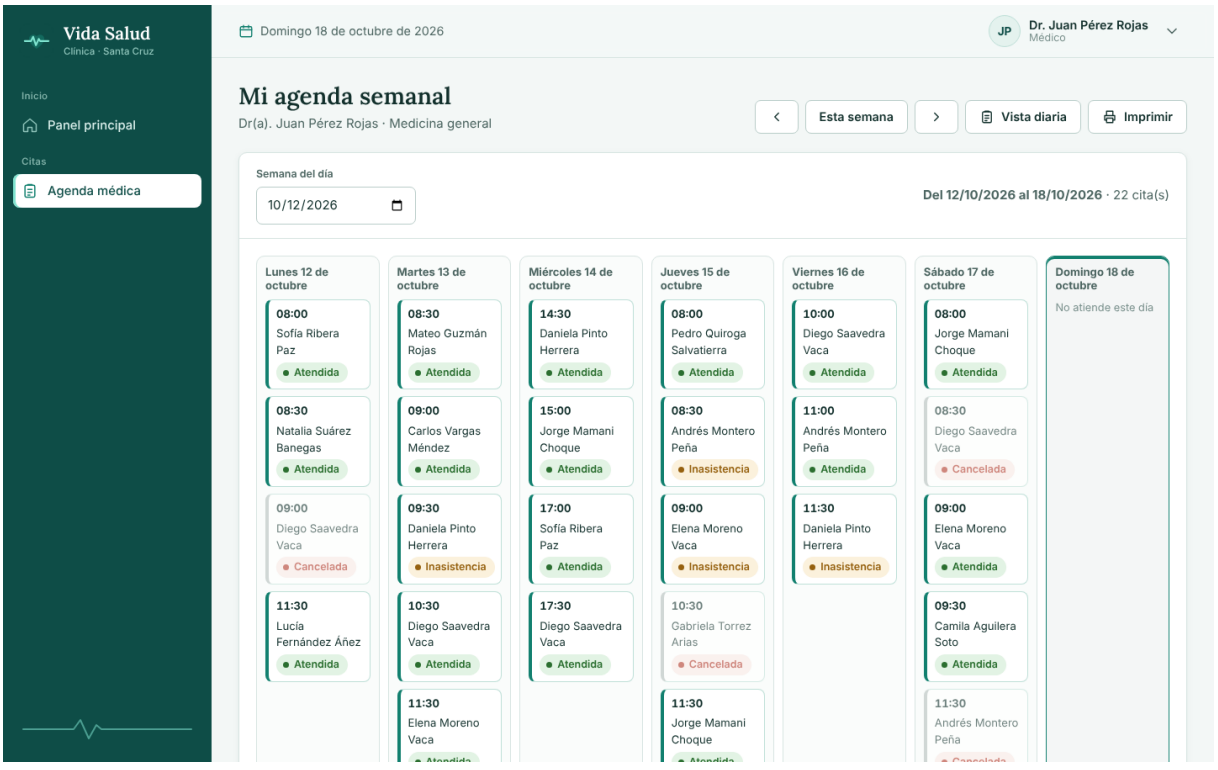

Figura 54. *CU13 — Agenda semanal del médico: siete días con sus citas y días sin atención.*

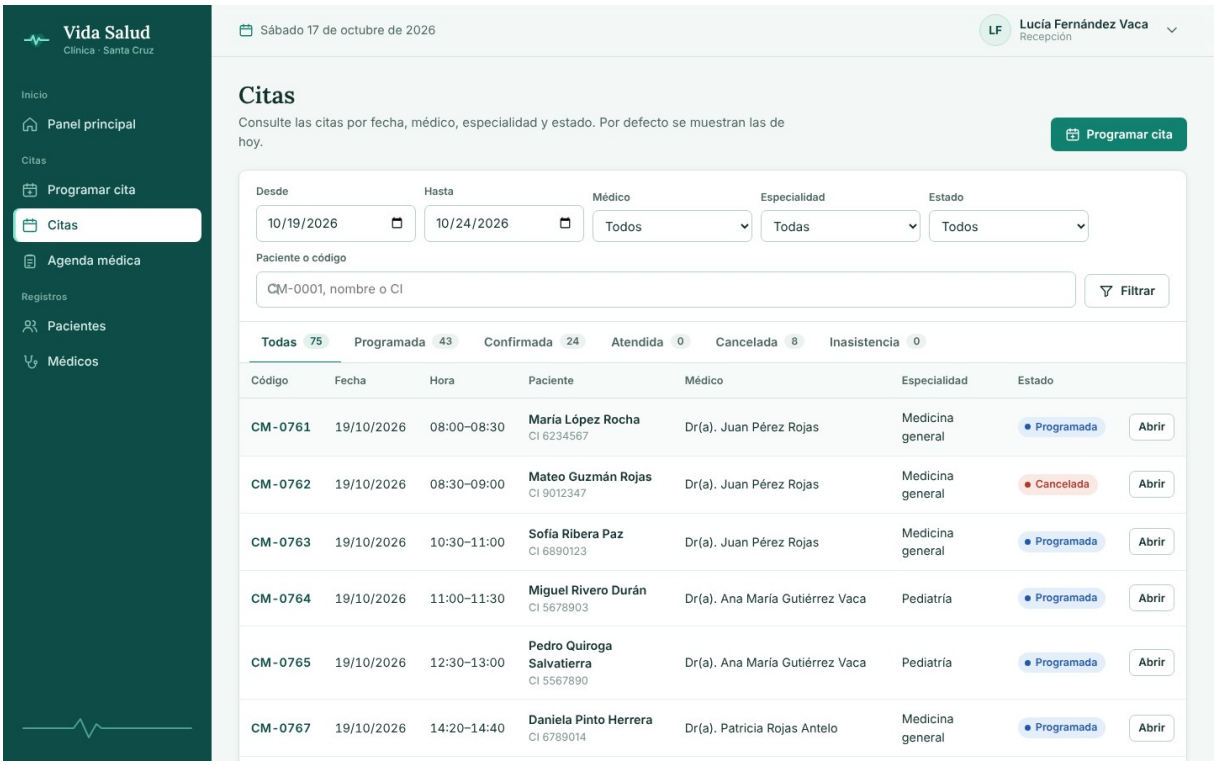

Figura 55. *CU13 — Citas por fecha, médico, especialidad y estado.*

Prototipo CU14: Generar reportes

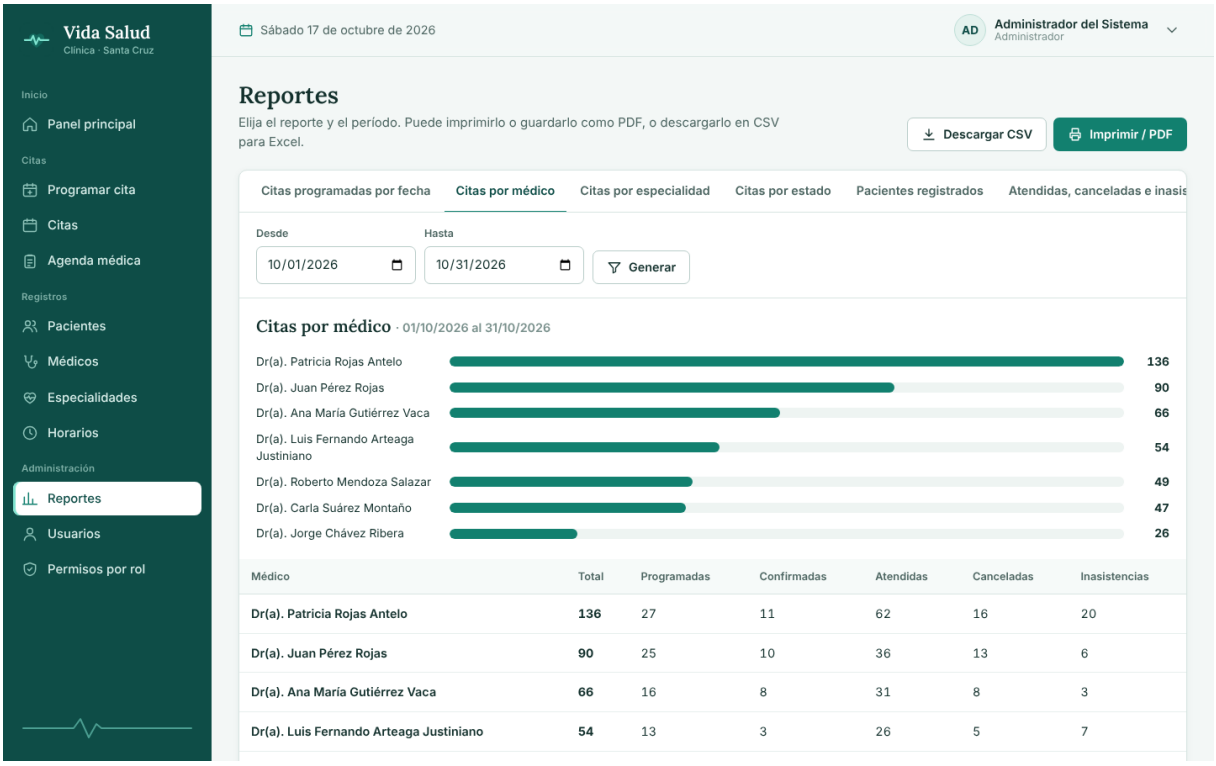

Figura 56. *CU14 — Reporte de citas por médico con totales.*

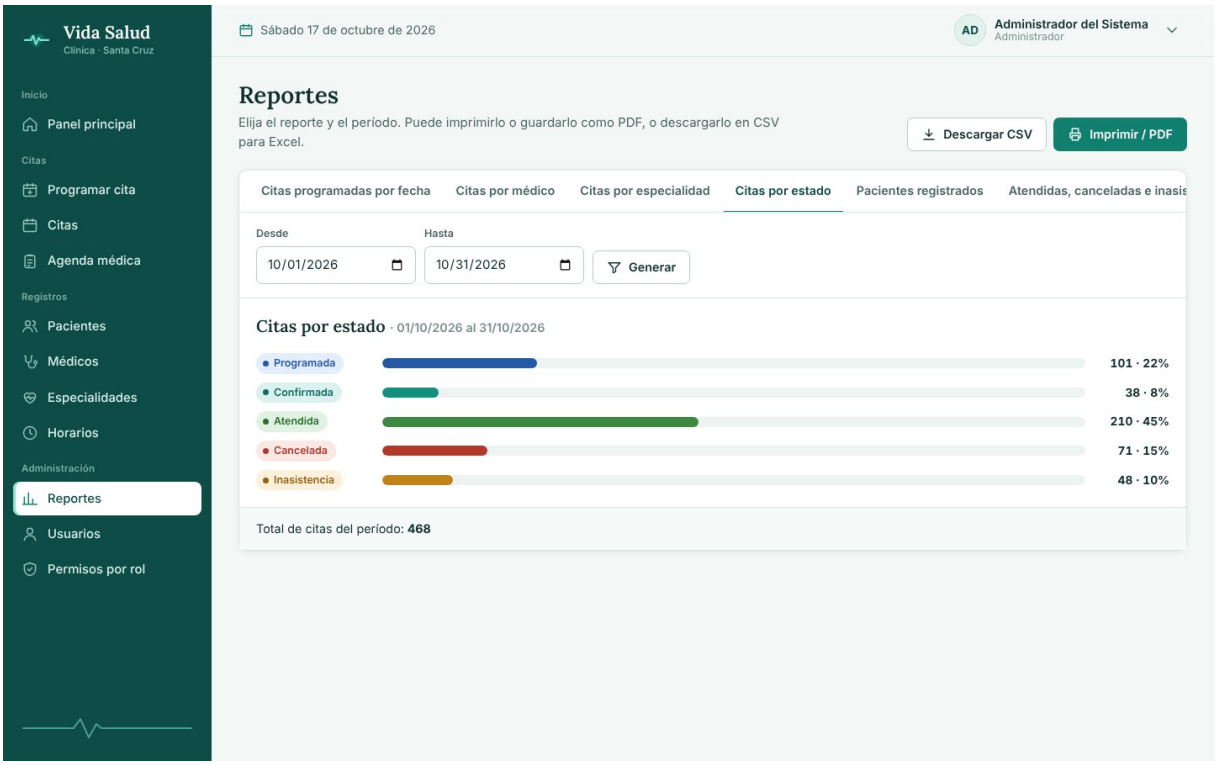

Figura 57. *CU14 — Reporte de citas por estado.*

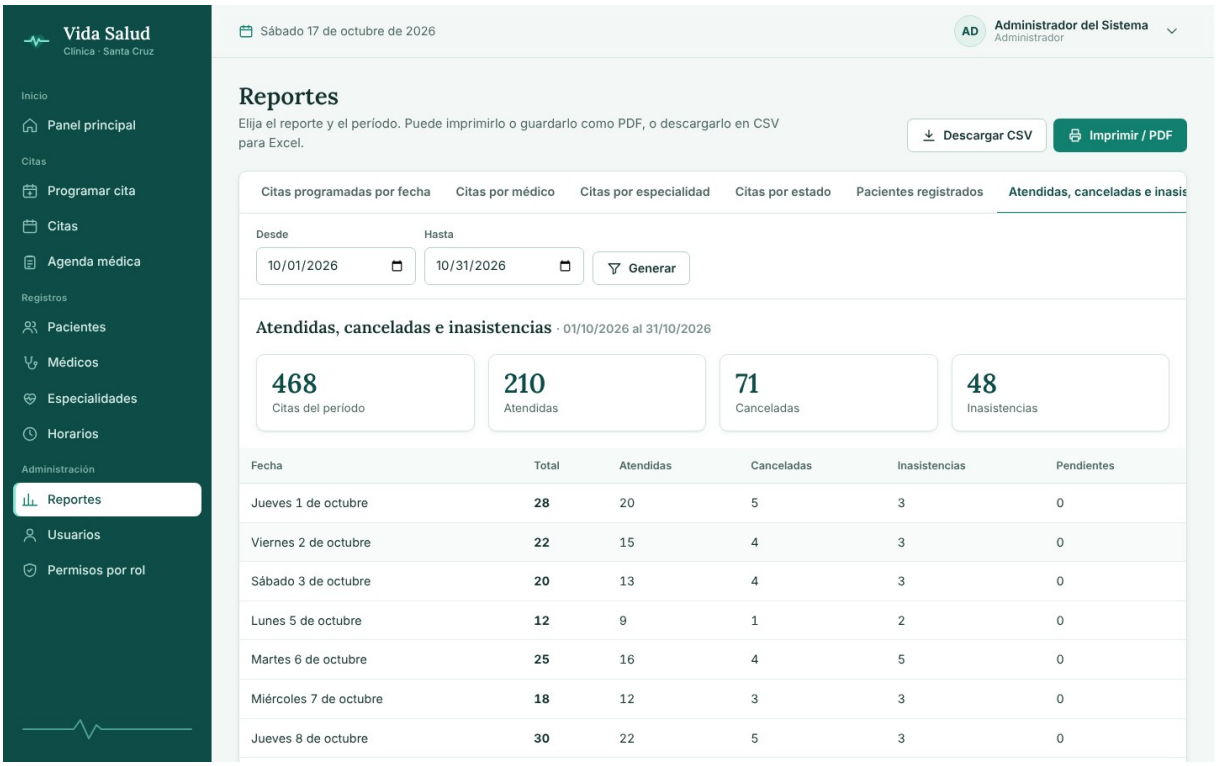

Figura 58. *CU14 — Atendidas, canceladas e inasistencias del período.*

Vista en celular. La misma aplicación se adapta a pantallas pequeñas: el menú lateral se oculta en un botón y las tablas se desplazan.

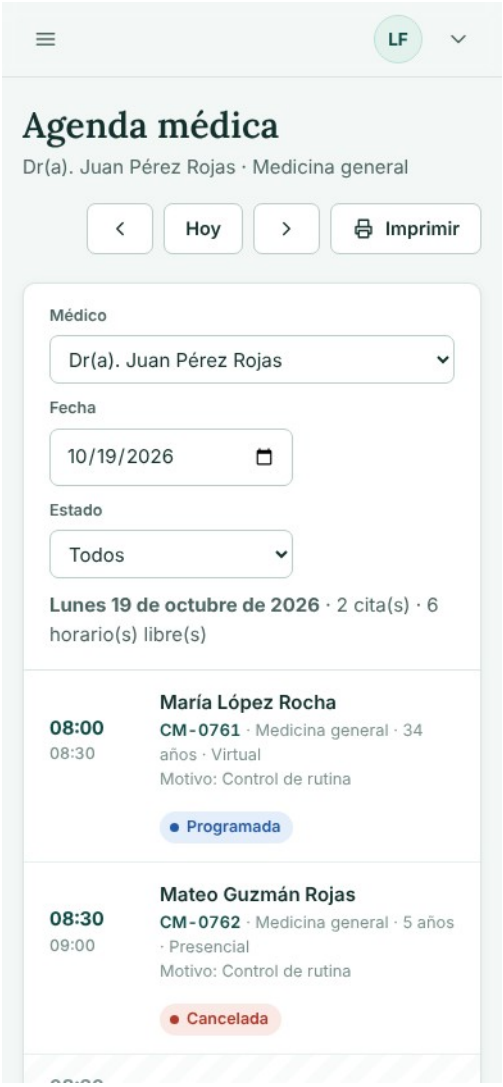

Figura 59. *Agenda médica diaria vista desde un celular.*

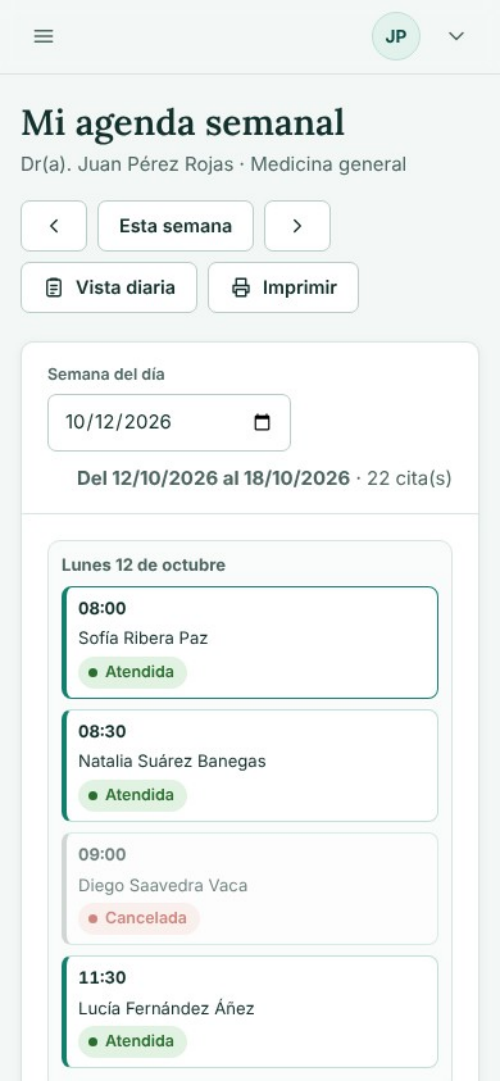

Figura 60. *Agenda semanal vista desde un celular.*

4.7 Estructurar el modelo de casos de uso

Ciclo 1 — Seguridad, registros base y programación de citas

Todos los usuarios inician sesión (CU01). El administrador gestiona usuarios y permisos y consulta la bitácora de acciones (CU02), especialidades (CU06), médicos (CU07) y sus horarios (CU08). La recepcionista registra a los pacientes (CU03) y programa las citas (CU09) mostrando solo horarios disponibles; el paciente también puede reservar en línea. Con este ciclo la clínica ya puede agendar citas sin choques de horario.

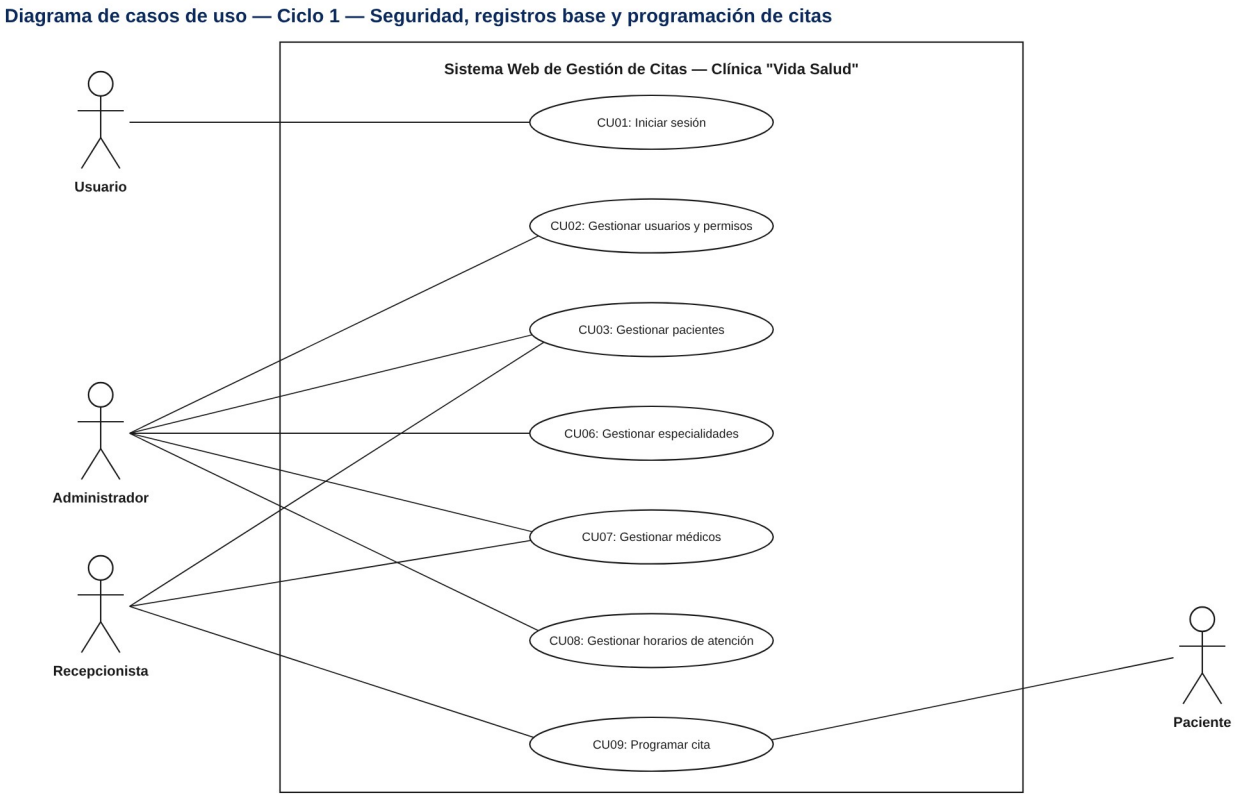

Figura 61. *Diagrama de casos de uso — Ciclo 1 — Seguridad, registros base y programación de citas.*

Ciclo 2 — Seguimiento de citas, agenda, portal del paciente y reportes

El paciente gestiona su perfil (CU04) y sigue sus citas (CU05). La recepcionista reprograma (CU10), cancela (CU11) y registra la asistencia (CU12); el médico marca sus citas como atendidas y consulta su agenda diaria o semanal (CU13). El administrador genera los reportes (CU14). Con este ciclo el sistema queda completo.

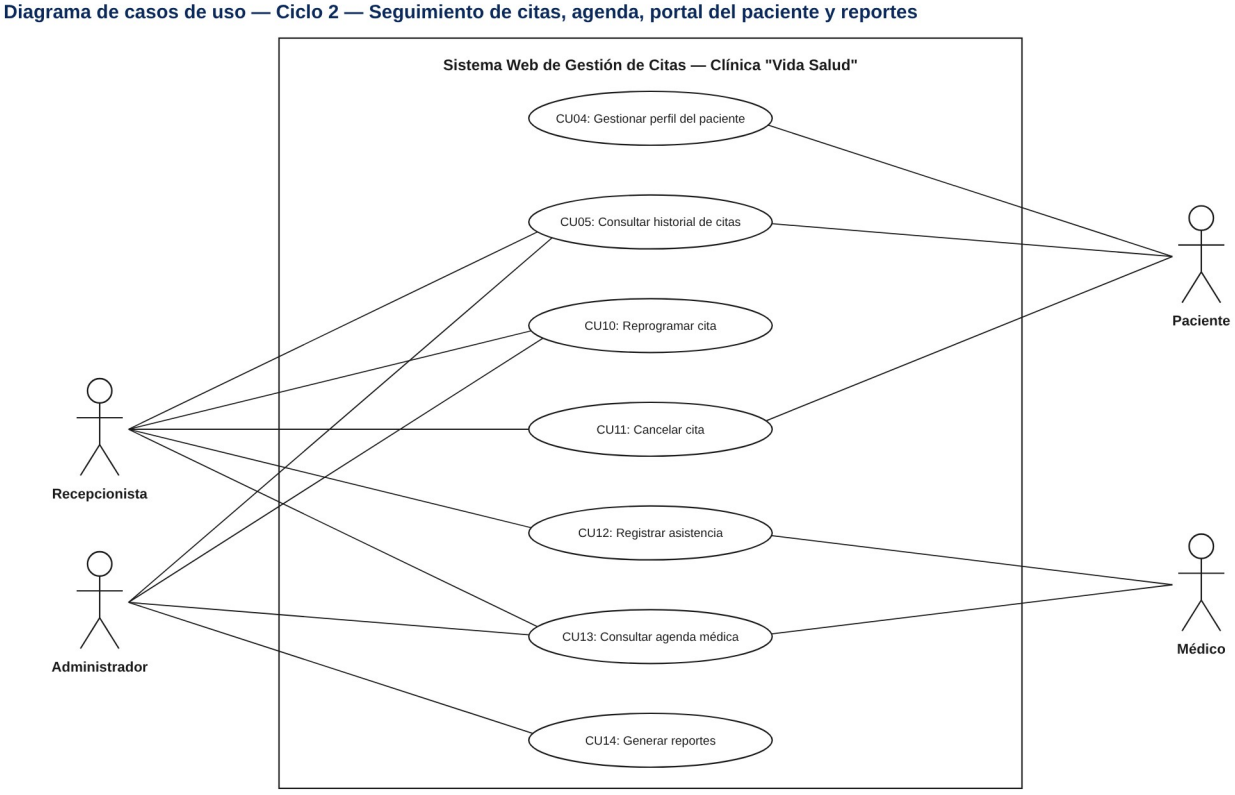

Figura 62. *Diagrama de casos de uso — Ciclo 2 — Seguimiento de citas, agenda, portal del paciente y reportes.*

Escenario completo

El diagrama muestra los 14 casos de uso de los 5 paquetes con los 4 actores (más el actor general Usuario).

Diagrama de casos de uso — escenario completo (14 casos de uso)

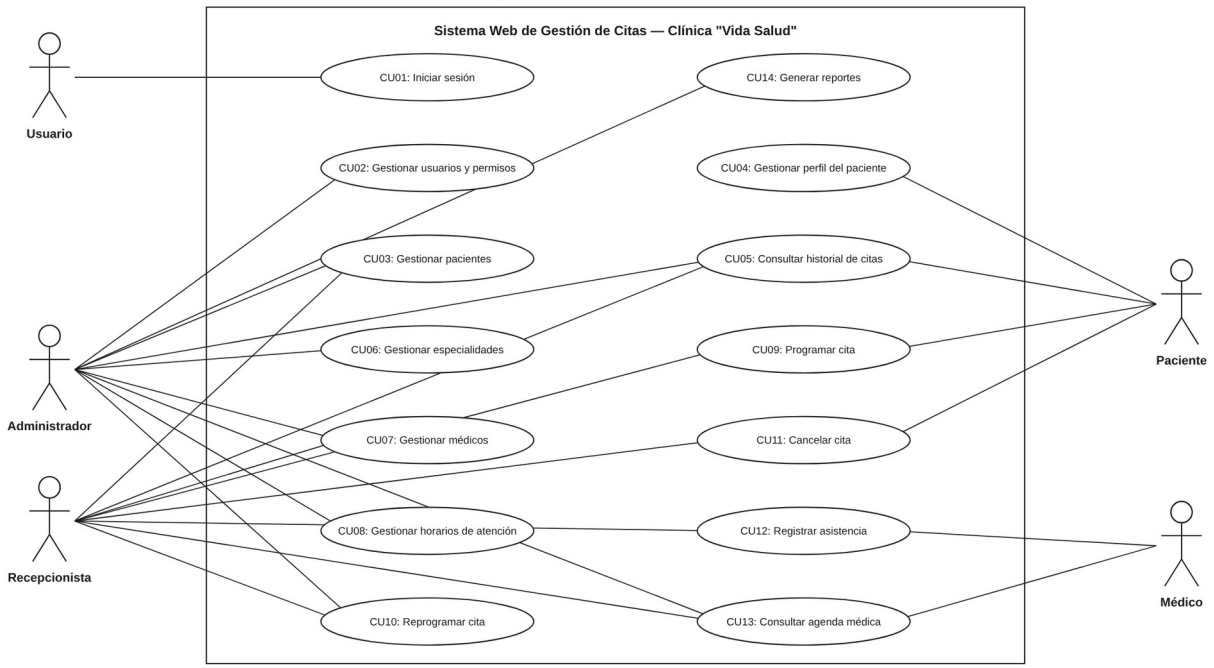

Figura 63. *Diagrama de casos de uso — escenario completo.*

5. FLUJO DE TRABAJO: ANÁLISIS

En el análisis se refinan los casos de uso capturados y se describen con clases de análisis de tres tipos: **interfaz (IU)**, que representa las pantallas con las que interactúa el actor; **control (CTR)**, que coordina el caso de uso y aplica las reglas de negocio; y **entidad (CE)**, que representa la información persistente. Las clases se agrupan en los mismos cinco paquetes que luego se usan en el diseño y en el código.

5.1 Análisis de arquitectura

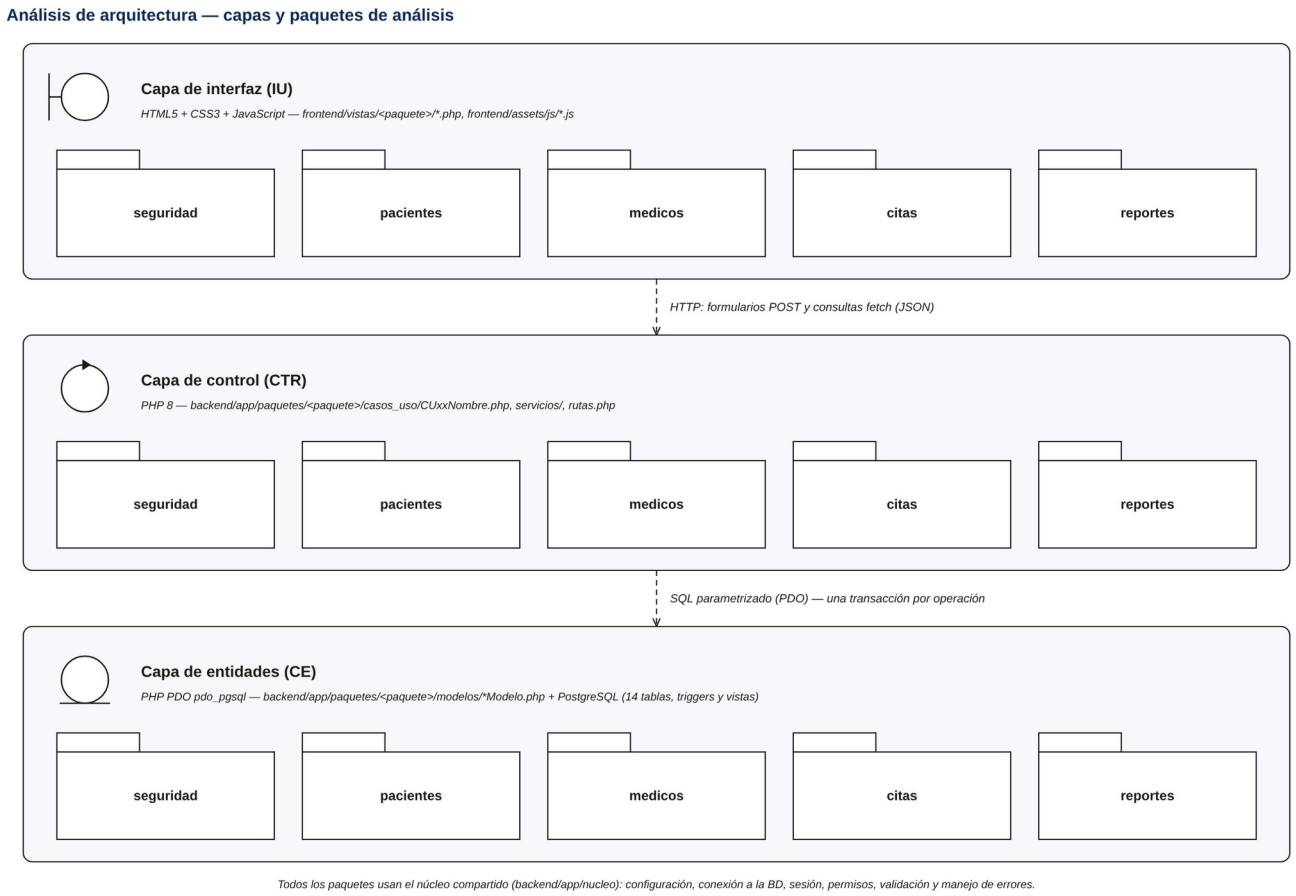

Figura 64. *Análisis de arquitectura: capas y paquetes de análisis.*

- **Capa de interfaz (IU):** plantillas HTML5 con CSS3 y JavaScript (`frontend/vistas/<paquete>/`). Capturan los datos y muestran resultados; no contienen reglas de negocio.
- **Capa de control (CTR):** un archivo PHP por caso de uso (`casos_uso/CUxxNombre.php`) más los servicios de reglas (`Disponibilidad`, `EstadoCita`). Coordinan el flujo, validan y aplican permisos.
- **Capa de entidades (CE):** modelos de acceso a datos (`modelos/*Modelo.php`) y las tablas de PostgreSQL, que garantizan la integridad con PK, FK, CHECK, UNIQUE, EXCLUDE y triggers.

5.2 Análisis de casos de uso

Para cada caso de uso se elaboró un diagrama de comunicación con las clases de análisis que participan y los mensajes numerados en el orden en que ocurren: los números enteros (1, 2, 3…) son acciones del actor y los decimales (1.1, 1.2…) son los mensajes internos que provoca cada acción. Reglas aplicadas:

- El actor solo se comunica con clases de interfaz (IU).
- Una interfaz se comunica con otras interfaces y con clases de control (CTR).
- Una clase de control se comunica con interfaces, con otras clases de control y con clases entidad (CE).
- No existe comunicación directa entre clases entidad: toda consulta o cambio pasa por una clase de control.

Ciclo 1 — Seguridad, registros base y programación de citas

CU01: Iniciar sesión

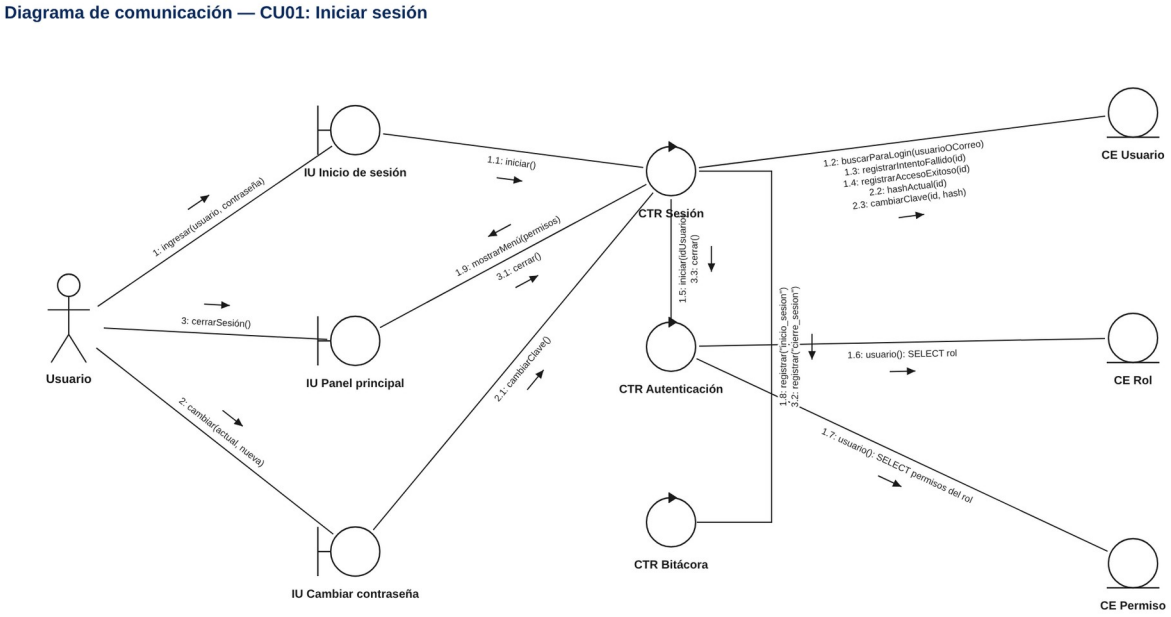

Figura 65. *Diagrama de comunicación — CU01: Iniciar sesión.*

CU02: Gestionar usuarios y permisos

Diagrama de comunicación — CU02: Gestionar usuarios y permisos

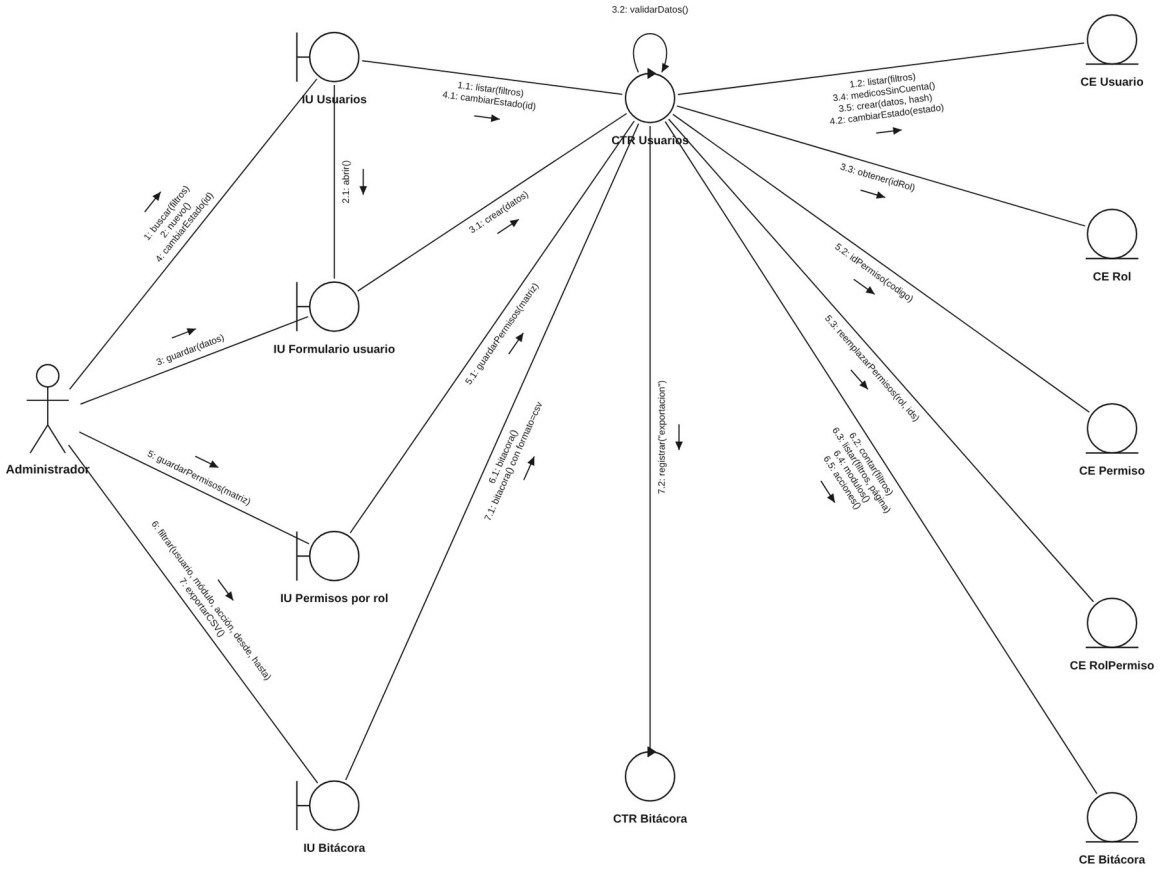

Figura 66. *Diagrama de comunicación — CU02: Gestionar usuarios y permisos.*

CU03: Gestionar pacientes

Diagrama de comunicación — CU03: Gestionar pacientes

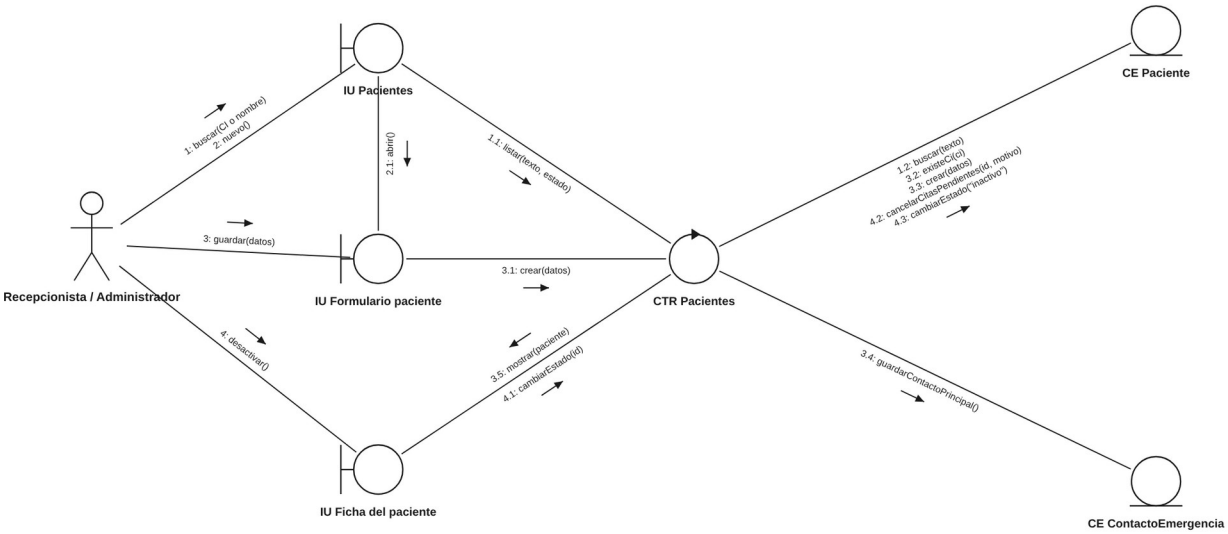

Figura 67. *Diagrama de comunicación — CU03: Gestionar pacientes.*

CU06: Gestionar especialidades

Diagrama de comunicación — CU06: Gestionar especialidades

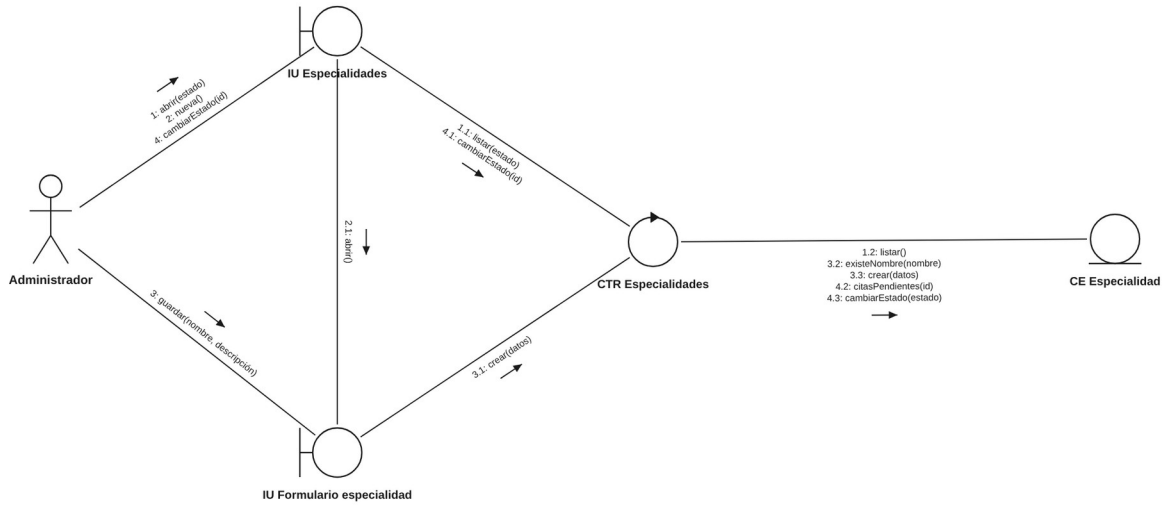

Figura 68. *Diagrama de comunicación — CU06: Gestionar especialidades.*

CU07: Gestionar médicos

Diagrama de comunicación — CU07: Gestionar médicos

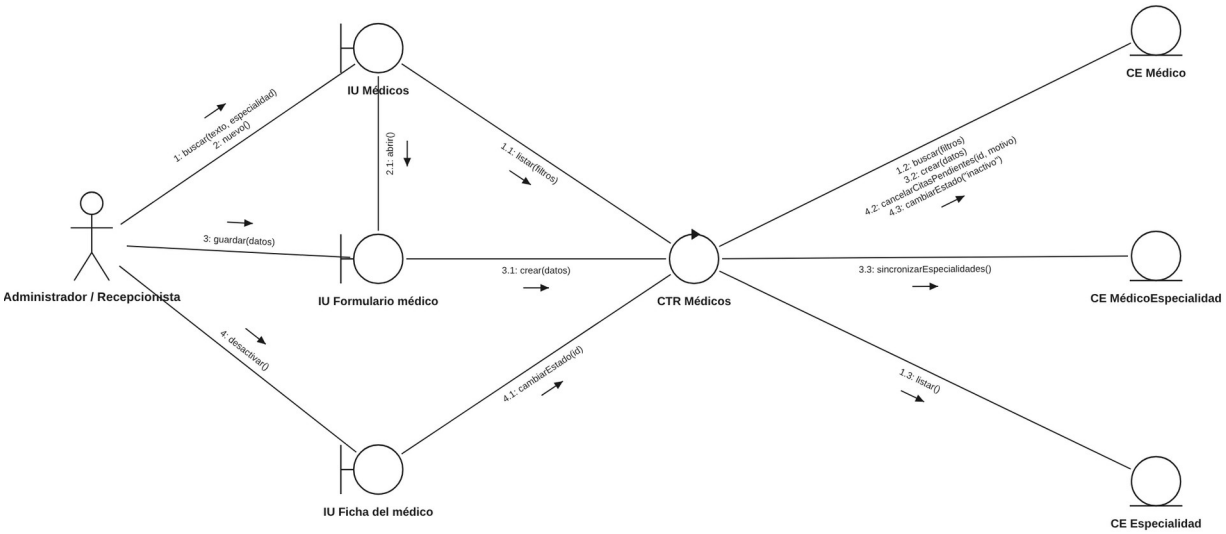

Figura 69. *Diagrama de comunicación — CU07: Gestionar médicos.*

CU08: Gestionar horarios de atención

Diagrama de comunicación — CU08: Gestionar horarios de atención

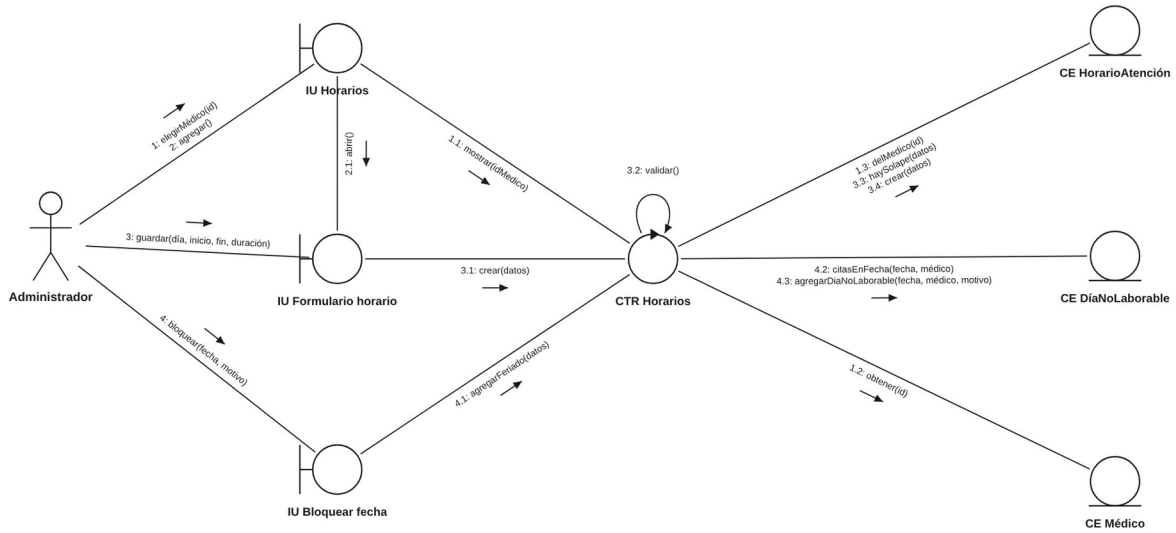

Figura 70. *Diagrama de comunicación — CU08: Gestionar horarios de atención.*

CU09: Programar cita

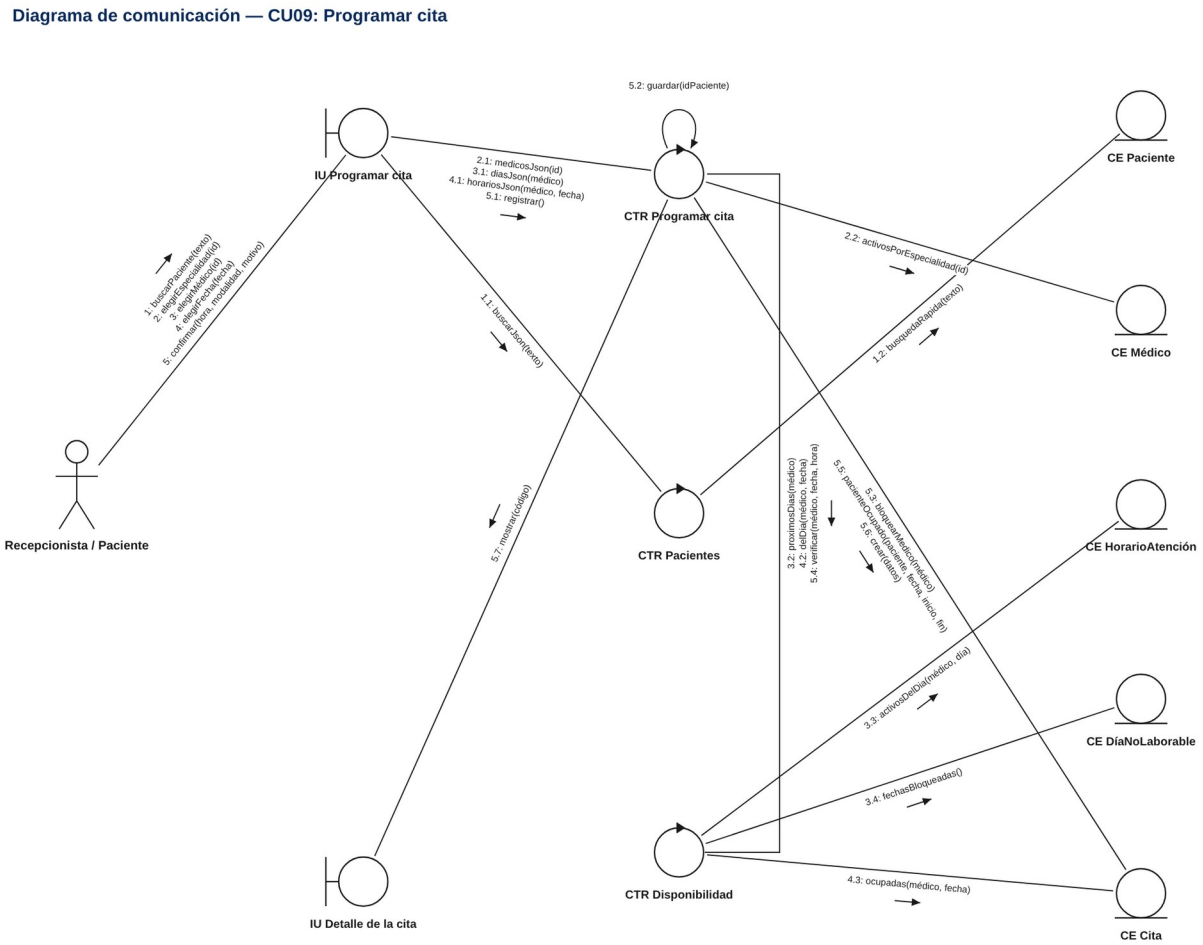

Figura 71. *Diagrama de comunicación — CU09: Programar cita.*

Ciclo 2 — Seguimiento de citas, agenda, portal del paciente y reportes

CU04: Gestionar perfil del paciente

Diagrama de comunicación — CU04: Gestionar perfil del paciente

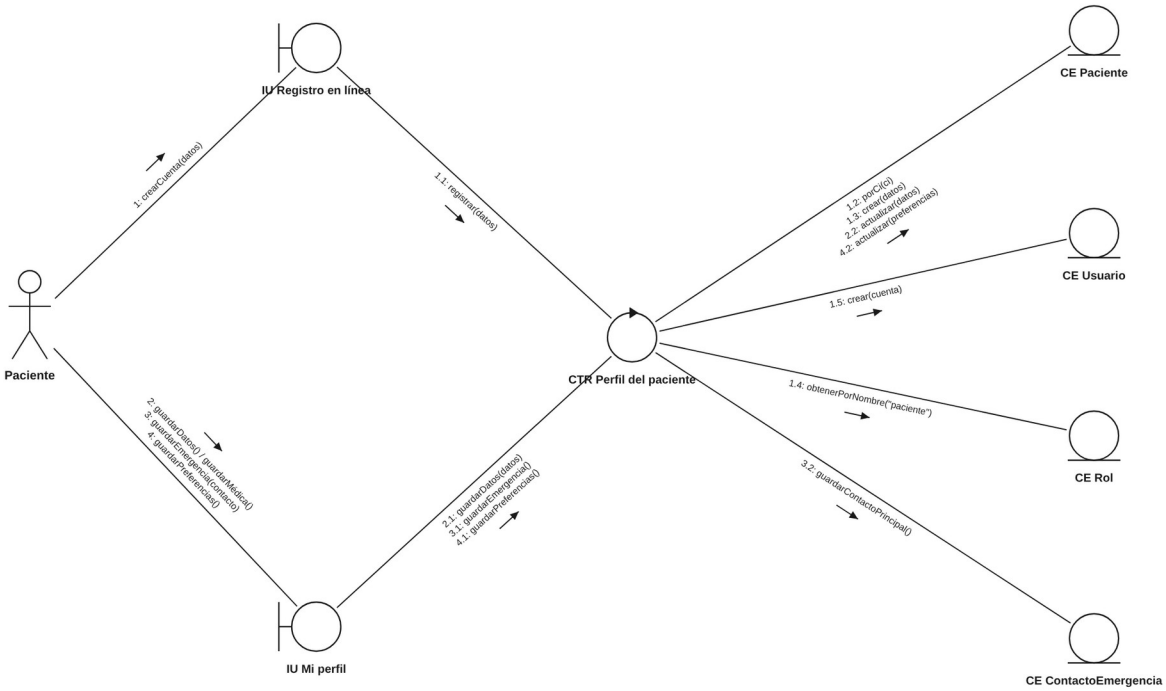

Figura 72. *Diagrama de comunicación — CU04: Gestionar perfil del paciente.*

CU05: Consultar historial de citas

Diagrama de comunicación — CU05: Consultar historial de citas

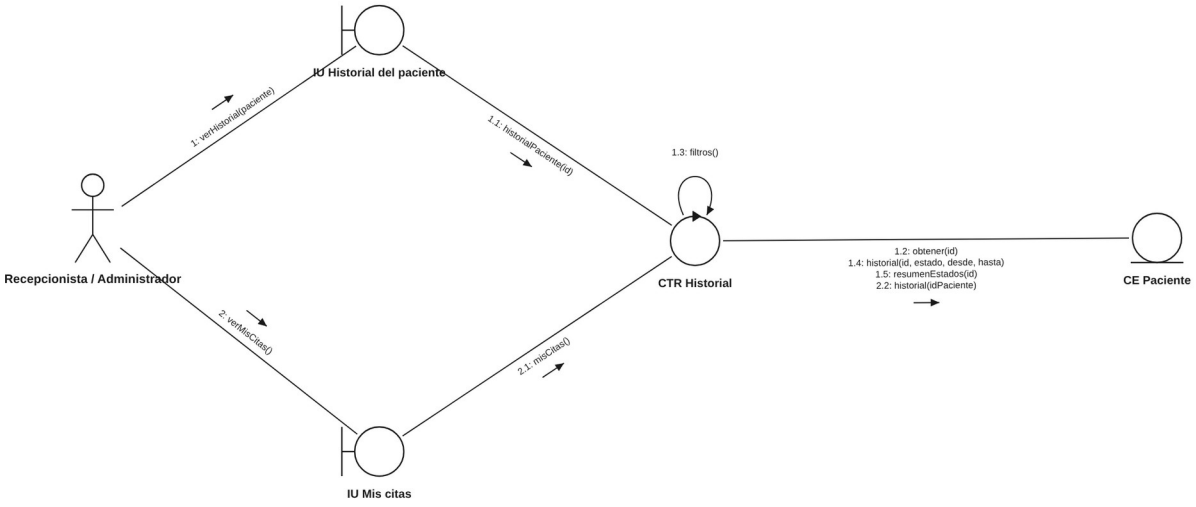

Figura 73. *Diagrama de comunicación — CU05: Consultar historial de citas.*

CU10: Reprogramar cita

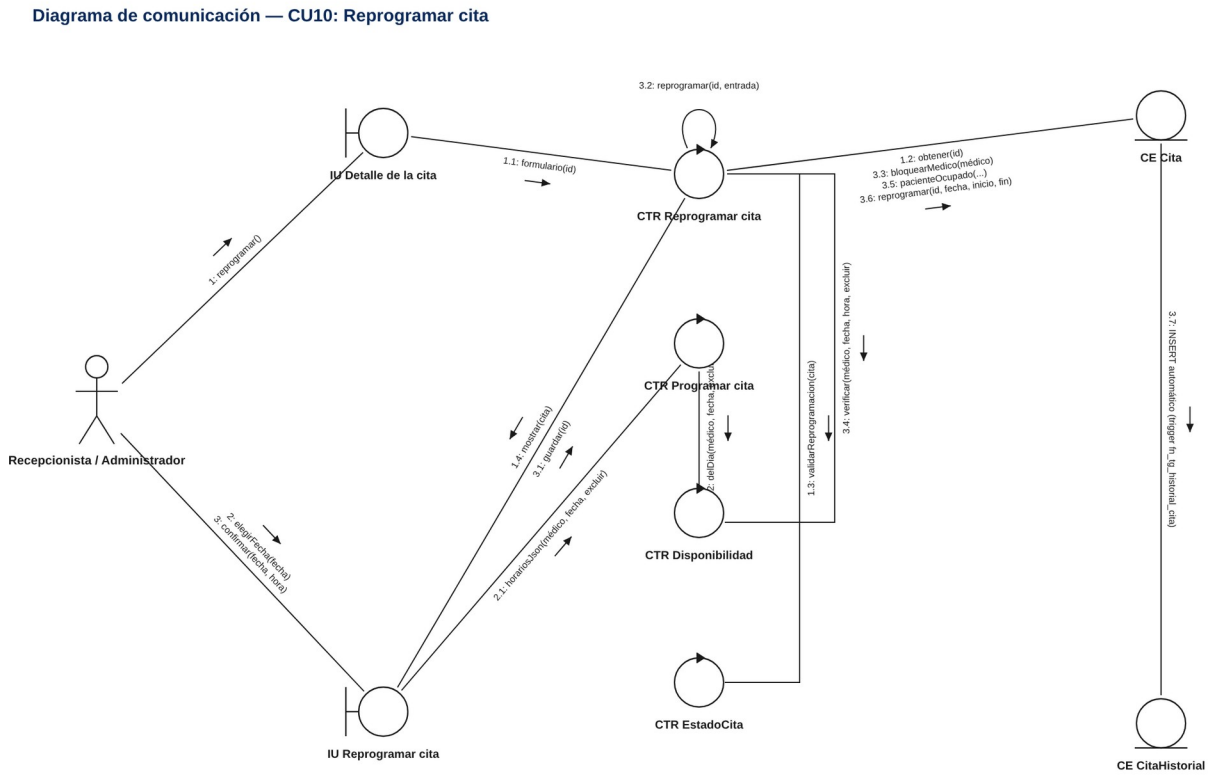

Figura 74. *Diagrama de comunicación — CU10: Reprogramar cita.*

CU11: Cancelar cita

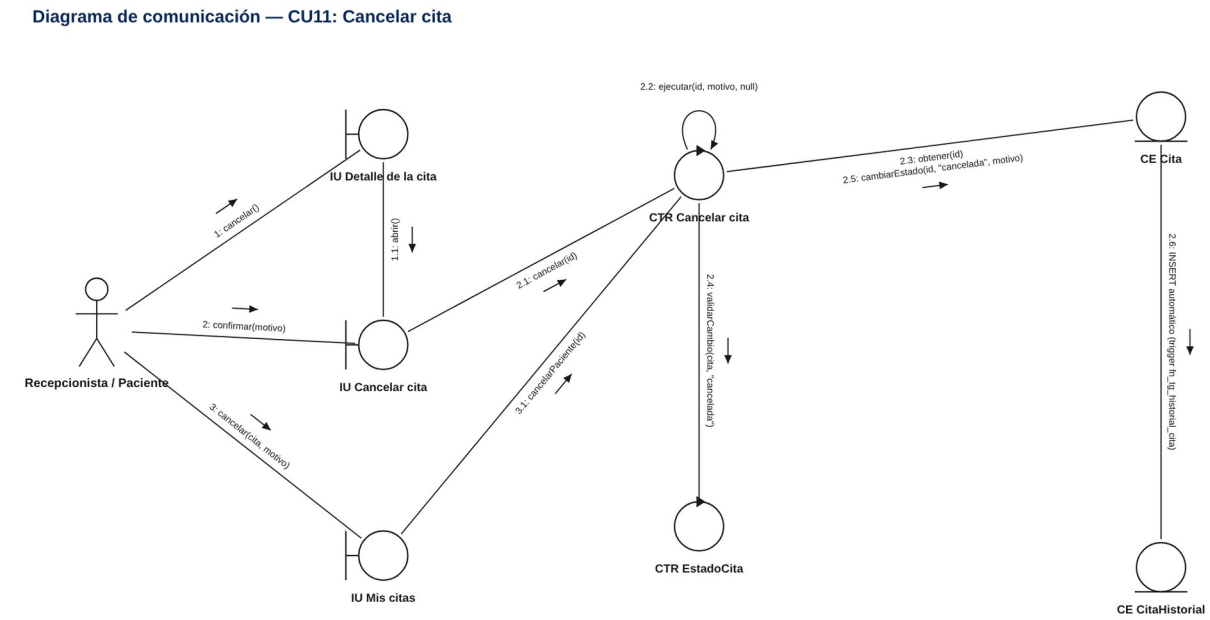

Figura 75. *Diagrama de comunicación — CU11: Cancelar cita.*

CU12: Registrar asistencia

Diagrama de comunicación — CU12: Registrar asistencia

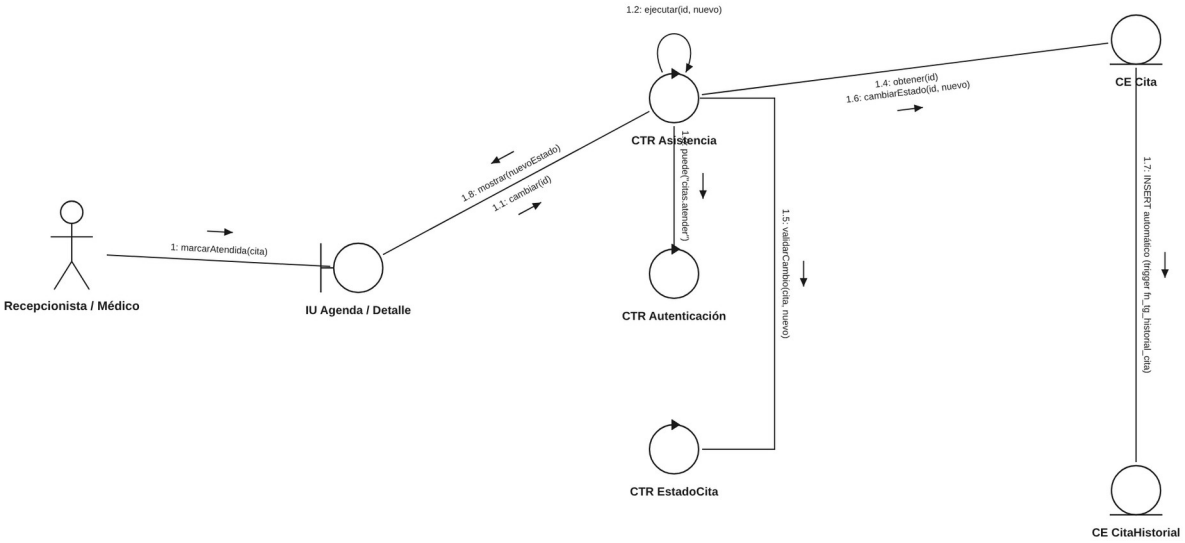

Figura 76. *Diagrama de comunicación — CU12: Registrar asistencia.*

CU13: Consultar agenda médica

Diagrama de comunicación — CU13: Consultar agenda médica

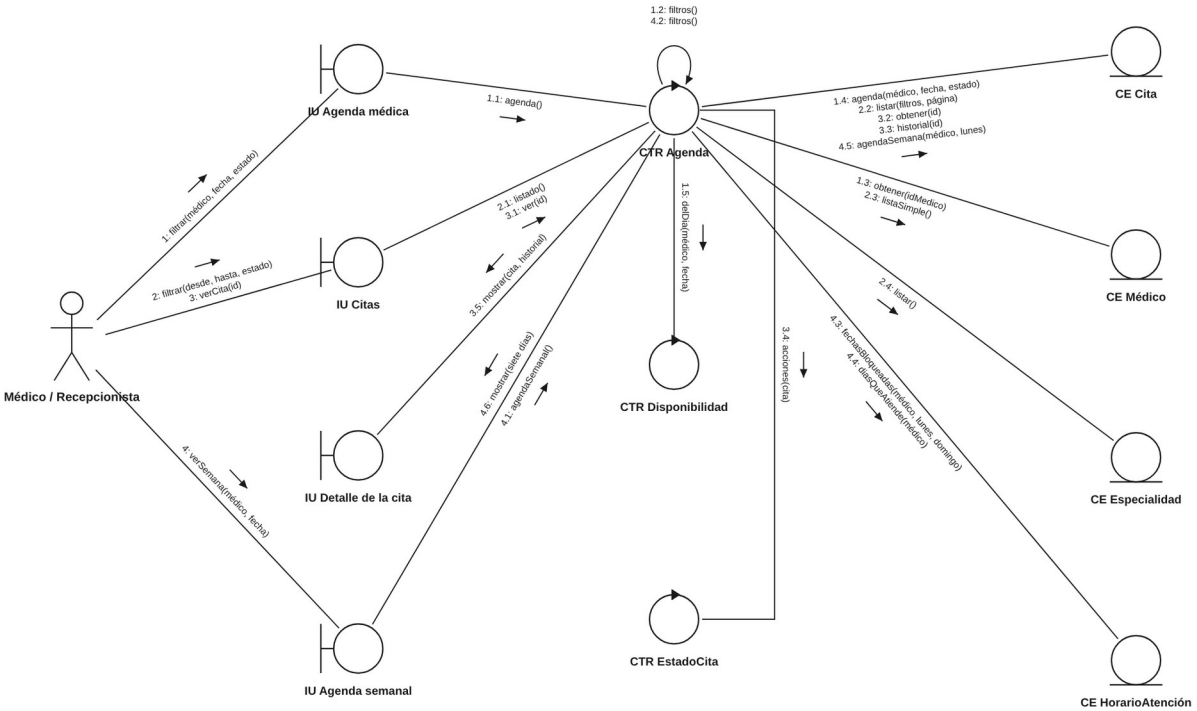

Figura 77. *Diagrama de comunicación — CU13: Consultar agenda médica.*

CU14: Generar reportes

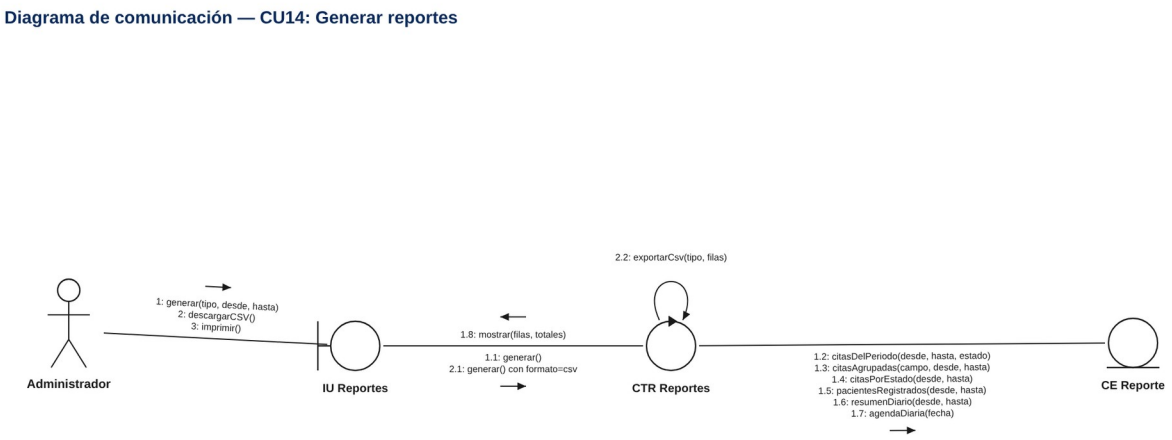

Figura 78. *Diagrama de comunicación — CU14: Generar reportes.*

5.3 Análisis de clases

Para cada caso de uso se identifican sus clases de interfaz, de control y de entidad. La tabla resume, por paquete, el nombre de cada clase de análisis y el archivo que la implementa; luego se muestra el diagrama de clases de análisis de cada caso de uso con sus atributos, operaciones y relaciones.

Tabla 9. *Clases de análisis — paquete seguridad*

CU	Interfaz (IU)	Control (CTR)	Entidad (CE)	Archivo de control
CU01	IU Inicio de sesión IU Panel principal IU Cambiar contraseña	CTR Sesión CTR Autenticación CTR Bitácora	CE Usuario CE Rol CE Permiso	CU01IniciarSesion.php
CU02	IU Usuarios IU Formulario usuario IU Permisos por rol IU Bitácora	CTR Usuarios CTR Bitácora	CE Usuario CE Rol CE Permiso CE RolPermiso CE Médico CE Bitácora	CU02GestionarUsuariosPermisos.php

Tabla 10. *Clases de análisis — paquete pacientes*

CU	Interfaz (IU)	Control (CTR)	Entidad (CE)	Archivo de control
CU03	IU Pacientes IU Formulario paciente IU Ficha del paciente	CTR Pacientes	CE Paciente CE ContactoEmergencia CE Cita	CU03GestionarPacientes.php
CU04	IU Registro en línea IU Mi perfil	CTR Perfil del paciente	CE Paciente CE Usuario CE Rol	CU04GestionarPerfilPaciente.php

CU	Interfaz (IU)	Control (CTR)	Entidad (CE)	Archivo de control
			CE ContactoEmergencia	
CU05	IU Historial del paciente IU Mis citas	CTR Historial	CE Paciente CE Cita CE Médico CE Especialidad	CU05ConsultarHistorialCitas.php

Tabla 11. *Clases de análisis — paquete medicos*

CU	Interfaz (IU)	Control (CTR)	Entidad (CE)	Archivo de control
CU06	IU Especialidades IU Formulario especialidad	CTR Especialidades	CE Especialidad CE Cita	CU06GestionarEspecialidades.php
CU07	IU Médicos IU Formulario médico IU Ficha del médico	CTR Médicos	CE Médico CE MédicoEspecialidad CE Especialidad CE Cita	CU07GestionarMedicos.php
CU08	IU Horarios IU Formulario horario IU Bloquear fecha	CTR Horarios	CE HorarioAtención CE DíaNoLaborable CE Médico CE Cita	CU08GestionarHorarios.php

Tabla 12. *Clases de análisis — paquete citas*

CU	Interfaz (IU)	Control (CTR)	Entidad (CE)	Archivo de control
CU09	IU Programar cita IU Detalle de la cita	CTR Programar cita CTR Pacientes CTR Disponibilidad	CE Paciente CE Médico CE HorarioAtención CE DíaNoLaborable CE Cita	CU09ProgramarCita.php
CU10	IU Detalle de la cita IU Reprogramar cita	CTR Reprogramar cita CTR Programar cita CTR Disponibilidad CTR EstadoCita	CE Cita CE CitaHistorial	CU10ReprogramarCita.php
CU11	IU Detalle de la cita IU Cancelar cita IU Mis citas	CTR Cancelar cita CTR EstadoCita	CE Cita CE CitaHistorial	CU11CancelarCita.php
CU12	IU Agenda / Detalle	CTR Asistencia CTR Autenticación CTR EstadoCita	CE Cita CE CitaHistorial	CU12RegistrarAsistencia.php
CU13	IU Agenda médica IU Citas IU Detalle de la cita IU Agenda semanal	CTR Agenda CTR Disponibilidad CTR EstadoCita	CE Cita CE Médico CE Especialidad CE CitaHistorial CE HorarioAtención	CU13ConsultarAgendaMedica.php

Tabla 13. *Clases de análisis — paquete reportes*

CU	Interfaz (IU)	Control (CTR)	Entidad (CE)	Archivo de control
CU14	IU Reportes	CTR Reportes	CE Reporte	CU14GenerarReportes.ph

CU	Interfaz (IU)	Control (CTR)	Entidad (CE)	Archivo de control
				p

Ciclo 1 — Seguridad, registros base y programación de citas

Clases de análisis CU01: Iniciar sesión

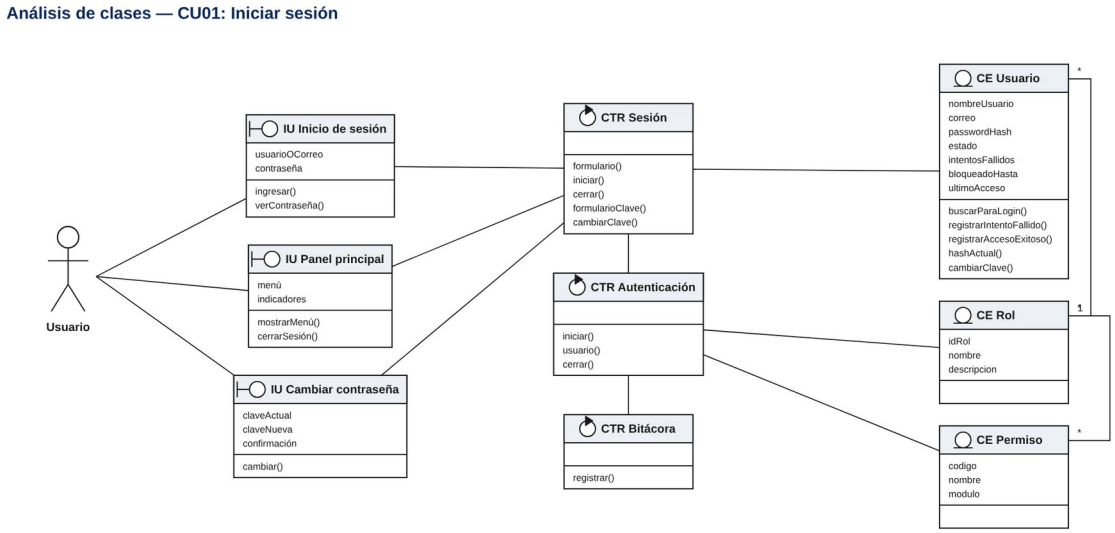

Figura 79. *Análisis de clases — CU01: Iniciar sesión.*

Clases de análisis CU02: Gestionar usuarios y permisos

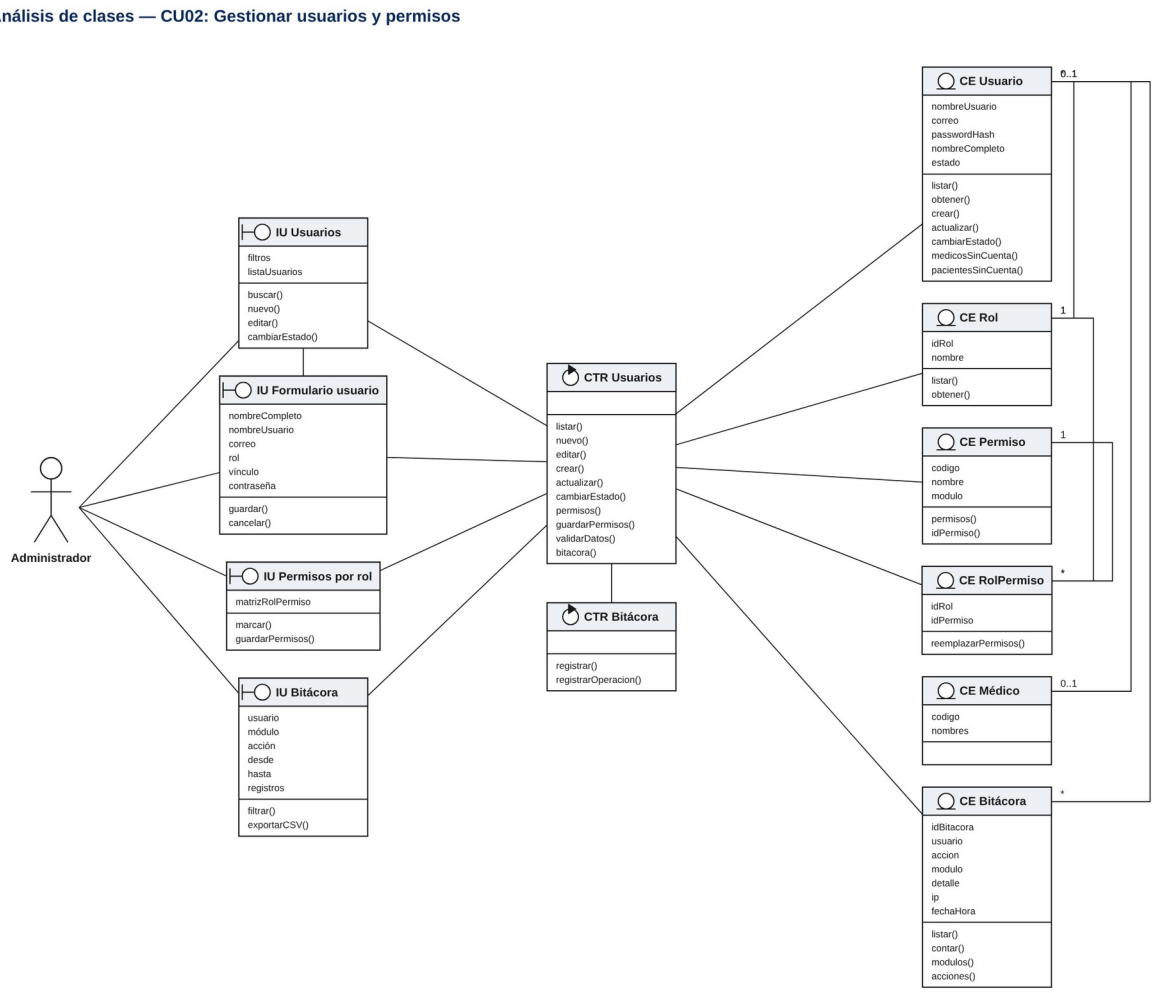

Figura 80. *Análisis de clases — CU02: Gestionar usuarios y permisos.*

Clases de análisis CU03: Gestionar pacientes

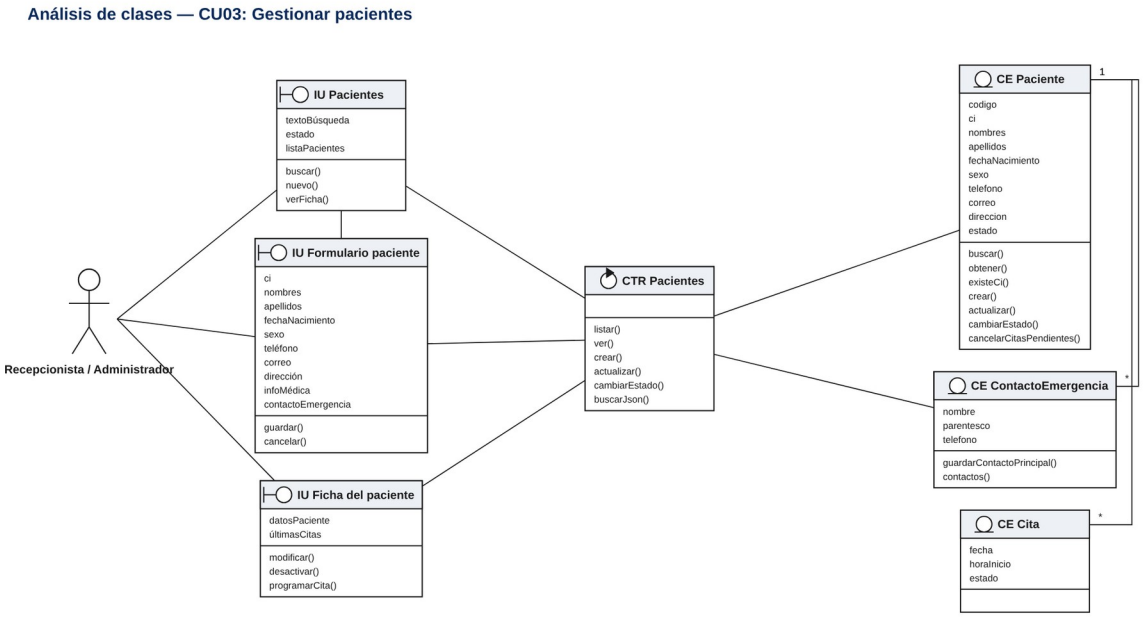

Figura 81. *Análisis de clases — CU03: Gestionar pacientes.*

Clases de análisis CU06: Gestionar especialidades

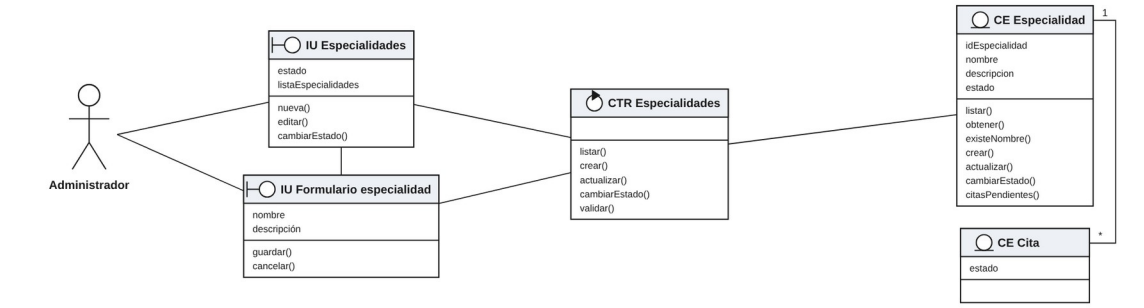

Figura 82. *Análisis de clases — CU06: Gestionar especialidades.*

Clases de análisis CU07: Gestionar médicos

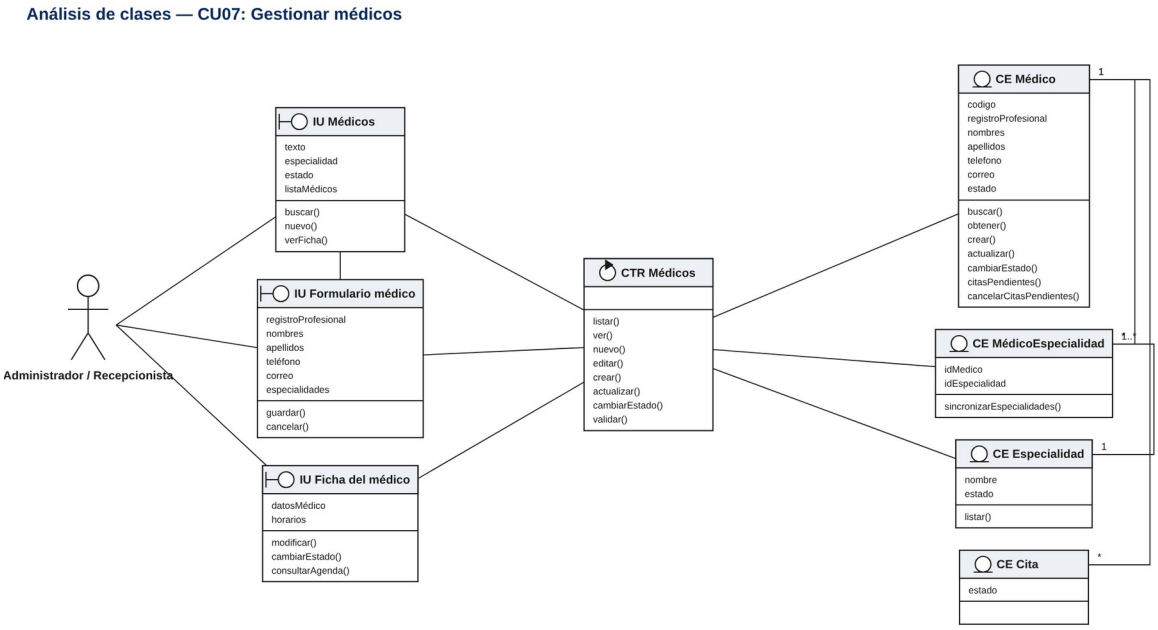

Figura 83. *Análisis de clases — CU07: Gestionar médicos.*

Clases de análisis CU08: Gestionar horarios de atención

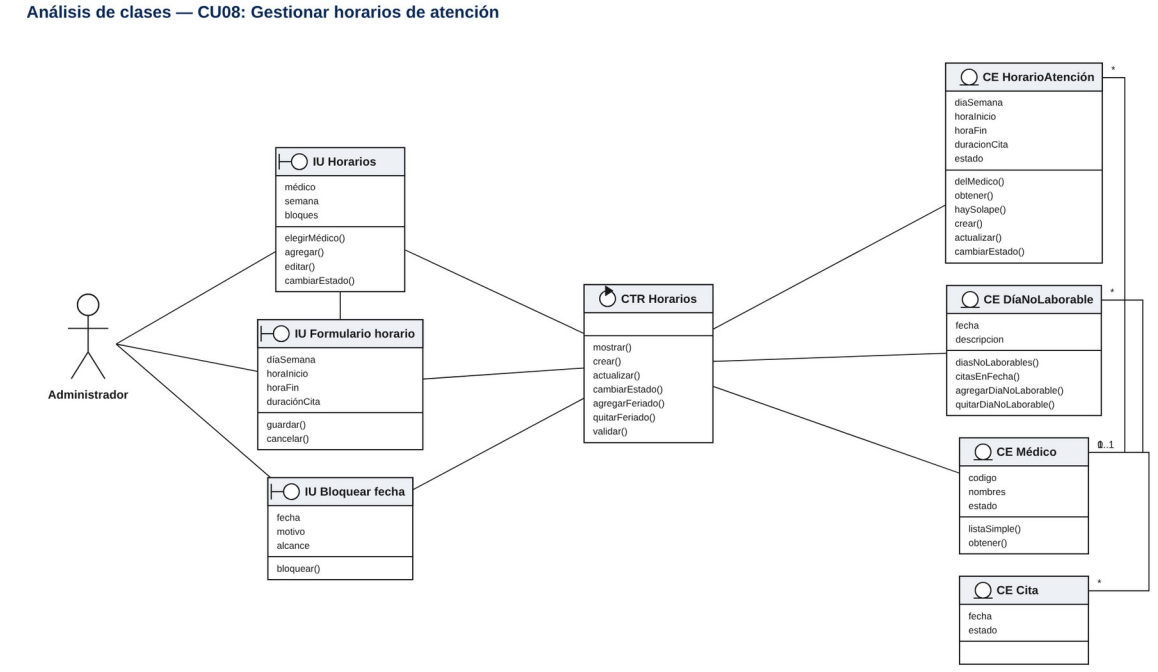

Figura 84. *Análisis de clases — CU08: Gestionar horarios de atención.*

Clases de análisis CU09: Programar cita

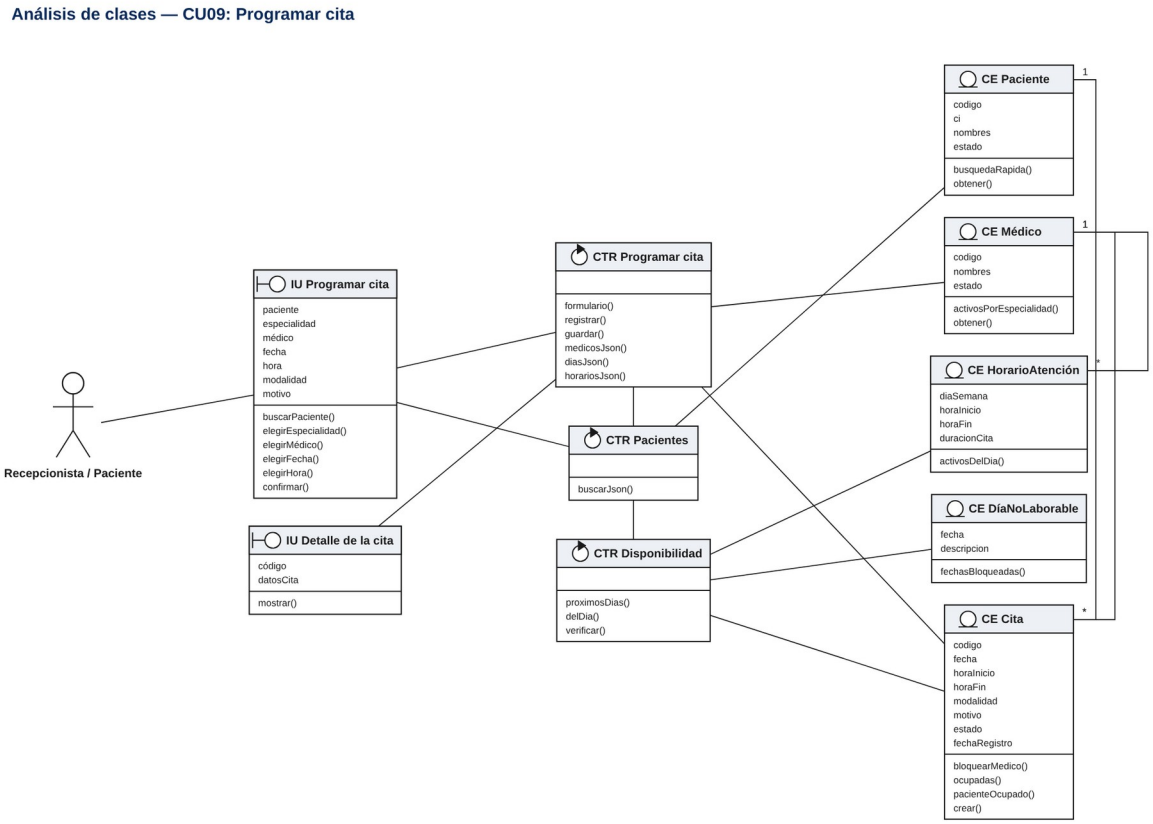

Figura 85. *Análisis de clases — CU09: Programar cita.*

Ciclo 2 — Seguimiento de citas, agenda, portal del paciente y reportes

Clases de análisis CU04: Gestionar perfil del paciente

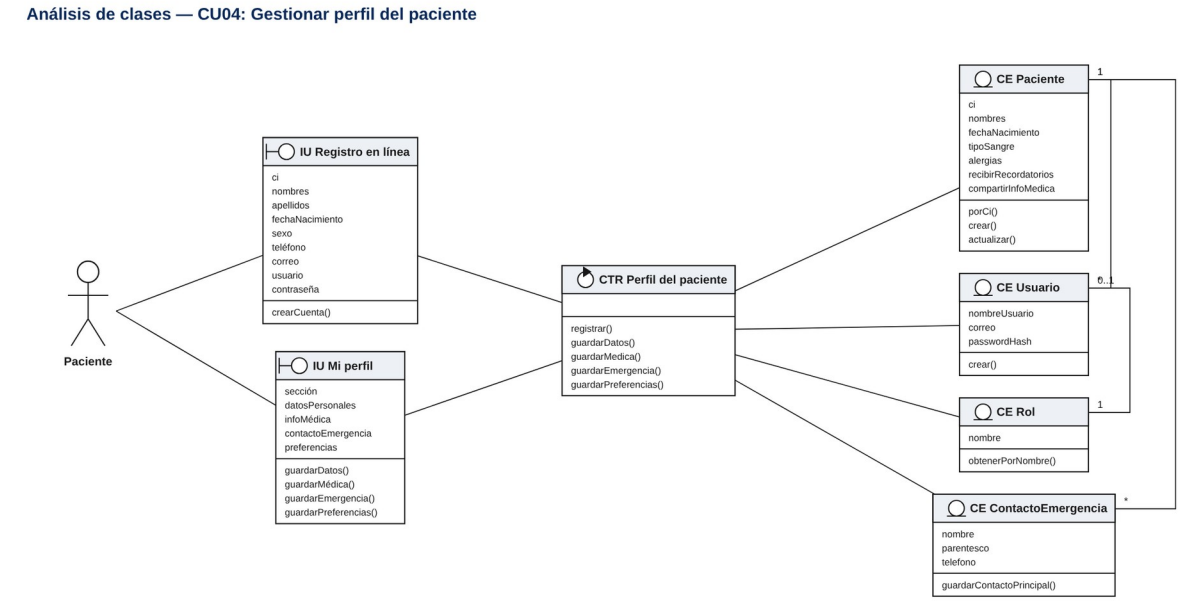

Figura 86. *Análisis de clases — CU04: Gestionar perfil del paciente.*

Clases de análisis CU05: Consultar historial de citas

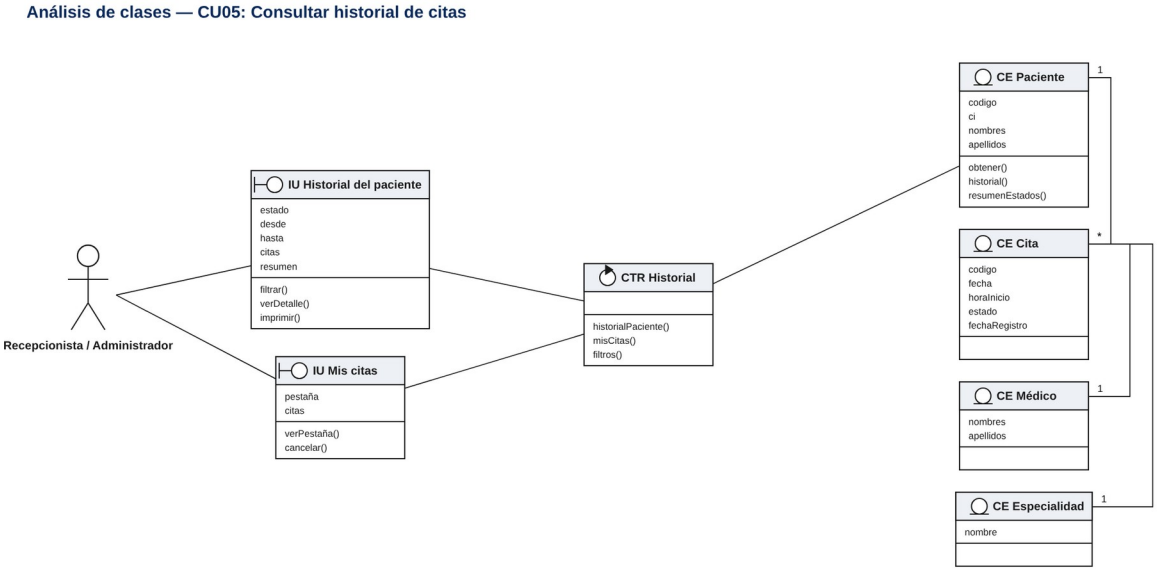

Figura 87. *Análisis de clases — CU05: Consultar historial de citas.*

Clases de análisis CU10: Reprogramar cita

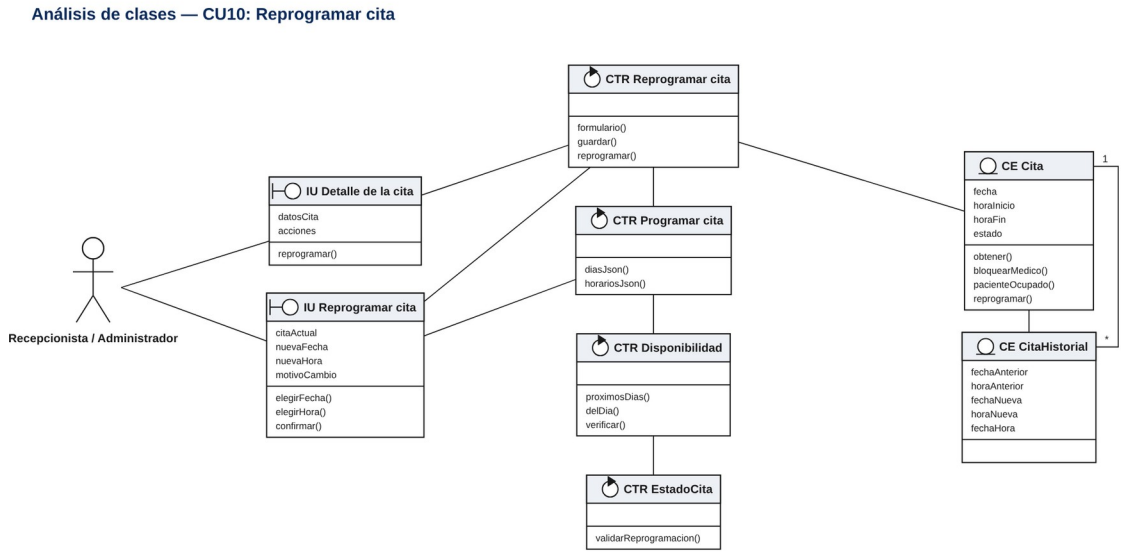

Figura 88. *Análisis de clases — CU10: Reprogramar cita.*

Clases de análisis CU11: Cancelar cita

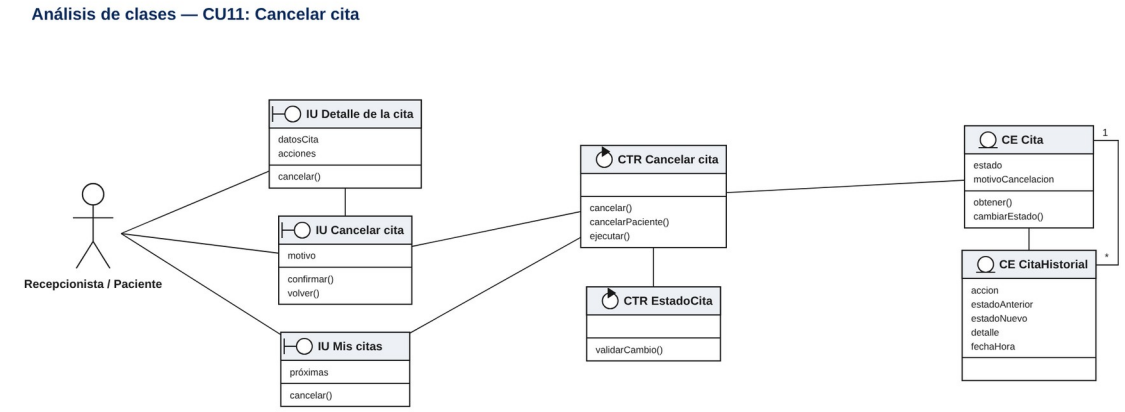

Figura 89. *Análisis de clases — CU11: Cancelar cita.*

Clases de análisis CU12: Registrar asistencia

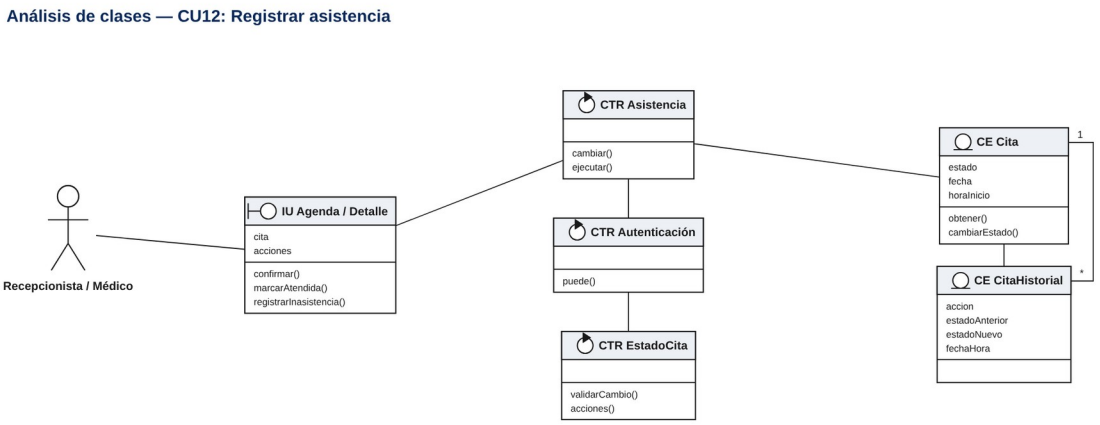

Figura 90. *Análisis de clases — CU12: Registrar asistencia.*

Clases de análisis CU13: Consultar agenda médica

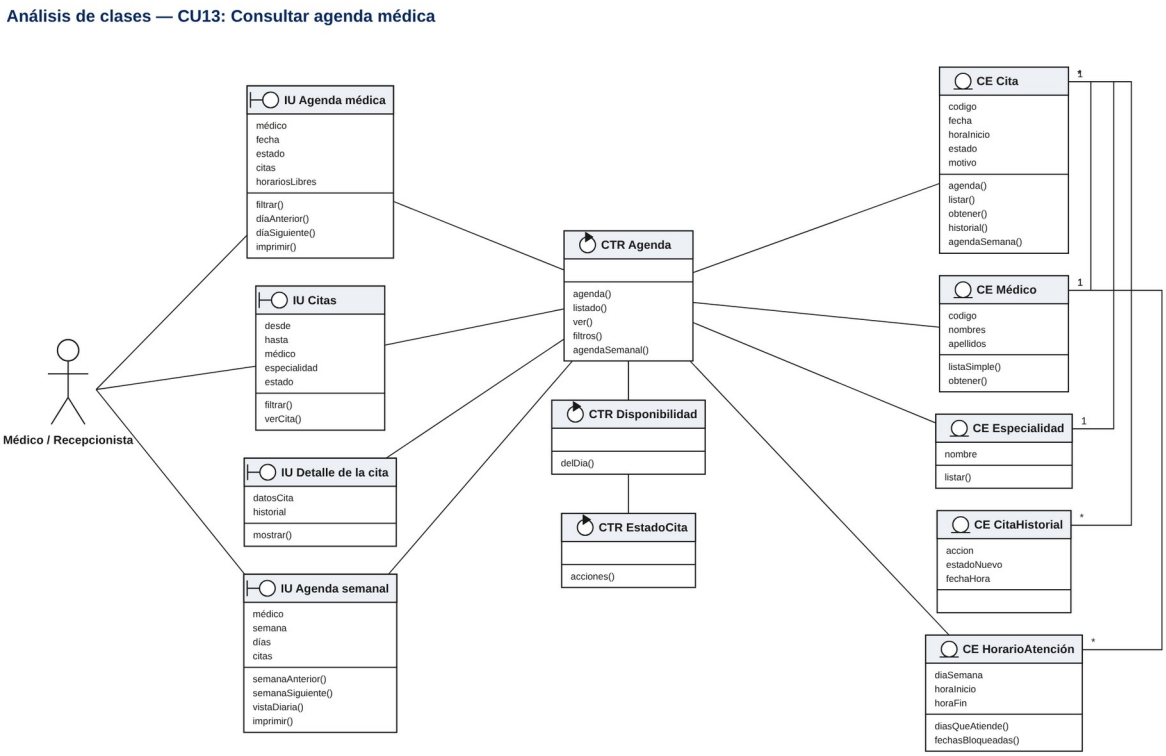

Figura 91. *Análisis de clases — CU13: Consultar agenda médica.*

Clases de análisis CU14: Generar reportes

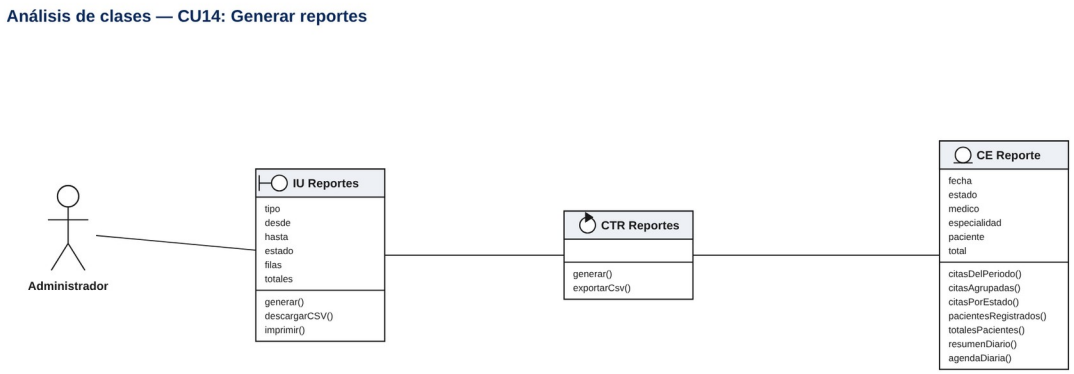

Figura 92. *Análisis de clases — CU14: Generar reportes.*

5.4 Análisis de paquetes

Tabla 14. *Paquetes de análisis*

Paquete	Responsabilidad	Casos de uso	Tablas principales
seguridad	Acceso, usuarios, roles, permisos y bitácora	CU01, CU02	usuario, rol, permiso, rol_permiso, bitacora
pacientes	Registro de pacientes, perfil e historial	CU03, CU04, CU05	paciente, contacto_emergencia
medicos	Médicos, especialidades y horarios de atención	CU06, CU07, CU08	medico, especialidad, medico_especialidad, horario_atencion, dia_no_laborable
citas	Programación, cambios, estados y agenda de citas	CU09, CU10, CU11, CU12, CU13	cita, cita_historial
reportes	Reportes administrativos	CU14	v_cita_detalle, v_resumen_citas_dia

El diagrama muestra los paquetes de análisis y sus dependencias. Una dependencia indica que un paquete usa clases entidad o servicios de otro; nunca usa sus interfaces.

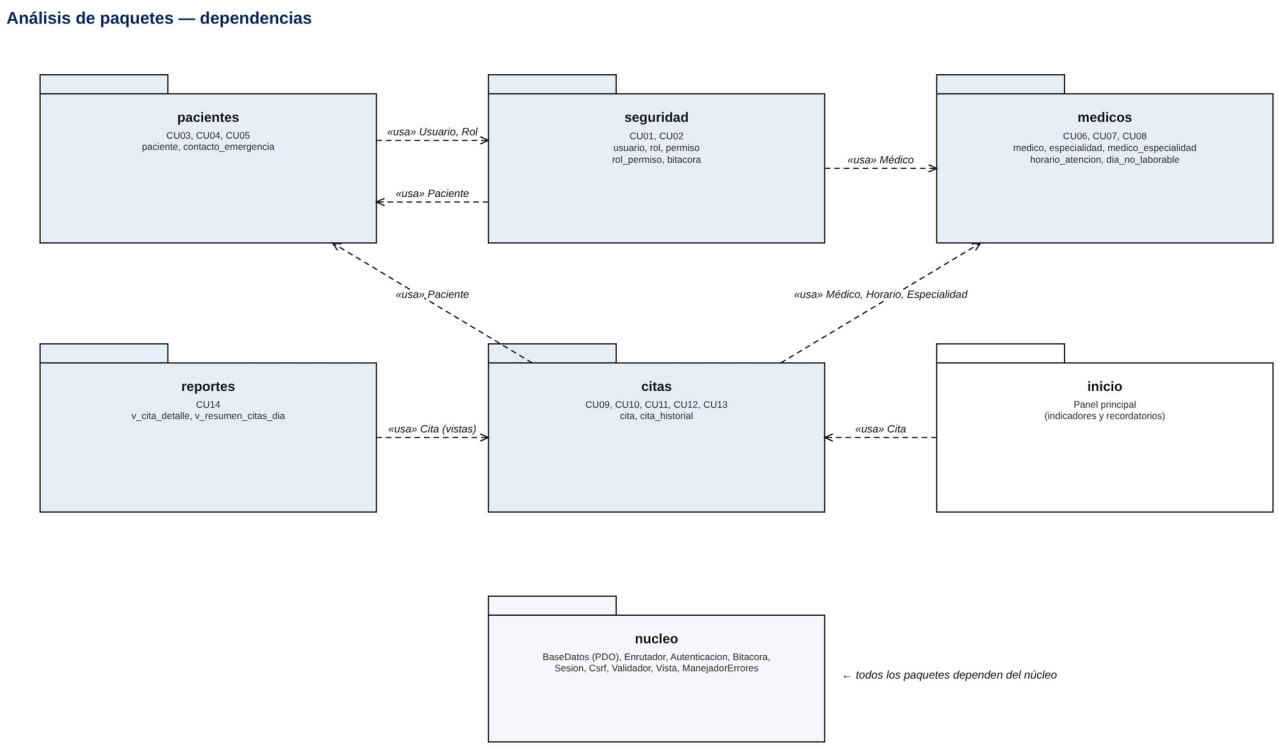

Figura 93. *Análisis de paquetes: dependencias.*

5.4.1 Relación entre paquetes: acoplamiento y cohesión

- **citas usa pacientes y medicos:** para programar una cita necesita al paciente (activo), al médico, sus especialidades, sus horarios y los días no laborables.
- **pacientes usa seguridad:** el registro en línea (CU04) crea la cuenta del paciente con el rol «paciente».
- **seguridad usa pacientes y medicos:** al crear una cuenta con rol Médico o Paciente se la vincula a un médico o paciente sin cuenta.
- **reportes usa citas:** los reportes leen las vistas de citas (v_cita_detalle, v_resumen_citas_dia).
- **Todos usan el núcleo:** conexión PDO, enrutador, autenticación, sesión, CSRF, validación, vistas y manejo de errores.

Cohesión alta: cada paquete agrupa casos de uso, tablas y reglas de un mismo tema (por ejemplo, todo el ciclo de vida de la cita está en *citas*). **Acoplamiento bajo:** los paquetes se comunican solo mediante clases modelo y servicios; no existen dependencias circulares entre interfaces, y el paquete *reportes* solo lee datos.

5.5 Reglas de negocio por paquete

Paquete seguridad

- Solo el administrador gestiona usuarios y permisos.
- Los 4 roles son fijos; el administrador siempre conserva «usuarios.gestionar».
- Un usuario inactivo, bloqueado o vinculado a un médico/paciente inactivo no inicia sesión.
- No puede quedar el sistema sin administradores activos.

- Toda acción (inicio y cierre de sesión, intento fallido, operación POST) queda en la bitácora, que no se puede modificar ni borrar.

Paquete pacientes

- El CI es único cuando se proporciona.
- No se eliminan pacientes: se desactivan y sus citas pendientes se cancelan.
- El paciente en línea solo ve y edita su propio perfil; el CI y la fecha de nacimiento los corrige recepción.

Paquete medicos

- No se repiten especialidades con el mismo nombre normalizado.
- Un médico inactivo no recibe citas; su historial se conserva.
- Los bloques horarios no se superponen; la hora final es posterior a la inicial.
- No se desactiva un bloque o se bloquea una fecha con citas pendientes.

Paquete citas

- Solo se ofrecen horarios libres, dentro del horario de atención y no pasados.
- Al guardar se vuelve a verificar la disponibilidad dentro de una transacción.
- Transiciones válidas: programada → confirmada/atendida/cancelada/inasistencia; confirmada → atendida/cancelada/inasistencia.
- Atendida, cancelada e inasistencia son finales; no se reprograman ni se borran.
- El médico solo marca atendidas sus citas y si tiene el permiso.

Paquete reportes

- Solo el administrador genera reportes.
- Los reportes no modifican datos; filtran por fecha o rango y muestran totales.

6. FLUJO DE TRABAJO: DISEÑO

El diseño transforma el modelo de análisis en una solución concreta para la tecnología elegida: PHP 8 con PDO, PostgreSQL, HTML5/CSS3/JavaScript y Apache (XAMPP). Se define la arquitectura física y lógica, el diseño de cada caso de uso mediante diagramas de secuencia con las clases reales del código, y el diseño lógico y físico de la base de datos.

6.1 Diseño de arquitectura (física y lógica)

6.1.1 Arquitectura física

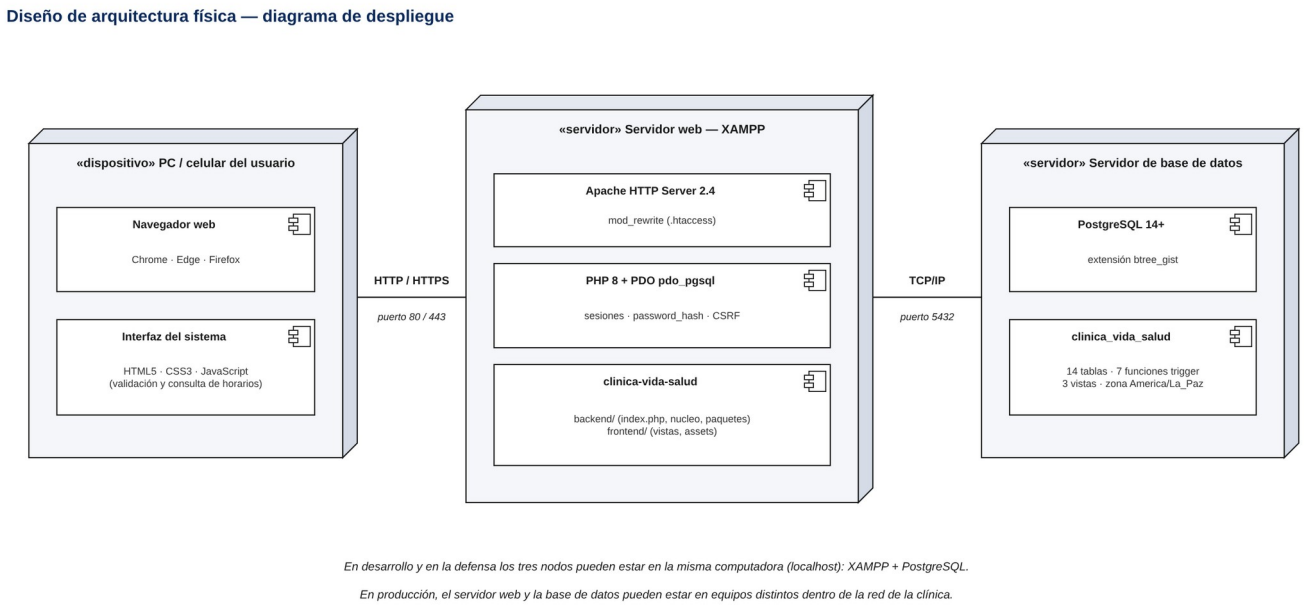

Figura 94. *Diagrama de despliegue: arquitectura física del sistema.*

Tabla 15. *Nodos de la arquitectura física*

Nodo	Software	Función
PC / celular del usuario	Navegador web moderno	Muestra la interfaz HTML5/CSS3 y ejecuta JavaScript.
Servidor web	XAMPP: Apache 2.4, PHP 8 con pdo_pgsql	Recibe las peticiones HTTP, ejecuta los casos de uso y genera las páginas.
Servidor de base de datos	PostgreSQL 14 o superior (btree_gist)	Almacena los datos y garantiza su integridad.

6.1.2 Arquitectura lógica

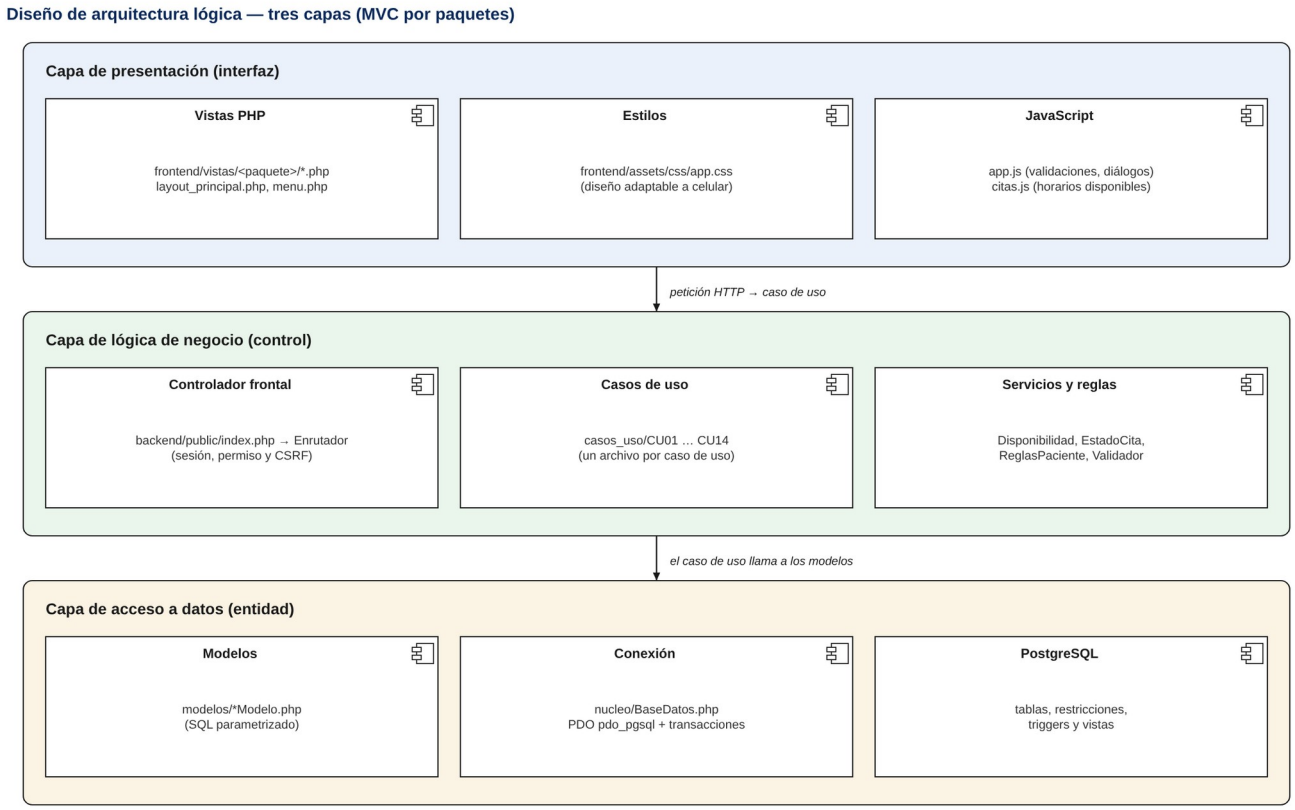

Figura 95. *Arquitectura lógica en tres capas.*

Tabla 16. *Capas de la arquitectura lógica*

Capa	Componentes	Responsabilidad
Presentación	frontend/vistas/<paquete>/*.php, layout, app.css, app.js, citas.js	Mostrar la información y capturar datos; validación inmediata en el navegador.
Lógica de negocio	backend/public/index.php, Enrutador, casos_uso/CU01–CU14, Disponibilidad, EstadoCita, ReglasPaciente, Validador, Autenticacion	Controlar el acceso, ejecutar cada caso de uso y aplicar las reglas de negocio en el servidor.
Acceso a datos	modelos/*Modelo.php, BaseDatos (PDO pdo_pgsql), PostgreSQL	Consultas parametrizadas, transacciones e integridad de los datos.

Flujo de una petición: el navegador pide una URL → Apache la envía a `backend/public/index.php` → el `Enrutador` busca la ruta, comprueba la sesión, el permiso y el token CSRF → ejecuta el método del caso de uso (`CUxx`) → el caso de uso valida, aplica las reglas y llama a los modelos → los modelos ejecutan SQL parametrizado con PDO → el caso de uso muestra una vista o redirige con un mensaje.

El siguiente diagrama de clases (elaborado en StarUML) muestra las clases del núcleo con sus atributos y métodos reales y sus dependencias principales. Los métodos subrayados son estáticos; Controlador es abstracta y de ella heredan todas las clases de caso de uso.

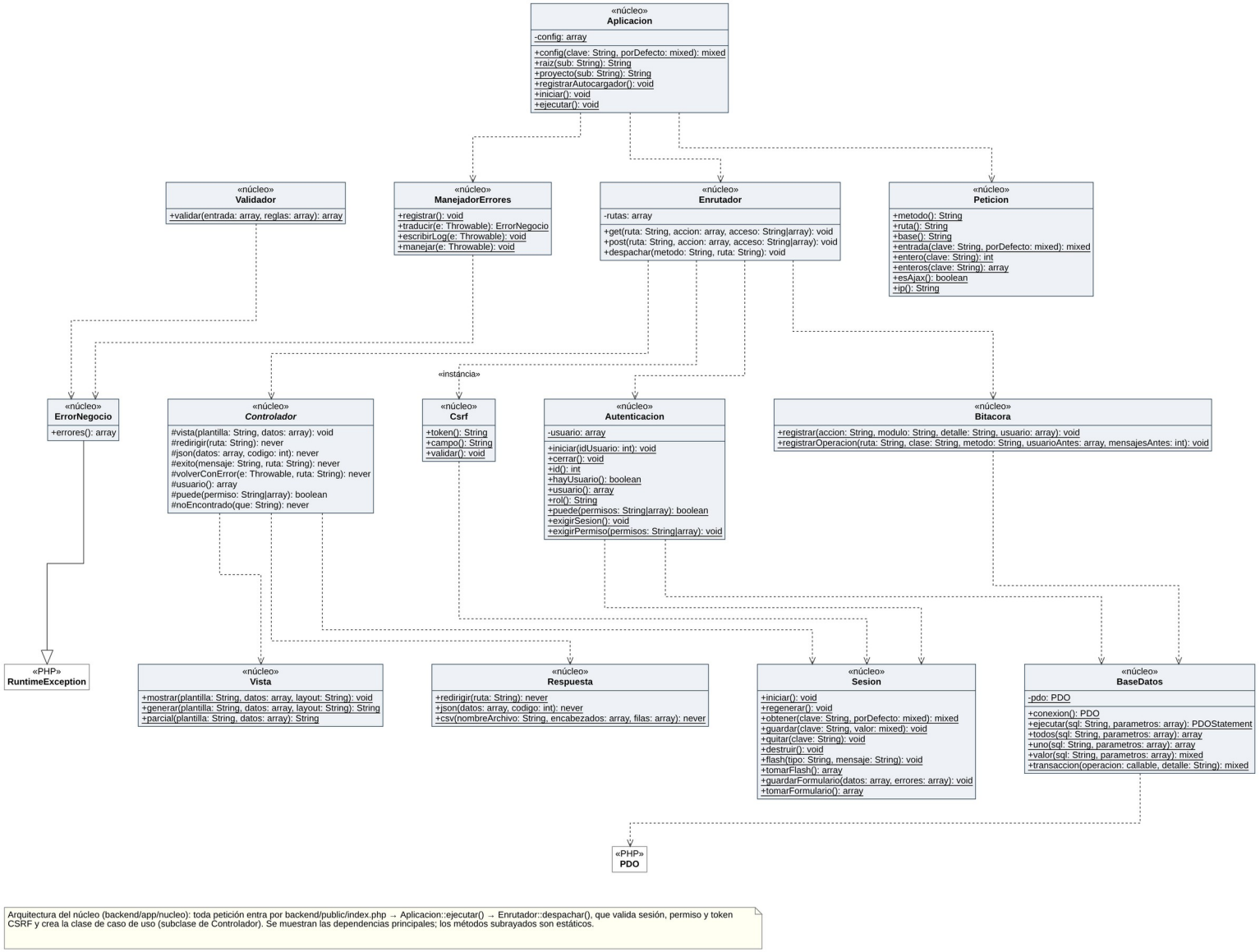

Figura 96. *Diagrama de clases de diseño — arquitectura del núcleo (backend/app/nucleo).*

6.2 Diseñar casos de uso (diagramas de clases de diseño y de secuencia)

Para cada caso de uso se presenta primero su **diagrama de clases de diseño** (StarUML): la clase «control» del caso de uso con sus atributos y métodos públicos, la clase Controlador de la que hereda y las clases «servicio» y «modelo» que usa, tal como están en el código. Las flechas punteadas son dependencias (la clase crea o llama a la otra) y la flecha con triángulo es herencia.

Cada diagrama de secuencia muestra la interacción en el tiempo entre el actor, la vista, la clase del caso de uso, los servicios, los modelos y PostgreSQL, usando los nombres reales de los archivos. Los fragmentos **alt** muestran caminos alternativos (por ejemplo, horario ocupado o datos válidos), **opt** pasos opcionales y **loop** repeticiones.

Ciclo 1 — Seguridad, registros base y programación de citas

CU01: Iniciar sesión

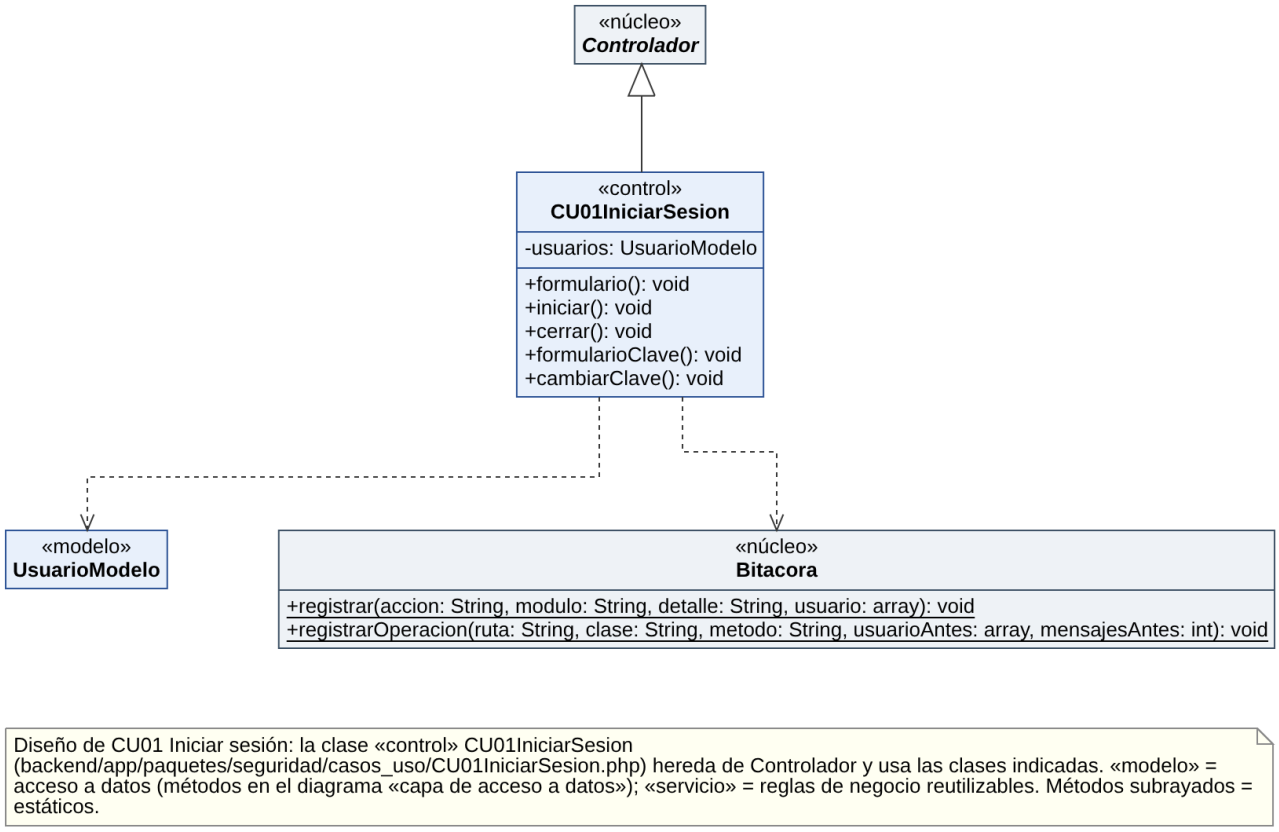

Figura 97. *Diagrama de clases de diseño — CU01: Iniciar sesión.*

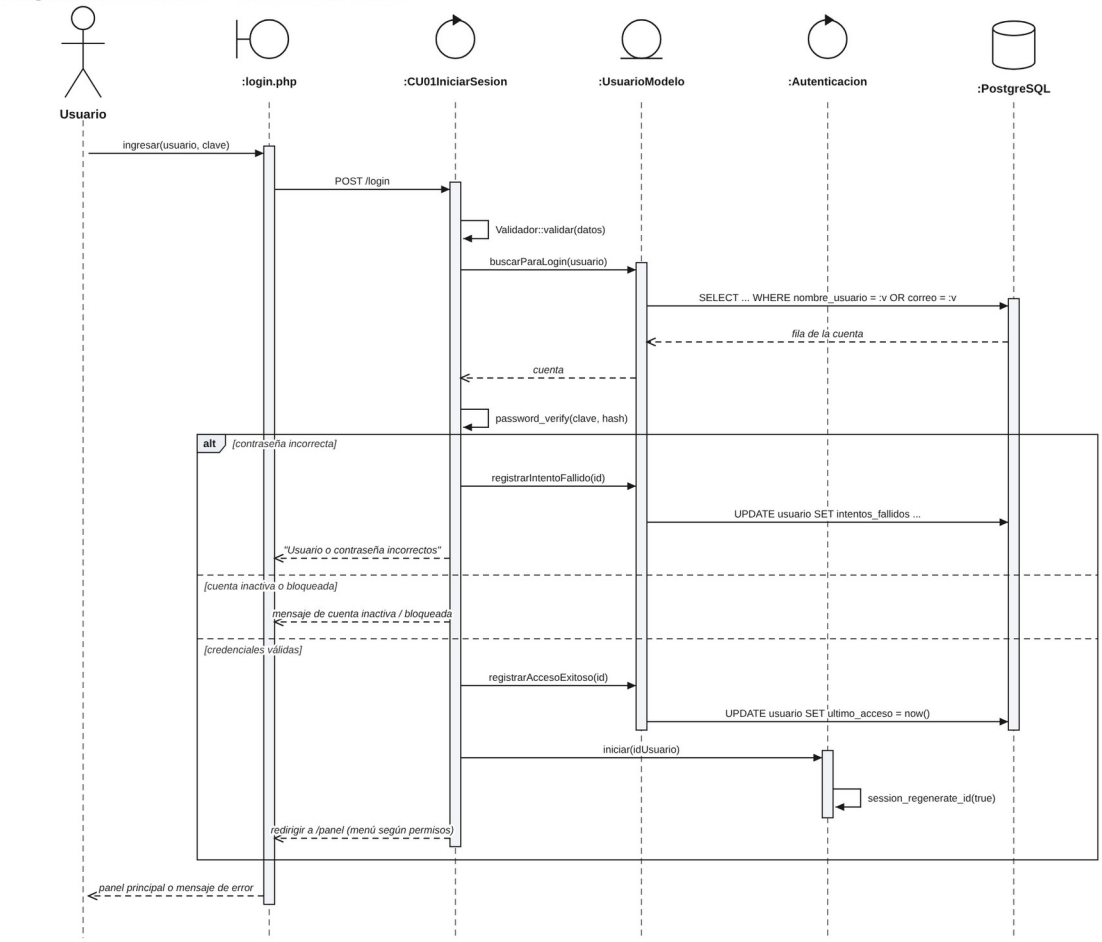

Figura 98. *Diagrama de secuencia — CU01: Iniciar sesión.*

CU02: Gestionar usuarios y permisos

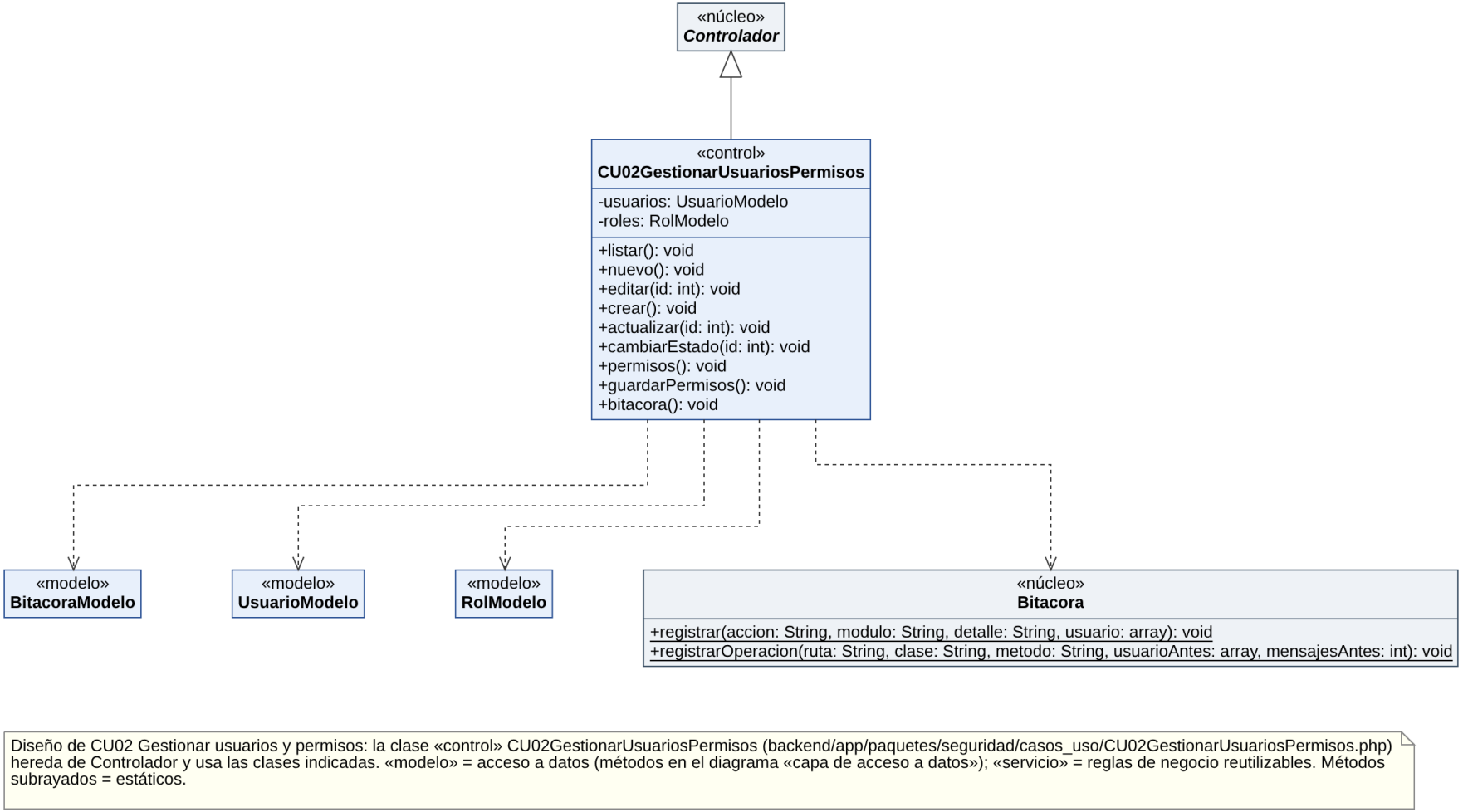

Figura 99. *Diagrama de clases de diseño — CU02: Gestionar usuarios y permisos.*

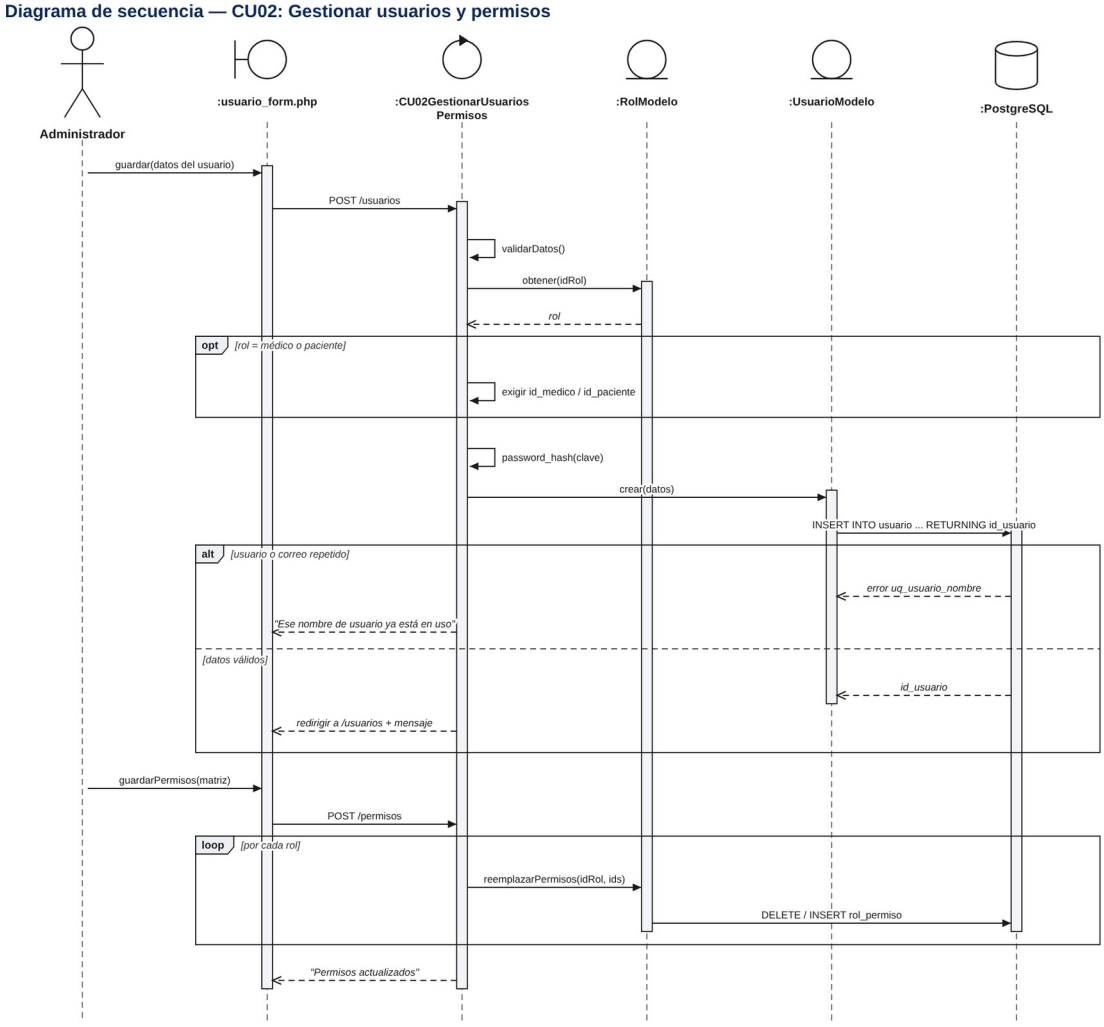

Figura 100. *Diagrama de secuencia — CU02: Gestionar usuarios y permisos.*

CU03: Gestionar pacientes

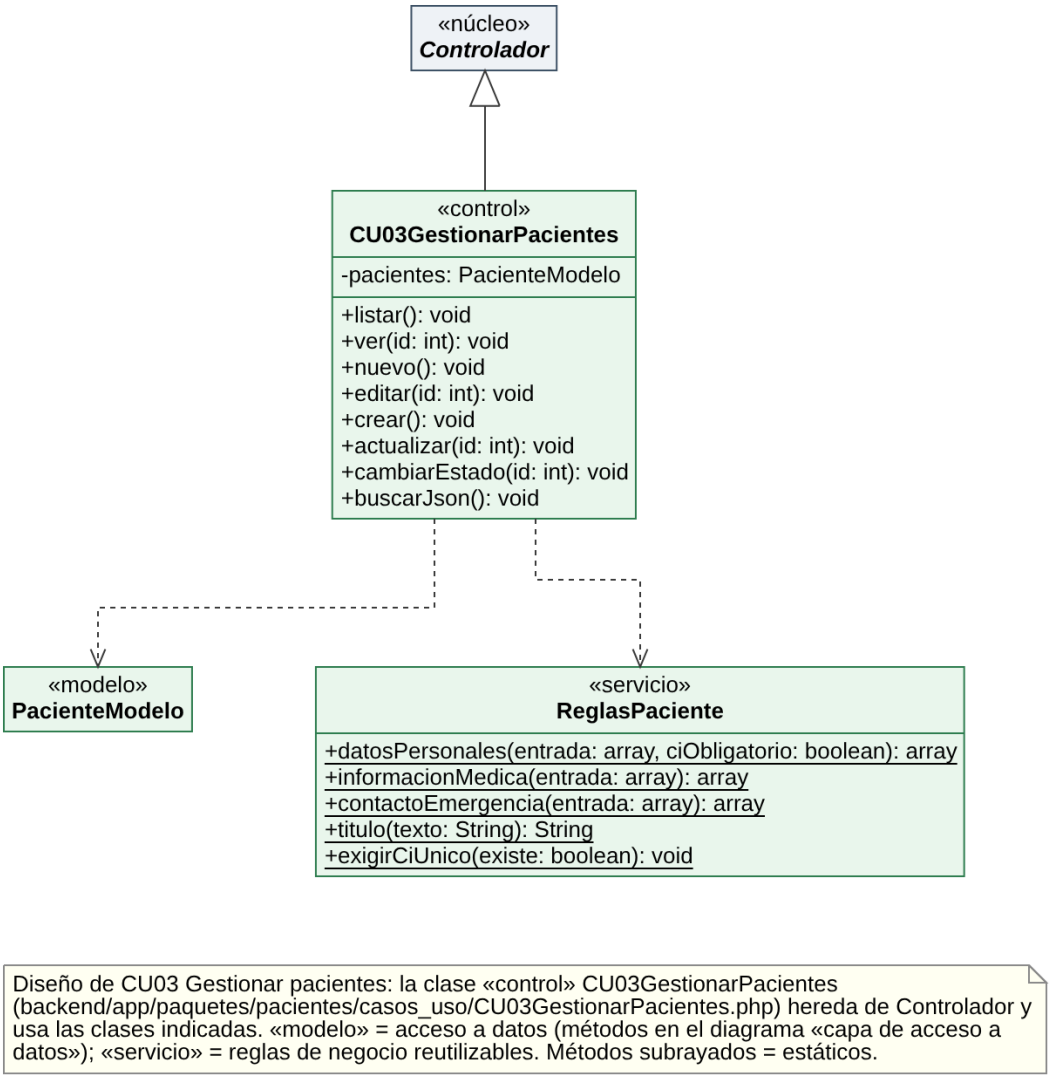

Figura 101. *Diagrama de clases de diseño — CU03: Gestionar pacientes.*

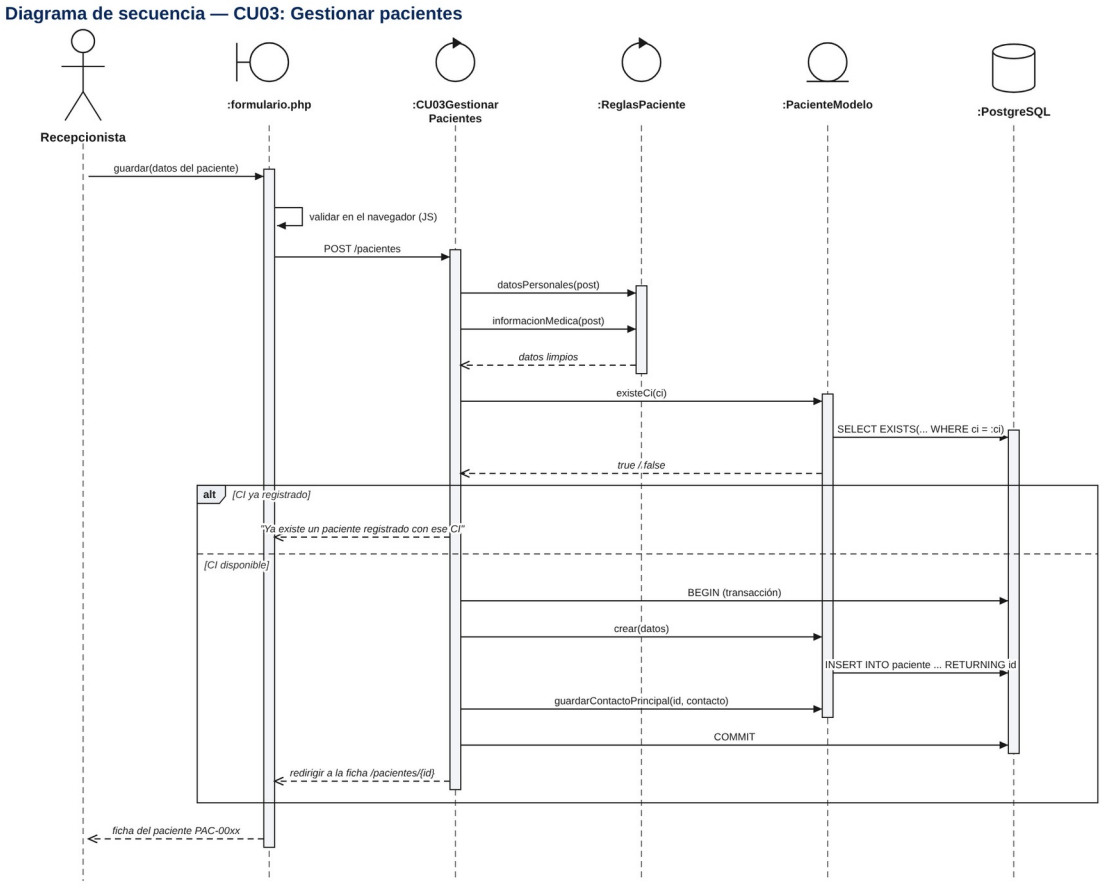

Figura 102. *Diagrama de secuencia — CU03: Gestionar pacientes.*

CU06: Gestionar especialidades

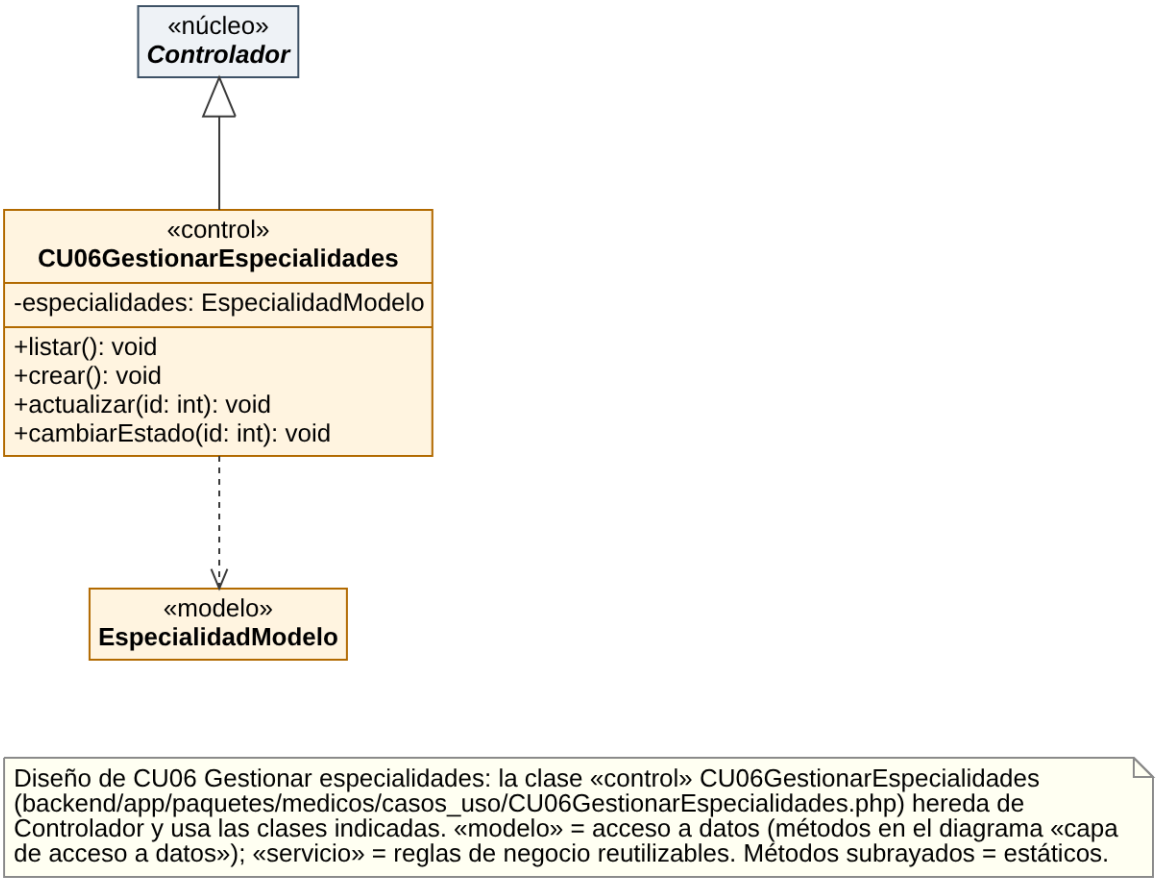

Figura 103. *Diagrama de clases de diseño — CU06: Gestionar especialidades.*

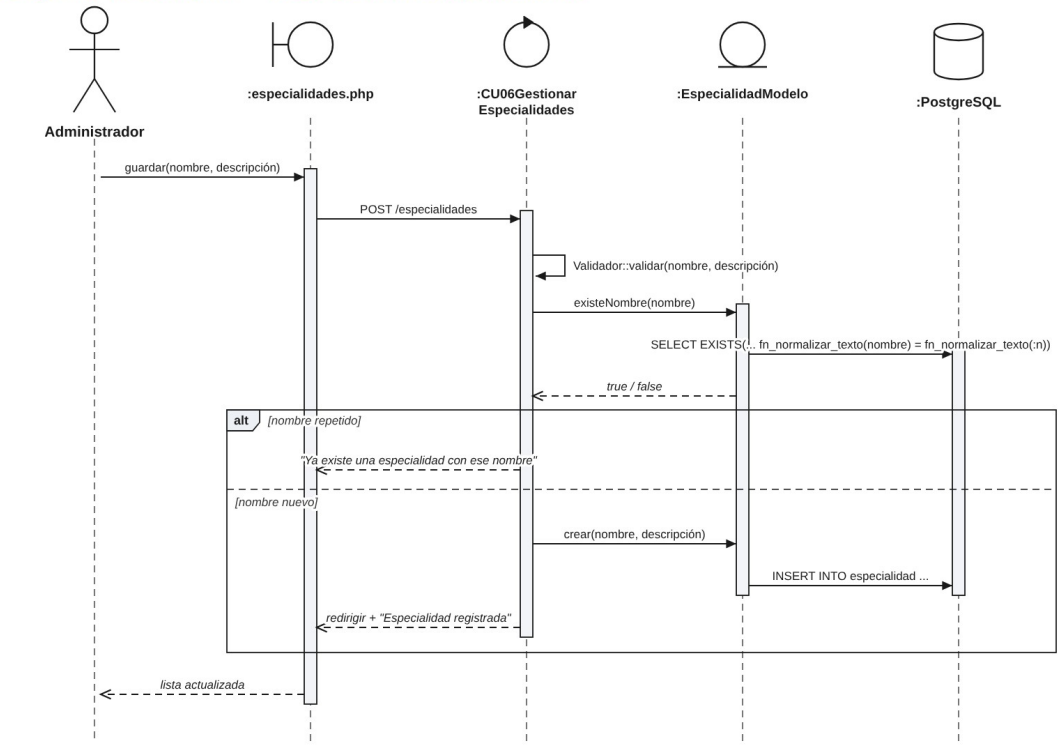

Figura 104. *Diagrama de secuencia — CU06: Gestionar especialidades.*

CU07: Gestionar médicos

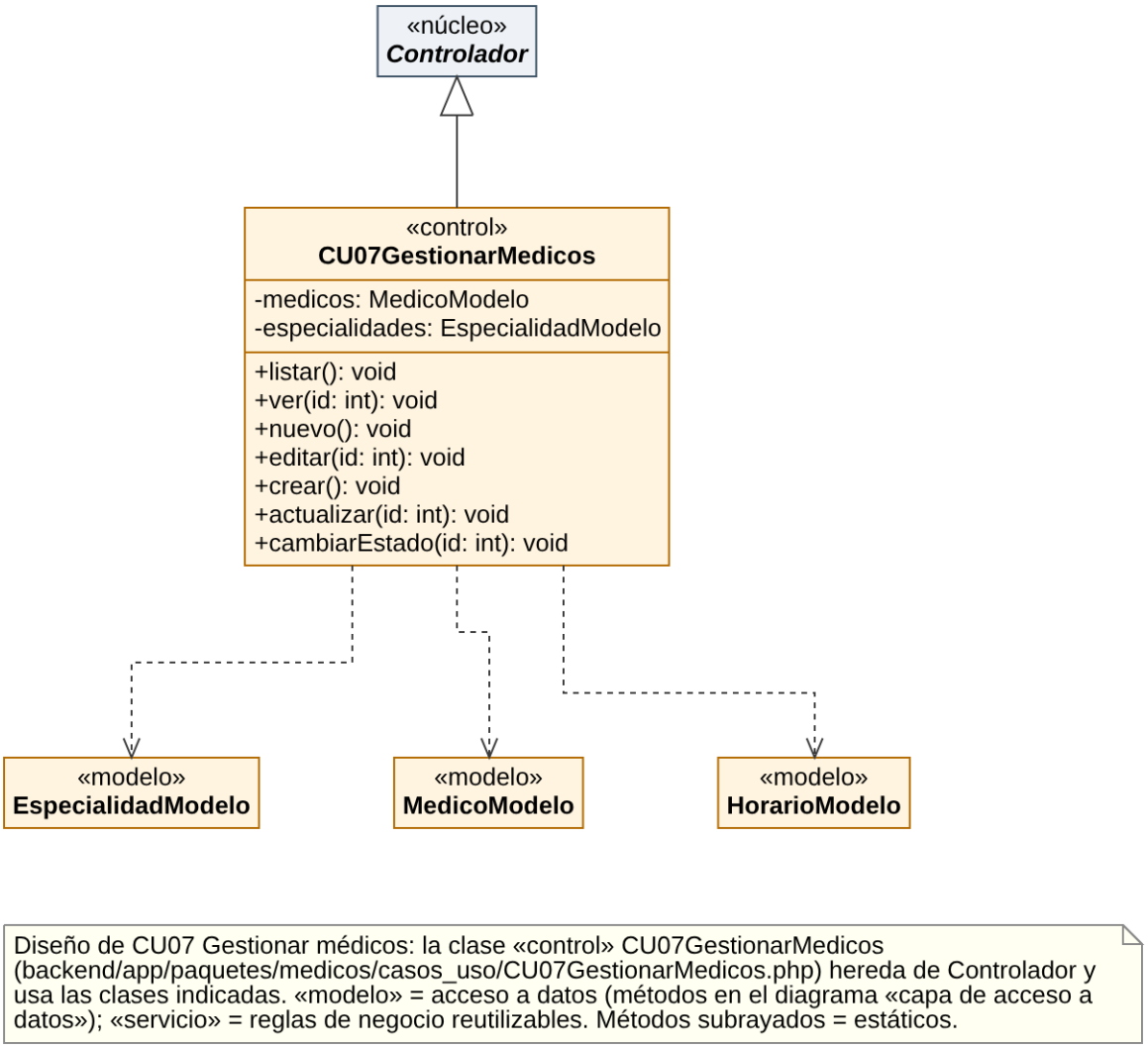

Figura 105. *Diagrama de clases de diseño — CU07: Gestionar médicos.*

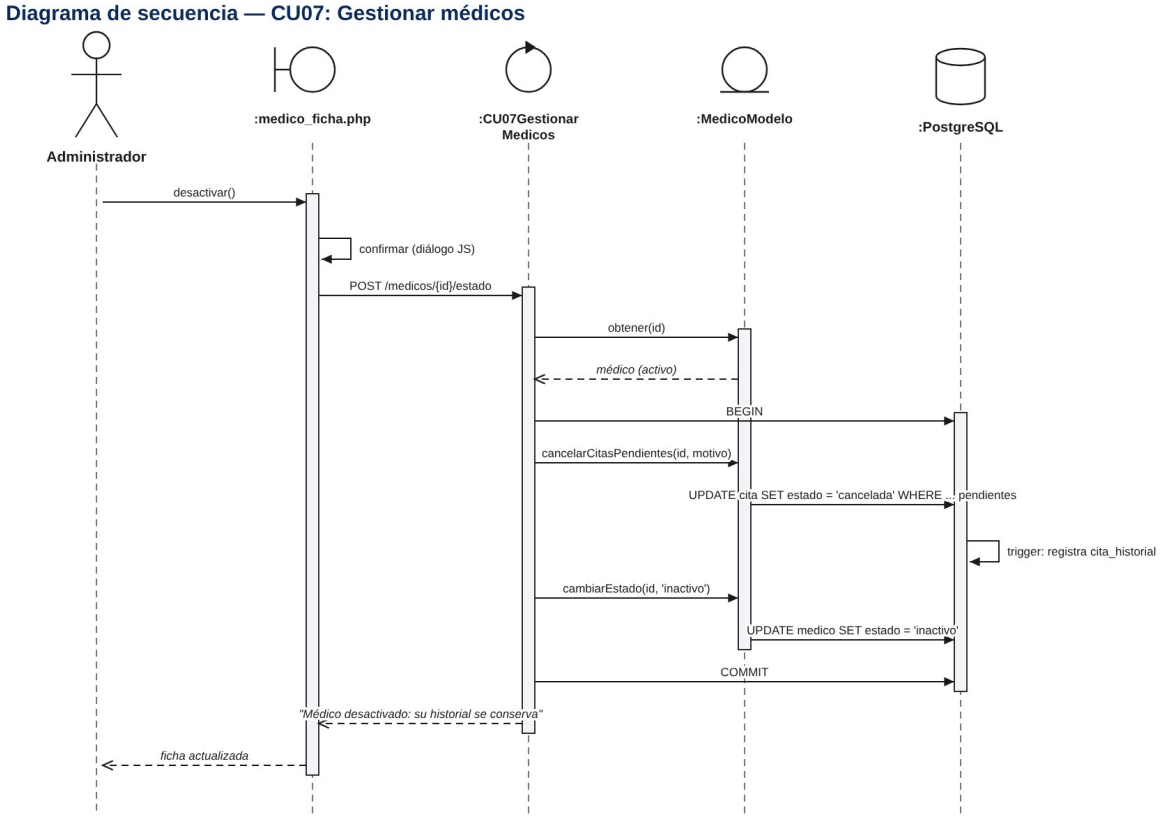

Figura 106. *Diagrama de secuencia — CU07: Gestionar médicos.*

CU08: Gestionar horarios de atención

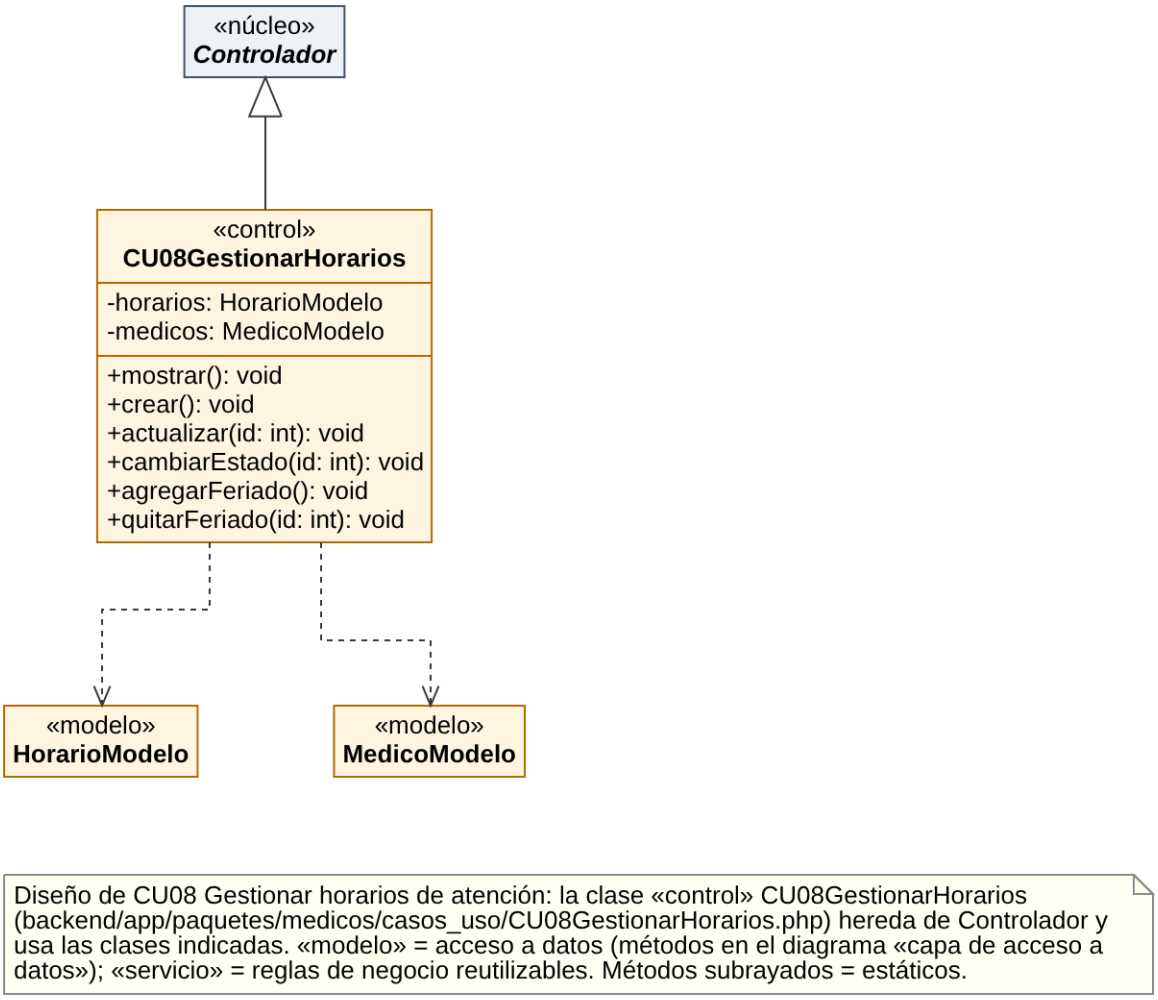

Figura 107. *Diagrama de clases de diseño — CU08: Gestionar horarios de atención.*

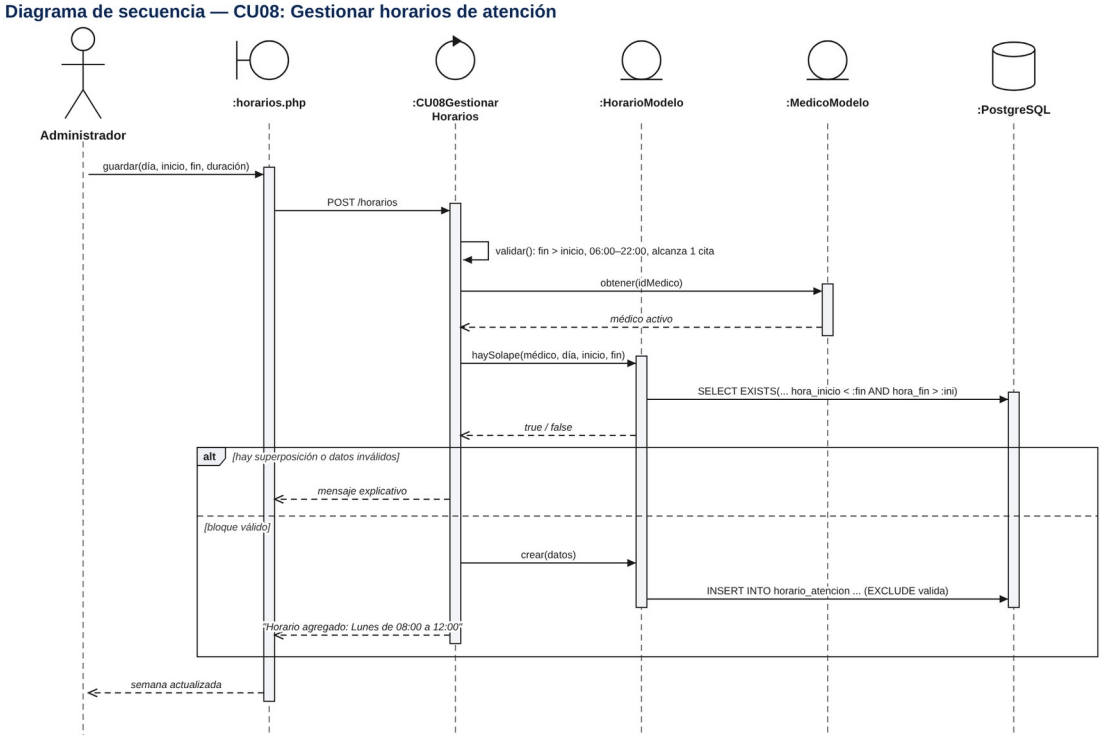

Figura 108. *Diagrama de secuencia — CU08: Gestionar horarios de atención.*

CU09: Programar cita

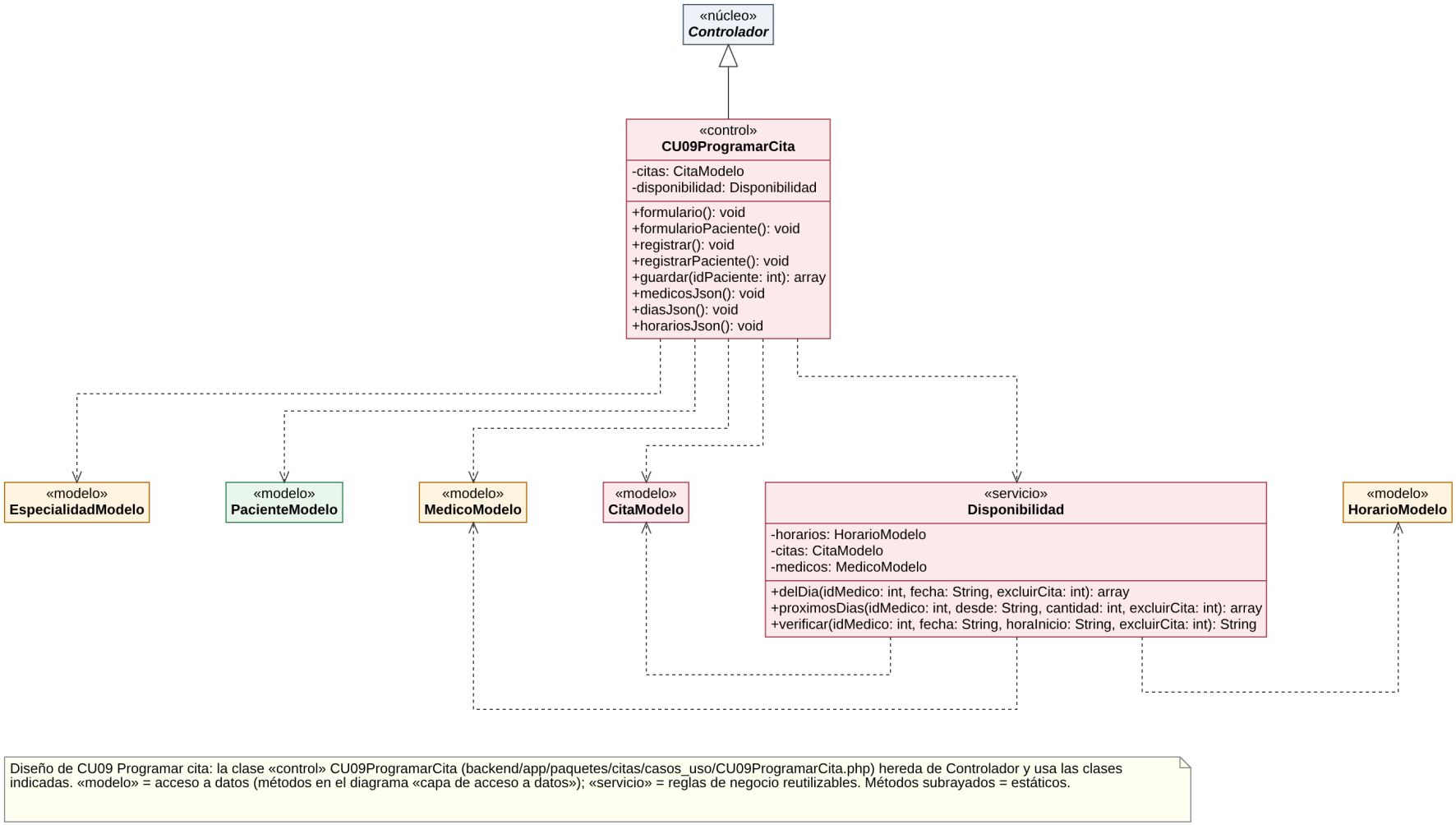

Figura 109. *Diagrama de clases de diseño — CU09: Programar cita.*

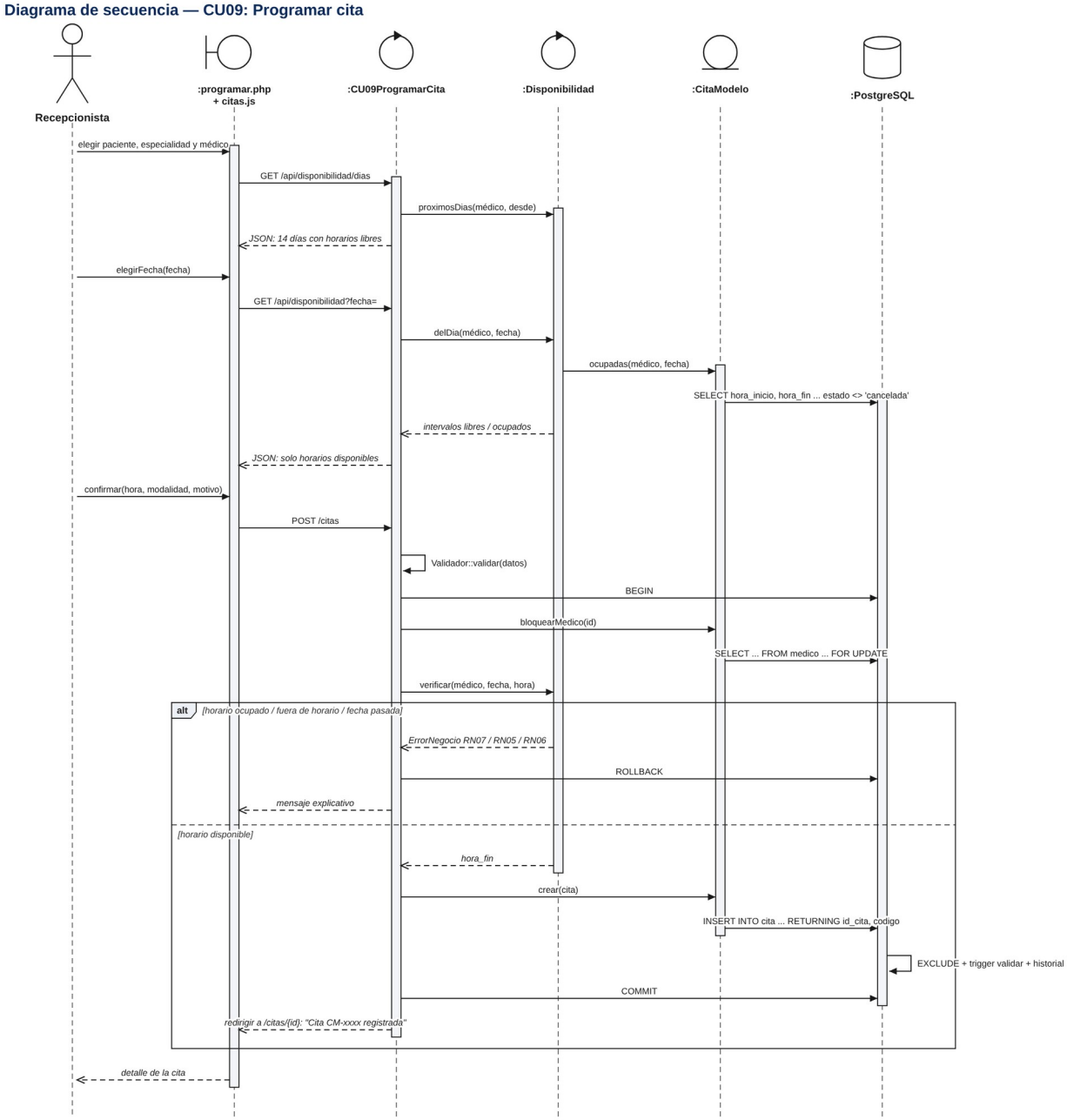

Figura 110. *Diagrama de secuencia — CU09: Programar cita.*

Ciclo 2 — Seguimiento de citas, agenda, portal del paciente y reportes

CU04: Gestionar perfil del paciente

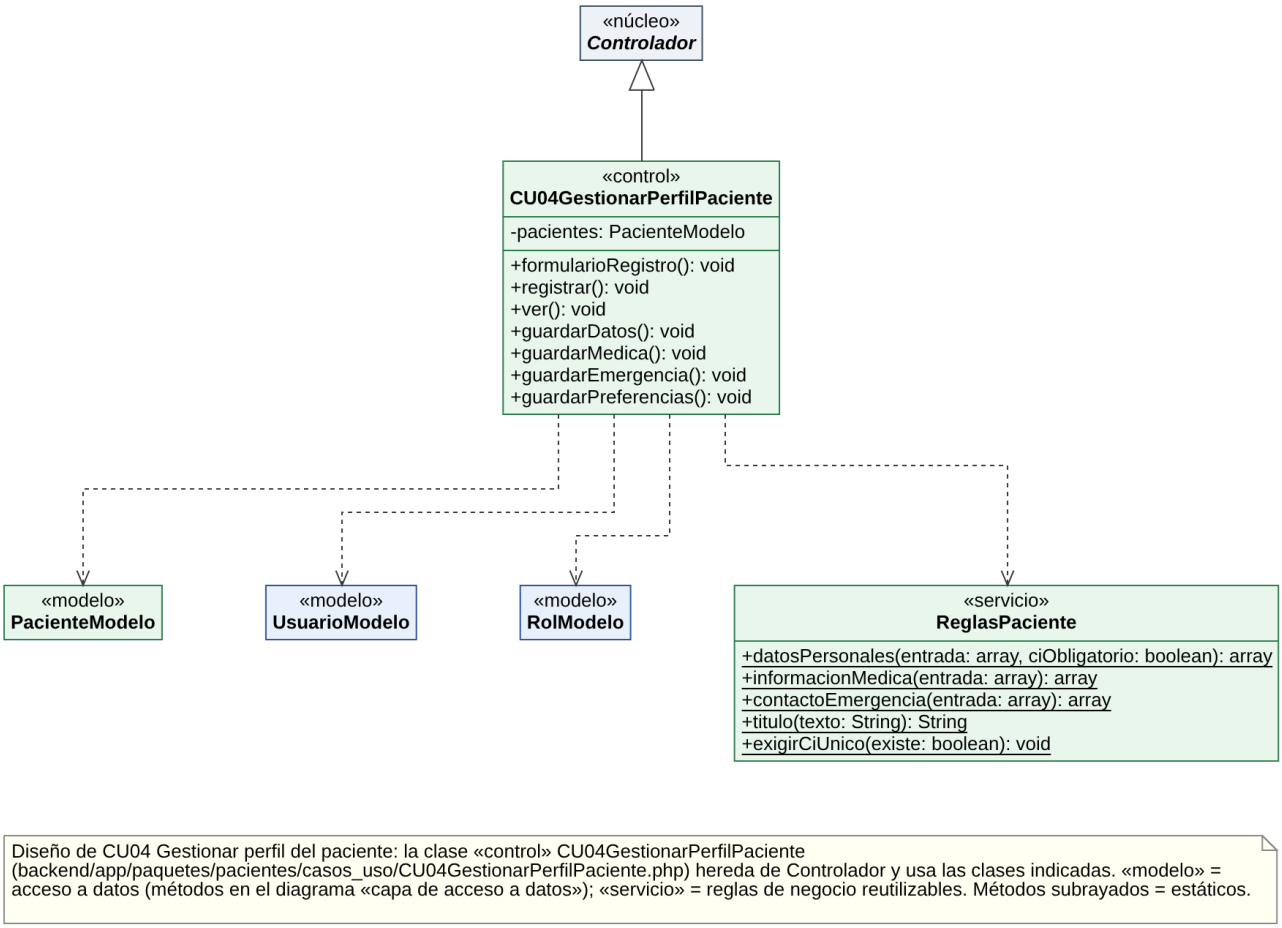

Figura 111. *Diagrama de clases de diseño — CU04: Gestionar perfil del paciente.*

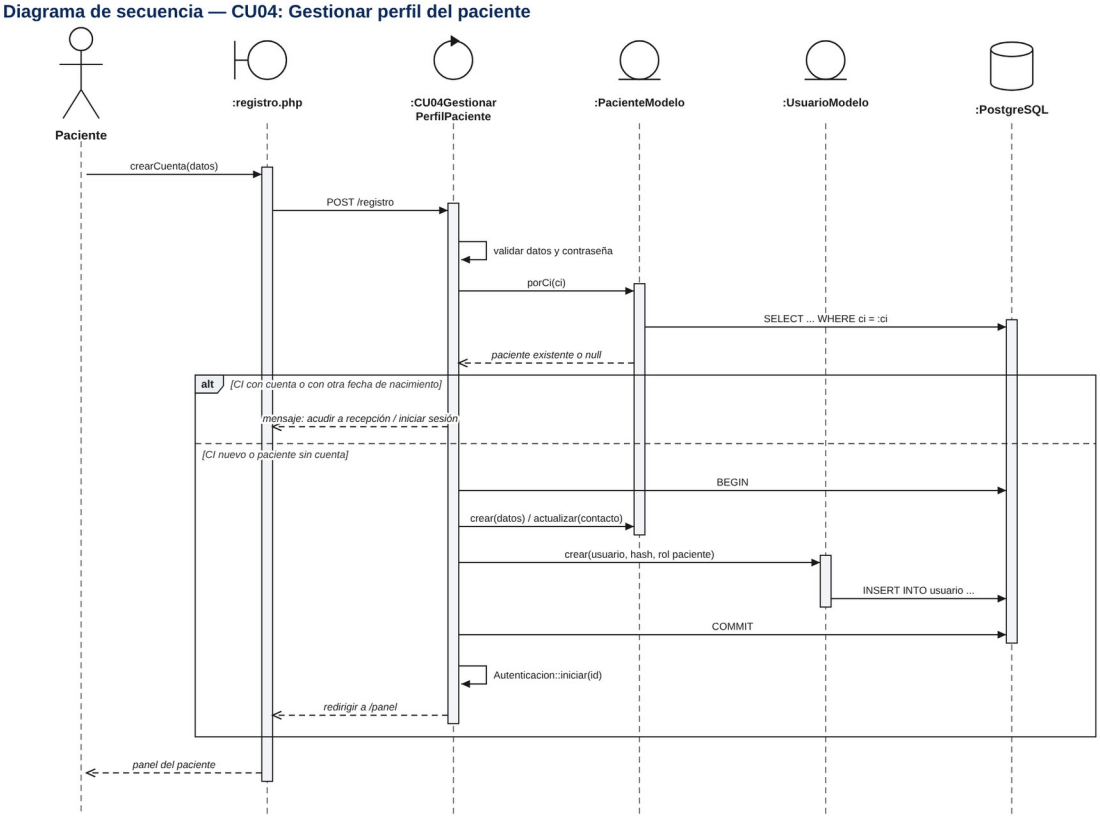

Figura 112. *Diagrama de secuencia — CU04: Gestionar perfil del paciente.*

CU05: Consultar historial de citas

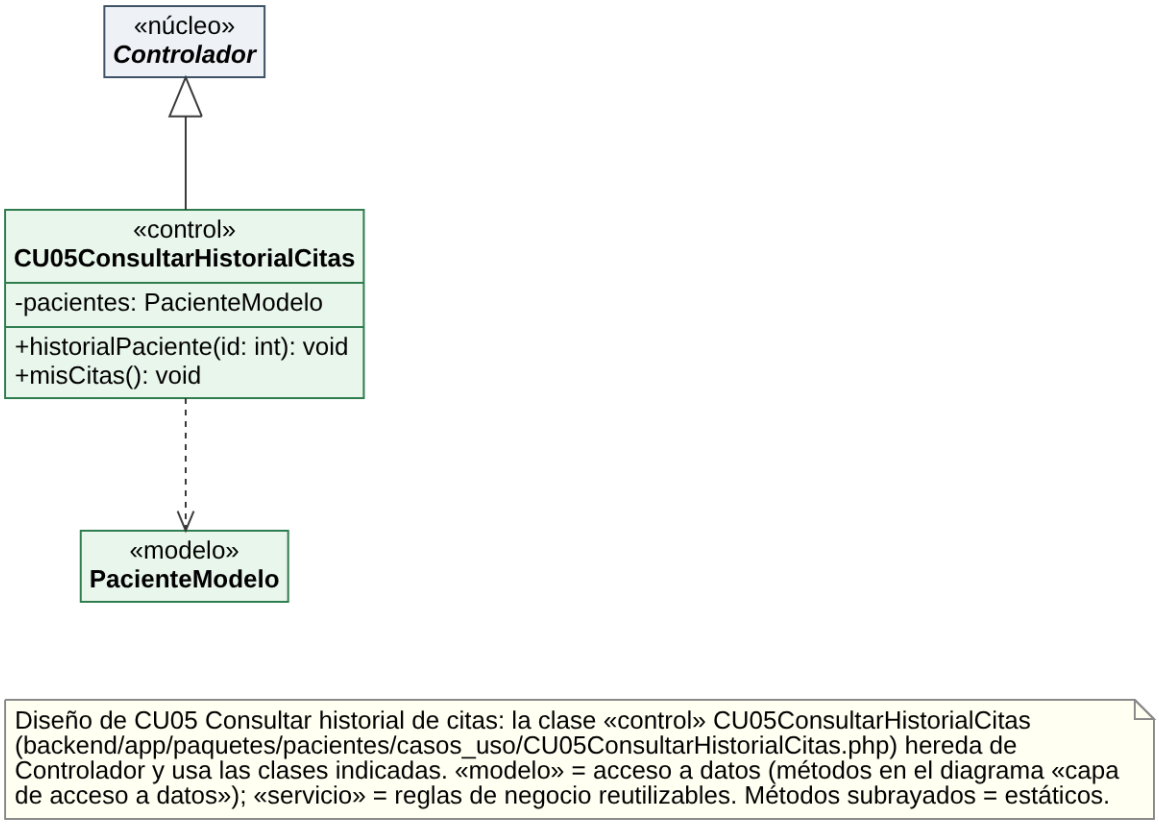

Figura 113. *Diagrama de clases de diseño — CU05: Consultar historial de citas.*

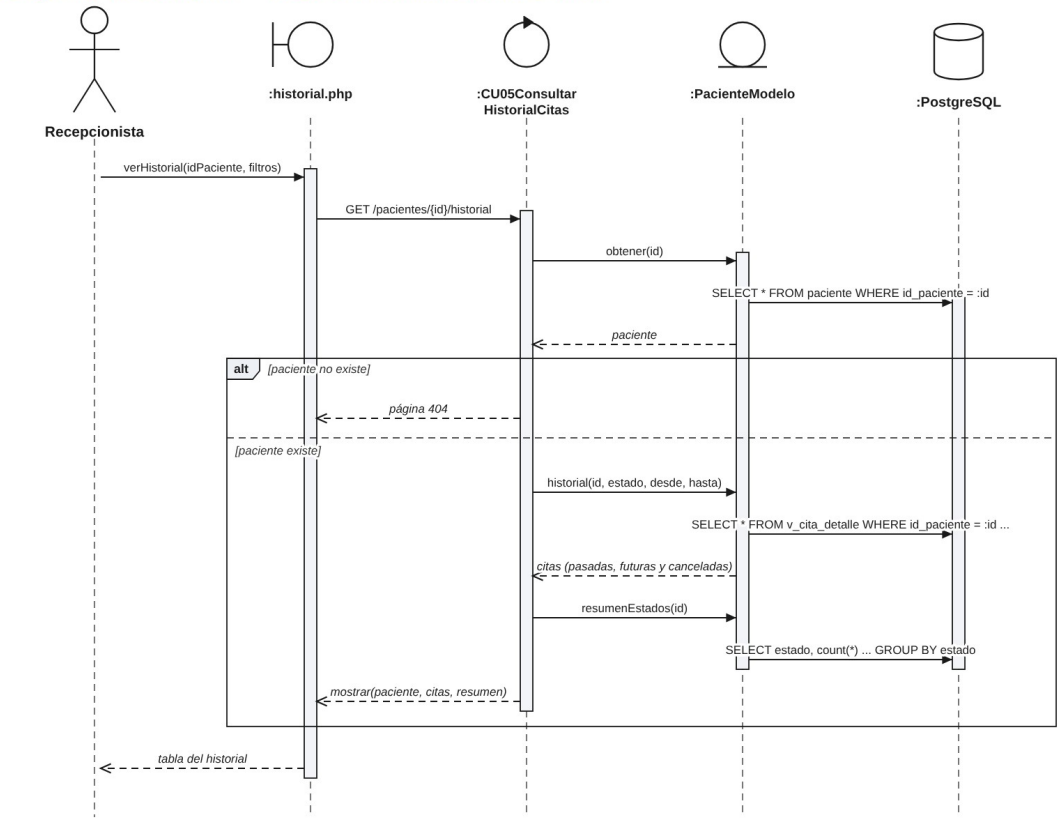

Figura 114. *Diagrama de secuencia — CU05: Consultar historial de citas.*

CU10: Reprogramar cita

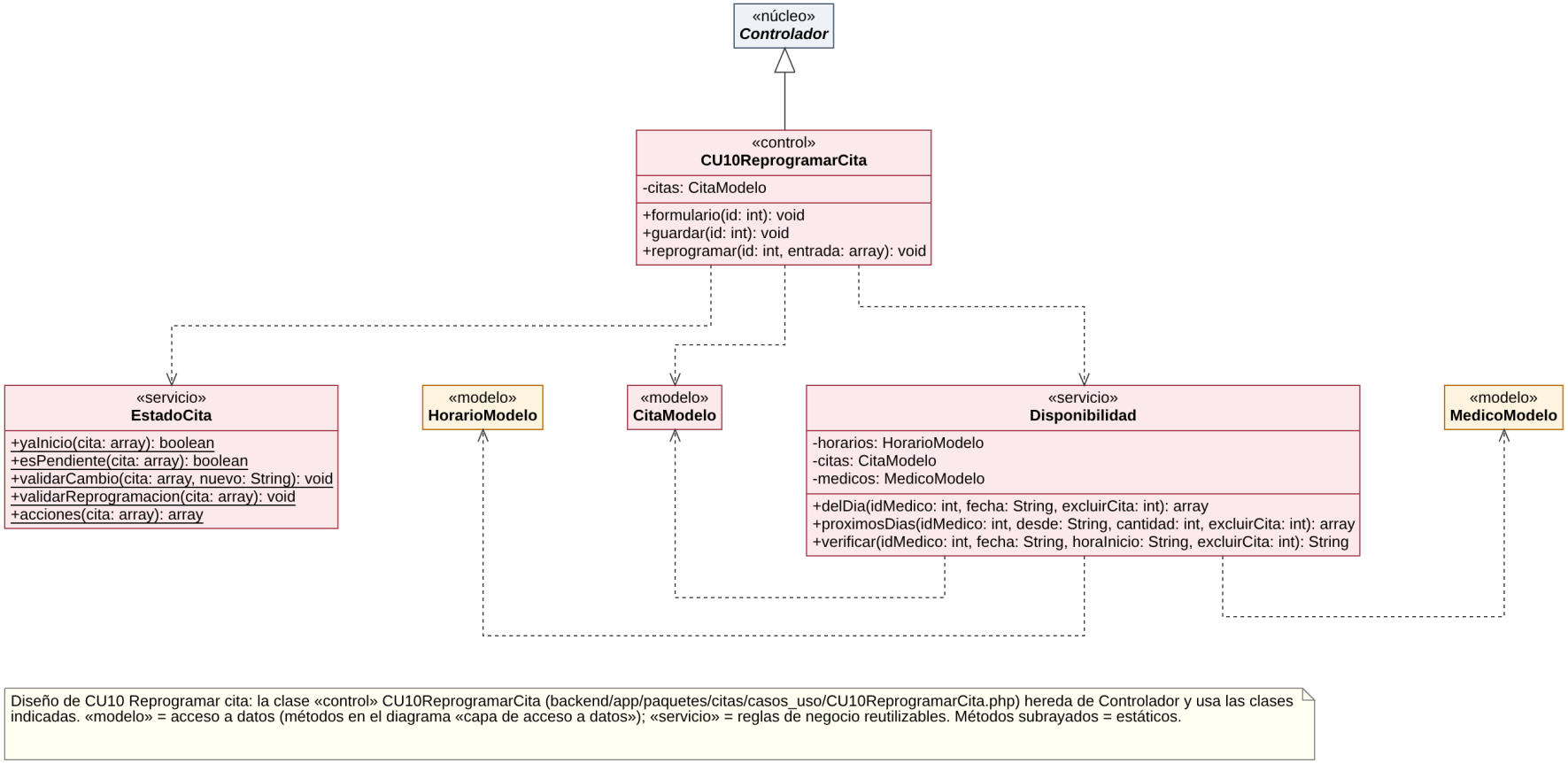

Figura 115. *Diagrama de clases de diseño — CU10: Reprogramar cita.*

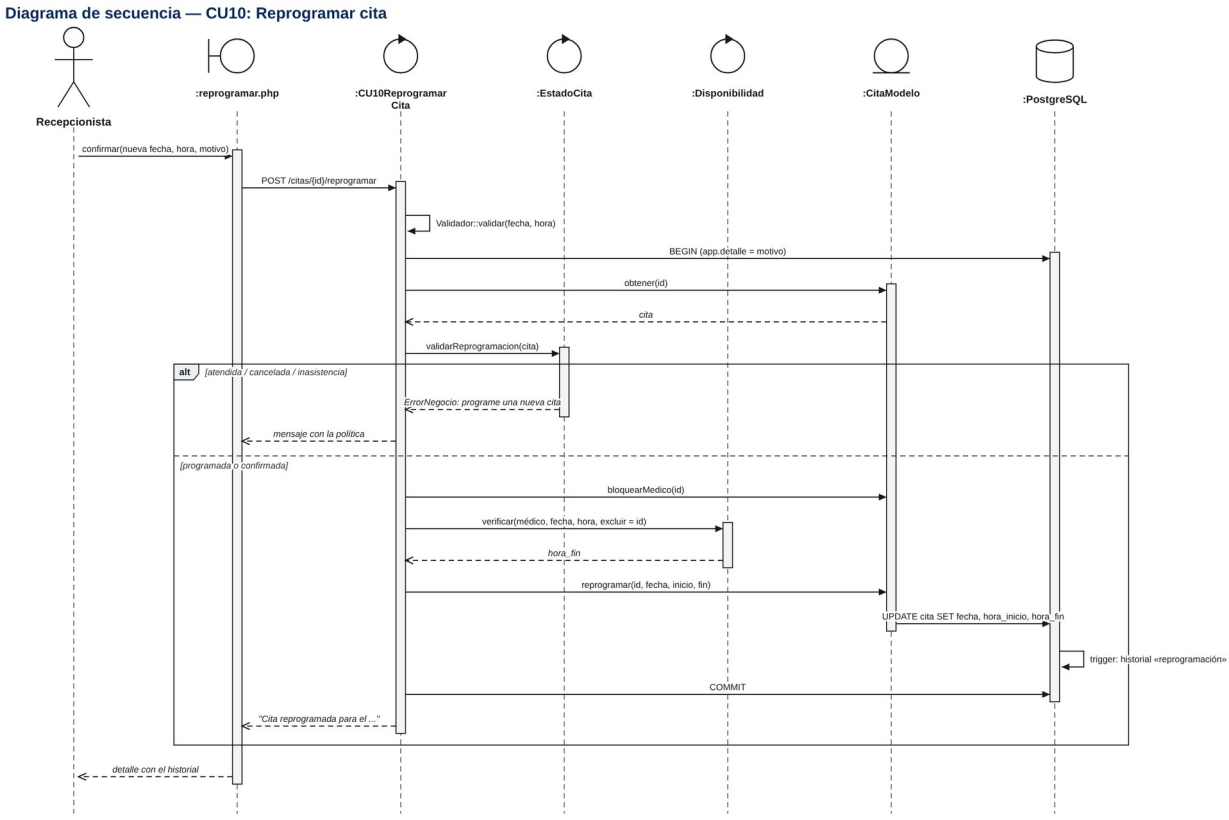

Figura 116. *Diagrama de secuencia — CU10: Reprogramar cita.*

CU11: Cancelar cita

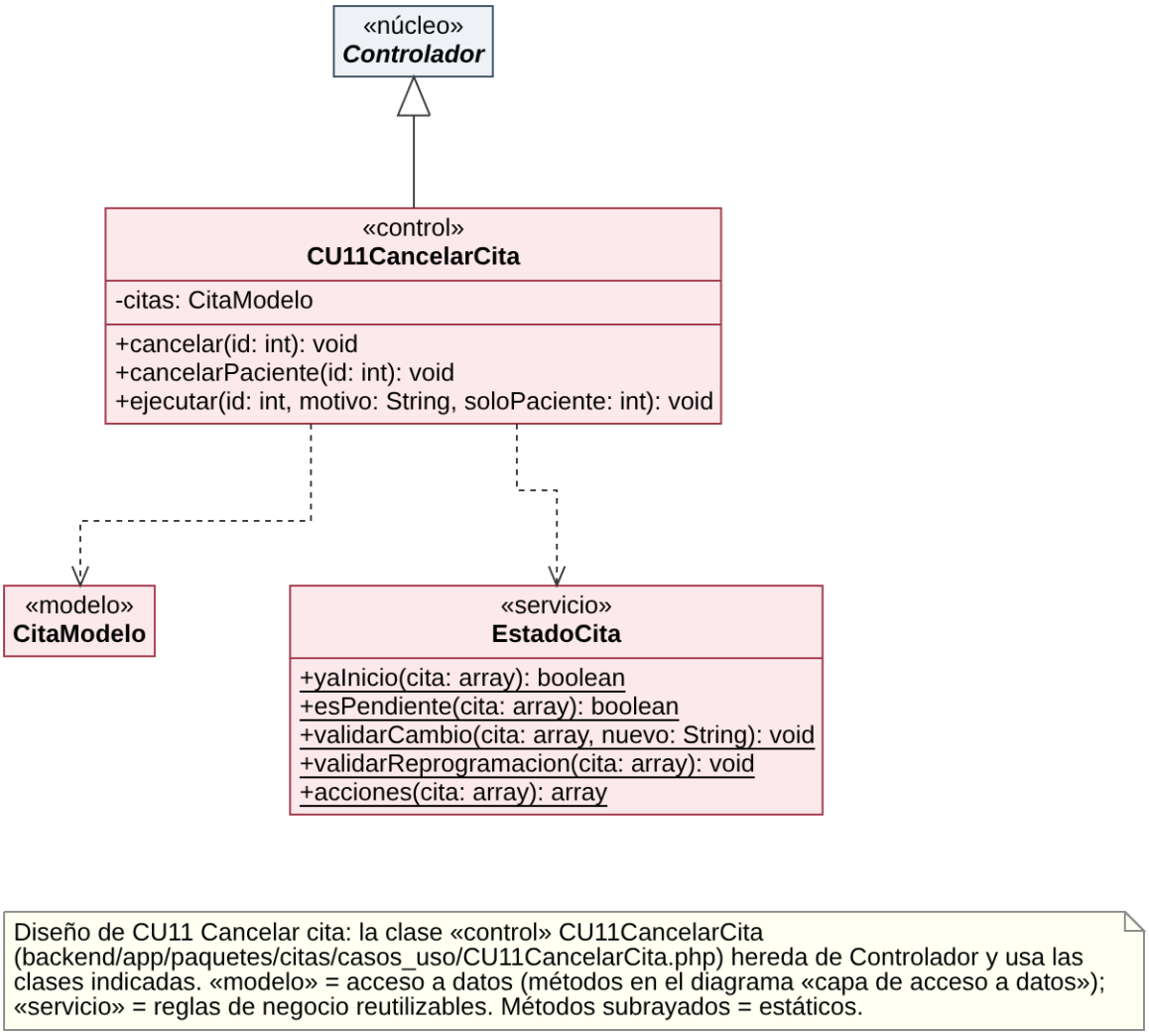

Figura 117. *Diagrama de clases de diseño — CU11: Cancelar cita.*

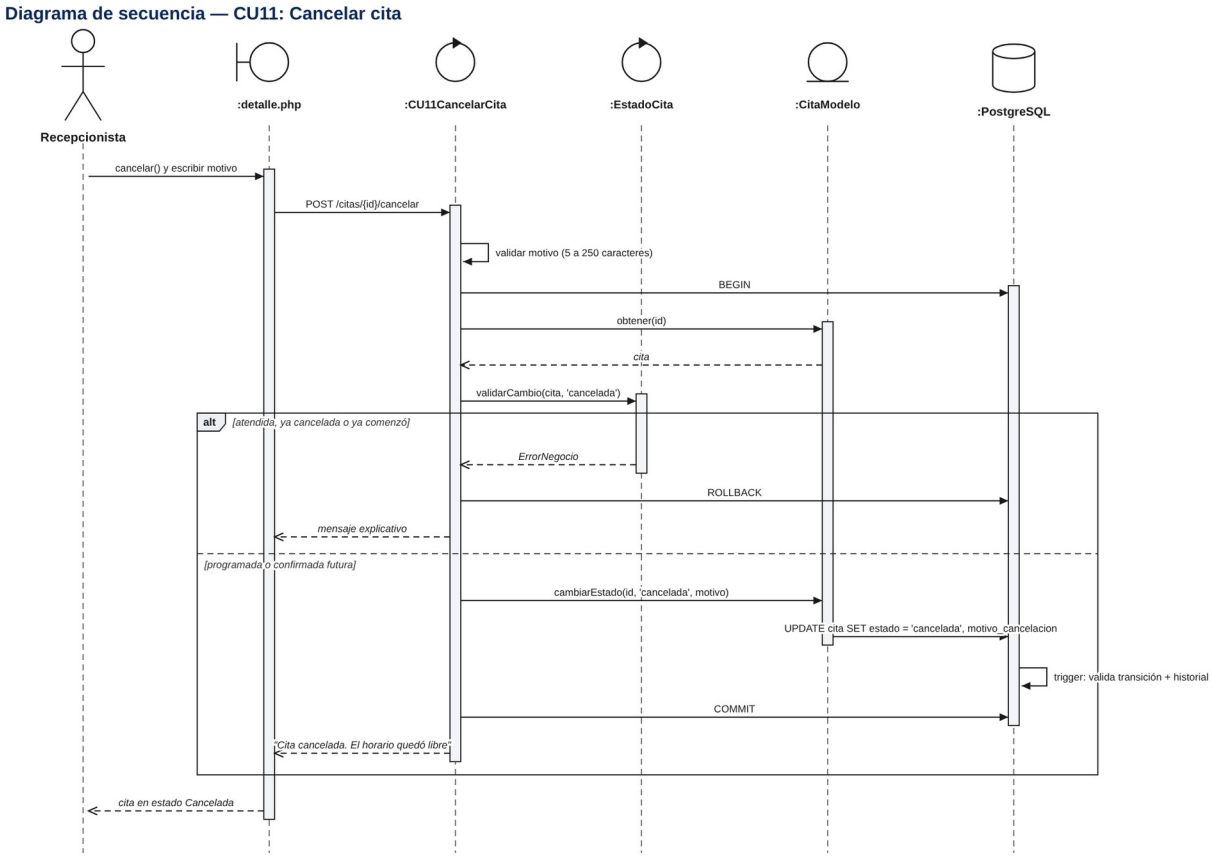

Figura 118. *Diagrama de secuencia — CU11: Cancelar cita.*

CU12: Registrar asistencia

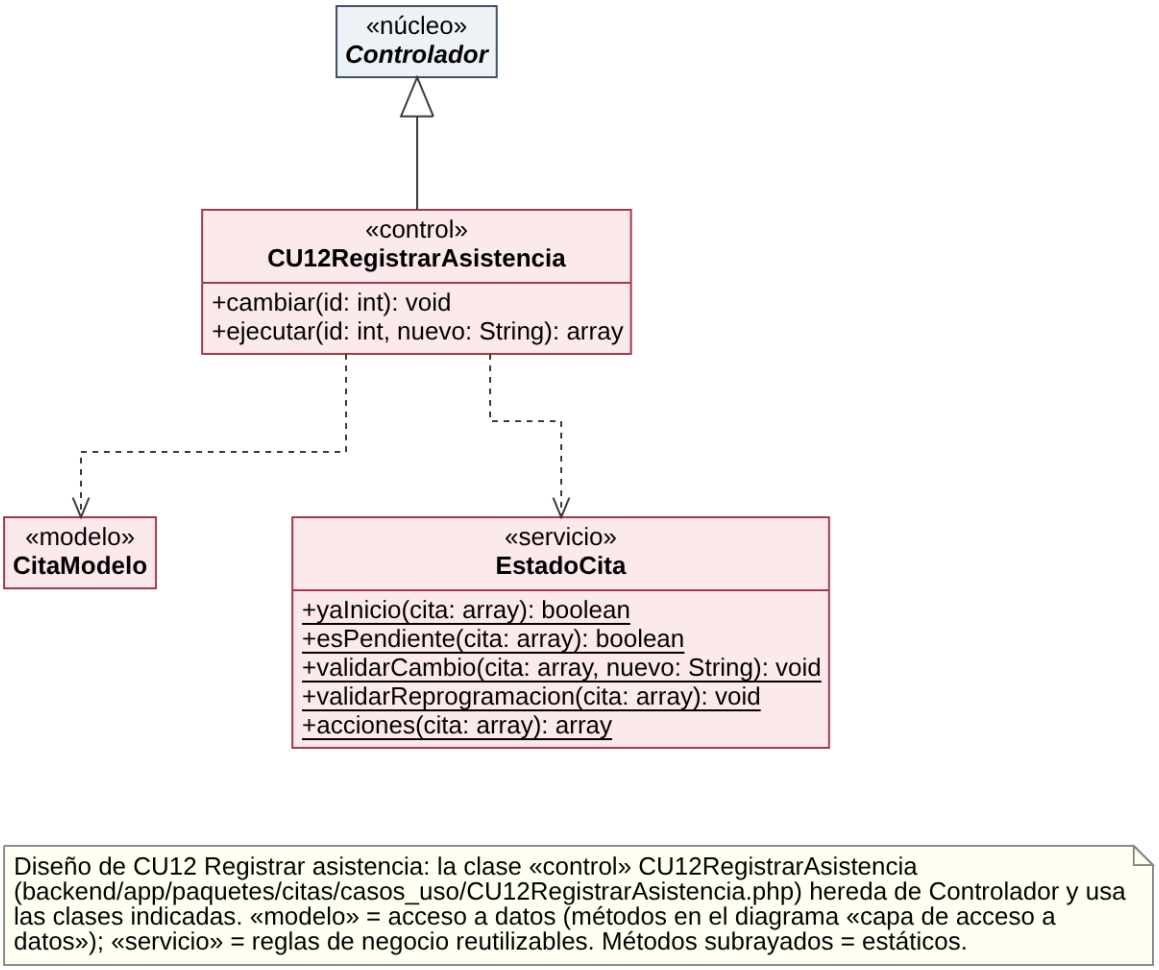

Figura 119. *Diagrama de clases de diseño — CU12: Registrar asistencia.*

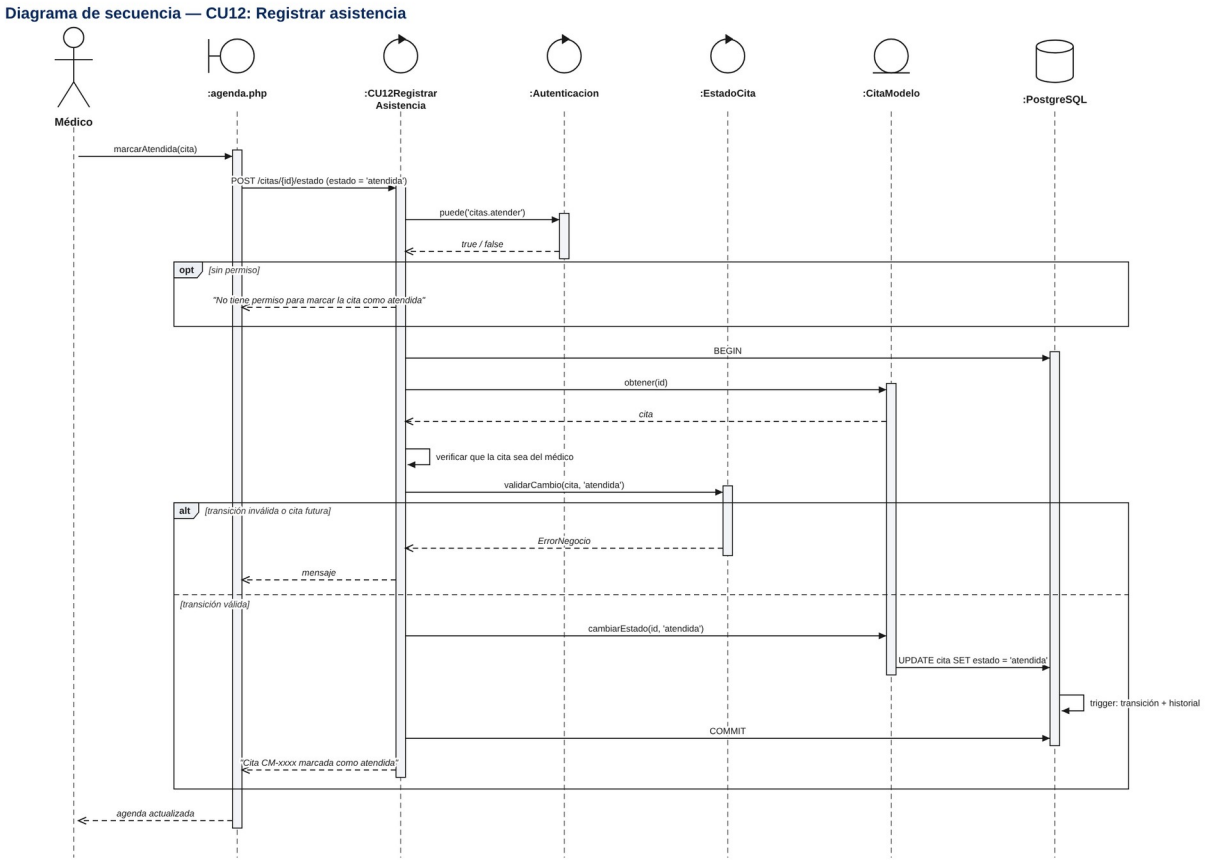

Figura 120. *Diagrama de secuencia — CU12: Registrar asistencia.*

CU13: Consultar agenda médica

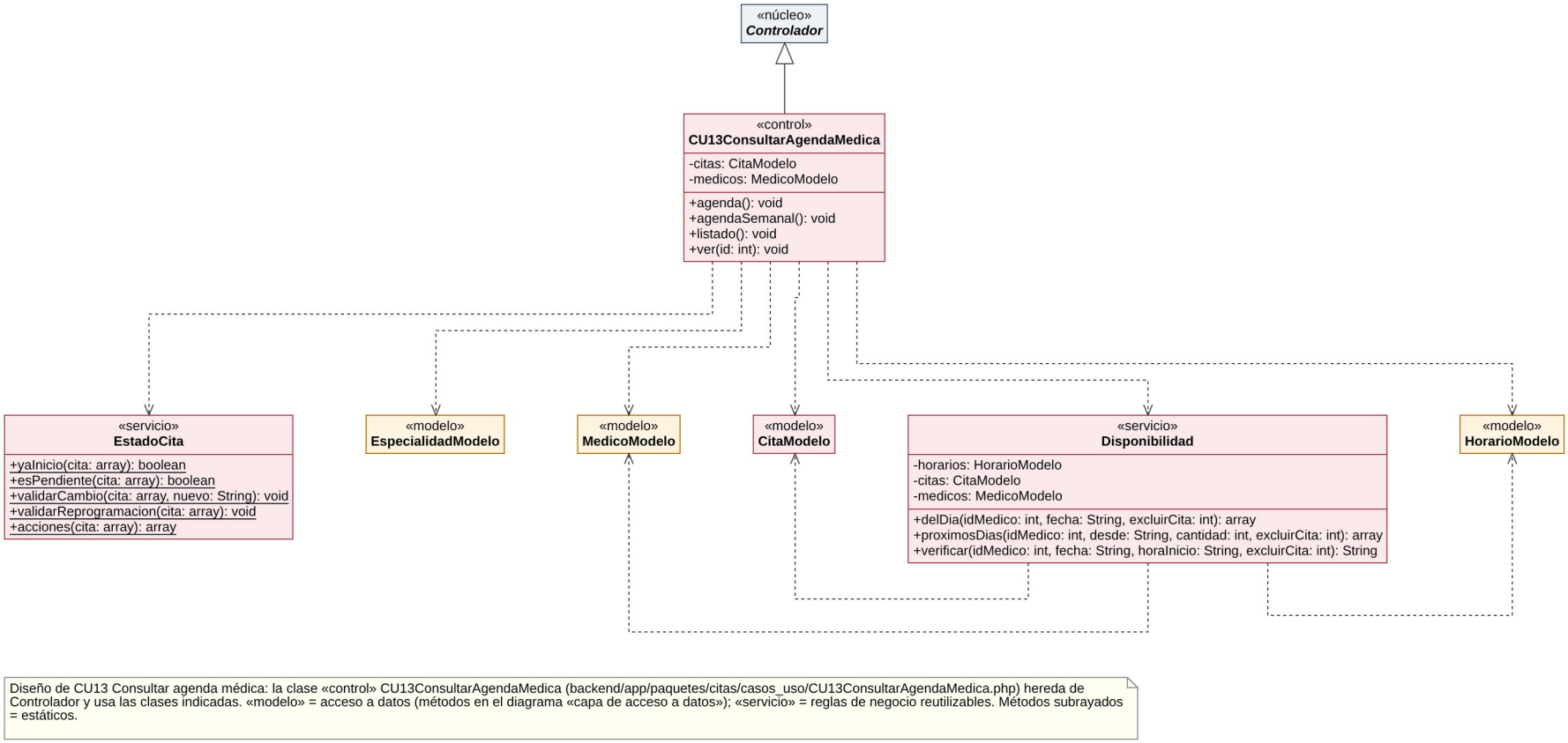

Figura 121. *Diagrama de clases de diseño — CU13: Consultar agenda médica.*

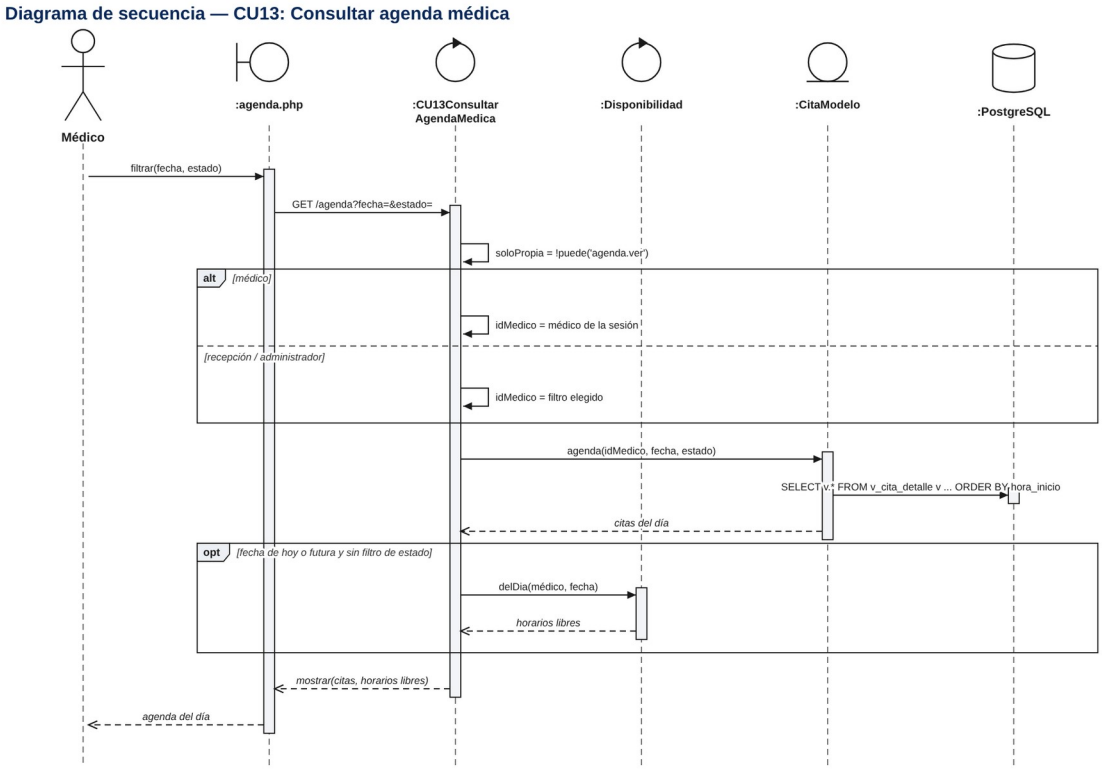

Figura 122. *Diagrama de secuencia — CU13: Consultar agenda médica.*

CU14: Generar reportes

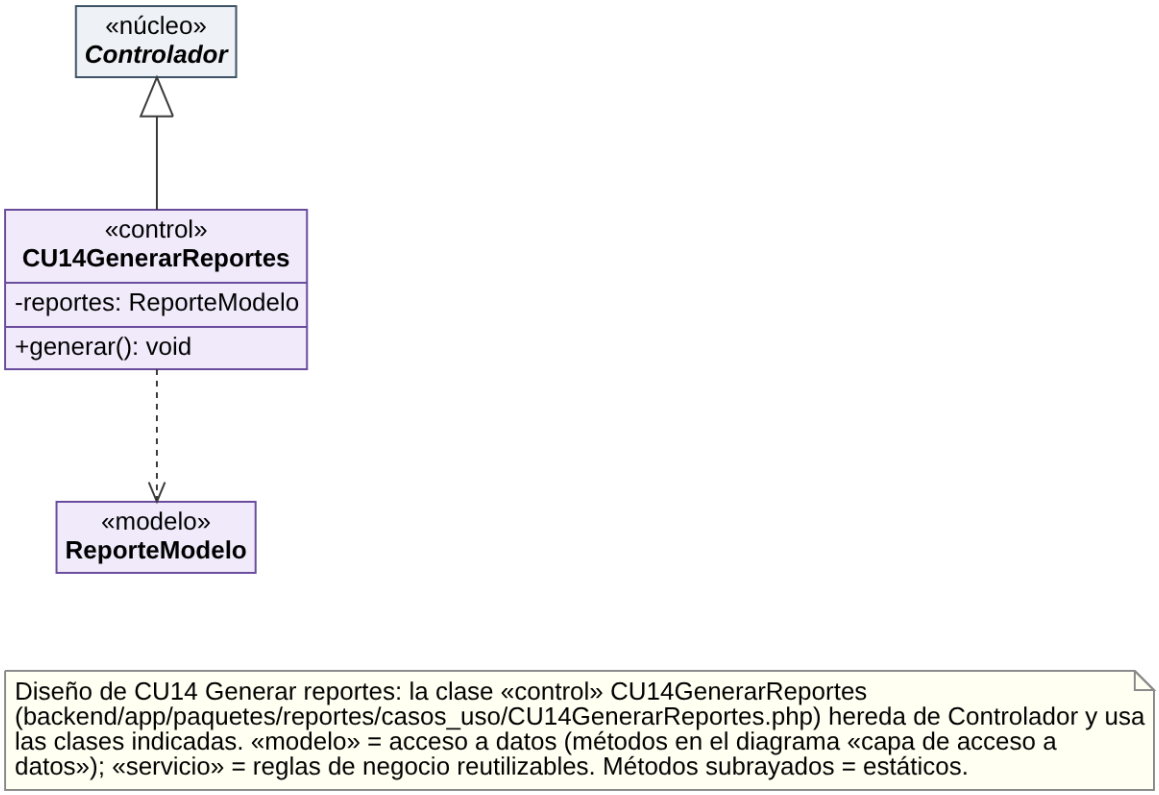

Figura 123. *Diagrama de clases de diseño — CU14: Generar reportes.*

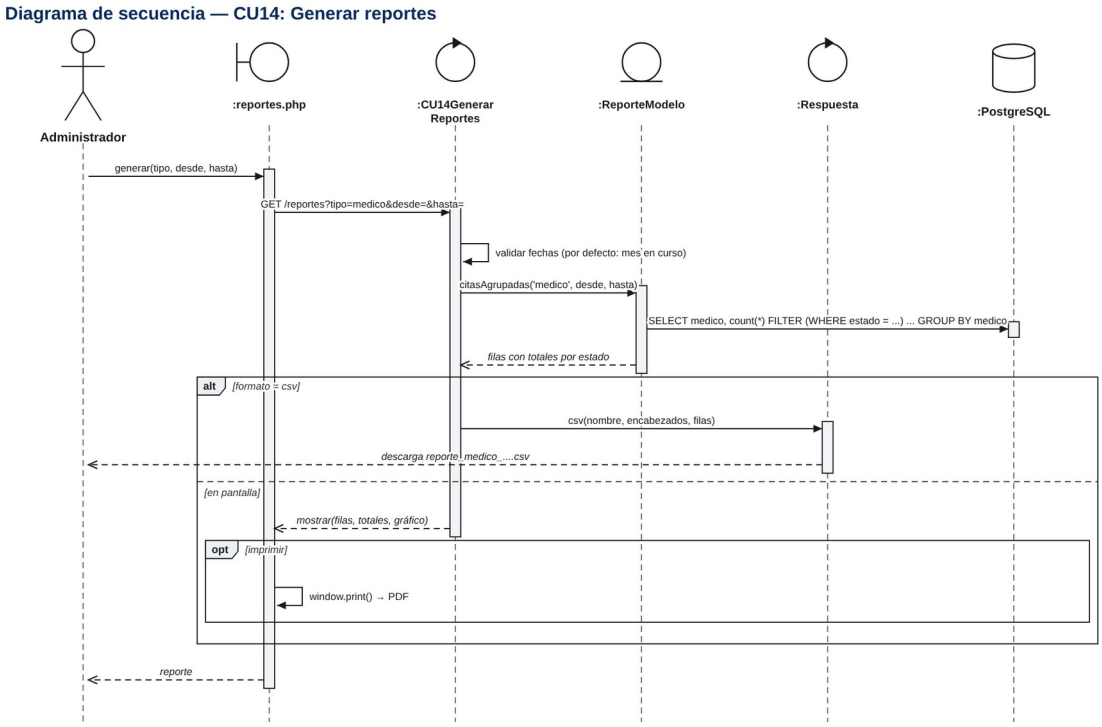

Figura 124. *Diagrama de secuencia — CU14: Generar reportes.*

La siguiente tabla resume, para cada caso de uso, dónde se implementa en cada capa:

Tabla 17. *Diseño por capas — paquete seguridad*

CU	Control (PHP)	Rutas	Vistas	Objetos de BD
CU01	seguridad/casos_uso/ CU01IniciarSesion.php	GET /login POST /login POST /salir GET/POST /cuenta/clave	login.php cambiar_clave.php panel.php	usuario, rol, permiso, rol_permiso
CU02	seguridad/casos_uso/ CU02GestionarUsuariosPermisos.php	GET /usuarios GET/POST /usuarios/nuevo · /usuarios GET/POST /usuarios/{id}/editar · /usuarios/{id} POST /usuarios/{id}/estado GET/POST /permisos GET /bitacora? usuario=&modulo=&accion=&desde=&hasta=&formato=csv	usuarios_lista.php usuario_form.php permisos.php bitacora.php	usuario, rol, permiso, rol_permiso, bitacora (triggers fn_tg_validar_usuario_rol y fn_tg_bitacora_inalterable)

Tabla 18. *Diseño por capas — paquete pacientes*

CU	Control (PHP)	Rutas	Vistas	Objetos de BD
CU03	pacientes/casos_uso/ CU03GestionarPacientes.php	GET /pacientes GET/POST /pacientes/nuevo ·	lista.php formulario.php ficha.php	paciente (UNIQUE uq_paciente_ci), contacto_emergencia

CU	Control (PHP)	Rutas	Vistas	Objetos de BD
		/pacientes GET /pacientes/{id} GET/POST /pacientes/{id}/editar · /pacientes/{id} POST /pacientes/{id}/estado GET /api/pacientes?q=		
CU04	pacientes/casos_uso/CU04GestionarPerfilPaciente.php	GET/POST /registro GET /mi-perfil POST /mi-perfil/personales · /medica · /emergencia · /cuenta	registro.php mi_perfil.php	paciente, usuario, rol, contacto_emergencia
CU05	pacientes/casos_uso/CU05ConsultarHistorialCitas.php	GET /pacientes/{id}/historial GET /mis-citas	historial.php mis_citas.php	vista v_cita_detalle (cita + paciente + médico + especialidad)

Tabla 19. *Diseño por capas — paquete medicos*

CU	Control (PHP)	Rutas	Vistas	Objetos de BD
CU06	medicos/casos_uso/CU06GestionarEspecialidades.php	GET /especialidades POST /especialidades POST /especialidades/{id} POST /especialidades/{id}/estado	especialidades.php	especialidad (índice único uq_especialidad_nombre_normalizado sobre fn_normalizar_texto(nombre))
CU07	medicos/casos_uso/CU07GestionarMedicos.php	GET /medicos GET /medicos/{id} GET/POST /medicos/nuevo · /medicos GET/POST /medicos/{id}/editar · /medicos/{id} POST /medicos/{id}/estado	medicos_lista.php medico_form.php medico_ficha.php	medico, medico_especialidad, vista v_medico_especialidades
CU08	medicos/casos_uso/CU08GestionarHorarios.php	GET /horarios?medico= POST /horarios POST /horarios/{id} · /horarios/{id}/estado POST /feriados · /feriados/{id}/quitar	horarios.php	horario_atencion (CHECK ck_horario_rango, EXCLUDE ex_horario_sin_solape), dia_no_laborable

Tabla 20. *Diseño por capas — paquete citas*

CU	Control (PHP)	Rutas	Vistas	Objetos de BD
CU09	citas/casos_uso/CU09ProgramarCita.php	GET /citas/nueva POST /citas GET/POST /reservar GET /api/medicos?especialidad= GET /api/disponibilidad/dias?medico= GET /api/disponibilidad?medico=&fecha=	programar.php citas.js detalle.php	cita (EXCLUDE ex_cita_medico_sin_solape y ex_cita_paciente_sin_solape, FK compuesta médico–especialidad, trigger fn_tg_validar_cita, trigger fn_tg_historial_cita)

CU	Control (PHP)	Rutas	Vistas	Objetos de BD
CU10	citas/casos_uso/ CU10ReprogramarCita.php	GET /citas/{id}/reprogramar POST /citas/{id}/reprogramar	reprogramar.php citas.js	cita, cita_historial (trigger fn_tg_historial_cita registra la reprogramación)
CU11	citas/casos_uso/ CU11CancelarCita.php	POST /citas/{id}/cancelar POST /mis-citas/{id}/cancelar	detalle.php mis_citas.php	cita (CHECK ck_cita_cancelada exige motivo; trigger impide DELETE), cita_historial
CU12	citas/casos_uso/ CU12RegistrarAsistencia.php	POST /citas/{id}/estado	detalle.php agenda.php citas_hoy.php	cita (trigger fn_tg_validar_transicion_cita), cita_historial
CU13	citas/casos_uso/ CU13ConsultarAgendaMedica.php	GET /agenda?medico=&fecha=&estado= GET /agenda/semana?medico=&fecha= GET /citas?desde=&hasta=&medico=&estado= GET /citas/{id}	agenda.php agenda_semana.php listado.php detalle.php	vista v_cita_detalle, cita_historial, horario_atencion, dia_no_laborable

Tabla 21. *Diseño por capas — paquete reportes*

CU	Control (PHP)	Rutas	Vistas	Objetos de BD
CU14	reportes/casos_uso/ CU14GenerarReportes.php	GET /reportes?tipo=&desde=&hasta=[&formato=csv]	reportes.php	vistas v_cita_detalle, v_resumen_citas_dia; tablas cita y paciente

6.3 Diseño de datos (lógico y físico)

6.3.1 Diseño de datos lógico — diagrama de clases

El diagrama de clases del dominio (elaborado en StarUML, archivo `doc/staruml/Clinica_Vida_Salud_Diagramas_Clases.mdj`) representa la estructura estática de los datos: 14 clases persistentes con sus atributos, 17 asociaciones con multiplicidad y 7 enumeraciones con los valores que permiten las restricciones CHECK de la base de datos (EstadoDeCita, Modalidad, AccionHistorial, Sexo, TipoSangre, EstadoUsuario y EstadoRegistro). Las clases asociativas RolPermiso y MedicoEspecialidad resuelven las relaciones muchos a muchos. Las multiplicidades 0..1 corresponden a llaves foráneas que admiten NULL (por ejemplo, una cita registrada por el propio paciente desde el portal o un día no laborable de toda la clínica).

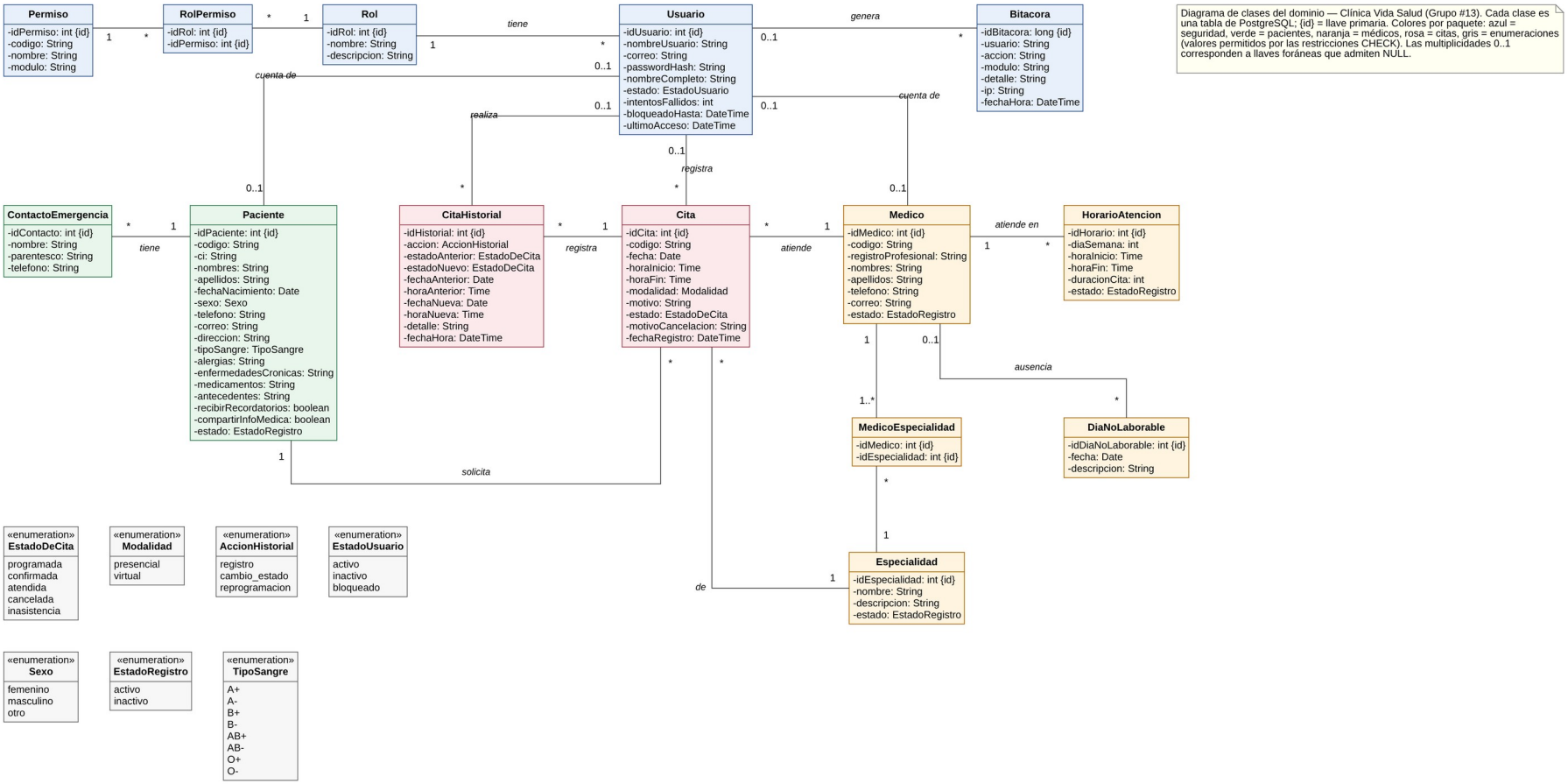

Figura 125. *Diagrama de clases del dominio — diseño lógico de datos.*

Relaciones principales: ROL 1—* USUARIO; ROL *—* PERMISO (ROL_PERMISO); USUARIO 0..1—0..1 MEDICO y PACIENTE (cuenta); PACIENTE 1—* CONTACTO_EMERGENCIA; PACIENTE 1—* CITA; MEDICO 1—* CITA; ESPECIALIDAD 1—* CITA; MEDICO *—* ESPECIALIDAD (MEDICO_ESPECIALIDAD); MEDICO 1—* HORARIO_ATENCION; MEDICO 0..1—* DIA_NO_LABORABLE; CITA 1—* CITA_HISTORIAL; USUARIO 0..1—* CITA (registra); USUARIO 0..1—* CITA_HISTORIAL (realiza); USUARIO 0..1—* BITACORA (genera).

Capa de acceso a datos. Cada entidad se lee y se guarda mediante una clase «modelo» que reúne sus consultas SQL parametrizadas y las ejecuta con BaseDatos (PDO pdo_pgsql). El siguiente diagrama muestra las 9 clases modelo con todos sus métodos públicos.

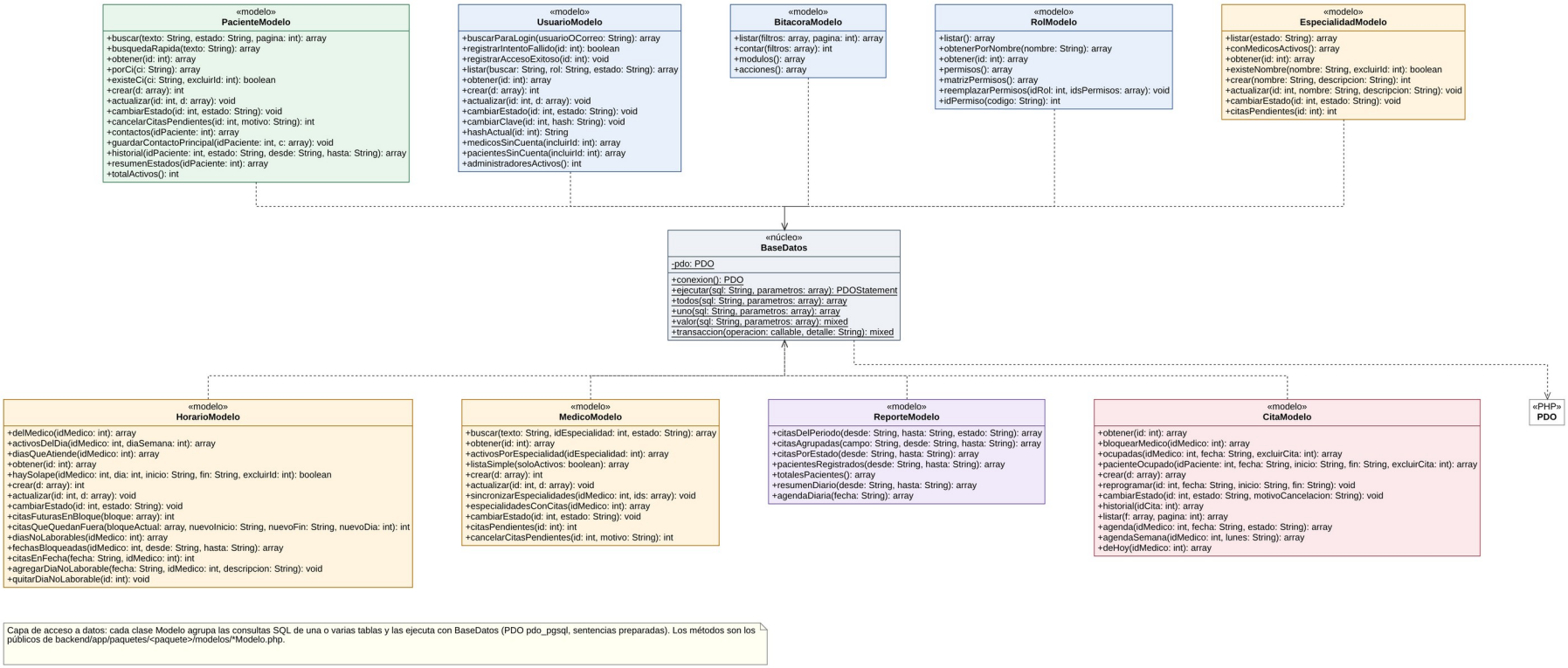

Figura 126. *Diagrama de clases de diseño — capa de acceso a datos (modelos y BaseDatos).*

Mapeo de clases a tablas

Tabla 22. *Mapeo de clases a tablas*

Clase	Tabla	PK	FK	Demás columnas
Rol	ROL	id_rol	—	nombre, descripcion
Permiso	PERMISO	id_permiso	—	codigo, nombre, modulo
RolPermiso	ROL_PERMISO	id_rol, id_permiso	id_rol, id_permiso	
Usuario	USUARIO	id_usuario	id_rol, id_medico, id_paciente	nombre_usuario, correo, password_hash, nombre_completo, estado, intentos_fallidos, bloqueado_hasta, ultimo_acceso, creado_en, actualizado_en
Paciente	PACIENTE	id_paciente	—	codigo, ci, nombres, apellidos, fecha_nacimiento, sexo, telefono, correo, direccion, tipo_sangre, alergias, enfermedades_cronicas, medicamentos, antecedentes, recibir_recordatorios, compartir_info_medica, estado, creado_en, actualizado_en
ContactoEmergencia	CONTACTO_EMERGENCIA	id_contacto	id_paciente	nombre, parentesco, telefono
Especialidad	ESPECIALIDAD	id_especialidad	—	nombre, descripcion, estado, creado_en, actualizado_en
Medico	MEDICO	id_medico	—	codigo, registro_profesional, nombres, apellidos, telefono, correo, estado, creado_en, actualizado_en
MedicoEspecialidad	MEDICO_ESPECIALIDAD	id_medico, id_especialidad	id_medico, id_especialidad	
HorarioAtencion	HORARIO_ATENCION	id_horario	id_medico	dia_semana, hora_inicio, hora_fin, duracion_cita, estado, creado_en, actualizado_en
DiaNoLaborable	DIA_NO_LABORABLE	id_dia_no_laborable	id_medico	fecha, descripcion
Cita	CITA	id_cita	id_paciente, id_medico, id_especialidad, id_usuario_registro	codigo, fecha, hora_inicio, hora_fin, modalidad, motivo, estado, motivo_cancelacion, fecha_registro, actualizado_en
CitaHistorial	CITA_HISTORIAL	id_historial	id_cita, id_usuario	accion, estado_anterior, estado_nuevo, fecha_anterior, hora_anterior, fecha_nueva, hora_nueva, detalle, fecha_hora

Clase	Tabla	PK	FK	Demás columnas
Bitacora	BITACORA	id_bitacora	id_usuario	usuario, accion, modulo, detalle, ip, fecha_hora

Normalización

Tabla 23. *Normalización del modelo*

Forma normal	Cómo se cumple en la base de datos
1FN	Todos los atributos son atómicos: nombres y apellidos separados; las especialidades de un médico no se guardan como lista sino en MEDICO_ESPECIALIDAD; los contactos de emergencia en su propia tabla.
2FN	En las tablas con llave compuesta (ROL_PERMISO, MEDICO_ESPECIALIDAD) no hay atributos que dependan solo de una parte de la llave.
3FN	No hay dependencias transitivas: la cita guarda id_medico e id_especialidad, no el nombre del médico ni de la especialidad; el nombre del rol se obtiene de ROL.
FNBC	Todo determinante es llave candidata: código, CI, registro profesional y correo son UNIQUE; el horario se identifica por su id y su no solapamiento se controla con EXCLUDE.

Ejemplos: si el nombre del médico se guardara en cada cita, al corregirlo habría que actualizar cientos de filas (anomalía de actualización); por eso la cita solo guarda la llave del médico y la vista v_cita_detalle une los datos para mostrarlos.

6.3.2 Diseño de datos físico

El diseño físico implementa el modelo en PostgreSQL: 14 tablas con llaves primarias autoincrementales (SERIAL y BIGSERIAL en la bitácora), llaves foráneas con ON DELETE RESTRICT para conservar el historial, restricciones CHECK y UNIQUE, tres restricciones EXCLUDE contra solapamientos, siete funciones de trigger, tres vistas y tres secuencias para los códigos legibles (PAC-0001, MED-001, CM-0001).

Diagrama relacional

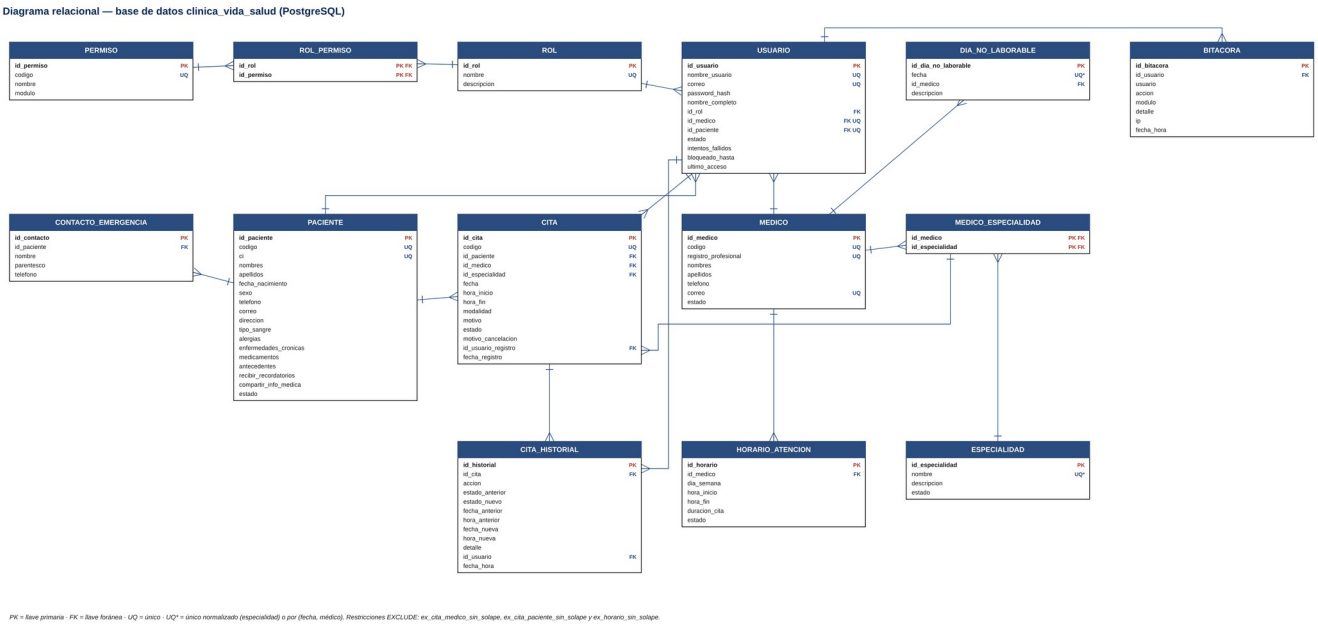

Figura 127. *Diagrama relacional de la base de datos clinica_vida_salud.*

Tabla de volumen

Tabla 24. *Tabla de volumen (estimación con 10 médicos y ~70 citas por día hábil)*

Tabla	Filas (datos de prueba)	Estimado 1er año	Crecimiento
ROL	4	4	Fijo
PERMISO	21	21	Fijo
ROL_PERMISO	33	33	~35
USUARIO	9	600	~800
PACIENTE	20	2 000	~2 500
CONTACTO_EMERGENCIA	7	1 500	~1 800
ESPECIALIDAD	5	5	~8
MEDICO	7	10	~12
MEDICO_ESPECIALIDAD	8	12	~15
HORARIO_ATENCION	29	50	~60
DIA_NO_LABORABLE	3	15	~15/año
CITA	977	18 000	~18 000/año
CITA_HISTORIAL	1806	40 000	~40 000/año
BITACORA	0	150 000	~150 000/año

Diccionario de datos

Tabla 25. *Diccionario de datos — tabla ROL*

Atributo	Tipo de dato	PK	FK	Not Null	Descripción
id_rol	smallint	SÍ	NO	SÍ	Identificador del rol.
nombre	varchar(30)	NO	NO	SÍ	administrador, recepcionista,

Atributo	Tipo de dato	PK	FK	Not Null	Descripción
					medico o paciente.
descripcion	varchar(150)	NO	NO	NO	Descripción de lo que hace el rol.

Tabla 26. *Diccionario de datos — tabla PERMISO*

Atributo	Tipo de dato	PK	FK	Not Null	Descripción
id_permiso	integer	SÍ	NO	SÍ	Identificador del permiso.
codigo	varchar(50)	NO	NO	SÍ	Código usado por el sistema (p. ej. citas.programar).
nombre	varchar(120)	NO	NO	SÍ	Nombre legible del permiso.
modulo	varchar(30)	NO	NO	SÍ	Paquete al que pertenece.

Tabla 27. *Diccionario de datos — tabla ROL_PERMISO*

Atributo	Tipo de dato	PK	FK	Not Null	Descripción
id_rol	smallint	SÍ	SÍ	SÍ	Rol que recibe el permiso.
id_permiso	integer	SÍ	SÍ	SÍ	Permiso asignado.

Tabla 28. *Diccionario de datos — tabla USUARIO*

Atributo	Tipo de dato	PK	FK	Not Null	Descripción
id_usuario	integer	SÍ	NO	SÍ	Identificador de la cuenta.
nombre_usuario	varchar(40)	NO	NO	SÍ	Usuario para iniciar sesión (único).
correo	varchar(120)	NO	NO	SÍ	Correo para iniciar sesión (único).
password_hash	varchar(255)	NO	NO	SÍ	Hash bcrypt generado con password_hash(); nunca la contraseña.
nombre_completo	varchar(120)	NO	NO	SÍ	Nombre que se muestra en el sistema.
id_rol	smallint	NO	SÍ	SÍ	Rol de la cuenta.
id_medico	integer	NO	SÍ	NO	Médico vinculado (rol médico).
id_paciente	integer	NO	SÍ	NO	Paciente vinculado (rol paciente).
estado	varchar(10)	NO	NO	SÍ	activo, inactivo o bloqueado.
intentos_fallidos	smallint	NO	NO	SÍ	Intentos de acceso fallidos seguidos.

Atributo	Tipo de dato	PK	FK	Not Null	Descripción
bloqueado_hasta	timestamptz	NO	NO	NO	Fin del bloqueo temporal.
ultimo_acceso	timestamptz	NO	NO	NO	Fecha y hora del último ingreso.
creado_en	timestamptz	NO	NO	SÍ	Fecha de creación.
actualizado_en	timestamptz	NO	NO	SÍ	Última modificación (trigger).

Tabla 29. *Diccionario de datos — tabla PACIENTE*

Atributo	Tipo de dato	PK	FK	Not Null	Descripción
id_paciente	integer	SÍ	NO	SÍ	Identificador interno.
codigo	varchar(10)	NO	NO	SÍ	Código único PAC-0001 (secuencia).
ci	varchar(15)	NO	NO	NO	Documento de identidad, único cuando se registra.
nombres	varchar(60)	NO	NO	SÍ	Nombres.
apellidos	varchar(60)	NO	NO	SÍ	Apellidos.
fecha_nacimiento	date	NO	NO	SÍ	Fecha de nacimiento (no futura).
sexo	varchar(10)	NO	NO	SÍ	femenino, masculino u otro.
telefono	varchar(20)	NO	NO	SÍ	Teléfono de contacto.
correo	varchar(120)	NO	NO	NO	Correo (opcional).
direccion	varchar(200)	NO	NO	NO	Domicilio (opcional).
tipo_sangre	varchar(3)	NO	NO	NO	Grupo sanguíneo.
alergias	varchar(250)	NO	NO	NO	Alergias conocidas.
enfermedades_cronicas	varchar(250)	NO	NO	NO	Enfermedades crónicas.
medicamentos	varchar(250)	NO	NO	NO	Medicamentos que toma.
antecedentes	varchar(500)	NO	NO	NO	Antecedentes relevantes.
recibir_recordatorios	boolean	NO	NO	SÍ	Mostrar recordatorios de citas.
compartir_info_medica	boolean	NO	NO	SÍ	Permite que el médico vea su información médica.
estado	varchar(10)	NO	NO	SÍ	activo o inactivo.
creado_en	timestamptz	NO	NO	SÍ	Fecha de registro.
actualizado_en	timestamptz	NO	NO	SÍ	Última modificación.

Tabla 30. *Diccionario de datos — tabla CONTACTO_EMERGENCIA*

Atributo	Tipo de dato	PK	FK	Not Null	Descripción
id_contacto	integer	SÍ	NO	SÍ	Identificador.
id_paciente	integer	NO	SÍ	SÍ	Paciente al que pertenece.
nombre	varchar(120)	NO	NO	SÍ	Nombre del contacto.
parentesco	varchar(40)	NO	NO	SÍ	Parentesco.
telefono	varchar(20)	NO	NO	SÍ	Teléfono.

Tabla 31. *Diccionario de datos — tabla ESPECIALIDAD*

Atributo	Tipo de dato	PK	FK	Not Null	Descripción
id_especialidad	integer	SÍ	NO	SÍ	Identificador.
nombre	varchar(80)	NO	NO	SÍ	Nombre (único normalizado).
descripcion	varchar(250)	NO	NO	NO	Descripción opcional.
estado	varchar(10)	NO	NO	SÍ	activo o inactivo.
creado_en	timestamptz	NO	NO	SÍ	Fecha de creación.
actualizado_en	timestamptz	NO	NO	SÍ	Última modificación.

Tabla 32. *Diccionario de datos — tabla MEDICO*

Atributo	Tipo de dato	PK	FK	Not Null	Descripción
id_medico	integer	SÍ	NO	SÍ	Identificador interno.
codigo	varchar(10)	NO	NO	SÍ	Código único MED-001.
registro_profesional	varchar(30)	NO	NO	SÍ	Número de registro profesional (único).
nombres	varchar(60)	NO	NO	SÍ	Nombres.
apellidos	varchar(60)	NO	NO	SÍ	Apellidos.
telefono	varchar(20)	NO	NO	SÍ	Teléfono.
correo	varchar(120)	NO	NO	SÍ	Correo (único).
estado	varchar(10)	NO	NO	SÍ	activo o inactivo.
creado_en	timestamptz	NO	NO	SÍ	Fecha de creación.
actualizado_en	timestamptz	NO	NO	SÍ	Última modificación.

Tabla 33. *Diccionario de datos — tabla MEDICO_ESPECIALIDAD*

Atributo	Tipo de dato	PK	FK	Not Null	Descripción
id_medico	integer	SÍ	SÍ	SÍ	Médico.
id_especialidad	integer	SÍ	SÍ	SÍ	Especialidad que atiende.

Tabla 34. *Diccionario de datos — tabla HORARIO_ATENCION*

Atributo	Tipo de dato	PK	FK	Not Null	Descripción
id_horario	integer	SÍ	NO	SÍ	Identificador.
id_medico	integer	NO	SÍ	SÍ	Médico.
dia_semana	smallint	NO	NO	SÍ	1 = lunes … 7 = domingo.
hora_inicio	time	NO	NO	SÍ	Hora inicial del bloque.
hora_fin	time	NO	NO	SÍ	Hora final (posterior a la inicial).
duracion_cita	smallint	NO	NO	SÍ	Minutos por cita: 15, 20, 30, 45 o 60.
estado	varchar(10)	NO	NO	SÍ	activo o inactivo.
creado_en	timestamptz	NO	NO	SÍ	Fecha de creación.
actualizado_en	timestamptz	NO	NO	SÍ	Última modificación.

Tabla 35. *Diccionario de datos — tabla DIA_NO_LABORABLE*

Atributo	Tipo de dato	PK	FK	Not Null	Descripción
id_dia_no_laborable	integer	SÍ	NO	SÍ	Identificador.
fecha	date	NO	NO	SÍ	Fecha bloqueada.
id_medico	integer	NO	SÍ	NO	Médico (vacío = toda la clínica).
descripcion	varchar(120)	NO	NO	SÍ	Motivo (feriado, ausencia).

Tabla 36. *Diccionario de datos — tabla CITA*

Atributo	Tipo de dato	PK	FK	Not Null	Descripción
id_cita	integer	SÍ	NO	SÍ	Identificador interno.
codigo	varchar(10)	NO	NO	SÍ	Código único CM-0001.
id_paciente	integer	NO	SÍ	SÍ	Paciente.
id_medico	integer	NO	SÍ	SÍ	Médico.
id_especialidad	integer	NO	SÍ	SÍ	Especialidad (FK compuesta con médico).
fecha	date	NO	NO	SÍ	Fecha de la cita.
hora_inicio	time	NO	NO	SÍ	Hora de inicio.
hora_fin	time	NO	NO	SÍ	Hora de fin.
modalidad	varchar(12)	NO	NO	SÍ	presencial o virtual.
motivo	varchar(250)	NO	NO	NO	Motivo breve (opcional).
estado	varchar(15)	NO	NO	SÍ	programada, confirmada, atendida, cancelada o inasistencia.
motivo_cancelacion	varchar(250)	NO	NO	NO	Obligatorio si está

Atributo	Tipo de dato	PK	FK	Not Null	Descripción
					cancelada.
id_usuario_registro	integer	NO	SÍ	NO	Usuario que registró la cita.
fecha_registro	timestamptz	NO	NO	SÍ	Fecha y hora de creación.
actualizado_en	timestamptz	NO	NO	SÍ	Última modificación.

Tabla 37. *Diccionario de datos — tabla CITA_HISTORIAL*

Atributo	Tipo de dato	PK	FK	Not Null	Descripción
id_historial	bigint	SÍ	NO	SÍ	Identificador.
id_cita	integer	NO	SÍ	SÍ	Cita.
accion	varchar(20)	NO	NO	SÍ	registro, cambio_estado o reprogramacion.
estado_anterior	varchar(15)	NO	NO	NO	Estado previo.
estado_nuevo	varchar(15)	NO	NO	NO	Estado nuevo.
fecha_anterior	date	NO	NO	NO	Fecha previa (reprogramación).
hora_anterior	time	NO	NO	NO	Hora previa.
fecha_nueva	date	NO	NO	NO	Fecha nueva.
hora_nueva	time	NO	NO	NO	Hora nueva.
detalle	varchar(250)	NO	NO	NO	Motivo o detalle del cambio.
id_usuario	integer	NO	SÍ	NO	Usuario que hizo el cambio.
fecha_hora	timestamptz	NO	NO	SÍ	Momento del cambio.

Tabla 38. *Diccionario de datos — tabla BITACORA*

Atributo	Tipo de dato	PK	FK	Not Null	Descripción
id_bitacora	bigint	SÍ	NO	SÍ	Identificador (BIGSERIAL).
id_usuario	integer	NO	SÍ	NO	Usuario que hizo la acción (vacío si no inició sesión).
usuario	varchar(120)	NO	NO	NO	Nombre de usuario escrito o «anónimo» (se conserva aunque cambie la cuenta).
accion	varchar(40)	NO	NO	SÍ	inicio_sesion, inicio_fallido, cuenta_bloqueada, cierre_sesion, operacion (petición POST) o exportacion.
modulo	varchar(30)	NO	NO	SÍ	Paquete donde

Atributo	Tipo de dato	PK	FK	Not Null	Descripción
					ocurrió: seguridad, pacientes, medicos, citas, reportes o inicio.
detalle	text	NO	NO	NO	Ruta, caso de uso y resultado; nunca contraseñas ni datos del formulario.
ip	varchar(45)	NO	NO	NO	Dirección IP del equipo.
fecha_hora	timestamptz	NO	NO	SÍ	Fecha y hora de la acción.

Restricciones, triggers y vistas

Tabla 39. *Objetos de integridad de la base de datos*

Objeto	Tipo	Qué garantiza
ex_cita_medico_sin_solape	EXCLUDE (gist)	Un médico no tiene dos citas no canceladas que se crucen (RF07, RN07, RN09).
ex_cita_paciente_sin_solape	EXCLUDE (gist)	Un paciente no tiene dos citas a la misma hora.
ex_horario_sin_solape	EXCLUDE (gist)	Los bloques activos de un médico no se superponen el mismo día.
fk_cita_medico_especialidad	FK compuesta	El médico de la cita atiende la especialidad elegida (RN03).
uq_paciente_ci / uq_especialidad_nombre_normalizado	UNIQUE	CI único (RF02) y especialidad sin nombres repetidos (RF04).
ck_horario_rango, ck_cita_rango, ck_cita_estado, ck_cita_cancelada…	CHECK	Hora final posterior, estados válidos, motivo obligatorio al cancelar.
fn_tg_validar_cita	Trigger BEFORE INSERT/UPDATE	Médico, paciente y especialidad activos, dentro del horario, no en el pasado, fecha no bloqueada (RN03–RN06).
fn_tg_validar_transicion_cita	Trigger BEFORE UPDATE OF estado	Solo transiciones válidas; no marcar atendida una cita futura (RF10).
fn_tg_historial_cita	Trigger AFTER INSERT/UPDATE	Registra en cita_historial la creación, cada cambio de estado y cada reprogramación con el usuario.
fn_tg_impedir_eliminacion	Trigger BEFORE DELETE	Las citas y su historial nunca se borran (RN10).
fn_tg_validar_usuario_rol	Trigger BEFORE INSERT/UPDATE	Un usuario médico/paciente está vinculado a su médico/paciente (RN12).
fn_tg_bitacora_inalterable	Trigger BEFORE UPDATE OR DELETE	La bitácora de acciones no se puede modificar ni borrar (auditoría).
fn_tg_actualizar_fecha	Trigger BEFORE UPDATE	Mantiene la columna actualizado_en.
v_cita_detalle, v_medico_especialidades, v_resumen_citas_dia	Vistas	Consultas listas para agenda, historial y reportes.

Script de creación (extracto). El script completo está en `basededato/sql/01_esquema.sql` a `05_datos_prueba.sql` y se ejecuta en orden con `instalar.sql`. Se muestra la tabla principal CITA con sus restricciones:

01_esquema.sql — tabla CITA

```
CREATE TABLE cita (
    id_cita              SERIAL       PRIMARY KEY,
    codigo               VARCHAR(10)  NOT NULL DEFAULT ('CM-' ||
lpad(nextval('seq_codigo_cita')::TEXT, 4, '0')),
    id_paciente          INTEGER      NOT NULL REFERENCES paciente(id_paciente) ON DELETE RESTRICT,
    id_medico            INTEGER      NOT NULL,
    id_especialidad      INTEGER      NOT NULL,
    fecha                DATE         NOT NULL,
    hora_inicio          TIME         NOT NULL,
    hora_fin             TIME         NOT NULL,
    modalidad            VARCHAR(12)  NOT NULL DEFAULT 'presencial',
    motivo               VARCHAR(250),
    estado               VARCHAR(15)  NOT NULL DEFAULT 'programada',
    motivo_cancelacion   VARCHAR(250),
    id_usuario_registro  INTEGER      REFERENCES usuario(id_usuario),
    fecha_registro       TIMESTAMPTZ  NOT NULL DEFAULT now(),
    actualizado_en       TIMESTAMPTZ  NOT NULL DEFAULT now(),
    CONSTRAINT uq_cita_codigo     UNIQUE (codigo),
    -- RN03: la cita pertenece a un paciente y a un médico válidos; además el
    -- médico debe tener la especialidad elegida (FK compuesta).
    CONSTRAINT fk_cita_medico_especialidad FOREIGN KEY (id_medico, id_especialidad)
        REFERENCES medico_especialidad (id_medico, id_especialidad) ON DELETE RESTRICT,
    CONSTRAINT ck_cita_rango      CHECK (hora_fin > hora_inicio),
    CONSTRAINT ck_cita_modalidad  CHECK (modalidad IN ('presencial','virtual')),
    CONSTRAINT ck_cita_estado     CHECK (estado IN
('programada','confirmada','atendida','cancelada','inasistencia')),
    CONSTRAINT ck_cita_cancelada  CHECK (estado <> 'cancelada' OR motivo_cancelacion IS NOT NULL),
    -- RN07 / RF07: un médico no puede tener citas superpuestas.
    -- RN09: las citas canceladas no ocupan disponibilidad (cláusula WHERE).
    CONSTRAINT ex_cita_medico_sin_solape EXCLUDE USING gist (
        id_medico WITH =,
        tsrange(fecha + hora_inicio, fecha + hora_fin) WITH &&
    ) WHERE (estado <> 'cancelada'),
    -- Un paciente tampoco puede estar en dos citas al mismo tiempo.
    CONSTRAINT ex_cita_paciente_sin_solape EXCLUDE USING gist (
        id_paciente WITH =,
        tsrange(fecha + hora_inicio, fecha + hora_fin) WITH &&
    ) WHERE (estado <> 'cancelada')
);
```

02_funciones_triggers.sql — control de transiciones de estado

```
CREATE OR REPLACE FUNCTION fn_tg_validar_transicion_cita()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.estado = OLD.estado THEN
        RETURN NEW;
    END IF;

    IF NOT (
        (OLD.estado = 'programada' AND NEW.estado IN
('confirmada','atendida','cancelada','inasistencia')) OR
        (OLD.estado = 'confirmada' AND NEW.estado IN ('atendida','cancelada','inasistencia'))
    ) THEN
        RAISE EXCEPTION 'RF10: transición no permitida de "%" a "%".', OLD.estado, NEW.estado;
    END IF;

    IF NEW.estado IN ('atendida','inasistencia') AND NEW.fecha > CURRENT_DATE THEN
        RAISE EXCEPTION 'RF10: una cita futura no puede marcarse como %.', NEW.estado;
    END IF;
```

```
    RETURN NEW;
END;
$$;
```

Seguridad de la base de datos. La aplicación se conecta con un usuario de PostgreSQL configurado en `backend/config/config.local.php` (fuera del repositorio). Las contraseñas de los usuarios del sistema se guardan solo como hash bcrypt. La zona horaria de la base se fija en America/La_Paz. Se recomienda en producción un usuario con permisos solo sobre la base clinica_vida_salud y copias de seguridad diarias con `pg_dump`.

Población de datos. `04_datos_iniciales.sql` carga los 4 roles, 21 permisos (incluido bitacora.ver), la matriz de permisos por rol, las 5 especialidades y el usuario administrador. `05_datos_prueba.sql` (opcional) carga 7 médicos, sus horarios, 20 pacientes, usuarios de demostración y citas desde seis semanas atrás hasta tres semanas adelante, con estados coherentes con la fecha.

7. FLUJO DE TRABAJO: IMPLEMENTACIÓN

La implementación traduce el diseño en código ejecutable. Cada caso de uso tiene su propio archivo PHP dentro de su paquete, de modo que para mostrar «dónde está» un requisito basta abrir `backend/app/paquetes/<paquete>/casos_uso/CUxx…php`; su pantalla está en `frontend/vistas/<paquete>/`. El código está comentado y sigue las reglas técnicas del examen: PHP con PDO pdo_pgsql, consultas parametrizadas, validación en cliente y servidor, sesiones, errores sin detalles internos y separación entre interfaz, lógica y datos.

7.1 Herramientas de desarrollo de la aplicación web

Tabla 40. *Herramientas de desarrollo*

Herramienta	Versión	Uso en el proyecto
PHP	8.1 o superior (XAMPP 8.2)	Lenguaje del servidor: casos de uso, validación, sesiones.
PDO + pdo_pgsql	incluido en PHP	Conexión a PostgreSQL, sentencias preparadas y transacciones.
Apache HTTP Server (XAMPP)	2.4	Servidor web local; mod_rewrite para URL amigables.
PostgreSQL	14 o superior (probado en 16)	Base de datos relacional; extensión btree_gist.
pgAdmin 4 / psql	—	Administración de la base y ejecución de scripts.
HTML5 y CSS3	—	Estructura y diseño de la interfaz (sin frameworks CSS).
JavaScript (ES6)	—	Validaciones y consulta de horarios con fetch (sin librerías).
Visual Studio Code	1.9x	Editor de código y terminal integrada.
Claude Code	—	Asistente de programación usado desde la terminal de VS Code.
Git y GitHub	2.4x	Control de versiones y publicación del código fuente.
StarUML	6.x	Diagramas de clases del dominio y de diseño (doc/staruml/Clinica_Vida_Salud_Diagramas_Clases.mdj).
Google Chrome (DevTools)	—	Pruebas de la interfaz y depuración.
Fuentes Inter y Lora	OFL	Tipografía incluida en el proyecto (funciona sin internet).

7.2 Implementación de la arquitectura del sistema

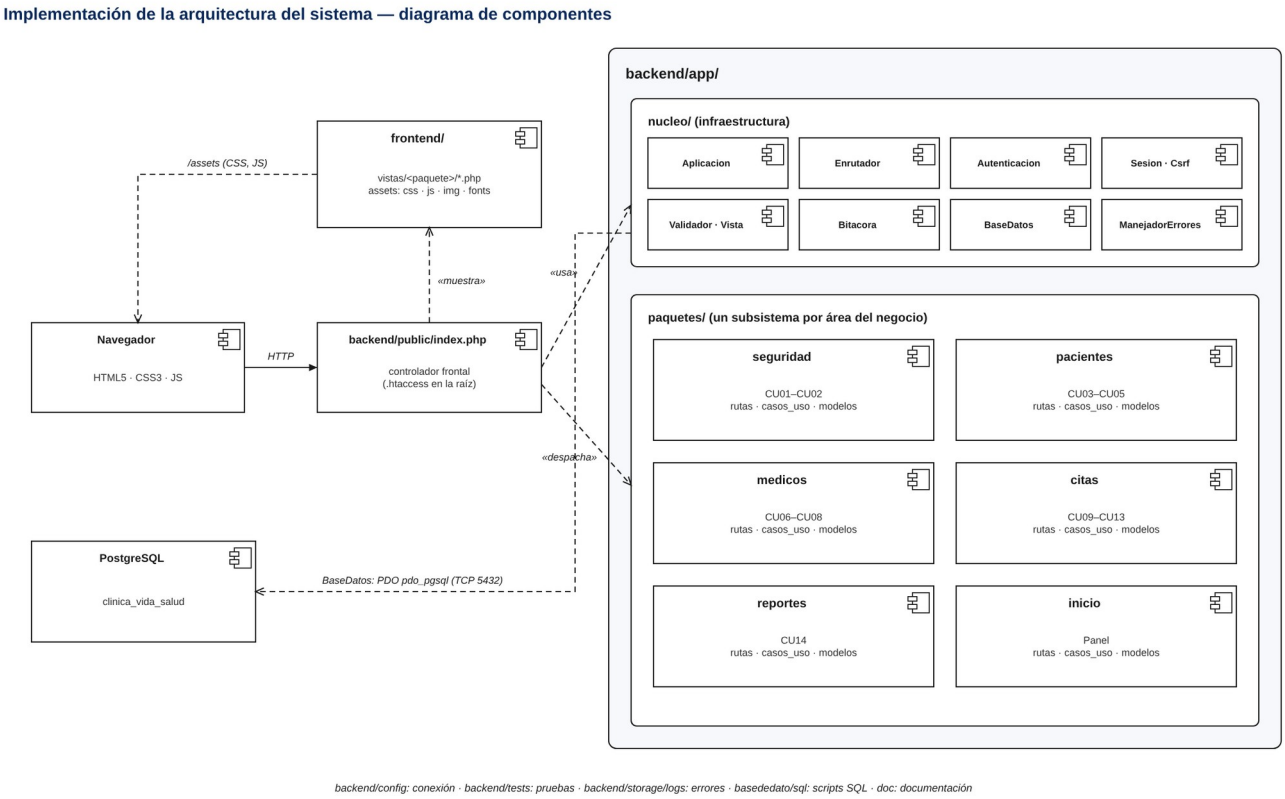

Figura 128. *Diagrama de componentes: implementación de la arquitectura del sistema.*

Estructura del proyecto

Árbol de carpetas

```
clinica-vida-salud/
├── .htaccess                  ← /assets → frontend/assets · todo lo demás → backend/public/index.php
├── README.md   LICENSE        ← guía general y licencia MIT (cualquiera puede modificar el proyecto)
├── backend/                   ← LÓGICA: PHP 8 + PDO pdo_pgsql
│   ├── public/index.php       ← controlador frontal: toda petición entra aquí (router.php: servidor de PHP)
│   ├── app/nucleo/            ← infraestructura común
│   │   ├── Aplicacion.php     (arranque, autocargador, configuración)
│   │   ├── BaseDatos.php      (PDO pdo_pgsql, consultas y transacciones)
│   │   ├── Enrutador.php      (URL → caso de uso, sesión, permiso, CSRF)
│   │   ├── Autenticacion.php  Sesion.php  Csrf.php  Validador.php
│   │   ├── Vista.php  Respuesta.php  Controlador.php
│   │   └── ManejadorErrores.php  ErrorNegocio.php  Peticion.php  ayudantes.php
│   ├── app/paquetes/
│   │   ├── seguridad/   rutas.php · casos_uso/CU01, CU02 · modelos/
│   │   ├── pacientes/   rutas.php · casos_uso/CU03–CU05 · modelos/ · ReglasPaciente.php
│   │   ├── medicos/     rutas.php · casos_uso/CU06–CU08 · modelos/
│   │   ├── citas/       rutas.php · casos_uso/CU09–CU13 · servicios/ · modelos/
│   │   ├── reportes/    rutas.php · casos_uso/CU14 · modelos/
│   │   └── inicio/      rutas.php · casos_uso/PanelPrincipal
│   ├── config/config.php      (conexión; config.local.php para la clave real)
│   ├── storage/logs/          (registro de errores)
│   └── tests/pruebas_sistema.php  (las 10 pruebas del examen)
├── frontend/                  ← INTERFAZ: HTML5, CSS3 y JavaScript
│   ├── vistas/compartido/     (layout, menú, paginación, página de error)
│   ├── vistas/<paquete>/      (una carpeta por paquete, igual que el backend)
│   └── assets/  css/app.css · js/app.js · js/citas.js · img/ · fonts/
├── basededato/sql/            ← DATOS: 01_esquema · 02_funciones_triggers · 03_vistas · 04_datos_iniciales
│                                05_datos_prueba · instalar.sql · instalar_bd.bat · pruebas/ · LEEME.md
└── doc/                       ← documento, diagramas StarUML, capturas, guía de defensa y prompt
```

Controlador frontal. El proyecto se organiza en `backend/` (lógica), `frontend/` (interfaz), `basededato/sql/` (datos) y `doc/`. El `.htaccess` de la raíz publica `frontend/assets` en `/assets` y envía toda otra petición a `backend/public/index.php`, que arranca la aplicación:

.htaccess (raíz del proyecto)

```
# ======================================================================
#  Apache (XAMPP): el proyecto vive en C:\xampp\htdocs\clinica-vida-salud
#  - /assets/...   -> frontend/assets/...      (CSS, JS, imágenes, fuentes)
#  - todo lo demás -> backend/public/index.php (controlador frontal)
#  Las carpetas internas (código, vistas, SQL, documentación) no se
#  pueden abrir desde el navegador.
# ======================================================================
Options -Indexes
DirectoryIndex disabled
<IfModule mod_rewrite.c>
    RewriteEngine On
    RewriteRule ^(backend|frontend|basededato|doc|\.git)(/|$) - [F,L]
    RewriteRule ^assets/(.+)$ frontend/assets/$1 [END]
    RewriteRule ^ backend/public/index.php [QSA,END]
</IfModule>
```

backend/public/index.php

```
<?php
/**
 * Controlador frontal: TODA petición entra por aquí.
 * El enrutador decide qué caso de uso la atiende (ver el archivo rutas.php de cada paquete).
 */
declare(strict_types=1);

require dirname(__DIR__) . '/app/nucleo/Aplicacion.php';

App\Nucleo\Aplicacion::ejecutar();
```

Enrutador. Relaciona cada URL con el caso de uso que la atiende y aplica el control de acceso ANTES de ejecutarlo (RF01, RN12): sin sesión redirige al inicio de sesión (Prueba 9); sin permiso muestra «Acceso denegado»; en todo POST valida el token CSRF.

backend/app/nucleo/Enrutador.php — despachar()

```
    public function despachar(string $metodo, string $ruta): void
    {
        $ruta = rtrim($ruta, '/') ?: '/';
        foreach ($this->rutas as $r) {
            if ($r['metodo'] !== $metodo || !preg_match($r['patron'], $ruta, $coincidencias)) {
                continue;
            }

            // Bitácora (CU02): toda petición POST se registra al terminar, con su resultado.
            if ($metodo === 'POST') {
                $usuarioAntes = Autenticacion::usuario();
                $mensajesAntes = count($_SESSION['flash'] ?? []);
                register_shutdown_function(static fn () => Bitacora::registrarOperacion(
                    $ruta, $r['accion'][0], $r['accion'][1], $usuarioAntes, $mensajesAntes
                ));
            }

            // Control de acceso ANTES de ejecutar el caso de uso (RF01, RN12)
            if ($r['acceso'] !== 'publico') {
                Autenticacion::exigirSesion();
                if ($r['acceso'] !== 'autenticado') {
                    Autenticacion::exigirPermiso($r['acceso']);
                }
```

```
            }
            // Protección CSRF en todo formulario enviado por POST
            if ($metodo === 'POST') {
                Csrf::validar();
            }

            $parametros = array_map('intval', array_filter($coincidencias, 'is_string',
ARRAY_FILTER_USE_KEY));
            [$clase, $accion] = $r['accion'];
            (new $clase())->$accion(...$parametros);
            return;
        }

        http_response_code(404);
        Vista::mostrar('compartido/error', [
            'titulo'  => 'Página no encontrada',
            'codigo'  => 404,
            'mensaje' => 'La dirección que escribió no existe en el sistema.',
        ], Autenticacion::hayUsuario() ? 'principal' : 'simple');
    }
```

Acceso a datos con PDO. Una sola conexión por petición, errores como excepciones, sentencias preparadas sin emulación y transacciones que informan a PostgreSQL qué usuario hace el cambio (lo usa el trigger del historial).

backend/app/nucleo/BaseDatos.php

```
    public static function conexion(): PDO
    {
        if (self::$pdo === null) {
            $c = Aplicacion::config('bd');
            $dsn = sprintf('pgsql:host=%s;port=%d;dbname=%s', $c['host'], $c['puerto'], $c['nombre']);
            self::$pdo = new PDO($dsn, $c['usuario'], $c['clave'], [
                PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES   => false,
            ]);
            // Codificación y zona horaria fijas: no dependen de la configuración de Windows.
            self::$pdo->exec("SET client_encoding TO 'UTF8'");
            self::$pdo->exec("SET TIME ZONE 'America/La_Paz'");
        }
        return self::$pdo;
    }

    /** Ejecuta una sentencia preparada y devuelve el statement. */
    public static function ejecutar(string $sql, array $parametros = []): PDOStatement
    {
        $sentencia = self::conexion()->prepare($sql);
        foreach ($parametros as $clave => $valor) {
            $nombre = is_int($clave) ? $clave + 1 : ':' . ltrim($clave, ':');
            $tipo = match (true) {
                is_int($valor)  => PDO::PARAM_INT,
                is_bool($valor) => PDO::PARAM_BOOL,
                $valor === null => PDO::PARAM_NULL,
                default         => PDO::PARAM_STR,
            };
            $sentencia->bindValue($nombre, $valor, $tipo);
        }
        $sentencia->execute();
        return $sentencia;
    }

    /** @return array<int,array<string,mixed>> todas las filas */
    public static function todos(string $sql, array $parametros = []): array
    {
        return self::ejecutar($sql, $parametros)->fetchAll();
    }
```

```
    /** @return array<string,mixed>|null una fila o null */
    public static function uno(string $sql, array $parametros = []): ?array
    {
        $fila = self::ejecutar($sql, $parametros)->fetch();
        return $fila === false ? null : $fila;
    }

    /** Devuelve el valor de la primera columna de la primera fila. */
    public static function valor(string $sql, array $parametros = []): mixed
    {
        $v = self::ejecutar($sql, $parametros)->fetchColumn();
        return $v === false ? null : $v;
    }

    /**
     * Ejecuta $operacion dentro de una transacción.
     * Informa a PostgreSQL qué usuario hace el cambio (lo usa el trigger
     * del historial de citas) y un detalle opcional.
     */
    public static function transaccion(callable $operacion, ?string $detalle = null): mixed
    {
        $pdo = self::conexion();
        $pdo->beginTransaction();
        try {
            $usuario = Autenticacion::id();
            self::ejecutar("SELECT set_config('app.id_usuario', :u, true)", ['u' => $usuario ?
(string) $usuario : '']);
            self::ejecutar("SELECT set_config('app.detalle', :d, true)", ['d' => $detalle ?? '']);
            $resultado = $operacion();
            $pdo->commit();
            return $resultado;
        } catch (Throwable $e) {
            if ($pdo->inTransaction()) {
                $pdo->rollBack();
            }
            throw $e;
        }
    }
```

Manejo de errores. Los errores de PostgreSQL se traducen a mensajes claros (por ejemplo, la restricción `ex_cita_medico_sin_solape` se muestra como «RN07: el médico ya tiene una cita en ese horario»); los errores inesperados se registran en `backend/storage/logs/errores.log` y el usuario solo ve un mensaje genérico, sin credenciales ni SQL.

backend/app/nucleo/ManejadorErrores.php — traducir()

```
    /** Nombre de restricción de la BD -> mensaje para el usuario. */
    private const MENSAJES = [
        'uq_paciente_ci'             => 'Ya existe un paciente registrado con ese CI.',
        'uq_usuario_nombre'          => 'Ese nombre de usuario ya está en uso.',
        'uq_usuario_correo'          => 'Ese correo ya está registrado en otra cuenta.',
        'uq_usuario_medico'          => 'Ese médico ya tiene una cuenta de usuario.',
        'uq_usuario_paciente'        => 'Ese paciente ya tiene una cuenta de usuario.',
        'uq_medico_registro'         => 'Ya existe un médico con ese número de registro profesional.',
        'uq_medico_correo'           => 'Ya existe un médico con ese correo.',
        'uq_especialidad_nombre_normalizado' => 'Ya existe una especialidad con ese nombre.',
        'uq_dia_no_laborable'        => 'Esa fecha ya está bloqueada.',
        'ex_cita_medico_sin_solape'  => 'RN07: el médico ya tiene una cita en ese horario. Elija otro
horario.',
        'ex_cita_paciente_sin_solape'=> 'El paciente ya tiene otra cita en ese mismo horario.',
        'ex_horario_sin_solape'      => 'El bloque se superpone con otro horario activo del médico ese
día.',
        'ck_horario_rango'           => 'La hora final debe ser posterior a la hora inicial.',
        'ck_cita_rango'              => 'La hora final de la cita debe ser posterior a la inicial.',
        'fk_cita_medico_especialidad'=> 'El médico no atiende la especialidad elegida.',
```

```
    ];

    public static function registrar(): void
    {
        $depurar = (bool) Aplicacion::config('app.depuracion', false);
        ini_set('display_errors', $depurar ? '1' : '0');
        error_reporting(E_ALL);
        set_exception_handler([self::class, 'manejar']);
        set_error_handler(static function (int $nivel, string $msg, string $archivo, int $linea): bool
{
            if (!(error_reporting() & $nivel)) {
                return false;            // error silenciado con @
            }
            throw new \ErrorException($msg, 0, $nivel, $archivo, $linea);
        });
    }

    /** Convierte un error de PostgreSQL en ErrorNegocio con mensaje claro. */
    public static function traducir(Throwable $e): ErrorNegocio
    {
        if ($e instanceof ErrorNegocio) {
            return $e;
        }
        if ($e instanceof PDOException) {
            $estado = $e->errorInfo[0] ?? (string) $e->getCode();
            $texto = $e->getMessage();
            foreach (self::MENSAJES as $restriccion => $mensaje) {
                if (str_contains($texto, $restriccion)) {
                    return new ErrorNegocio($mensaje);
                }
            }
            // Mensajes de los triggers: RAISE EXCEPTION 'RNxx: ...'
            if ($estado === 'P0001' && preg_match('/ERROR:\s+(.+?)(\n|CONTEXT|$)/s', $texto, $m)) {
                return new ErrorNegocio(trim($m[1]));
            }
            return match ($estado) {
                '23503' => new ErrorNegocio('No se puede completar: el registro está relacionado con
otros datos.'),
                '23505' => new ErrorNegocio('Ya existe un registro con esos datos.'),
                '23514', '22007', '22008', '22P02' => new ErrorNegocio('Algún dato no tiene el formato
correcto.'),
                default => self::registrarYGenerico($e),
            };
        }
        return self::registrarYGenerico($e);
    }
```

Rutas de un paquete. Cada paquete declara sus URL, el caso de uso que las atiende y el permiso exigido:

backend/app/paquetes/citas/rutas.php

```
<?php
/**
 * Rutas del paquete CITAS (CU09 a CU13).
 */
use App\Nucleo\Enrutador;
use App\Paquetes\Citas\CasosUso\CU09ProgramarCita;
use App\Paquetes\Citas\CasosUso\CU10ReprogramarCita;
use App\Paquetes\Citas\CasosUso\CU11CancelarCita;
use App\Paquetes\Citas\CasosUso\CU12RegistrarAsistencia;
use App\Paquetes\Citas\CasosUso\CU13ConsultarAgendaMedica;

return static function (Enrutador $r): void {
    // CU09 · Programar cita (recepción y paciente en línea) + consultar disponibilidad
    $r->get('/citas/nueva',  [CU09ProgramarCita::class, 'formulario'],         'citas.programar');
    $r->post('/citas',       [CU09ProgramarCita::class, 'registrar'],          'citas.programar');
    $r->get('/reservar',     [CU09ProgramarCita::class, 'formularioPaciente'], 'portal.reservar');
    $r->post('/reservar',    [CU09ProgramarCita::class, 'registrarPaciente'],  'portal.reservar');
    $r->get('/api/medicos',               [CU09ProgramarCita::class, 'medicosJson'], ['citas.programar',
'portal.reservar']);
```

```
    $r->get('/api/disponibilidad/dias',    [CU09ProgramarCita::class, 'diasJson'],    ['citas.programar',
'citas.reprogramar', 'portal.reservar']);
    $r->get('/api/disponibilidad',         [CU09ProgramarCita::class, 'horariosJson'],['citas.programar',
'citas.reprogramar', 'portal.reservar']);

    // CU10 · Reprogramar cita
    $r->get('/citas/{id}/reprogramar',  [CU10ReprogramarCita::class, 'formulario'], 'citas.reprogramar');
    $r->post('/citas/{id}/reprogramar', [CU10ReprogramarCita::class, 'guardar'],    'citas.reprogramar');

    // CU11 · Cancelar cita
    $r->post('/citas/{id}/cancelar',     [CU11CancelarCita::class, 'cancelar'],         'citas.cancelar');
    $r->post('/mis-citas/{id}/cancelar', [CU11CancelarCita::class, 'cancelarPaciente'], 'portal.mis_citas');

    // CU12 · Registrar asistencia / estado de la cita
    $r->post('/citas/{id}/estado', [CU12RegistrarAsistencia::class, 'cambiar'], ['citas.confirmar',
'citas.atender']);

    // CU13 · Consultar agenda médica y citas
    $r->get('/agenda',     [CU13ConsultarAgendaMedica::class, 'agenda'],  ['agenda.ver', 'agenda.propia']);
    $r->get('/agenda/semana', [CU13ConsultarAgendaMedica::class, 'agendaSemanal'], ['agenda.ver', 'agenda.propia']);
    $r->get('/citas',      [CU13ConsultarAgendaMedica::class, 'listado'], 'citas.ver');
    $r->get('/citas/{id}', [CU13ConsultarAgendaMedica::class, 'ver'],     ['citas.ver', 'agenda.propia',
'portal.mis_citas']);
};
```

7.3 Implementación de la arquitectura de los subsistemas

Cada paquete es un subsistema con la misma estructura: `rutas.php` (URL y permisos), `casos_uso/` (control), `modelos/` (acceso a datos) y, en citas, `servicios/` con las reglas de disponibilidad y de estados. Sus pantallas están en `frontend/vistas/<paquete>/`.

7.3.1 Subsistema seguridad

Acceso, usuarios, roles, permisos y bitácora.

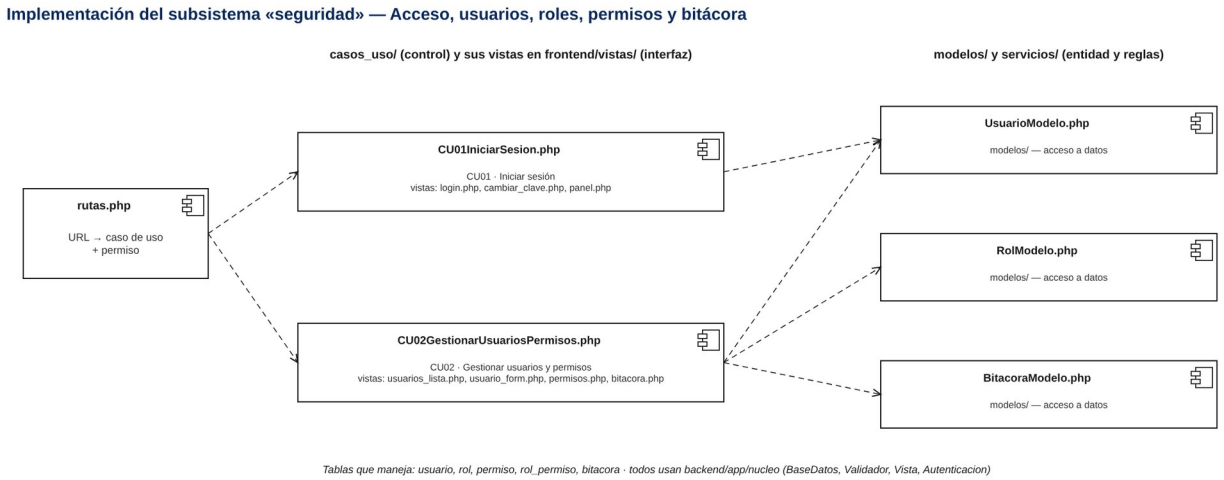

Figura 129. *Implementación del subsistema seguridad.*

Tabla 41. *Implementación de los casos de uso — seguridad*

CU	Archivo	Métodos públicos	Modelos
CU01	CU01IniciarSesion.php	formulario(), iniciar(), cerrar(), formularioClave(), cambiarClave()	UsuarioModelo.php
CU02	CU02GestionarUsuariosPermisos.php	listar(), nuevo(), editar(), crear(), actualizar(), cambiarEstado(), permisos(), guardarPermisos(),	UsuarioModelo.php, RolModelo.php, BitacoraModelo.php

CU	Archivo	Métodos públicos	Modelos
		bitacora()	

CU01 — iniciar(): password_verify, bloqueo y sesión segura

```
    /**
     * Hash ficticio: si el usuario no existe igual se ejecuta
     * password_verify(), así el tiempo de respuesta no revela
     * qué usuarios existen.
     */
    private const HASH_FICTICIO = '$2y$10$iRnbJMJ7z9wdzYQpMF33TeR.1L1s2ge1aTzeByX6.j5famAgwMlxW';

    private UsuarioModelo $usuarios;

    public function __construct()
    {
        $this->usuarios = new UsuarioModelo();
    }

    /** GET /login */
    public function formulario(): void
    {
        if (Autenticacion::hayUsuario()) {
            $this->redirigir('/panel');
        }
        \App\Nucleo\Vista::mostrar('seguridad/login', ['titulo' => 'Iniciar sesión'], 'simple');
    }

    /** POST /login */
    public function iniciar(): void
    {
        try {
            $datos = Validador::validar($_POST, [
                'usuario' => ['Usuario o correo', 'requerido|texto:3:120'],
                'clave'   => ['Contraseña',       'requerido|texto:1:64'],
            ]);

            $cuenta = $this->usuarios->buscarParaLogin($datos['usuario']);

            // Paso 3: verificación de la contraseña (siempre se ejecuta)
            $claveCorrecta = password_verify($datos['clave'], $cuenta['password_hash'] ?? self::HASH_FICTICIO);

            // Bitácora: a nombre de la cuenta encontrada o, si no existe, de lo que se escribió.
            $quien = ['id_usuario' => $cuenta['id_usuario'] ?? null, 'nombre_usuario' =>
$cuenta['nombre_usuario'] ?? $datos['usuario']];
            if ($cuenta && $cuenta['esta_bloqueado']) {
                Bitacora::registrar('cuenta_bloqueada', 'seguridad', 'Intento de ingreso con la cuenta bloqueada.',
$quien);
                throw new ErrorNegocio(sprintf(
                    'La cuenta está bloqueada por varios intentos fallidos. Intente de nuevo en %d minuto(s).',
                    max(1, (int) $cuenta['minutos_restantes'])
                ));
            }
            if (!$cuenta || !$claveCorrecta) {
                Bitacora::registrar('inicio_fallido', 'seguridad',
                    $cuenta ? 'Contraseña incorrecta.' : 'El usuario o correo escrito no existe.', $quien);
                if ($cuenta && $this->usuarios->registrarIntentoFallido((int) $cuenta['id_usuario'])) {
                    Bitacora::registrar('cuenta_bloqueada', 'seguridad', sprintf(
                        'Cuenta bloqueada %d minuto(s) por %d intentos fallidos.',
                        (int) \App\Nucleo\Aplicacion::config('seguridad.minutos_bloqueo', 15),
                        (int) \App\Nucleo\Aplicacion::config('seguridad.max_intentos', 5)
                    ), $quien);
                }
                // Mismo mensaje en ambos casos: no se revela si el usuario existe.
                throw new ErrorNegocio('Usuario o contraseña incorrectos.');
            }

            // Paso 4: la cuenta (y su médico o paciente vinculado) debe estar activa.
            if ($cuenta['estado'] !== 'activo'
                || ($cuenta['estado_medico'] ?? 'activo') !== 'activo'
                || ($cuenta['estado_paciente'] ?? 'activo') !== 'activo') {
                Bitacora::registrar('inicio_fallido', 'seguridad', 'Cuenta inactiva.', $quien);
                throw new ErrorNegocio('Su cuenta está inactiva. Comuníquese con la administración de la clínica.');
            }

            // Si PHP mejoró su algoritmo, se actualiza el hash sin molestar al usuario.
```

```
            if (password_needs_rehash($cuenta['password_hash'], PASSWORD_DEFAULT)) {
                $this->usuarios->cambiarClave((int) $cuenta['id_usuario'], password_hash($datos['clave'],
PASSWORD_DEFAULT));
            }

            // Paso 5: sesión segura
            $this->usuarios->registrarAccesoExitoso((int) $cuenta['id_usuario']);
            Autenticacion::iniciar((int) $cuenta['id_usuario']);
            Bitacora::registrar('inicio_sesion', 'seguridad', 'Ingresó al sistema.');

            $destino = Sesion::obtener('volver_a', '/panel');
            Sesion::quitar('volver_a');
            $this->redirigir(is_string($destino) && str_starts_with($destino, '/') ? $destino : '/panel');
        } catch (Throwable $e) {
            $this->volverConError($e, '/login');
        }
    }
```

7.3.2 Subsistema pacientes

Registro de pacientes, perfil e historial.

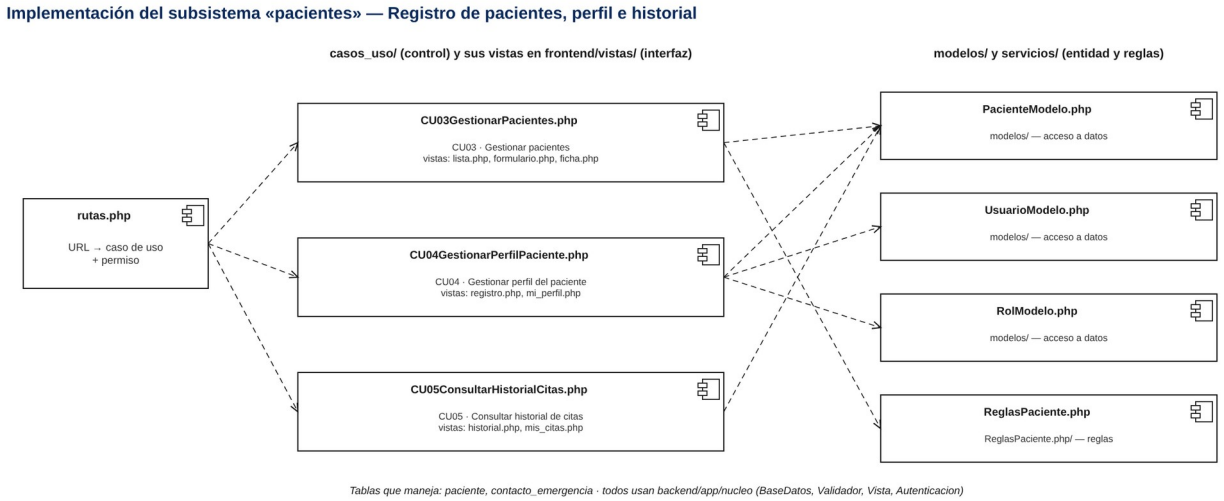

Figura 130. *Implementación del subsistema pacientes.*

Tabla 42. *Implementación de los casos de uso — pacientes*

CU	Archivo	Métodos públicos	Modelos
CU03	CU03GestionarPacientes.php	listar(), ver(), nuevo(), editar(), crear(), actualizar(), cambiarEstado(), buscarJson()	PacienteModelo.php
CU04	CU04GestionarPerfilPaciente.php	formularioRegistro(), registrar(), ver(), guardarDatos(), guardarMedica(), guardarEmergencia(), guardarPreferencias()	PacienteModelo.php, UsuarioModelo.php, RolModelo.php
CU05	CU05ConsultarHistorialCitas.php	historialPaciente(), misCitas()	PacienteModelo.php

CU03 — crear(): CI único y registro en transacción

```
    /** GET /pacientes */
    public function listar(): void
    {
        $buscar = Peticion::entrada('q');
```

```
        $estado = Peticion::entrada('estado');
        $pagina = max(1, Peticion::entero('pagina') ?? 1);
        $r = $this->pacientes->buscar($buscar, $estado, $pagina);
        $this->vista('pacientes/lista', [
            'titulo'    => 'Pacientes',
            'pacientes' => $r['filas'],
            'total'     => $r['total'],
            'pagina'    => $pagina,
            'paginas'   => max(1, (int) ceil($r['total'] / PacienteModelo::POR_PAGINA)),
            'filtros'   => compact('buscar', 'estado'),
            'puedeGestionar' => $this->puede('pacientes.gestionar'),
        ]);
    }

    /** GET /pacientes/{id} — ficha del paciente */
    public function ver(int $id): void
    {
        $p = $this->pacientes->obtener($id) ?? $this->noEncontrado('El paciente');
        $this->vista('pacientes/ficha', [
            'titulo'    => $p['nombres'] . ' ' . $p['apellidos'],
            'paciente'  => $p,
            'contactos' => $this->pacientes->contactos($id),
            'resumen'   => $this->pacientes->resumenEstados($id),
            'recientes' => array_slice($this->pacientes->historial($id), 0, 5),
            'puedeGestionar' => $this->puede('pacientes.gestionar'),
        ]);
    }

    /** GET /pacientes/nuevo */
    public function nuevo(): void
    {
        $this->vista('pacientes/formulario', ['titulo' => 'Registrar paciente', 'paciente' => null, 'contacto' =>
null]);
    }

    /** GET /pacientes/{id}/editar */
    public function editar(int $id): void
    {
        $p = $this->pacientes->obtener($id) ?? $this->noEncontrado('El paciente');
        $this->vista('pacientes/formulario', [
            'titulo'   => 'Modificar paciente',
            'paciente' => $p,
            'contacto' => $this->pacientes->contactos($id)[0] ?? null,
        ]);
    }

    /** POST /pacientes — Prueba 1 (registrar) y Prueba 2 (CI duplicado) */
    public function crear(): void
    {
        try {
            $d = ReglasPaciente::datosPersonales($_POST, false) + ReglasPaciente::informacionMedica($_POST);
            $contacto = ReglasPaciente::contactoEmergencia($_POST);
            if ($d['ci']) {
                ReglasPaciente::exigirCiUnico($this->pacientes->existeCi($d['ci']));
            }
            $id = BaseDatos::transaccion(function () use ($d, $contacto) {
                $id = $this->pacientes->crear($d);
                $this->pacientes->guardarContactoPrincipal($id, $contacto);
                return $id;
            });
            $p = $this->pacientes->obtener($id);
            $this->exito("Paciente {$p['codigo']} · {$p['nombres']} {$p['apellidos']} registrado.",
"/pacientes/{$id}");
        } catch (Throwable $e) {
            $this->volverConError($e, '/pacientes/nuevo');
        }
    }
```

7.3.3 Subsistema medicos

Médicos, especialidades y horarios de atención.

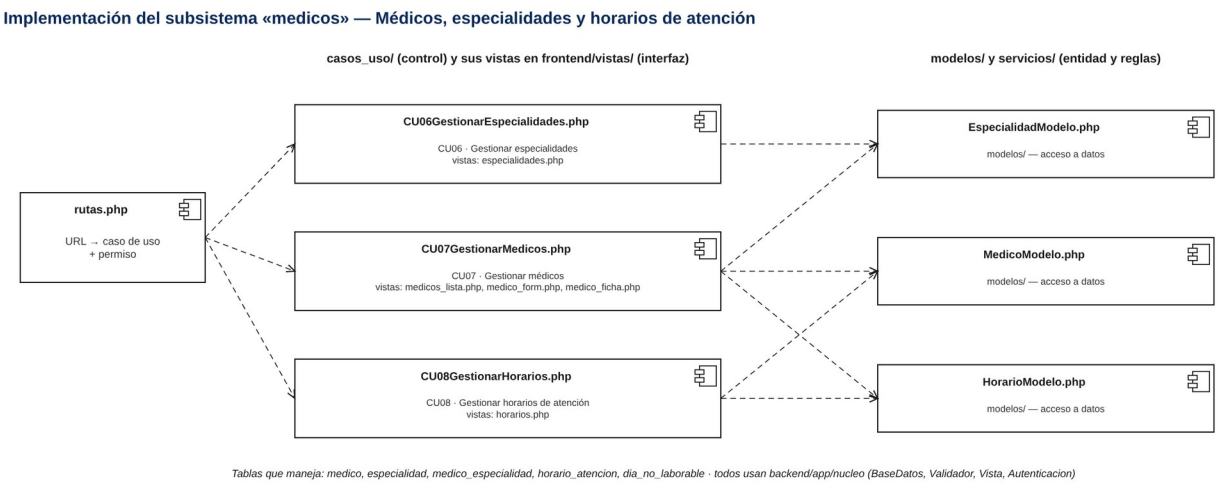

Figura 131. *Implementación del subsistema medicos.*

Tabla 43. *Implementación de los casos de uso — medicos*

CU	Archivo	Métodos públicos	Modelos
CU06	CU06GestionarEspecialidades.php	listar(), crear(), actualizar(), cambiarEstado()	EspecialidadModelo.php
CU07	CU07GestionarMedicos.php	listar(), ver(), nuevo(), editar(), crear(), actualizar(), cambiarEstado()	MedicoModelo.php, EspecialidadModelo.php, HorarioModelo.php
CU08	CU08GestionarHorarios.php	mostrar(), crear(), actualizar(), cambiarEstado(), agregarFeriado(), quitarFeriado()	HorarioModelo.php, MedicoModelo.php

CU08 — validar(): reglas del horario de atención

```
    /** GET /horarios?medico=ID */
    public function mostrar(): void
    {
        $lista = $this->medicos->listaSimple();
        $idMedico = Peticion::entero('medico') ?? (int) ($lista[0]['id_medico'] ?? 0);
        $medico = $idMedico ? $this->medicos->obtener($idMedico) : null;

        $this->vista('medicos/horarios', [
            'titulo'    => 'Horarios de atención',
            'medicos'   => $lista,
            'medico'    => $medico,
            'bloques'   => $medico ? $this->horarios->delMedico($idMedico) : [],
            'feriados'  => $this->horarios->diasNoLaborables($medico ? $idMedico : null),
        ]);
    }

    /** POST /horarios — agregar bloque */
    public function crear(): void
    {
        $idMedico = Peticion::entero('id_medico') ?? 0;
        try {
            $d = $this->validar();
            $d['id_medico'] = $idMedico;
            $medico = $this->medicos->obtener($idMedico) ?? throw new ErrorNegocio('Seleccione un médico válido.');
            if ($medico['estado'] !== 'activo') {
                throw new ErrorNegocio('El médico está inactivo: actívelo antes de asignarle horarios.');
            }
            if ($this->horarios->haySolape($idMedico, $d['dia_semana'], $d['hora_inicio'], $d['hora_fin'])) {
                throw new ErrorNegocio('El bloque se superpone con otro horario del médico ese día.');
            }
            BaseDatos::transaccion(fn () => $this->horarios->crear($d));
            $this->exito('Horario agregado: ' . DIAS_SEMANA[$d['dia_semana']] . " de {$d['hora_inicio']} a
```

```
{$d['hora_fin']}.", "/horarios?medico={$idMedico}");
        } catch (Throwable $e) {
            $this->volverConError($e, "/horarios?medico={$idMedico}");
        }
    }

    /** POST /horarios/{id} — modificar bloque */
    public function actualizar(int $id): void
    {
        $bloque = $this->horarios->obtener($id);
        $idMedico = (int) ($bloque['id_medico'] ?? 0);
        try {
            $bloque ?? throw new ErrorNegocio('El horario no existe.');
            $d = $this->validar();
            if ($this->horarios->haySolape($idMedico, $d['dia_semana'], $d['hora_inicio'], $d['hora_fin'], $id)) {
                throw new ErrorNegocio('El bloque se superpone con otro horario del médico ese día.');
            }
            if (($n = $this->horarios->citasQueQuedanFuera($bloque, $d['hora_inicio'], $d['hora_fin'],
$d['dia_semana'])) > 0) {
                throw new ErrorNegocio("El cambio deja fuera {$n} cita(s) pendiente(s). Reprográmelas antes de
modificar el horario.");
            }
            BaseDatos::transaccion(fn () => $this->horarios->actualizar($id, $d));
            $this->exito('Horario actualizado.', "/horarios?medico={$idMedico}");
        } catch (Throwable $e) {
            $this->volverConError($e, "/horarios?medico={$idMedico}");
        }
    }

    /** POST /horarios/{id}/estado — activar / desactivar bloque */
    public function cambiarEstado(int $id): void
    {
        $bloque = $this->horarios->obtener($id);
        $idMedico = (int) ($bloque['id_medico'] ?? 0);
        try {
            $bloque ?? throw new ErrorNegocio('El horario no existe.');
            $nuevo = $bloque['estado'] === 'activo' ? 'inactivo' : 'activo';
            if ($nuevo === 'inactivo' && ($n = $this->horarios->citasFuturasEnBloque($bloque)) > 0) {
                throw new ErrorNegocio("No se puede desactivar: hay {$n} cita(s) pendiente(s) en este horario.
Reprográmelas primero.");
            }
            if ($nuevo === 'activo' && $this->horarios->haySolape($idMedico, (int) $bloque['dia_semana'],
$bloque['hora_inicio'], $bloque['hora_fin'], $id)) {
                throw new ErrorNegocio('No se puede activar: se superpone con otro horario activo.');
            }
            BaseDatos::transaccion(fn () => $this->horarios->cambiarEstado($id, $nuevo));
            $this->exito($nuevo === 'activo' ? 'Horario activado.' : 'Horario desactivado.', "/horarios?
medico={$idMedico}");
        } catch (Throwable $e) {
            $this->volverConError($e, "/horarios?medico={$idMedico}");
        }
    }

    /** POST /feriados — bloquear una fecha (toda la clínica o un médico) */
    public function agregarFeriado(): void
    {
        $idMedicoVolver = Peticion::entero('volver_medico');
        try {
            $d = Validador::validar($_POST, [
                'fecha'       => ['Fecha', 'requerido|fecha|no_pasada'],
                'descripcion' => ['Descripción', 'requerido|texto:3:120'],
                'id_medico'   => ['Médico', 'opcional|entero:1'],
            ]);
            if (($n = $this->horarios->citasEnFecha($d['fecha'], $d['id_medico'])) > 0) {
                throw new ErrorNegocio("Hay {$n} cita(s) pendiente(s) ese día. Reprográmelas o cancélelas antes de
bloquear la fecha.");
            }
            BaseDatos::transaccion(fn () => $this->horarios->agregarDiaNoLaborable($d['fecha'], $d['id_medico'],
$d['descripcion']));
            $this->exito('Fecha bloqueada: ' . fecha_corta($d['fecha']) . '.', '/horarios' . ($idMedicoVolver ? "?
medico={$idMedicoVolver}" : ''));
        } catch (Throwable $e) {
            $this->volverConError($e, '/horarios' . ($idMedicoVolver ? "?medico={$idMedicoVolver}" : ''));
        }
    }

    /** POST /feriados/{id}/quitar */
    public function quitarFeriado(int $id): void
    {
```

```
        $idMedicoVolver = Peticion::entero('volver_medico');
        BaseDatos::transaccion(fn () => $this->horarios->quitarDiaNoLaborable($id));
        $this->exito('La fecha volvió a estar disponible.', '/horarios' . ($idMedicoVolver ? "?
medico={$idMedicoVolver}" : ''));
    }

    /** Validación del bloque horario (RF05). */
    private function validar(): array
    {
        $d = Validador::validar($_POST, [
            'dia_semana'    => ['Día', 'requerido|entero:1:7'],
            'hora_inicio'   => ['Hora inicial', 'requerido|hora'],
            'hora_fin'      => ['Hora final', 'requerido|hora'],
            'duracion_cita' => ['Duración de cada cita', 'requerido|en:15,20,30,45,60'],
        ]);
        $d['duracion_cita'] = (int) $d['duracion_cita'];

        if ($d['hora_fin'] <= $d['hora_inicio']) {
            throw new ErrorNegocio('Revise los datos.', ['hora_fin' => 'La hora final debe ser posterior a la hora
inicial.']);
        }
        if ($d['hora_inicio'] < '06:00' || $d['hora_fin'] > '22:00') {
            throw new ErrorNegocio('Revise los datos.', ['hora_inicio' => 'La clínica atiende entre 06:00 y
22:00.']);
        }
        $minutos = (strtotime($d['hora_fin']) - strtotime($d['hora_inicio'])) / 60;
        if ($minutos < $d['duracion_cita']) {
            throw new ErrorNegocio('Revise los datos.', ['duracion_cita' => 'El bloque es más corto que la duración
de una cita.']);
        }
        return $d;
    }
```

7.3.4 Subsistema citas

Programación, cambios, estados y agenda de citas.

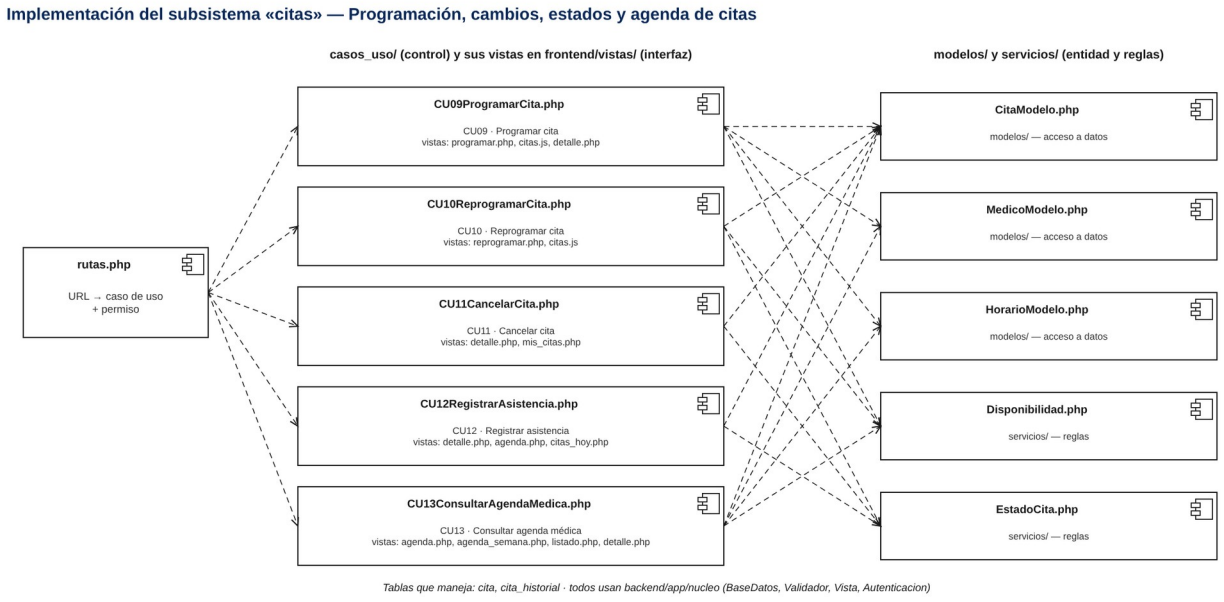

Figura 132. *Implementación del subsistema citas.*

Tabla 44. *Implementación de los casos de uso — citas*

CU	Archivo	Métodos públicos	Modelos
CU09	CU09ProgramarCita.php	formulario(), formularioPaciente(), registrar(), registrarPaciente(), guardar(),	CitaModelo.php, MedicoModelo.php,

CU	Archivo	Métodos públicos	Modelos
		medicosJson(), diasJson(), horariosJson()	HorarioModelo.php
CU10	CU10ReprogramarCita.php	formulario(), guardar(), reprogramar()	CitaModelo.php
CU11	CU11CancelarCita.php	cancelar(), cancelarPaciente(), ejecutar()	CitaModelo.php
CU12	CU12RegistrarAsistencia.php	cambiar(), ejecutar()	CitaModelo.php
CU13	CU13ConsultarAgendaMedica.php	agenda(), agendaSemanal(), listado(), ver()	CitaModelo.php, MedicoModelo.php, HorarioModelo.php

CU09 — guardar(): verificación en el servidor dentro de una transacción

```
    /** GET /citas/nueva (recepción) */
    public function formulario(): void
    {
        $paciente = null;
        if ($id = Peticion::entero('paciente')) {
            $paciente = (new PacienteModelo())->obtener($id);
            if ($paciente && $paciente['estado'] !== 'activo') {
                $paciente = null;
            }
        }
        $this->vista('citas/programar', [
            'titulo'         => 'Programar cita',
            'modo'           => 'recepcion',
            'paciente'       => $paciente,
            'especialidades' => (new EspecialidadModelo())->conMedicosActivos(),
            'accion'         => url('/citas'),
            'scripts'        => ['citas.js'],
        ]);
    }

    /** GET /reservar (paciente en línea) */
    public function formularioPaciente(): void
    {
        $this->vista('citas/programar', [
            'titulo'         => 'Reservar cita',
            'modo'           => 'portal',
            'paciente'       => (new PacienteModelo())->obtener((int) $this->usuario()['id_paciente']),
            'especialidades' => (new EspecialidadModelo())->conMedicosActivos(),
            'accion'         => url('/reservar'),
            'scripts'        => ['citas.js'],
        ]);
    }

    /** POST /citas — Prueba 3 (horario disponible), 4 (superpuesta), 5 (fuera de horario) */
    public function registrar(): void
    {
        try {
            $cita = $this->guardar(Peticion::entero('id_paciente') ?? 0);
            $this->exito("Cita {$cita['codigo']} registrada.", "/citas/{$cita['id_cita']}");
        } catch (Throwable $e) {
            $this->volverConError($e, '/citas/nueva' . (Peticion::entero('id_paciente') ? '?paciente=' .
Peticion::entero('id_paciente') : ''));
        }
    }

    /** POST /reservar — el paciente solo puede reservar para sí mismo */
    public function registrarPaciente(): void
    {
        try {
            $cita = $this->guardar((int) $this->usuario()['id_paciente']);
            $this->exito("Su cita {$cita['codigo']} quedó reservada. Le esperamos.", '/mis-citas');
        } catch (Throwable $e) {
            $this->volverConError($e, '/reservar');
        }
    }

    /**
```

```
     * Lógica común de registro (validación + transacción).
     * @return array{id_cita:int,codigo:string}
     */
    public function guardar(int $idPaciente): array
    {
        $d = Validador::validar($_POST + ['id_paciente' => $idPaciente], [
            'id_paciente'     => ['Paciente', 'requerido|entero:1'],
            'id_especialidad' => ['Especialidad', 'requerido|entero:1'],
            'id_medico'       => ['Médico', 'requerido|entero:1'],
            'fecha'           => ['Fecha', 'requerido|fecha|no_pasada'],
            'hora_inicio'     => ['Hora', 'requerido|hora'],
            'modalidad'       => ['Modalidad', 'requerido|en:presencial,virtual'],
            'motivo'          => ['Motivo', 'opcional|texto:3:250'],
        ]);
        $d['id_paciente'] = $idPaciente ?: $d['id_paciente'];

        // RN03: paciente y médico válidos y activos; el médico atiende la especialidad.
        $paciente = (new PacienteModelo())->obtener($d['id_paciente']);
        if (!$paciente || $paciente['estado'] !== 'activo') {
            throw new ErrorNegocio('Revise los datos.', ['id_paciente' => 'Seleccione un paciente activo.']);
        }
        $medico = (new MedicoModelo())->obtener($d['id_medico']);
        if (!$medico || !in_array($d['id_especialidad'], $medico['ids_especialidades'], true)) {
            throw new ErrorNegocio('El médico elegido no atiende esa especialidad.');
        }
        if ($medico['estado'] !== 'activo') {
            throw new ErrorNegocio('RN04: el médico está inactivo y no puede recibir nuevas citas.');
        }
        $especialidad = (new EspecialidadModelo())->obtener($d['id_especialidad']);
        if (!$especialidad || $especialidad['estado'] !== 'activo') {
            throw new ErrorNegocio('La especialidad elegida no está disponible.');
        }

        return BaseDatos::transaccion(function () use ($d) {
            $this->citas->bloquearMedico($d['id_medico']);
            // RN13: se vuelve a comprobar la disponibilidad dentro de la transacción.
            $d['hora_fin'] = $this->disponibilidad->verificar($d['id_medico'], $d['fecha'], $d['hora_inicio']);
            if ($otra = $this->citas->pacienteOcupado($d['id_paciente'], $d['fecha'], $d['hora_inicio'],
$d['hora_fin'])) {
                throw new ErrorNegocio("El paciente ya tiene la cita {$otra['codigo']} a esa misma hora.");
            }
            $d['id_usuario'] = \App\Nucleo\Autenticacion::id();
            return $this->citas->crear($d);
        }, 'Cita registrada');
    }
```

Disponibilidad — delDia(): cálculo de horarios disponibles

```
    /**
     * Intervalos del día con su situación.
     * @return array{bloqueo:?string, intervalos:array<int,array{inicio:string,fin:string,libre:bool}>}
     */
    public function delDia(int $idMedico, string $fecha, ?int $excluirCita = null): array
    {
        $medico = $this->medicos->obtener($idMedico);
        if (!$medico || $medico['estado'] !== 'activo') {
            return ['bloqueo' => 'El médico está inactivo y no recibe nuevas citas.', 'intervalos' => []];
        }
        $hoy = date('Y-m-d');
        if ($fecha < $hoy) {
            return ['bloqueo' => 'No se ofrecen citas en fechas pasadas.', 'intervalos' => []];
        }
        $limite = date('Y-m-d', strtotime('+' . (int) Aplicacion::config('citas.dias_maximos_reserva', 60) . '
days'));
        if ($fecha > $limite) {
            return ['bloqueo' => 'Solo se reservan citas hasta el ' . fecha_corta($limite) . '.', 'intervalos' =>
[]];
        }
        $feriado = $this->horarios->fechasBloqueadas($idMedico, $fecha, $fecha)[$fecha] ?? null;
        if ($feriado) {
            return ['bloqueo' => "Fecha bloqueada: {$feriado}.", 'intervalos' => []];
        }

        $bloques = $this->horarios->activosDelDia($idMedico, (int) date('N', strtotime($fecha)));
        if (!$bloques) {
            return ['bloqueo' => 'El médico no atiende ese día.', 'intervalos' => []];
        }
```

```
        $ocupadas = $this->citas->ocupadas($idMedico, $fecha, $excluirCita);
        $ahora = date('H:i');
        $intervalos = [];
        foreach ($bloques as $b) {
            $paso = (int) $b['duracion_cita'] * 60;
            $fin = strtotime($b['hora_fin']);
            for ($t = strtotime($b['hora_inicio']); $t + $paso <= $fin; $t += $paso) {
                $ini = date('H:i', $t);
                $fi = date('H:i', $t + $paso);
                if ($fecha === $hoy && $ini <= $ahora) {
                    continue; // RN06: ya pasó
                }
                $libre = true;
                foreach ($ocupadas as $c) {
                    if (substr($c['hora_inicio'], 0, 5) < $fi && substr($c['hora_fin'], 0, 5) > $ini) {
                        $libre = false;
                        break;
                    }
                }
                $intervalos[] = ['inicio' => $ini, 'fin' => $fi, 'libre' => $libre];
            }
        }
        return ['bloqueo' => $intervalos ? null : 'Ya no quedan horarios ese día.', 'intervalos' => $intervalos];
    }
```

EstadoCita — validarCambio(): transiciones de estado (RF10)

```
    /** ¿Ya empezó la hora de la cita? */
    public static function yaInicio(array $cita): bool
    {
        return ($cita['fecha'] . ' ' . substr($cita['hora_inicio'], 0, 5)) <= date('Y-m-d H:i');
    }

    public static function esPendiente(array $cita): bool
    {
        return in_array($cita['estado'], ['programada', 'confirmada'], true);
    }

    /** Valida el cambio de estado y explica por qué no se permite. */
    public static function validarCambio(array $cita, string $nuevo): void
    {
        $actual = $cita['estado'];
        if (!in_array($nuevo, self::TRANSICIONES[$actual] ?? [], true)) {
            throw new ErrorNegocio(match ($actual) {
                'cancelada'    => 'Una cita cancelada no puede marcarse como ' . $nuevo . '. Programe una nueva
cita.',
                'atendida'     => 'Una cita atendida ya está cerrada y conserva su registro histórico.',
                'inasistencia' => 'La cita ya fue registrada como inasistencia.',
                default        => "No se puede pasar de «{$actual}» a «{$nuevo}».",
            });
        }
        if (in_array($nuevo, ['atendida', 'inasistencia'], true) && $cita['fecha'] > date('Y-m-d')) {
            throw new ErrorNegocio('Una cita futura no puede marcarse como ' . $nuevo . '.');
        }
        if ($nuevo === 'inasistencia' && !self::yaInicio($cita)) {
            throw new ErrorNegocio('La inasistencia se registra cuando ya pasó la hora de la cita.');
        }
        if ($nuevo === 'confirmada' && self::yaInicio($cita)) {
            throw new ErrorNegocio('Esta cita ya comenzó: registre si fue atendida o si hubo inasistencia.');
        }
        if ($nuevo === 'cancelada' && self::yaInicio($cita)) {
            throw new ErrorNegocio('La cita ya comenzó y no puede cancelarse; registre la asistencia o la
inasistencia.');
        }
    }
```

7.3.5 Subsistema reportes

Reportes administrativos.

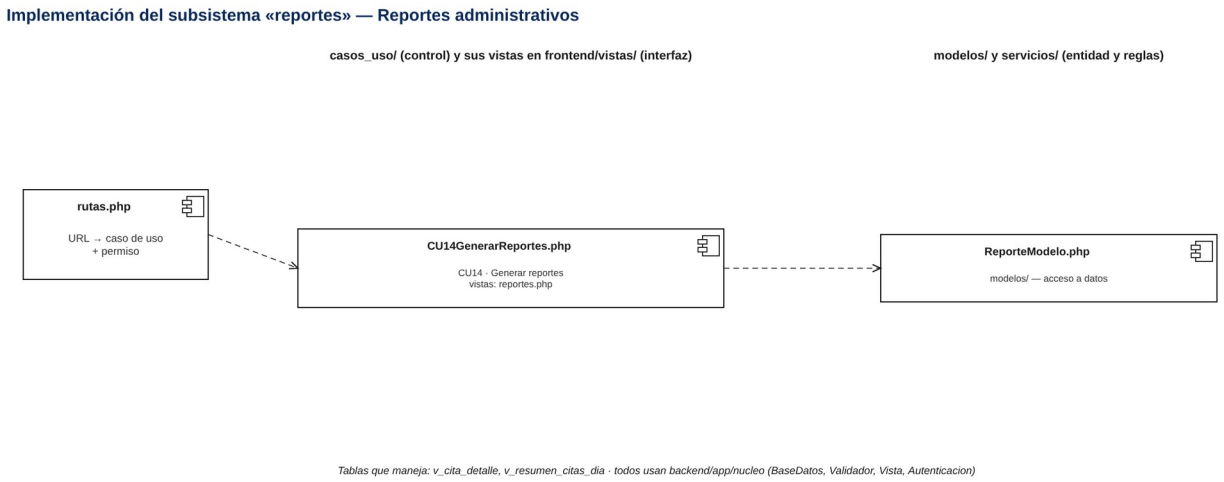

Figura 133. *Implementación del subsistema reportes.*

Tabla 45. *Implementación de los casos de uso — reportes*

CU	Archivo	Métodos públicos	Modelos
CU14	CU14GenerarReportes.php	generar()	ReporteModelo.php

ReporteModelo — citasAgrupadas(): citas por médico o especialidad

```
    /** 1. Citas programadas para una fecha o rango (detalle). */
    public function citasDelPeriodo(string $desde, string $hasta, ?string $estado = null): array
    {
        $sql = "SELECT * FROM v_cita_detalle WHERE fecha BETWEEN CAST(:a AS DATE) AND CAST(:b AS DATE)";
        $p = ['a' => $desde, 'b' => $hasta];
        if ($estado) {
            $sql .= " AND estado = :e";
            $p['e'] = $estado;
        }
        return BaseDatos::todos($sql . " ORDER BY fecha, hora_inicio, medico", $p);
    }

    /** 2 y 3. Citas agrupadas por médico o por especialidad, con su desglose por estado. */
    public function citasAgrupadas(string $campo, string $desde, string $hasta): array
    {
        $col = $campo === 'medico' ? 'medico' : 'especialidad';
        return BaseDatos::todos(
            "SELECT $col AS grupo,
                    count(*) AS total,
                    count(*) FILTER (WHERE estado = 'programada')   AS programadas,
                    count(*) FILTER (WHERE estado = 'confirmada')   AS confirmadas,
                    count(*) FILTER (WHERE estado = 'atendida')     AS atendidas,
                    count(*) FILTER (WHERE estado = 'cancelada')    AS canceladas,
                    count(*) FILTER (WHERE estado = 'inasistencia') AS inasistencias
               FROM v_cita_detalle
              WHERE fecha BETWEEN CAST(:a AS DATE) AND CAST(:b AS DATE)
              GROUP BY $col
              ORDER BY total DESC, grupo",
            ['a' => $desde, 'b' => $hasta]
        );
    }
```

7.4 Trazabilidad del análisis al código

La siguiente matriz relaciona cada clase de análisis de los diagramas de comunicación y de clases de análisis (capítulo 5) con el archivo que la implementa. Los métodos de las clases de control (CTR) y de entidad (CE) tienen exactamente el mismo nombre en el documento y en el código; cada CE se

implementa con la clase Modelo que guarda sus consultas SQL y se almacena en la tabla indicada. Los registros de CE CitaHistorial los inserta automáticamente el trigger fn_tg_historial_cita, y los de la bitácora los escribe CTR Bitácora (núcleo Bitacora.php) en cada inicio o cierre de sesión y en cada petición POST.

Tabla 46. *Trazabilidad análisis → código — Ciclo 1*

Clase de análisis	Implementación	Métodos (mismos nombres en el código)
CU01: Iniciar sesión		
IU Inicio de sesión **IU Panel principal** **IU Cambiar contraseña**	Pantallas (frontend/)	vistas/seguridad/login.php, vistas/seguridad/cambiar_clave.php, vistas/inicio/panel.php
CTR Sesión	CU01IniciarSesion.php	formulario(), iniciar(), cerrar(), formularioClave(), cambiarClave()
CTR Autenticación	Autenticacion.php	iniciar(), usuario(), cerrar()
CTR Bitácora	Bitacora.php	registrar()
CE Usuario	UsuarioModelo.php · tabla usuario	buscarParaLogin(), registrarIntentoFallido(), registrarAccesoExitoso(), hashActual(), cambiarClave()
CE Rol	RolModelo.php · tabla rol	(solo atributos: idRol, nombre, descripcion)
CE Permiso	RolModelo.php · tabla permiso	(solo atributos: codigo, nombre, modulo)
CU02: Gestionar usuarios y permisos		
IU Usuarios **IU Formulario usuario** **IU Permisos por rol** **IU Bitácora**	Pantallas (frontend/)	vistas/seguridad/usuarios_lista.php, vistas/seguridad/usuario_form.php, vistas/seguridad/permisos.php, vistas/seguridad/bitacora.php
CTR Usuarios	CU02GestionarUsuariosPermisos.php	listar(), nuevo(), editar(), crear(), actualizar(), cambiarEstado(), permisos(), guardarPermisos(), validarDatos(), bitacora()
CTR Bitácora	Bitacora.php	registrar(), registrarOperacion()
CE Usuario	UsuarioModelo.php · tabla usuario	listar(), obtener(), crear(), actualizar(), cambiarEstado(), medicosSinCuenta(), pacientesSinCuenta()
CE Rol	RolModelo.php · tabla rol	listar(), obtener()
CE Permiso	RolModelo.php · tabla permiso	permisos(), idPermiso()
CE RolPermiso	RolModelo.php · tabla rol_permiso	reemplazarPermisos()
CE Médico	MedicoModelo.php · tabla medico	(solo atributos: codigo, nombres)
CE Bitácora	BitacoraModelo.php · tabla bitacora	listar(), contar(), modulos(), acciones()
CU03: Gestionar pacientes		
IU Pacientes **IU Formulario paciente** **IU Ficha del paciente**	Pantallas (frontend/)	vistas/pacientes/lista.php, vistas/pacientes/formulario.php, vistas/pacientes/ficha.php
CTR Pacientes	CU03GestionarPacientes.php	listar(), ver(), crear(), actualizar(), cambiarEstado(), buscarJson()
CE Paciente	PacienteModelo.php · tabla paciente	buscar(), obtener(), existeCi(), crear(), actualizar(), cambiarEstado(), cancelarCitasPendientes()
CE ContactoEmergencia	PacienteModelo.php · tabla contacto_emergencia	guardarContactoPrincipal(), contactos()
CE Cita	CitaModelo.php · tabla cita	(solo atributos: fecha, horaInicio, estado)
CU06: Gestionar especialidades		

Clase de análisis	Implementación	Métodos (mismos nombres en el código)
IU Especialidades **IU Formulario especialidad**	Pantallas (frontend/)	vistas/medicos/especialidades.php
CTR Especialidades	CU06GestionarEspecialidades.php	listar(), crear(), actualizar(), cambiarEstado(), validar()
CE Especialidad	EspecialidadModelo.php · tabla especialidad	listar(), obtener(), existeNombre(), crear(), actualizar(), cambiarEstado(), citasPendientes()
CE Cita	CitaModelo.php · tabla cita	(solo atributos: estado)
CU07: Gestionar médicos		
IU Médicos **IU Formulario médico** **IU Ficha del médico**	Pantallas (frontend/)	vistas/medicos/medicos_lista.php, vistas/medicos/medico_form.php, vistas/medicos/medico_ficha.php
CTR Médicos	CU07GestionarMedicos.php	listar(), ver(), nuevo(), editar(), crear(), actualizar(), cambiarEstado(), validar()
CE Médico	MedicoModelo.php · tabla medico	buscar(), obtener(), crear(), actualizar(), cambiarEstado(), citasPendientes(), cancelarCitasPendientes()
CE MédicoEspecialidad	MedicoModelo.php · tabla medico_especialidad	sincronizarEspecialidades()
CE Especialidad	EspecialidadModelo.php · tabla especialidad	listar()
CE Cita	CitaModelo.php · tabla cita	(solo atributos: estado)
CU08: Gestionar horarios de atención		
IU Horarios **IU Formulario horario** **IU Bloquear fecha**	Pantallas (frontend/)	vistas/medicos/horarios.php
CTR Horarios	CU08GestionarHorarios.php	mostrar(), crear(), actualizar(), cambiarEstado(), agregarFeriado(), quitarFeriado(), validar()
CE HorarioAtención	HorarioModelo.php · tabla horario_atencion	delMedico(), obtener(), haySolape(), crear(), actualizar(), cambiarEstado()
CE DíaNoLaborable	HorarioModelo.php · tabla dia_no_laborable	diasNoLaborables(), citasEnFecha(), agregarDiaNoLaborable(), quitarDiaNoLaborable()
CE Médico	MedicoModelo.php · tabla medico	listaSimple(), obtener()
CE Cita	CitaModelo.php · tabla cita	(solo atributos: fecha, estado)
CU09: Programar cita		
IU Programar cita **IU Detalle de la cita**	Pantallas (frontend/)	vistas/citas/programar.php, assets/js/citas.js, vistas/citas/detalle.php
CTR Programar cita	CU09ProgramarCita.php	formulario(), registrar(), guardar(), medicosJson(), diasJson(), horariosJson()
CTR Pacientes	CU03GestionarPacientes.php	buscarJson()
CTR Disponibilidad	Disponibilidad.php	proximosDias(), delDia(), verificar()
CE Paciente	PacienteModelo.php · tabla paciente	busquedaRapida(), obtener()
CE Médico	MedicoModelo.php · tabla medico	activosPorEspecialidad(), obtener()
CE HorarioAtención	HorarioModelo.php · tabla horario_atencion	activosDelDia()
CE DíaNoLaborable	HorarioModelo.php · tabla dia_no_laborable	fechasBloqueadas()
CE Cita	CitaModelo.php · tabla cita	bloquearMedico(), ocupadas(), pacienteOcupado(), crear()

Tabla 47. *Trazabilidad análisis → código — Ciclo 2*

Clase de análisis	Implementación	Métodos (mismos nombres en el código)
CU04: Gestionar perfil del paciente		

Clase de análisis	Implementación	Métodos (mismos nombres en el código)
IU Registro en línea **IU Mi perfil**	Pantallas (frontend/)	vistas/pacientes/registro.php, vistas/pacientes/mi_perfil.php
CTR Perfil del paciente	CU04GestionarPerfilPaciente.php	registrar(), guardarDatos(), guardarMedica(), guardarEmergencia(), guardarPreferencias()
CE Paciente	PacienteModelo.php · tabla paciente	porCi(), crear(), actualizar()
CE Usuario	UsuarioModelo.php · tabla usuario	crear()
CE Rol	RolModelo.php · tabla rol	obtenerPorNombre()
CE ContactoEmergencia	PacienteModelo.php · tabla contacto_emergencia	guardarContactoPrincipal()
CU05: Consultar historial de citas		
IU Historial del paciente **IU Mis citas**	Pantallas (frontend/)	vistas/pacientes/historial.php, vistas/pacientes/mis_citas.php
CTR Historial	CU05ConsultarHistorialCitas.php	historialPaciente(), misCitas(), filtros()
CE Paciente	PacienteModelo.php · tabla paciente	obtener(), historial(), resumenEstados()
CE Cita	CitaModelo.php · tabla cita	(solo atributos: codigo, fecha, horaInicio, estado, fechaRegistro)
CE Médico	MedicoModelo.php · tabla medico	(solo atributos: nombres, apellidos)
CE Especialidad	EspecialidadModelo.php · tabla especialidad	(solo atributos: nombre)
CU10: Reprogramar cita		
IU Detalle de la cita **IU Reprogramar cita**	Pantallas (frontend/)	vistas/citas/reprogramar.php, assets/js/citas.js
CTR Reprogramar cita	CU10ReprogramarCita.php	formulario(), guardar(), reprogramar()
CTR Programar cita	CU09ProgramarCita.php	diasJson(), horariosJson()
CTR Disponibilidad	Disponibilidad.php	proximosDias(), delDia(), verificar()
CTR EstadoCita	EstadoCita.php	validarReprogramacion()
CE Cita	CitaModelo.php · tabla cita	obtener(), bloquearMedico(), pacienteOcupado(), reprogramar()
CE CitaHistorial	trigger fn_tg_historial_cita · tabla cita_historial	(solo atributos: fechaAnterior, horaAnterior, fechaNueva, horaNueva, fechaHora)
CU11: Cancelar cita		
IU Detalle de la cita **IU Cancelar cita** **IU Mis citas**	Pantallas (frontend/)	vistas/citas/detalle.php, vistas/pacientes/mis_citas.php
CTR Cancelar cita	CU11CancelarCita.php	cancelar(), cancelarPaciente(), ejecutar()
CTR EstadoCita	EstadoCita.php	validarCambio()
CE Cita	CitaModelo.php · tabla cita	obtener(), cambiarEstado()
CE CitaHistorial	trigger fn_tg_historial_cita · tabla cita_historial	(solo atributos: accion, estadoAnterior, estadoNuevo, detalle, fechaHora)
CU12: Registrar asistencia		
IU Agenda / Detalle	Pantallas (frontend/)	vistas/citas/detalle.php, vistas/citas/agenda.php, vistas/inicio/citas_hoy.php
CTR Asistencia	CU12RegistrarAsistencia.php	cambiar(), ejecutar()
CTR Autenticación	Autenticacion.php	puede()
CTR EstadoCita	EstadoCita.php	validarCambio(), acciones()
CE Cita	CitaModelo.php · tabla cita	obtener(), cambiarEstado()
CE CitaHistorial	trigger fn_tg_historial_cita · tabla cita_historial	(solo atributos: accion, estadoAnterior, estadoNuevo, fechaHora)

Clase de análisis	Implementación	Métodos (mismos nombres en el código)
CU13: Consultar agenda médica		
IU Agenda médica **IU Citas** **IU Detalle de la cita** **IU Agenda semanal**	Pantallas (frontend/)	vistas/citas/agenda.php, vistas/citas/agenda_semana.php, vistas/citas/listado.php, vistas/citas/detalle.php
CTR Agenda	CU13ConsultarAgendaMedica.php	agenda(), listado(), ver(), filtros(), agendaSemanal()
CTR Disponibilidad	Disponibilidad.php	delDia()
CTR EstadoCita	EstadoCita.php	acciones()
CE Cita	CitaModelo.php · tabla cita	agenda(), listar(), obtener(), historial(), agendaSemana()
CE Médico	MedicoModelo.php · tabla medico	listaSimple(), obtener()
CE Especialidad	EspecialidadModelo.php · tabla especialidad	listar()
CE CitaHistorial	trigger fn_tg_historial_cita · tabla cita_historial	(solo atributos: accion, estadoNuevo, fechaHora)
CE HorarioAtención	HorarioModelo.php · tabla horario_atencion	diasQueAtiende(), fechasBloqueadas()
CU14: Generar reportes		
IU Reportes	Pantallas (frontend/)	vistas/reportes/reportes.php
CTR Reportes	CU14GenerarReportes.php	generar(), exportarCsv()
CE Reporte	ReporteModelo.php · tabla v_cita_detalle, v_resumen_citas_dia, paciente	citasDelPeriodo(), citasAgrupadas(), citasPorEstado(), pacientesRegistrados(), totalesPacientes(), resumenDiario(), agendaDiaria()

7.5 Instalación y puesta en marcha

1. Instalar XAMPP (PHP 8.1+) y PostgreSQL 14 o superior.
2. En `C:\xampp\php\php.ini` quitar el `;` de `extension=pdo_pgsql` y `extension=pgsql`, y reiniciar Apache.
3. Clonar el repositorio en `C:\xampp\htdocs\clinica-vida-salud` (`git clone https://github.com/edisonayala000-netizen/clinica-vida-salud.git`).
4. Crear la base: ejecutar `basededato\sql\instalar_bd.bat` o `psql -U postgres -c "CREATE DATABASE clinica_vida_salud ENCODING 'UTF8' TEMPLATE template0"` y `psql -U postgres -d clinica_vida_salud -f instalar.sql` desde la carpeta basededato\sql.
5. Si la clave de postgres no es «postgres», crear `backend\config\config.local.php` con `<?php return ['bd' => ['clave' => 'SU_CLAVE']];`.
6. Abrir http://localhost/clinica-vida-salud e ingresar con `admin` / `Clinica2026*`.
7. Ejecutar las pruebas: `php backend\tests\pruebas_sistema.php http://localhost/clinica-vida-salud`.

8. FLUJO DE TRABAJO: PRUEBAS

Las pruebas verifican que el sistema cumpla los requisitos. Se aplicaron tres niveles: **(1)** las 10 pruebas mínimas del examen, ejecutadas tanto desde la interfaz como de forma automática con `backend/tests/pruebas_sistema.php` (pruebas de caja negra sobre el sistema en funcionamiento: inicia sesión, envía formularios y revisa la respuesta); **(2)** 16 pruebas de la base de datos (`basededato/sql/pruebas/pruebas_bd.sql`) que intentan guardar datos inválidos directamente en PostgreSQL para comprobar restricciones y triggers; y **(3)** pruebas de validación y permisos por caso de uso.

8.1 Pruebas mínimas del examen

Tabla 48. *Resultados de las 10 pruebas mínimas*

N.º	Prueba	Resultado esperado	Resultado obtenido	Estado
1	Registrar un paciente nuevo	El paciente queda guardado	Paciente PAC-0022 · Paciente De Prueba registrado. (CI 13170821)	Aprobada
2	Registrar un paciente con CI duplicado	El sistema rechaza el duplicado	Ya existe un paciente registrado con ese CI. | Este CI ya pertenece a otro paciente.	Aprobada
3	Programar una cita en horario disponible	La cita queda registrada	Cita CM-1001 registrada. (2026-10-19 08:00, Dr. Juan Pérez)	Aprobada
4	Programar otra cita superpuesta para el mismo médico	El sistema impide el conflicto	RN07: el médico ya tiene una cita de 08:00 a 08:30. Elija otro horario.	Aprobada
5	Intentar reservar fuera del horario del médico	El sistema rechaza la cita	RN05: la hora elegida está fuera del horario de atención del médico o no respeta los intervalos configurados. (lunes 2026-10-19 a las 20:00; atiende de 08:00 a 12:00)	Aprobada
6	Cancelar una cita	Cambia a estado cancelado y conserva el historial	Cita cancelada. El horario quedó libre y la cita se conserva en el historial. Historial: «Cita registrada» y «Cambio de estado de programada a cancelada».	Aprobada
7	Consultar el historial de un paciente	Aparecen sus citas registradas	El historial muestra la cita CM-1001 con estado Cancelada	Aprobada
8	Buscar las citas de un médico por fecha	Se muestran los resultados correctos	2 cita(s) del 2026-10-19, todas del Dr. Juan Pérez Rojas	Aprobada
9	Ingresar a una página interna sin iniciar sesión	El acceso es denegado	HTTP 302 → redirige a /login: «Acceso denegado: inicie sesión para continuar.»	Aprobada
10	Ejecutar una operación	El servidor valida y	Paciente: Revise los datos	Aprobada

N.º	Prueba	Resultado esperado	Resultado obtenido	Estado
	con datos inválidos	rechaza la operación	marcados en el formulario. | CI debe tener de 5 a 12 dígitos (puede llevar complemento, ej. 1234567-1B). | Nombres solo admite letras y espacio… · Cita: Revise los datos marcados en el formulario. | Fech…	

```
$ php backend/tests/pruebas_sistema.php http://127.0.0.1:8080

Sistema de citas — Clínica Vida Salud · pruebas sobre http://127.0.0.1:8080

[OK]   Prueba  1 · Registrar un paciente nuevo
      Esperado: El paciente queda guardado
      Obtenido: Paciente PAC-0022 · Paciente De Prueba registrado. (CI 13170821)

[OK]   Prueba  2 · Registrar un paciente con CI duplicado
      Esperado: El sistema rechaza el duplicado
      Obtenido: Ya existe un paciente registrado con ese CI. | Este CI ya pertenece a otro paciente.

[OK]   Prueba  3 · Programar una cita en horario disponible
      Esperado: La cita queda registrada
      Obtenido: Cita CM-1001 registrada. (2026-10-19 08:00, Dr. Juan Pérez)

[OK]   Prueba  4 · Programar otra cita superpuesta para el mismo médico
      Esperado: El sistema impide el conflicto
      Obtenido: RN07: el médico ya tiene una cita de 08:00 a 08:30. Elija otro horario.

[OK]   Prueba  5 · Intentar reservar fuera del horario del médico
      Esperado: El sistema rechaza la cita
      Obtenido: RN05: la hora elegida está fuera del horario de atención del médico o no respeta los intervalos configurados. (lunes 2026-10-19 a las
20:00; atiende de 08:00 a 12:00)

[OK]   Prueba  6 · Cancelar una cita
      Esperado: Cambia a estado cancelado y conserva el historial
      Obtenido: Cita cancelada. El horario quedó libre y la cita se conserva en el historial. Historial: «Cita registrada» y «Cambio de estado de
programada a cancelada».

[OK]   Prueba  7 · Consultar el historial de un paciente
      Esperado: Aparecen sus citas registradas
      Obtenido: El historial muestra la cita CM-1001 con estado Cancelada

[OK]   Prueba  8 · Buscar las citas de un médico por fecha
      Esperado: Se muestran los resultados correctos
      Obtenido: 2 cita(s) del 2026-10-19, todas del Dr. Juan Pérez Rojas

[OK]   Prueba  9 · Ingresar a una página interna sin iniciar sesión
      Esperado: El acceso es denegado
      Obtenido: HTTP 302 → redirige a /login: «Acceso denegado: inicie sesión para continuar.»

[OK]   Prueba 10 · Ejecutar una operación con datos inválidos
      Esperado: El servidor valida y rechaza la operación
      Obtenido: Paciente: Revise los datos marcados en el formulario. | CI debe tener de 5 a 12 dígitos (puede llevar complemento, ej. 1234567-1B). |
Nombres solo admite letras y espacio… · Cita: Revise los datos marcados en el formulario. | Fecha no puede ser una fecha pasada. | Hora no es una hora
válida (HH:MM).…

Resultado: 10 de 10 pruebas aprobadas.
```

Figura 134. *Ejecución automática de las 10 pruebas: 10 de 10 aprobadas.*

Prueba 1: Registrar un paciente nuevo

Se registra a Rodrigo Justiniano Paz; el sistema genera su código y muestra su ficha.

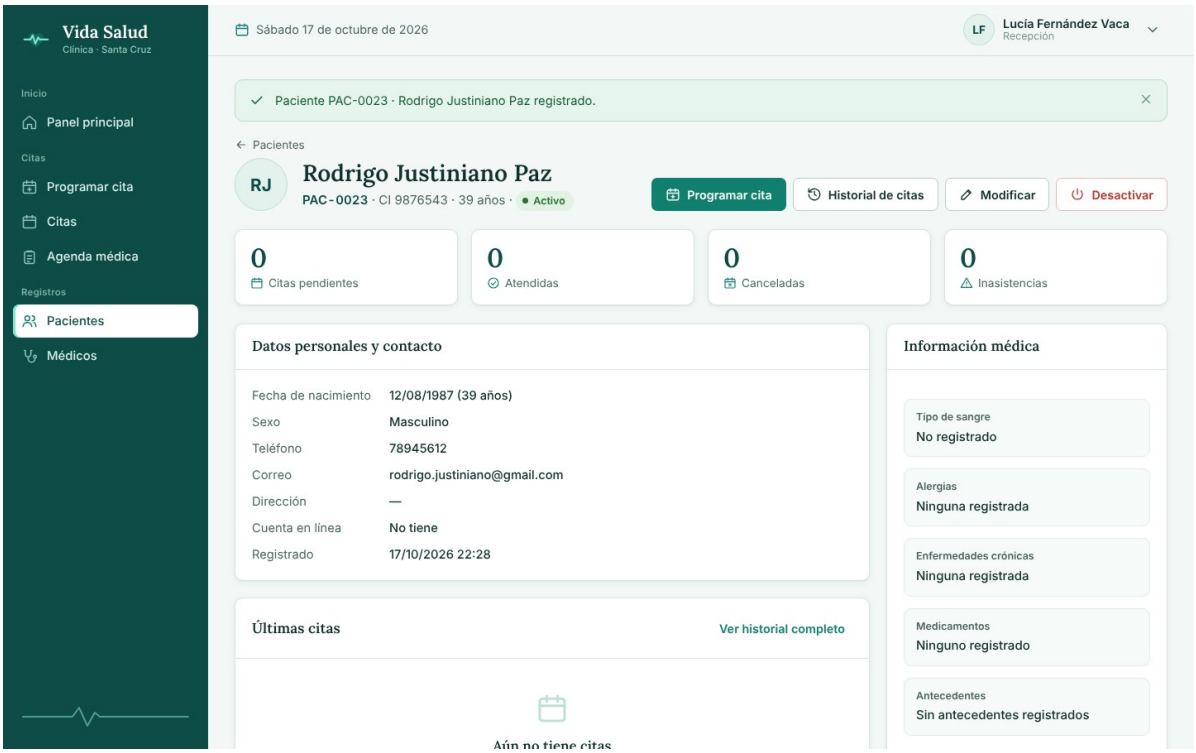

Figura 135. *Evidencia de la prueba 1: registrar un paciente nuevo.*

Prueba 2: Registrar un paciente con CI duplicado

Se intenta registrar otra vez el CI 6234567: el servidor rechaza el duplicado y marca el campo.

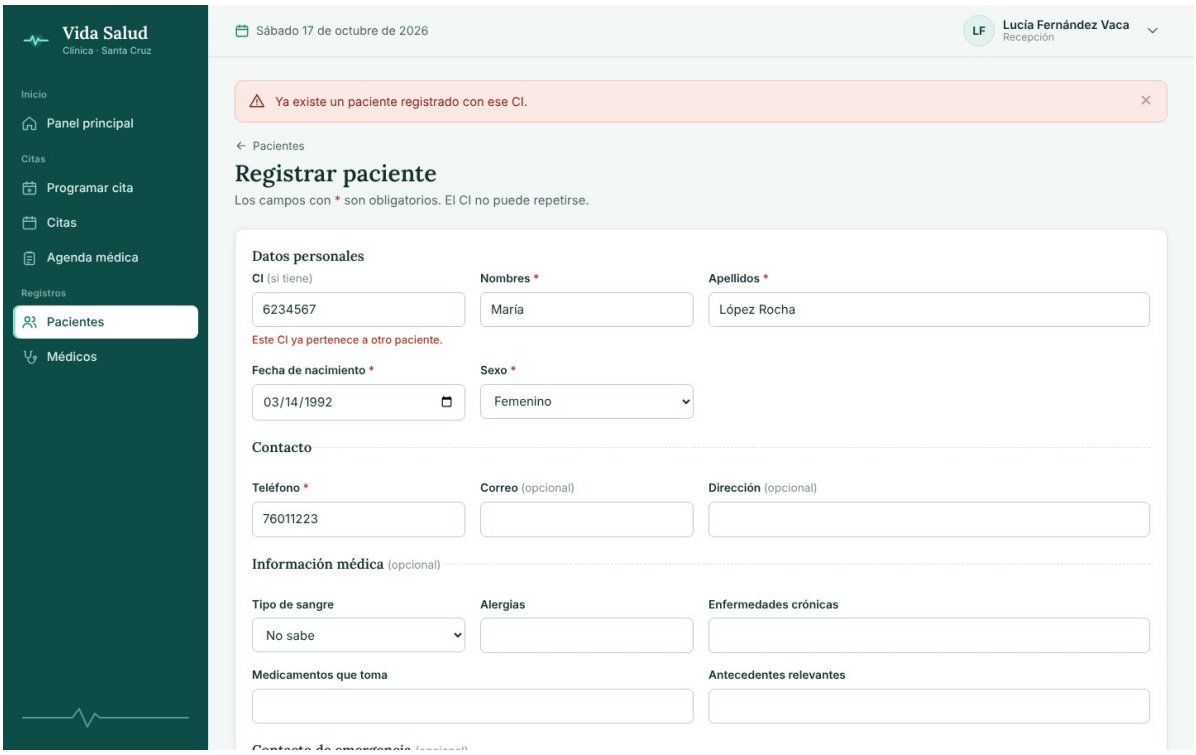

Figura 136. *Evidencia de la prueba 2: registrar un paciente con ci duplicado.*

Prueba 3: Programar una cita en horario disponible

La cita queda registrada con su código único y la fecha de creación.

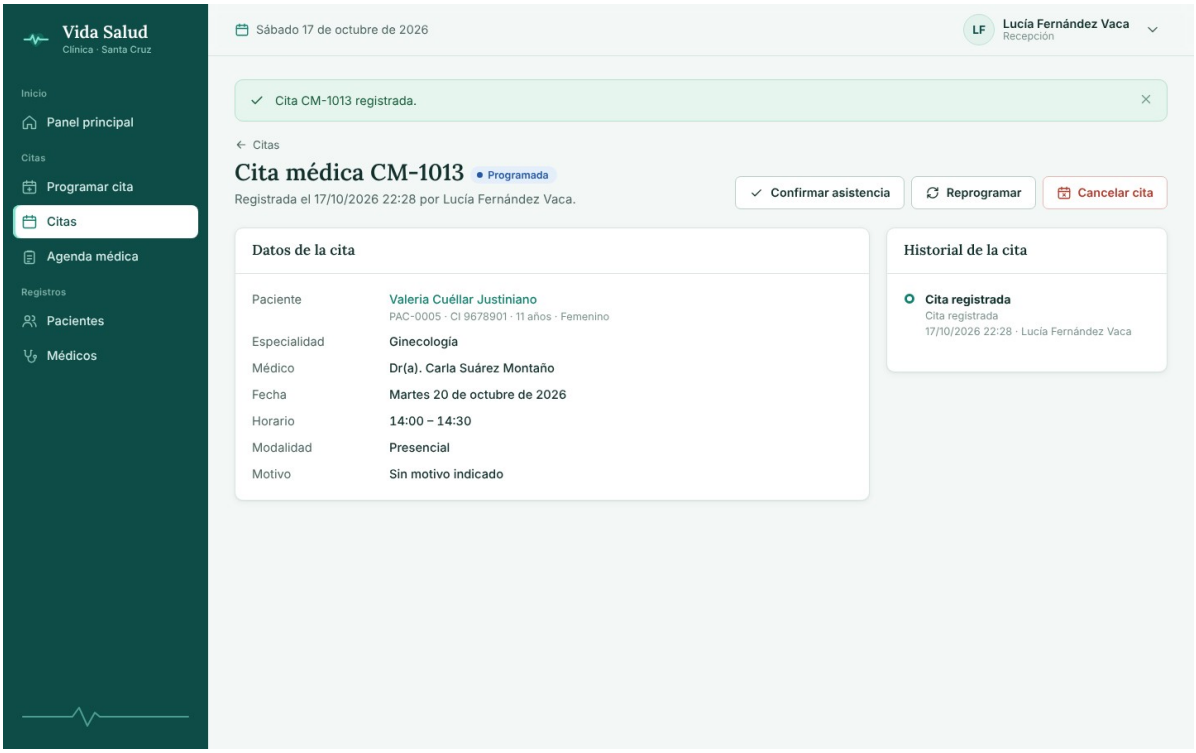

Figura 137. *Evidencia de la prueba 3: programar una cita en horario disponible.*

Prueba 4: Programar otra cita superpuesta para el mismo médico

Dos recepcionistas eligen el mismo horario al mismo tiempo; la primera lo registra y a la segunda el servidor le responde RN07 y no guarda la cita.

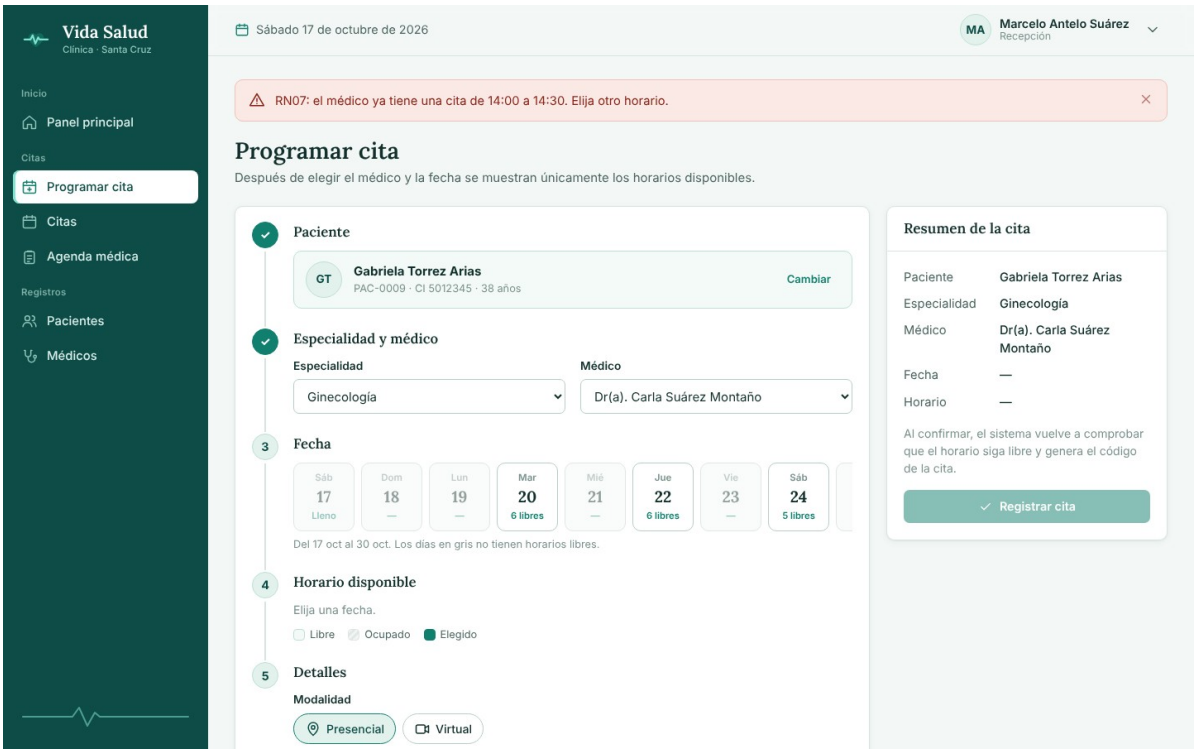

Figura 138. *Evidencia de la prueba 4: programar otra cita superpuesta para el mismo médico.*

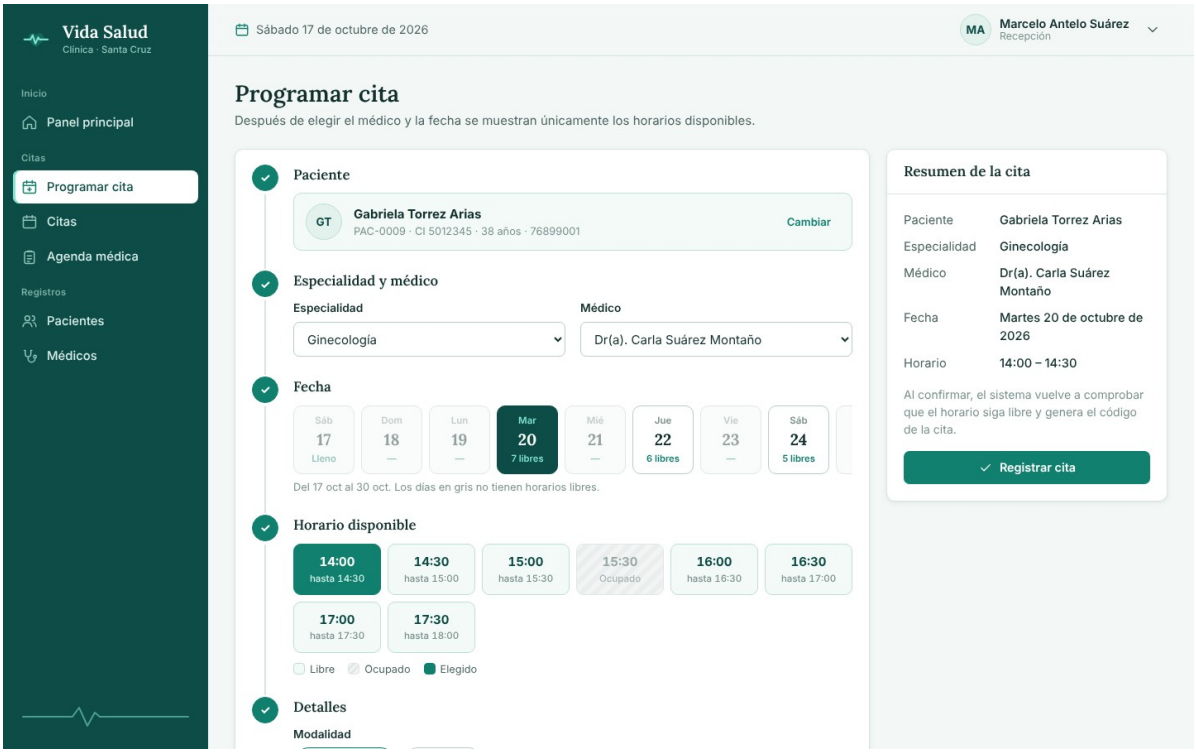

Figura 139. *Prueba 4: la segunda recepcionista había elegido el mismo horario antes de que se ocupara.*

Prueba 5: Intentar reservar fuera del horario médico

Se altera la hora a las 20:00 antes de enviar; el servidor responde RN05.

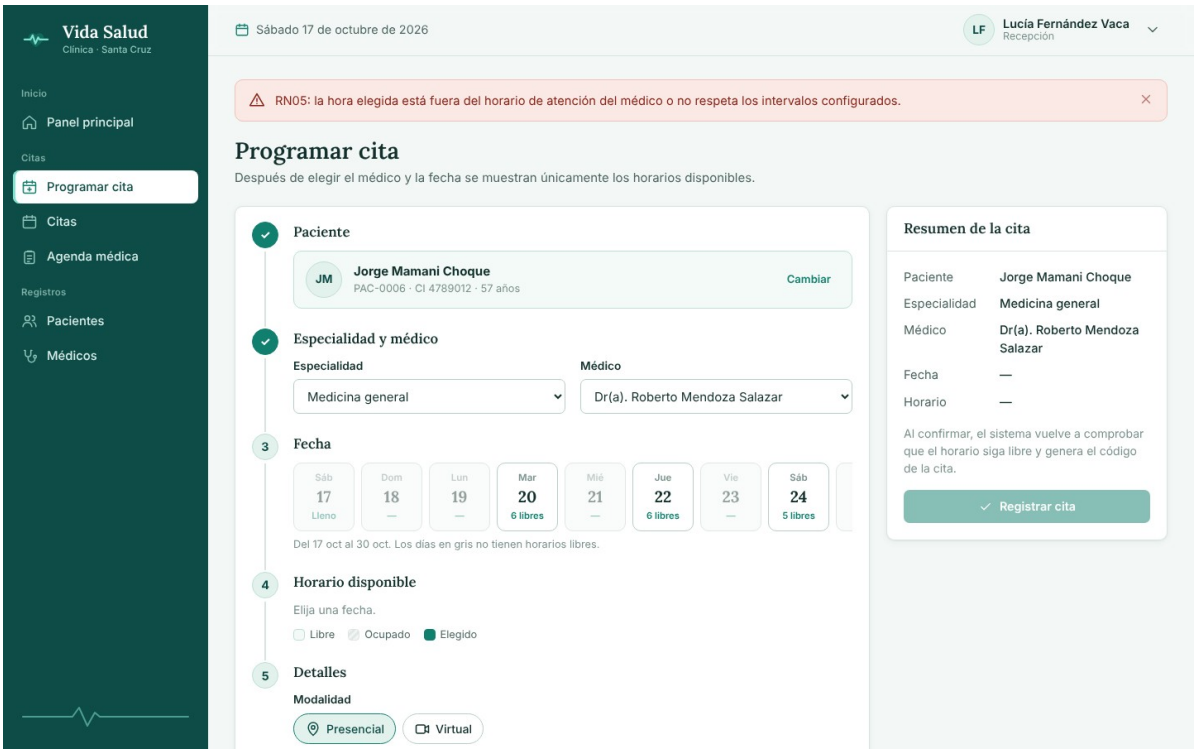

Figura 140. *Evidencia de la prueba 5: intentar reservar fuera del horario médico.*

Prueba 6: Cancelar una cita

La cita cambia a «Cancelada», guarda el motivo y su historial conserva el registro y el cambio de estado.

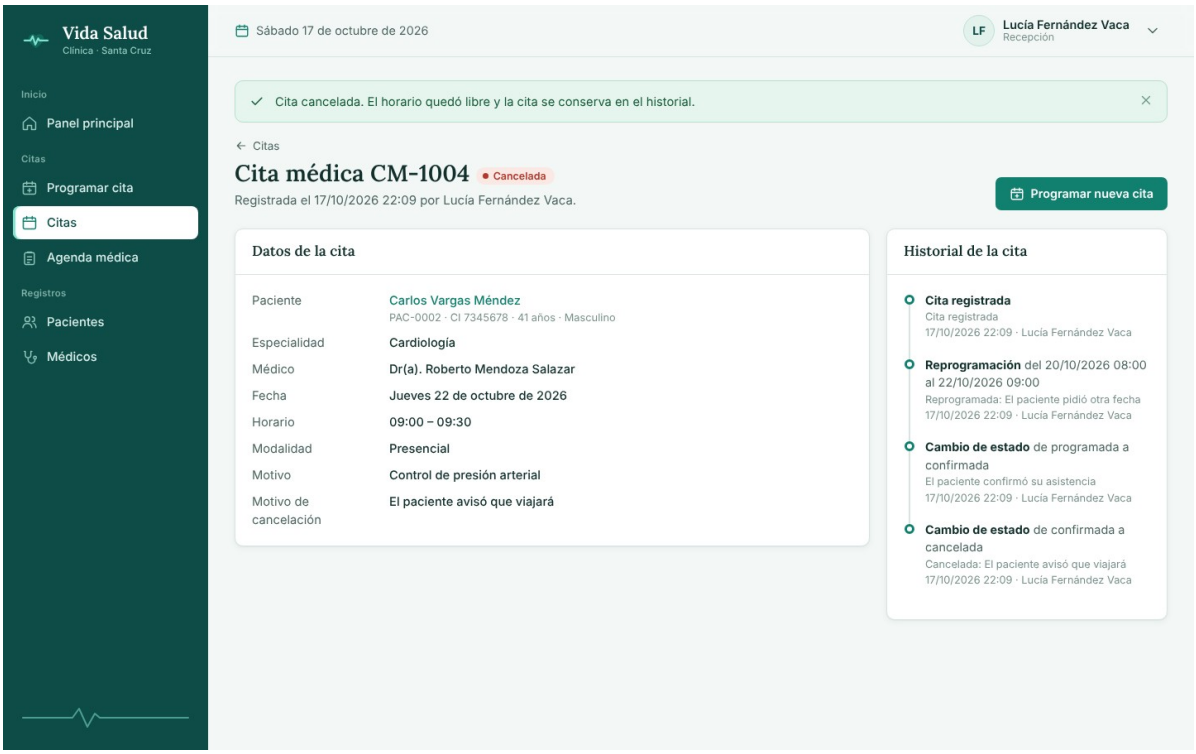

Figura 141. *Evidencia de la prueba 6: cancelar una cita.*

Prueba 7: Consultar el historial de un paciente

Aparecen sus citas pasadas, futuras, canceladas e inasistencias.

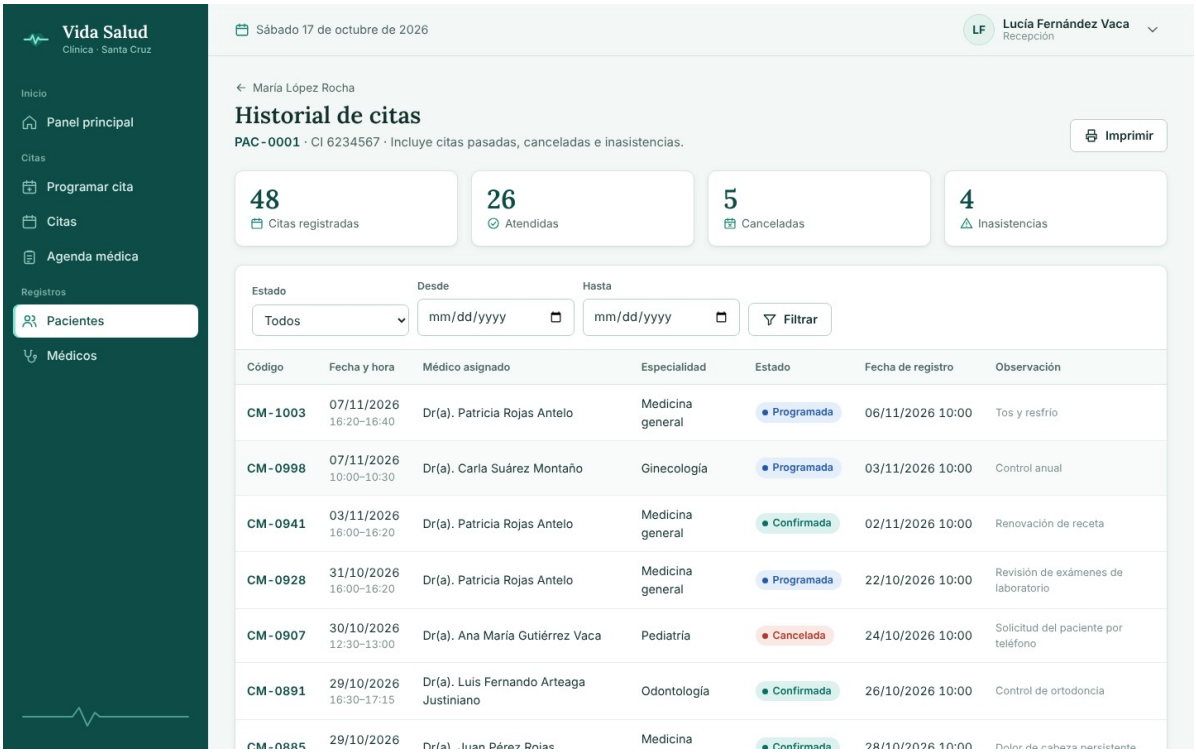

Figura 142. *Evidencia de la prueba 7: consultar el historial de un paciente.*

Prueba 8: Buscar las citas de un médico por fecha

Filtro por médico y fecha: solo aparecen las citas del Dr. Juan Pérez de ese día.

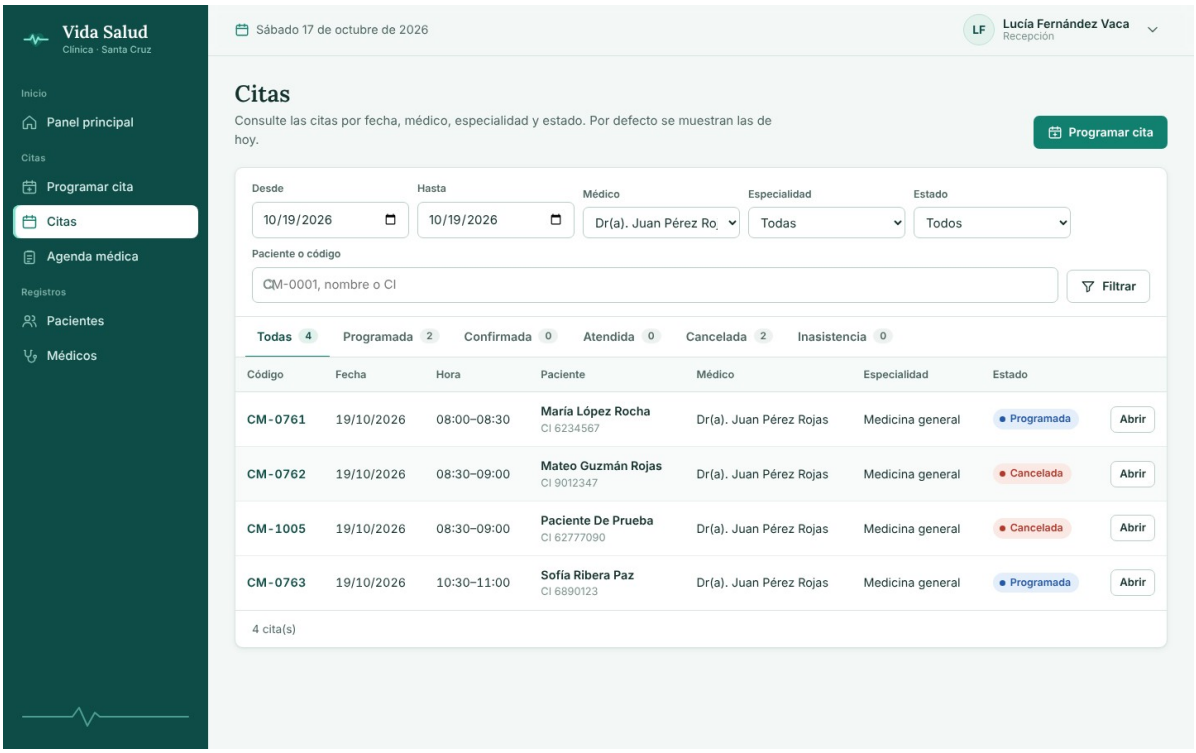

Figura 143. *Evidencia de la prueba 8: buscar las citas de un médico por fecha.*

Prueba 9: Ingresar a una página interna sin iniciar sesión

Al abrir /citas sin sesión el sistema redirige al inicio con «Acceso denegado».

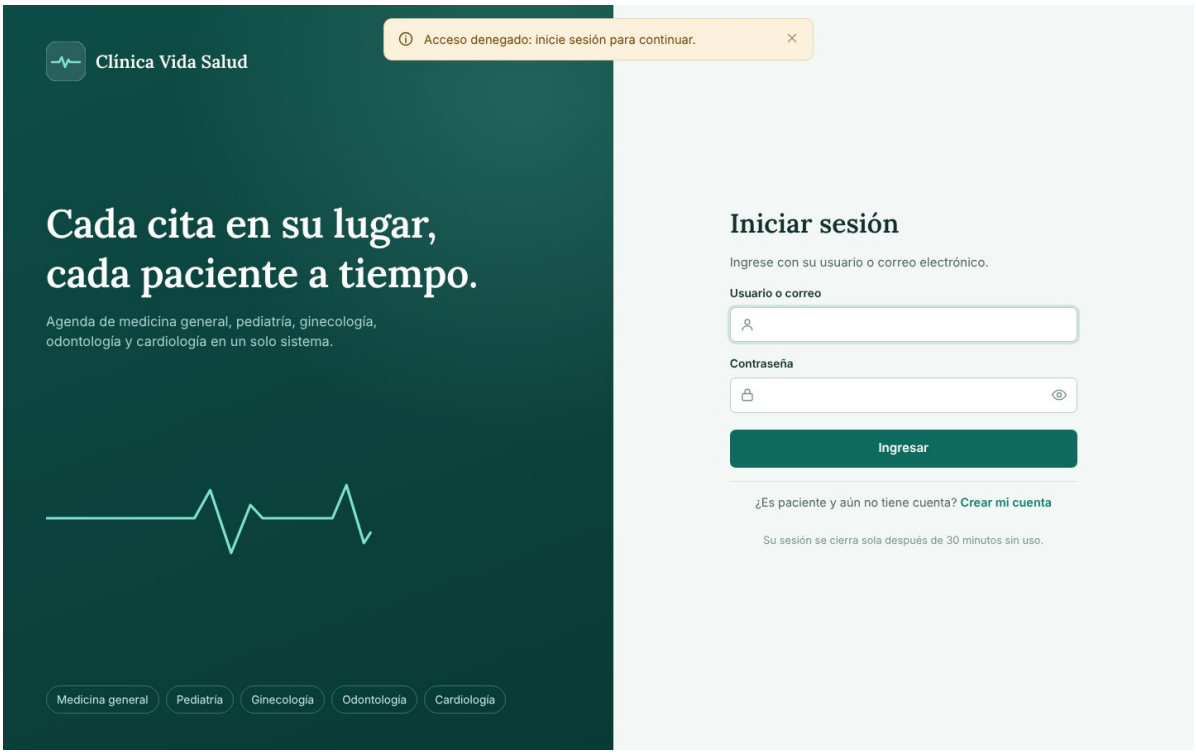

Figura 144. *Evidencia de la prueba 9: ingresar a una página interna sin iniciar sesión.*

Prueba 10: Ejecutar una operación con datos inválidos

Se envía el formulario saltando la validación del navegador: el servidor valida y rechaza cada dato inválido.

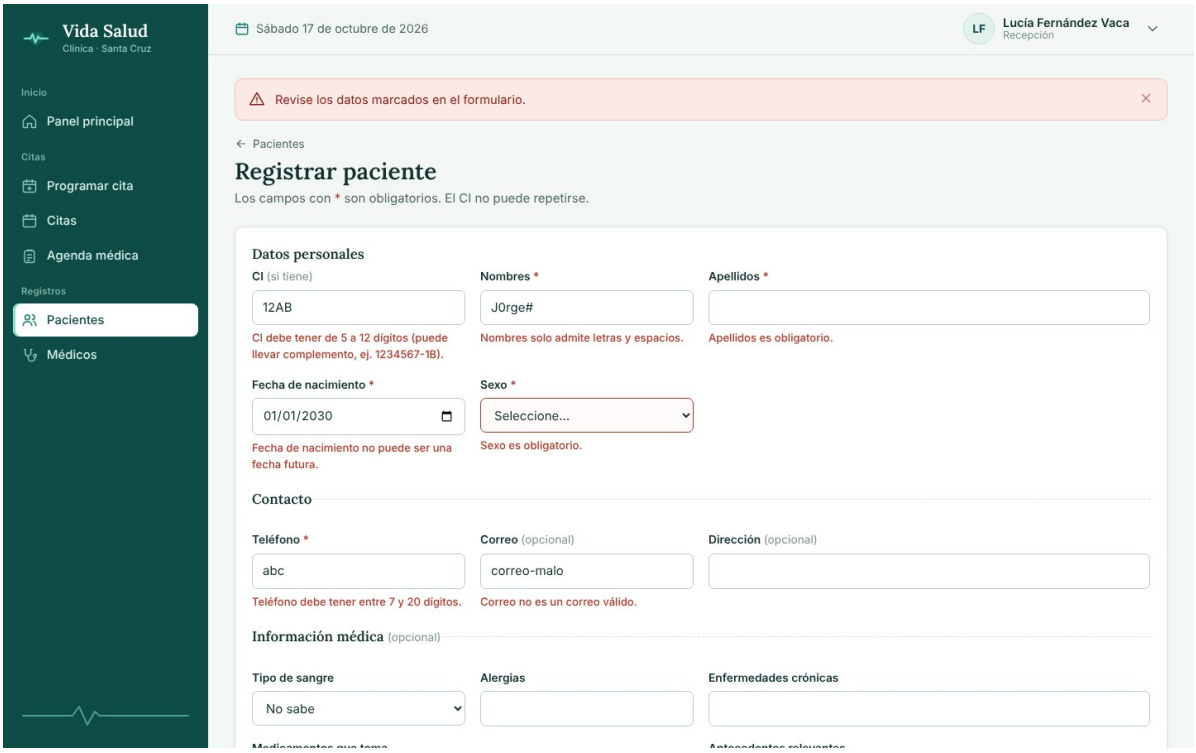

Figura 145. *Evidencia de la prueba 10: ejecutar una operación con datos inválidos.*

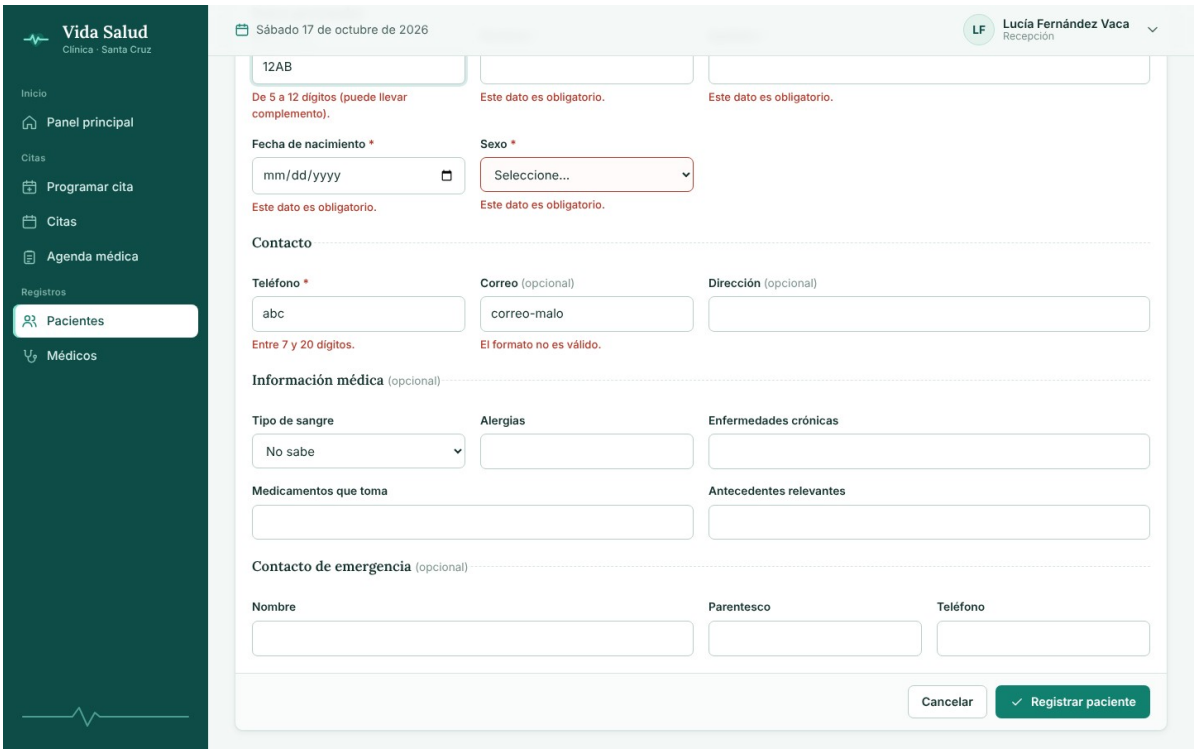

Figura 146. *Prueba 10: además, JavaScript valida en el navegador antes de enviar.*

8.2 Pruebas de la base de datos

El script `pruebas_bd.sql` se ejecuta dentro de una transacción que al final se revierte (no deja cambios). Cada prueba intenta una operación inválida y verifica que PostgreSQL la rechace con el mensaje esperado.

Tabla 49. *Pruebas de la base de datos (16 de 16 aprobadas)*

N.º	Prueba	Objeto que la detiene
1	CI duplicado	UNIQUE uq_paciente_ci
2	Cita superpuesta del mismo médico	EXCLUDE ex_cita_medico_sin_solape
3	Cita fuera del horario de atención	Trigger fn_tg_validar_cita (RN05)
4	Cita en fecha pasada	Trigger (RN06)
5	Cita con médico inactivo	Trigger (RN04)
6	Médico con especialidad que no atiende	FK compuesta fk_cita_medico_especialidad
7	Cita cancelada marcada como atendida	Trigger fn_tg_validar_transicion_cita (RF10)
8	Borrado físico de una cita	Trigger fn_tg_impedir_eliminacion (RN10)
9	Especialidad con el mismo nombre normalizado	Índice único sobre fn_normalizar_texto(nombre)
10	Horario con hora final anterior	CHECK ck_horario_rango
11	Horario superpuesto del mismo médico	EXCLUDE ex_horario_sin_solape
12	Eliminar un paciente con citas	FK ON DELETE RESTRICT (RN11)
13	Usuario médico sin vincular	Trigger fn_tg_validar_usuario_rol (RN12)
14	Cancelar libera el horario y conserva	EXCLUDE … WHERE estado <>

N.º	Prueba	Objeto que la detiene
	historial	cancelada + historial
15	Borrar un registro de la bitácora	Trigger fn_tg_bitacora_inalterable
16	Modificar un registro de la bitácora	Trigger fn_tg_bitacora_inalterable

```
$ psql -U postgres -d clinica_vida_salud -f basededato/sql/pruebas/pruebas_bd.sql
==================== RESULTADOS ====================
 N.º |                                Prueba                                |  Estado  |                                Mensaje de PostgreSQL
-----+----------------------------------------------------------------------+----------+-------------------------------------------------------------
------------------------------
   1 | CI duplicado (UNIQUE uq_paciente_ci)                                 | APROBADA | duplicate key value violates unique constraint
"uq_paciente_ci"
   2 | Cita superpuesta del mismo médico (EXCLUDE ex_cita_medico_sin_solape) | APROBADA | conflicting key value violates exclusion constraint
"ex_cita_medico_sin_solape"
   3 | Cita fuera del horario de atención (trigger RN05)                    | APROBADA | RN05: la cita está fuera del horario de atención del
médico.
   4 | Cita en fecha pasada (trigger RN06)                                  | APROBADA | RN06: no se pueden registrar citas en fechas u horas
pasadas.
   5 | Cita con médico inactivo (trigger RN04)                              | APROBADA | RN04: el médico está inactivo y no puede recibir nuevas
citas.
   6 | Médico con especialidad que no atiende (FK compuesta)                | APROBADA | insert or update on table "cita" violates foreign key
constraint "fk_cita_medico_especiali
   7 | Cita cancelada marcada como atendida (trigger RF10)                  | APROBADA | RF10: transición no permitida de "cancelada" a "atendida".
   8 | Borrado físico de una cita (trigger RN10)                            | APROBADA | RN10: los registros de cita no se eliminan; cambie su
estado.
   9 | Especialidad con el mismo nombre normalizado                         | APROBADA | duplicate key value violates unique constraint
"uq_especialidad_nombre_normalizado"
  10 | Horario con hora final anterior a la inicial (CHECK)                 | APROBADA | new row for relation "horario_atencion" violates check
constraint "ck_horario_rango"
  11 | Horario superpuesto del mismo médico (EXCLUDE)                       | APROBADA | conflicting key value violates exclusion constraint
"ex_horario_sin_solape"
  12 | Eliminar un paciente con citas (FK RESTRICT, RN11)                   | APROBADA | update or delete on table "paciente" violates foreign key
constraint "cita_id_paciente_fke
  13 | Usuario con rol médico sin vincular a un médico (trigger RN12)       | APROBADA | RN12: un usuario con rol médico debe estar vinculado a un
médico.
  14 | Cancelar libera el horario y conserva el historial (RN09)            | APROBADA | Horario reutilizado; la cita cancelada tiene 2 registros de
historial
  15 | Borrar una fila de la bitácora (trigger fn_tg_bitacora_inalterable)  | APROBADA | La bitácora no se modifica ni se borra: solo se agregan
registros.
  16 | Modificar una fila de la bitácora (trigger fn_tg_bitacora_inalterable) | APROBADA | La bitácora no se modifica ni se borra: solo se agregan
registros.
(16 rows)

          Resumen
---------------------------
 16 de 16 pruebas aprobadas
(1 row)

ROLLBACK
```

Figura 147. *Ejecución de las pruebas de la base de datos: 16 de 16 aprobadas.*

8.3 Pruebas de validación y permisos por caso de uso

Tabla 50. *Pruebas de validación y permisos*

CU	Caso de prueba	Resultado esperado	Estado
CU01	Contraseña incorrecta 5 veces seguidas	La cuenta se bloquea 15 minutos	Aprobada
CU01	Usuario inexistente	Mismo mensaje que contraseña incorrecta	Aprobada
CU02	Recepcionista abre /usuarios	Página 403 «Acceso denegado»	Aprobada
CU02	Quitar al médico el permiso «citas.atender»	Desaparece el botón «Atendida» y el servidor rechaza la acción	Aprobada
CU02	Iniciar sesión, fallar una	La bitácora muestra el inicio, el	Aprobada

CU	Caso de prueba	Resultado esperado	Estado
	contraseña y registrar una cita	intento fallido y la operación con usuario, IP y resultado	
CU02	Recepcionista abre /bitacora	Página 403: solo el administrador tiene «bitacora.ver»	Aprobada
CU04	Registro en línea con CI existente y otra fecha de nacimiento	Se pide acudir a recepción; no se vincula	Aprobada
CU06	Registrar «PEDIATRIA» cuando existe «Pediatría»	Rechazo por nombre normalizado repetido	Aprobada
CU07	Desactivar un médico con citas pendientes	Citas pendientes canceladas; historial conservado	Aprobada
CU08	Hora final 08:00 y hora inicial 12:00	«La hora final debe ser posterior a la hora inicial»	Aprobada
CU10	Reprogramar una cita atendida	Se rechaza y se indica programar una nueva	Aprobada
CU11	Paciente cancela a menos de 2 horas	Se rechaza y se indica llamar a recepción	Aprobada
CU12	Marcar atendida una cita cancelada	«Una cita cancelada no puede marcarse como atendida»	Aprobada
CU13	Médico intenta ver la agenda de otro médico	Solo ve su propia agenda	Aprobada
CU13	Abrir la agenda semanal y pasar a la semana siguiente	Siete días de lunes a domingo con sus citas; días sin atención marcados	Aprobada
CU14	Descargar reporte en CSV	Archivo .csv con encabezados y tildes correctas	Aprobada

8.4 Resultado de las pruebas

Las 10 pruebas mínimas del examen, las 16 pruebas de la base de datos y las pruebas de validación y permisos se aprobaron. Las reglas críticas —citas superpuestas, horarios fuera de atención, fechas pasadas, CI duplicado y transiciones de estado— se controlan en dos niveles: el servidor PHP explica el problema al usuario y PostgreSQL garantiza la integridad aunque alguien intente saltarse la aplicación.

CONCLUSIÓN

Se analizó el proceso de gestión de citas de la Clínica Vida Salud y se lo transformó en un modelo de 14 casos de uso organizados en 5 paquetes (seguridad, pacientes, médicos, citas y reportes), con sus requisitos funcionales y no funcionales y sus reglas de negocio.

Siguiendo el PUDS en dos ciclos se elaboraron los modelos de captura de requisitos, análisis (clases de interfaz, control y entidad, diagramas de comunicación y paquetes), diseño (arquitectura física y lógica, diagramas de secuencia y diseño de datos), implementación y pruebas.

Se diseñó e implementó una base de datos normalizada en PostgreSQL de 14 tablas que, además de llaves primarias y foráneas, usa restricciones CHECK, UNIQUE y EXCLUDE, siete funciones de trigger y tres vistas, garantizando que un médico nunca tenga citas superpuestas, que ninguna cita se pierda y que la bitácora de acciones no pueda alterarse.

Se desarrolló el sistema web con PHP, PDO pdo_pgsql, HTML5, CSS3 y JavaScript sobre Apache (XAMPP), con un archivo por caso de uso, validación en el navegador y en el servidor, contraseñas cifradas, sesiones seguras, permisos por rol y protección contra inyección SQL, XSS y CSRF. La interfaz es clara, profesional y adaptable al celular.

Las 10 pruebas mínimas del examen y las 16 pruebas de la base de datos se aprobaron, demostrando que el sistema resuelve los problemas identificados: elimina la duplicación de citas, mantiene datos correctos de los pacientes, muestra los horarios disponibles, controla cancelaciones e inasistencias, da acceso inmediato al historial y genera los reportes administrativos.

RECOMENDACIÓN

- Cambiar las contraseñas de los usuarios de demostración y la clave de PostgreSQL antes de usar el sistema con datos reales.
- Publicar el sistema con HTTPS y programar copias de seguridad diarias de la base de datos con pg_dump.
- Capacitar a la recepción y a los médicos con los usuarios de prueba antes de la puesta en marcha.
- En una siguiente versión, enviar los recordatorios por correo o WhatsApp y añadir la historia clínica (diagnóstico y receta).
- Mantener la regla de un archivo por caso de uso y agregar pruebas automáticas cuando se incorporen nuevas funciones.
- Revisar mensualmente los reportes de inasistencias y de citas por especialidad para ajustar los horarios de los médicos.

BIBLIOGRAFÍA

- Jacobson, I., Booch, G. y Rumbaugh, J. (2000). El Proceso Unificado de Desarrollo de Software. Addison-Wesley.
- Booch, G., Rumbaugh, J. y Jacobson, I. (2006). El Lenguaje Unificado de Modelado. Manual de referencia (2.ª ed.). Addison-Wesley.
- Object Management Group (2017). OMG Unified Modeling Language (UML), Version 2.5.1. https://www.omg.org/spec/UML/
- Pressman, R. y Maxim, B. (2021). Ingeniería del software: un enfoque práctico (9.ª ed.). McGraw-Hill.
- Laudon, K. y Laudon, J. (2016). Sistemas de información gerencial (14.ª ed.). Pearson.

- Elmasri, R. y Navathe, S. (2016). Fundamentos de sistemas de bases de datos (7.ª ed.). Pearson.
- The PHP Group (2026). Manual de PHP: PDO, password_hash y sesiones. https://www.php.net/manual/es/
- PostgreSQL Global Development Group (2026). PostgreSQL Documentation: Constraints, Triggers, btree_gist. https://www.postgresql.org/docs/
- Mozilla (2026). MDN Web Docs: HTML, CSS y JavaScript. https://developer.mozilla.org/es/
- OWASP Foundation (2021). OWASP Top 10: riesgos de seguridad en aplicaciones web. https://owasp.org/Top10/
- Apache Friends (2026). XAMPP. https://www.apachefriends.org/

CÓDIGO FUENTE

El código fuente organizado y comentado está publicado en GitHub:

https://github.com/edisonayala000-netizen/clinica-vida-salud

Figura 148. *Código QR de acceso al repositorio del proyecto.*

ANEXOS

Anexo A. Usuarios de demostración

Tabla 51. *Usuarios de demostración*

Usuario	Contraseña	Rol	Para probar
admin	Clinica2026*	Administrador	Usuarios, permisos, especialidades, médicos, horarios y reportes
recepcion	Clinica2026*	Recepcionista	Pacientes, programar, reprogramar, cancelar y asistencia
recepcion2	Clinica2026*	Recepcionista	Probar dos recepcionistas reservando a la vez (Prueba 4)
dr.perez	Clinica2026*	Médico	Mi agenda y marcar citas como atendidas
maria.lopez	Clinica2026*	Paciente	Mi perfil, reservar, mis citas y recordatorios

Anexo B. Reportes adicionales

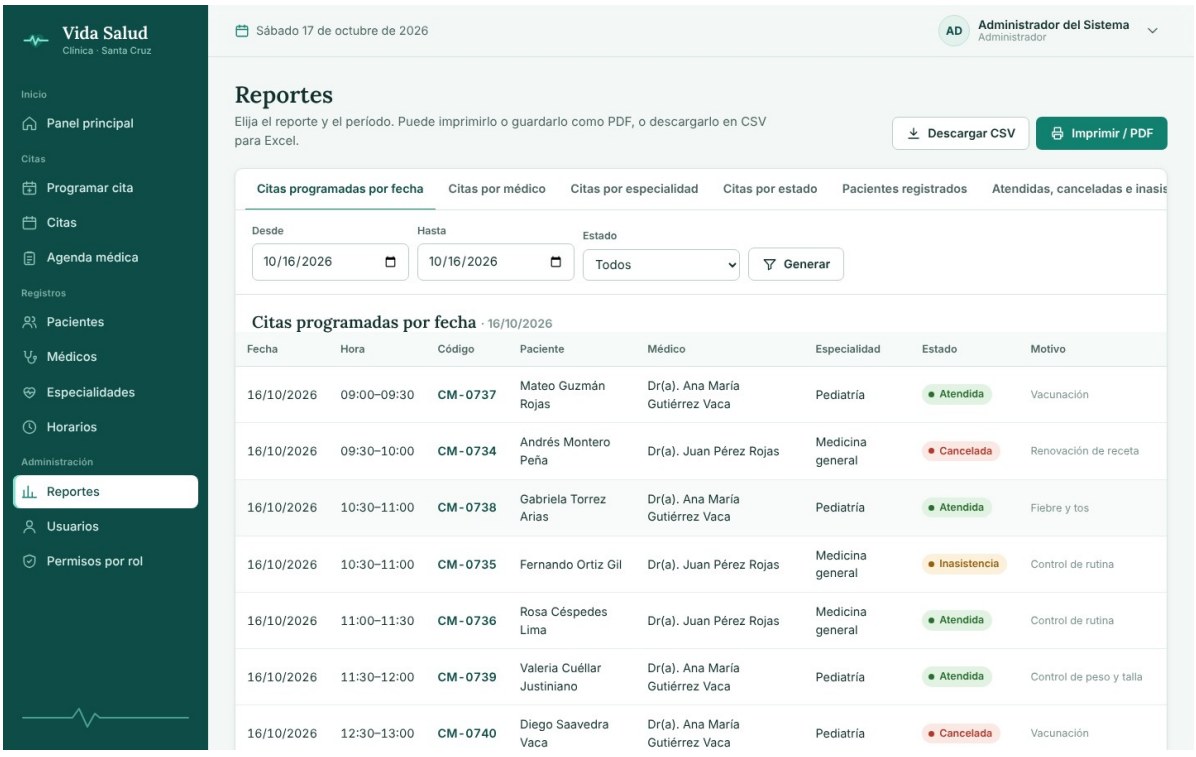

Figura 149. *Reporte de citas programadas para una fecha.*

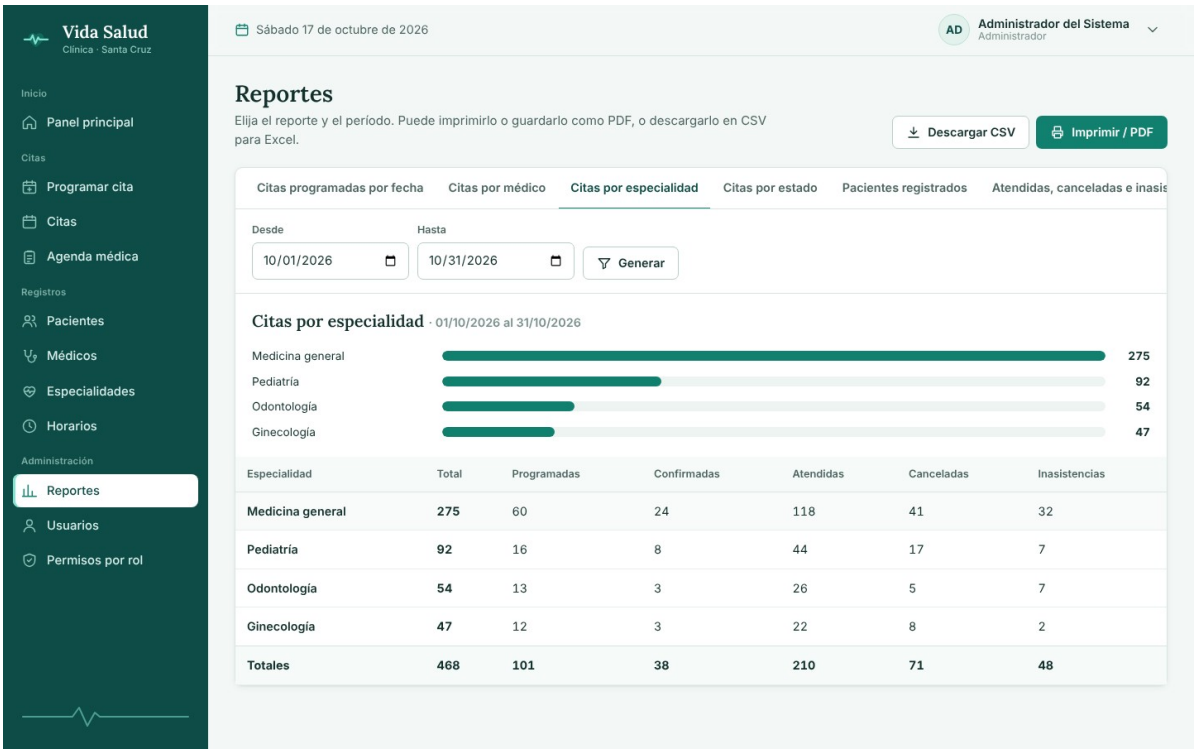

Figura 150. *Reporte de citas por especialidad.*

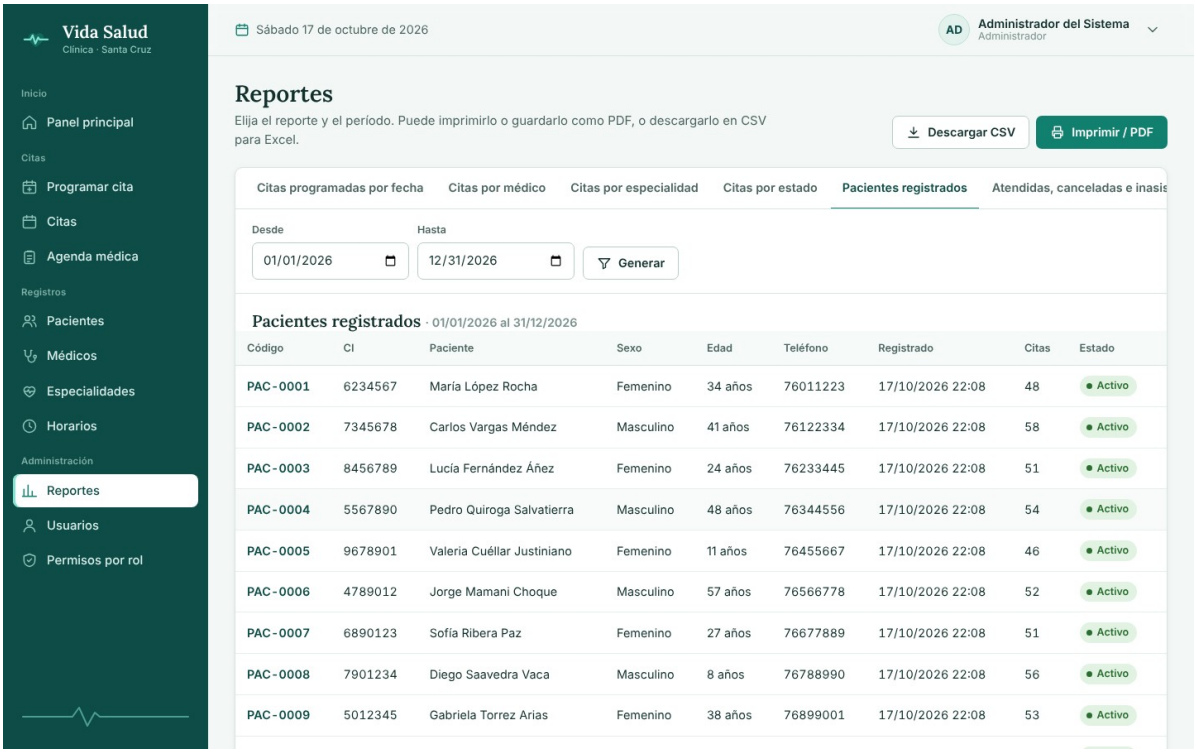

Figura 151. *Reporte de pacientes registrados.*

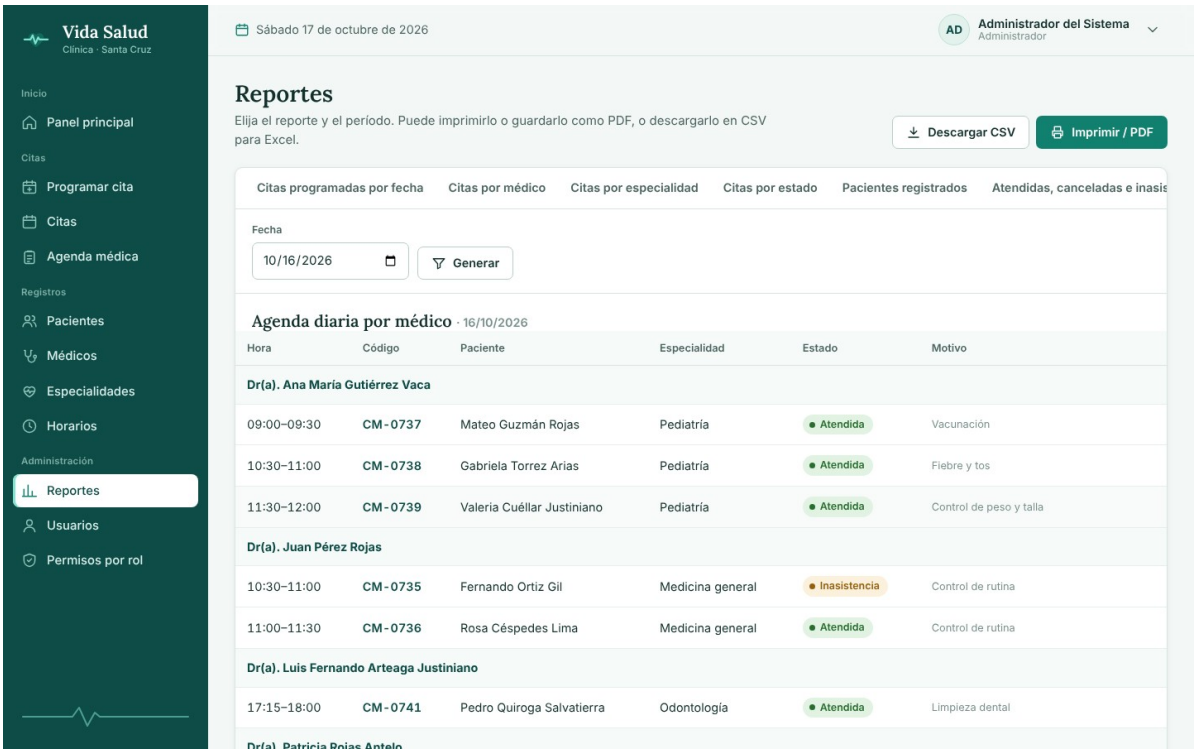

Figura 152. *Agenda diaria de cada médico.*

Anexo C. Paneles por rol

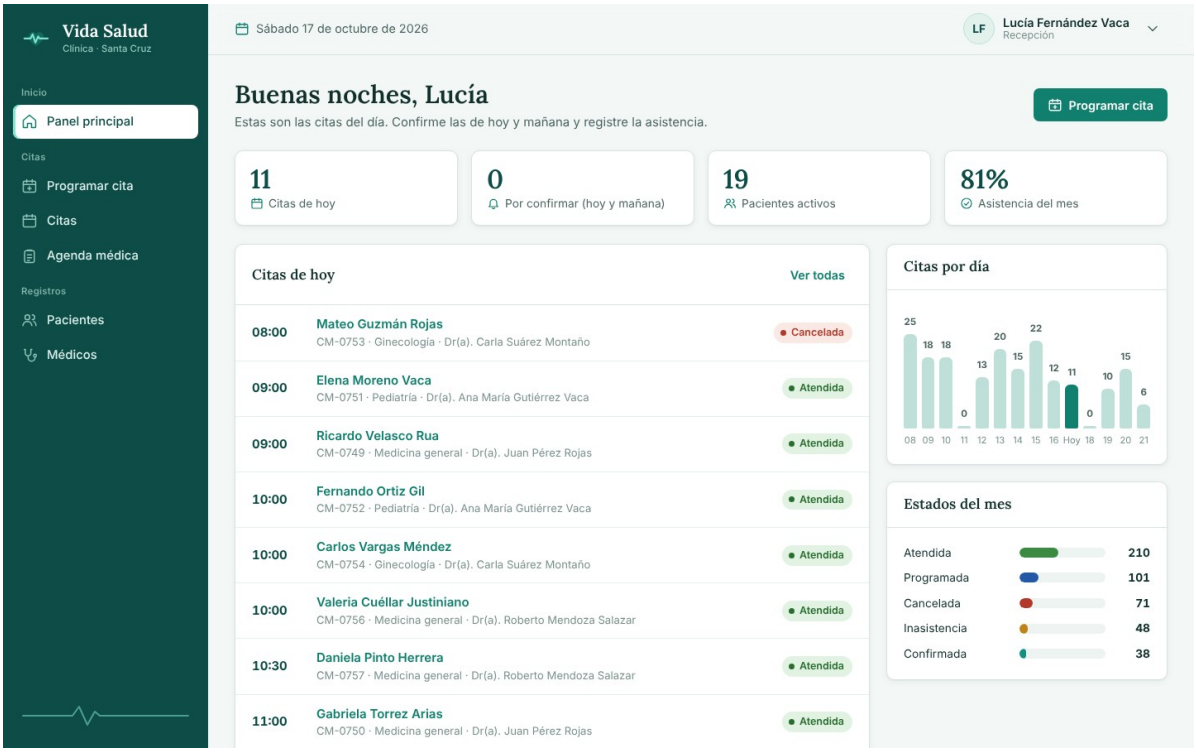

Figura 153. *Panel de la recepcionista.*

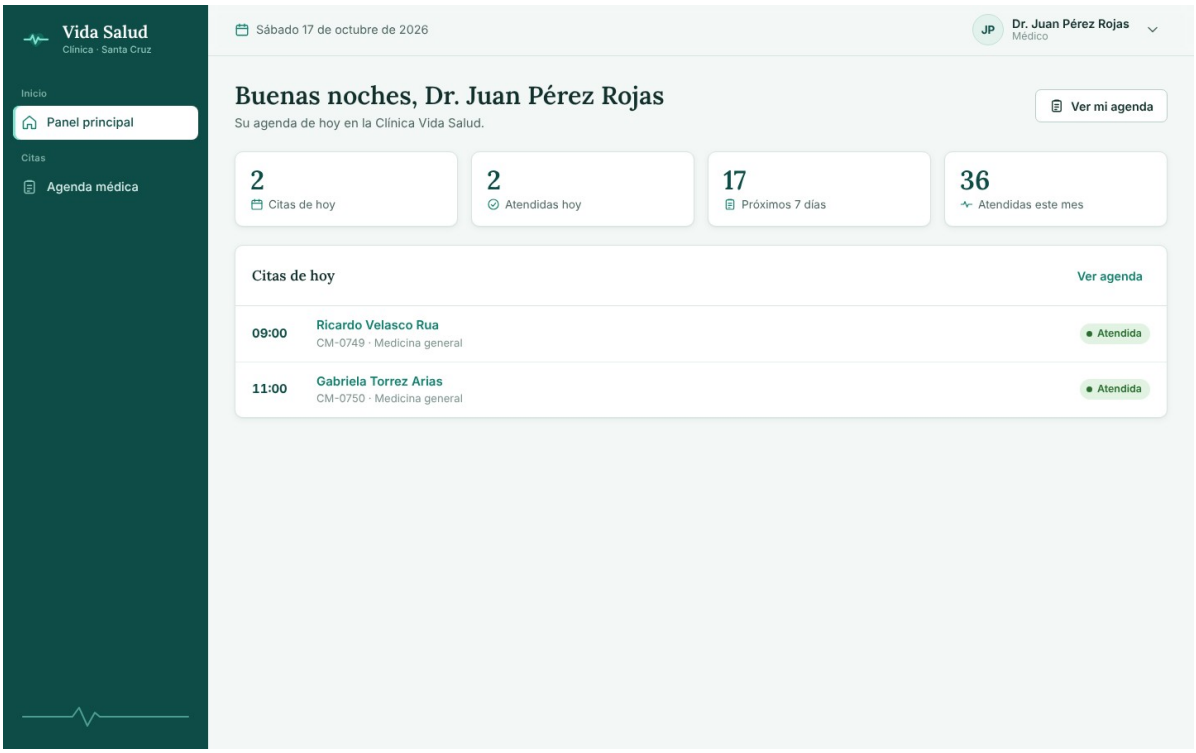

Figura 154. *Panel del médico.*

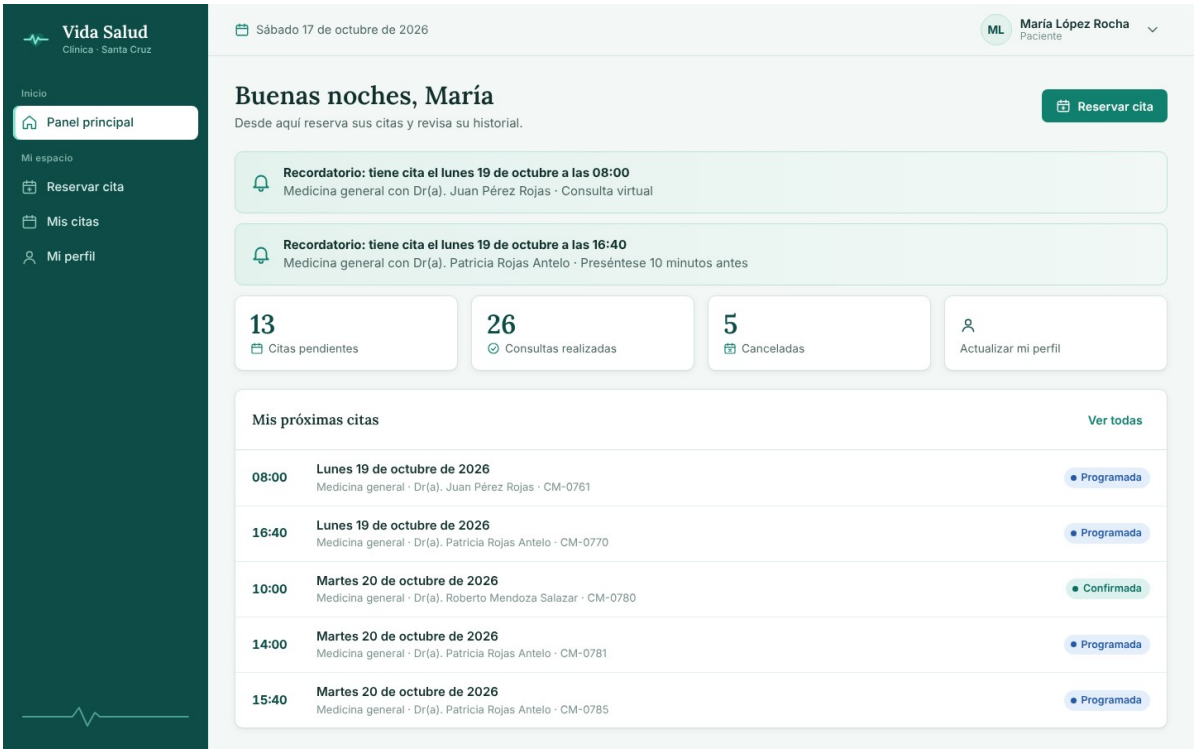

Figura 155. *Panel del paciente con recordatorios automáticos.*

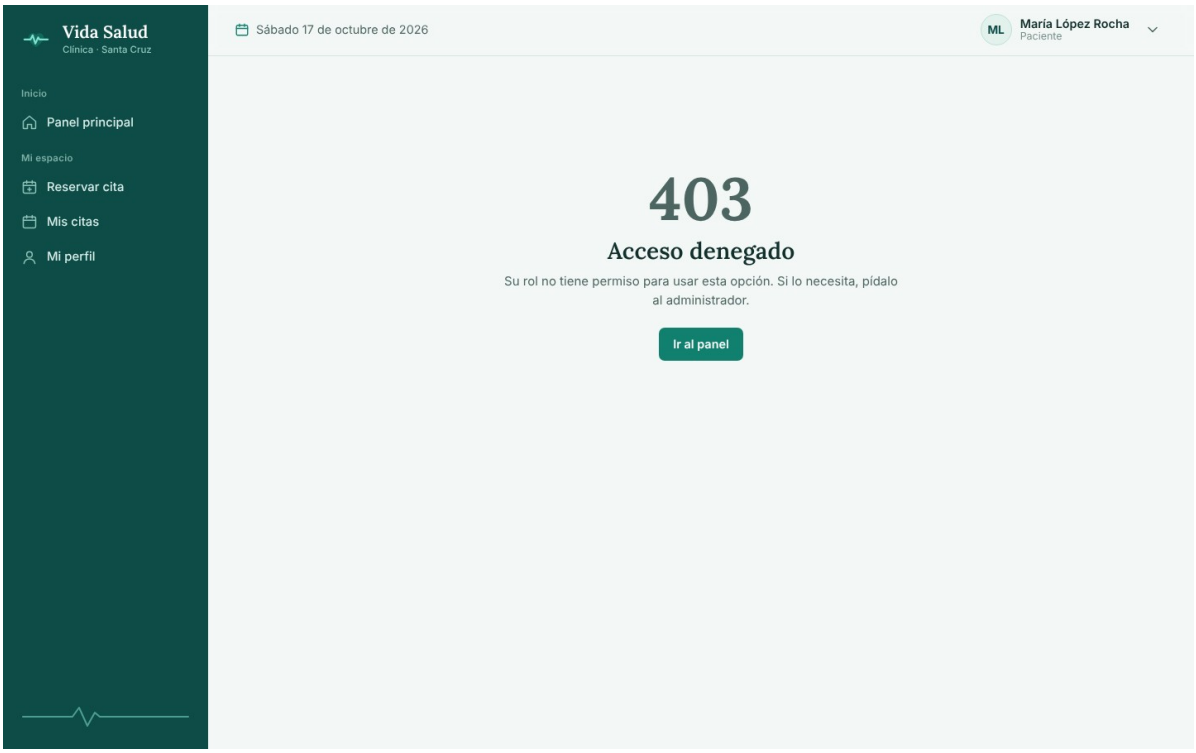

Figura 156. *Un paciente intenta abrir una página de administración: acceso denegado.*

Anexo D. Glosario

Tabla 52. *Glosario*

Término	Significado
Caso de uso (CU)	Funcionalidad que el sistema ofrece a un actor para lograr un objetivo.
IU / CTR / CE	Clases de análisis de interfaz, control y entidad.
PDO	Capa de PHP para conectarse a bases de datos con sentencias preparadas.
Transacción	Grupo de operaciones que se guardan todas o ninguna.
EXCLUDE	Restricción de PostgreSQL que impide filas con rangos que se cruzan.
Trigger	Función de la base de datos que se ejecuta automáticamente al insertar, modificar o borrar.
CSRF	Ataque que envía formularios en nombre del usuario; se evita con un token secreto.
XSS	Inyección de código en páginas; se evita escapando la salida.
Hash bcrypt	Transformación irreversible de la contraseña con sal aleatoria.